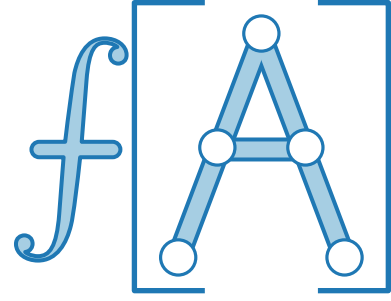


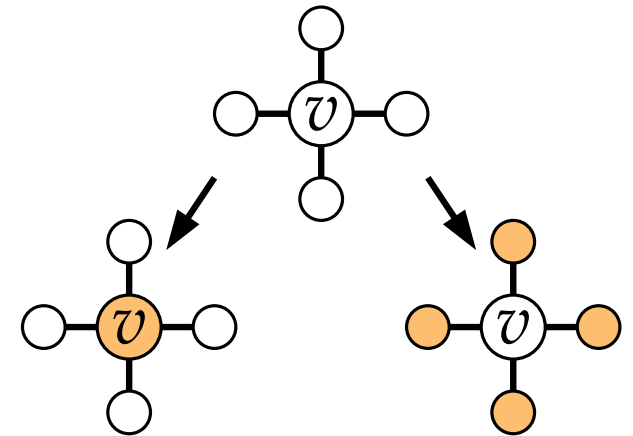
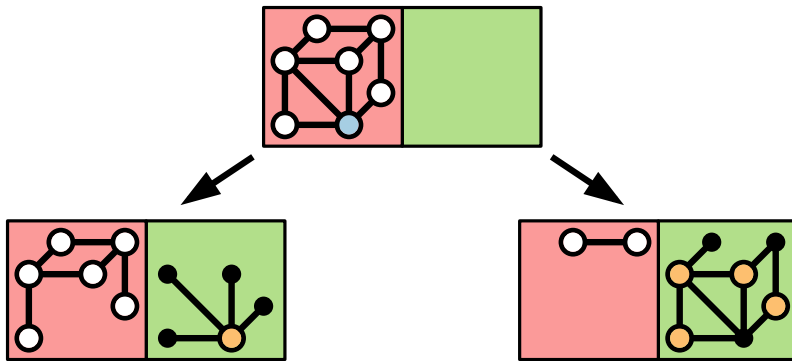


Fortgeschrittene Algorithmen

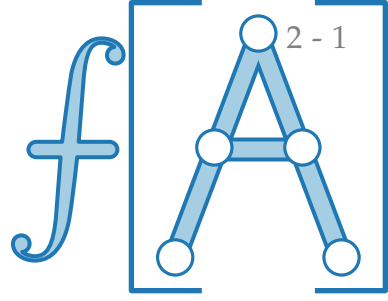
20. Vorlesung

Parametrisierte Algorithmen

Teil I: KNOTENÜBERDECKUNG

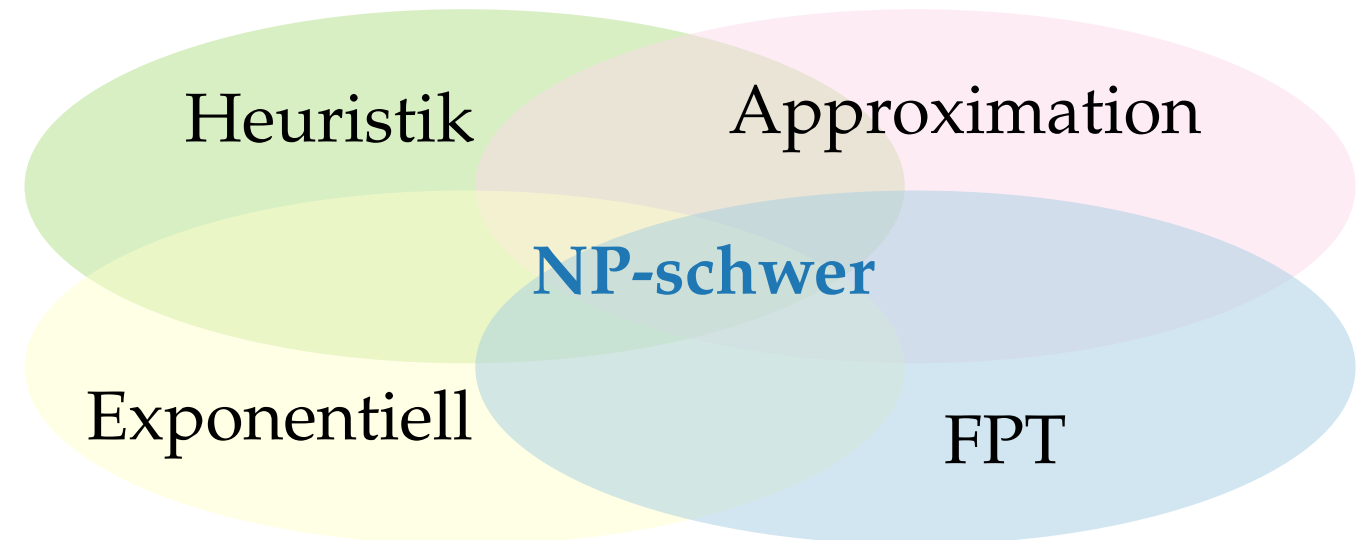


NP-schwere Probleme lösen

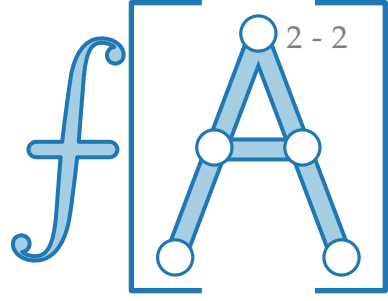


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

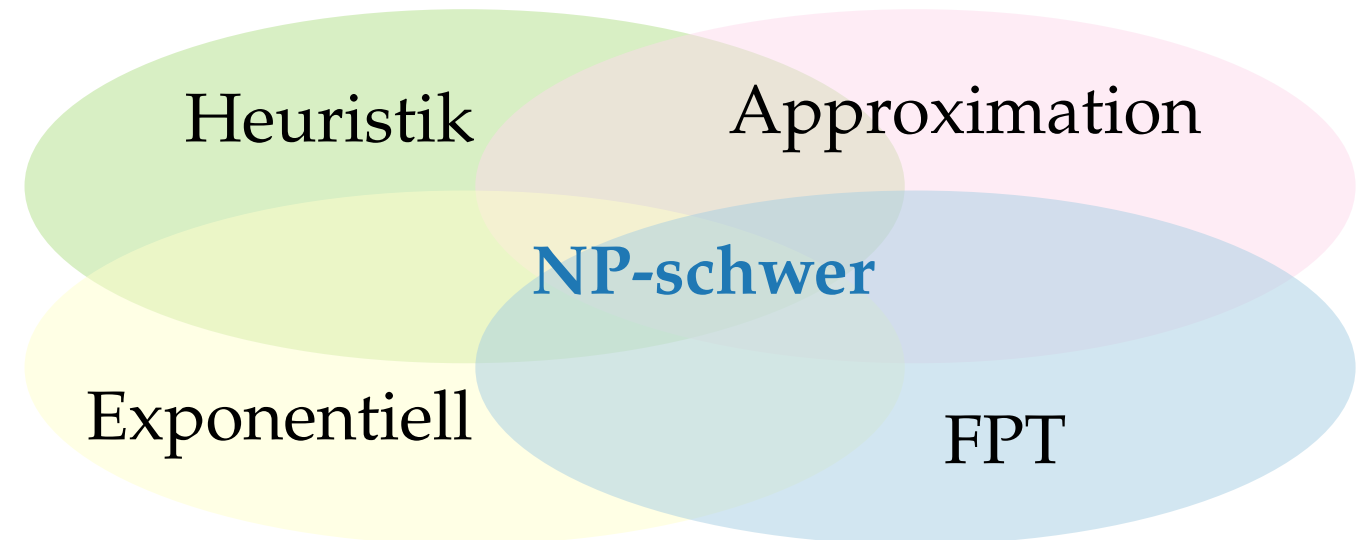


NP-schwere Probleme lösen

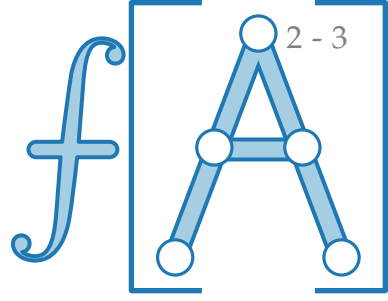


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen



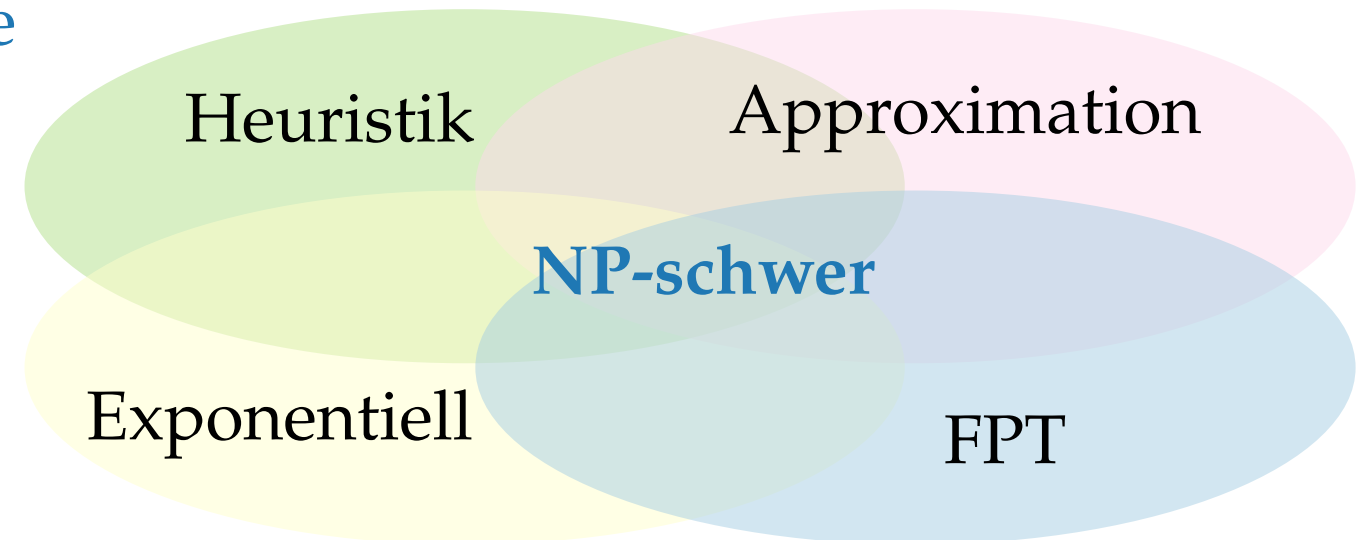
NP-schwere Probleme lösen



Was können wir tun?

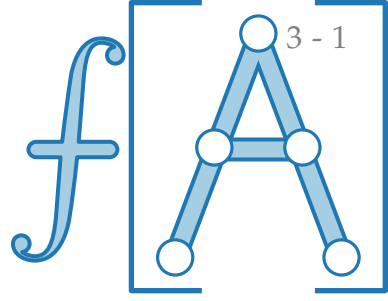
- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

heute



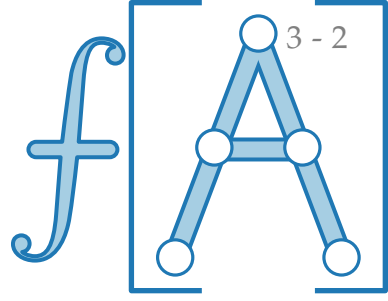
Ein Beispiel: VERTEXCOVER

Def. Sei $G = (V, E)$ ein ungerichteter Graph.



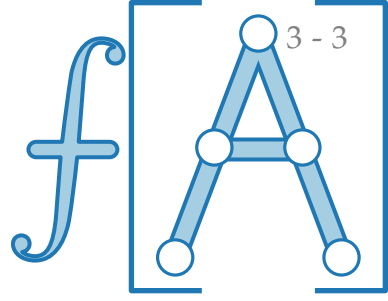
Ein Beispiel: VERTEXCOVER

Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt

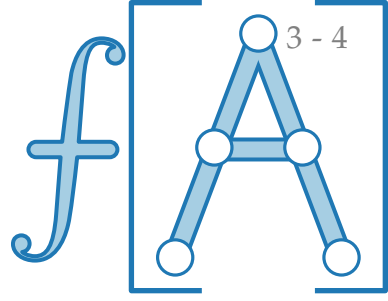


Ein Beispiel: VERTEXCOVER

Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.



Ein Beispiel: VERTEXCOVER



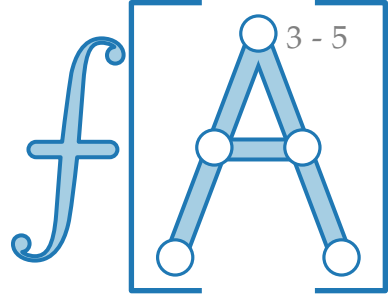
Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

Ein Beispiel: VERTEXCOVER



Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

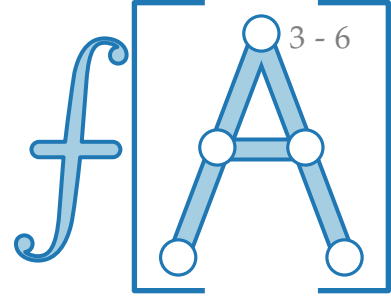
Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

Optimierungsproblem

Ein Beispiel: VERTEXCOVER



Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

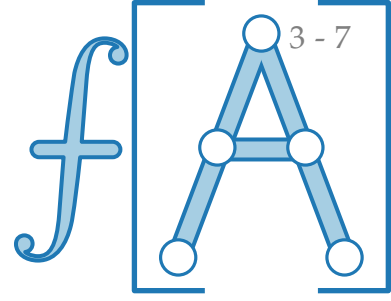
Ges. eine kleinste Knotenüberdeckung von G

Optimierungsproblem

Problem.

Entscheidungsproblem

Ein Beispiel: VERTEXCOVER



Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

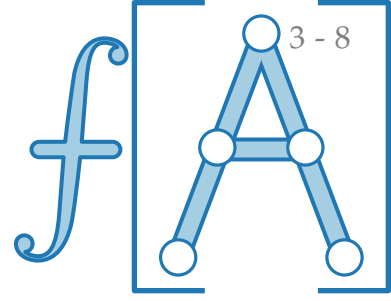
Ges. eine kleinste Knotenüberdeckung von G

Optimierungsproblem

Problem. k -KNOTENÜBERDECKUNG (k -VC)

Entscheidungsproblem

Ein Beispiel: VERTEXCOVER



Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

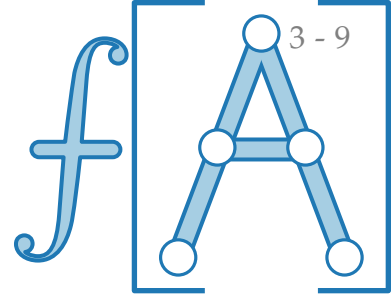
Optimierungsproblem

Problem. k -KNOTENÜBERDECKUNG (k -VC)

Geg. Graph G , natürliche Zahl k

Entscheidungsproblem

Ein Beispiel: VERTEXCOVER



Def. Sei $G = (V, E)$ ein ungerichteter Graph.
 $C \subseteq V$ heißt **Knotenüberdeckung** (engl. **vertex cover**)
von G , falls für alle $uv \in E$ gilt $\{u, v\} \cap C \neq \emptyset$.

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

Optimierungsproblem

Problem. k -KNOTENÜBERDECKUNG (k -VC)

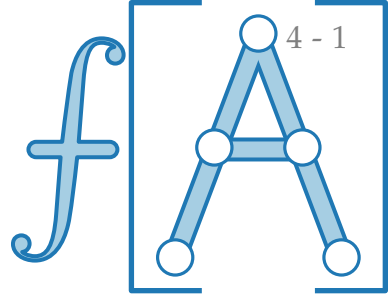
Geg. Graph G , natürliche Zahl k

Ges. Knotenüberdeckung der Größe $\leq k$ von G ,
falls eine solche existiert (sonst false)

Entscheidungsproblem

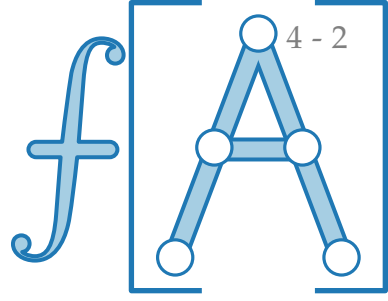
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde



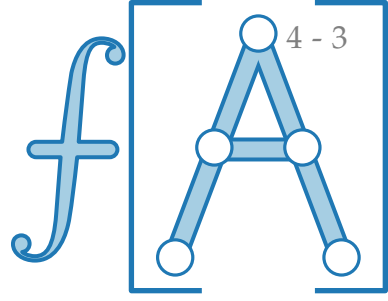
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$)

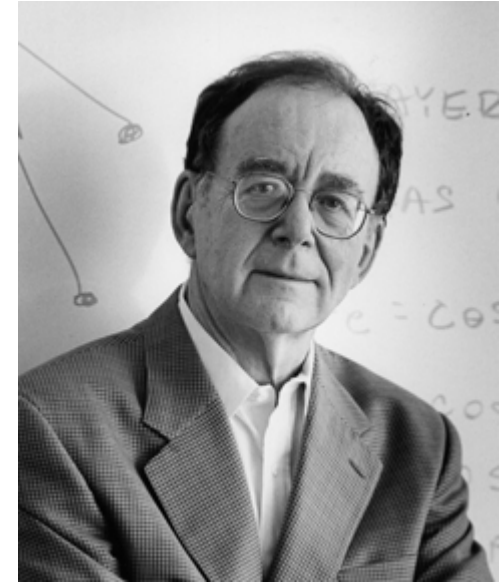


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

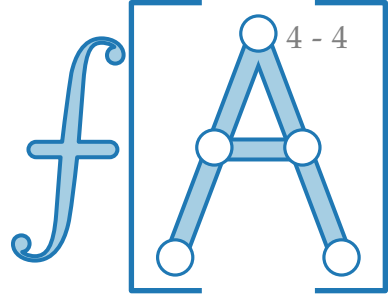


Richard M. Karp
*1935 Boston, MA

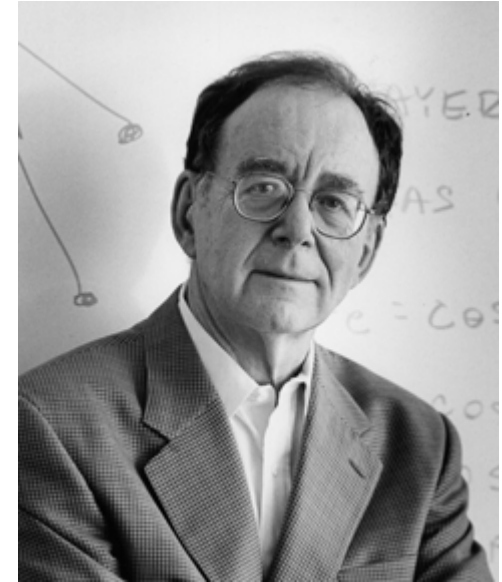


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

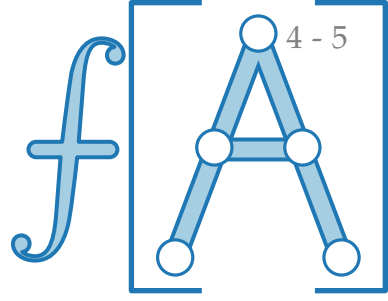


Richard M. Karp
*1935 Boston, MA

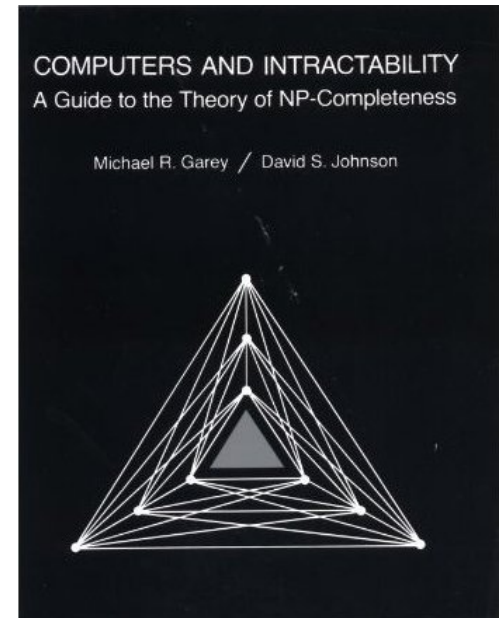
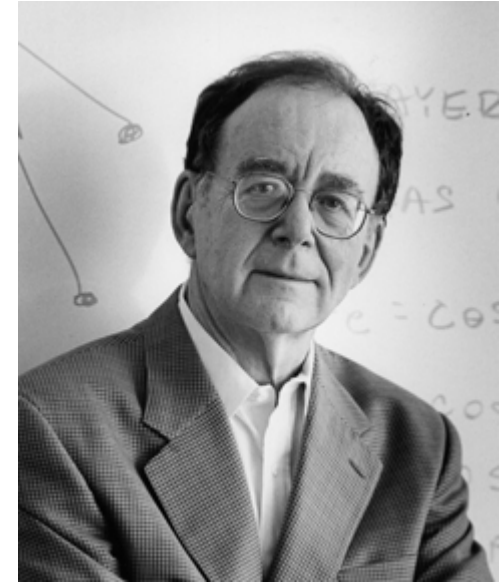


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

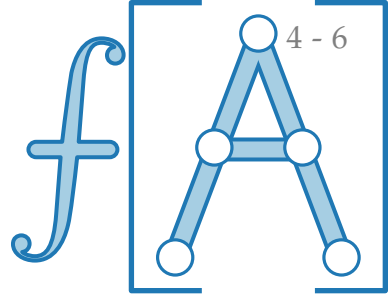


Richard M. Karp
*1935 Boston, MA

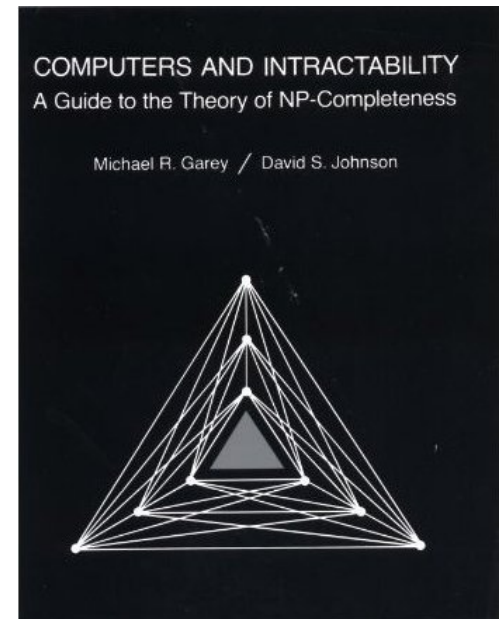
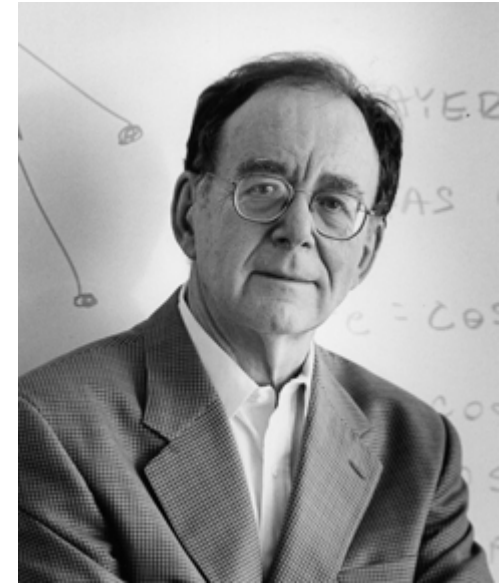


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]
- approximierbar:

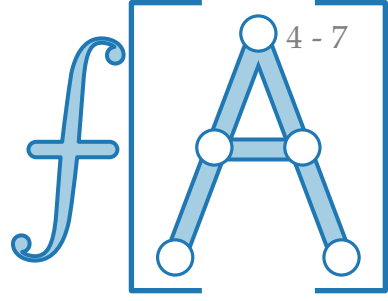


Richard M. Karp
*1935 Boston, MA

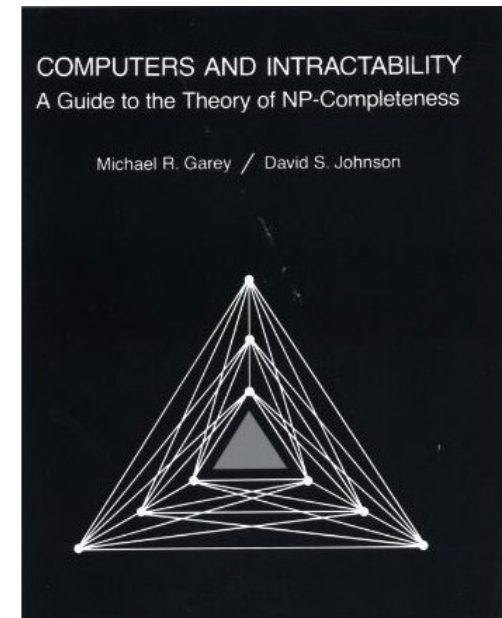
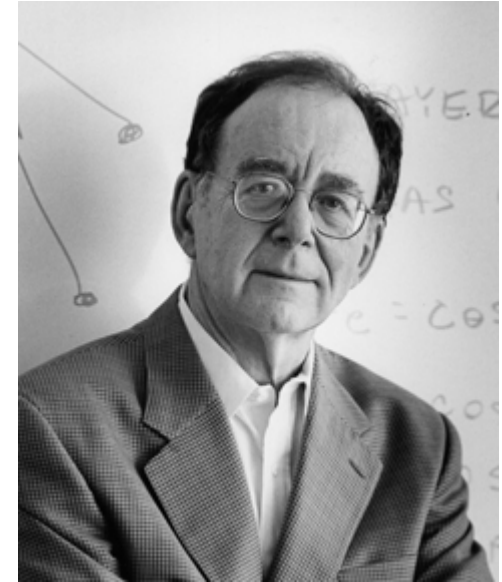


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]
- approximierbar:
Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.



Richard M. Karp
*1935 Boston, MA



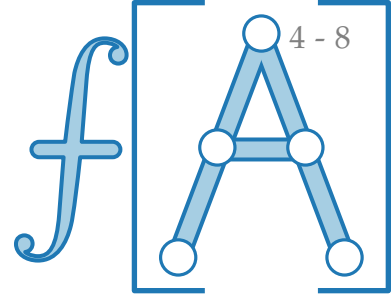
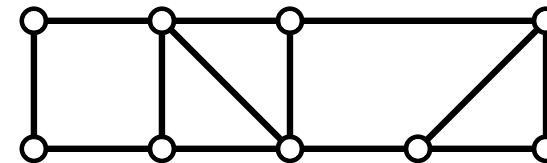
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

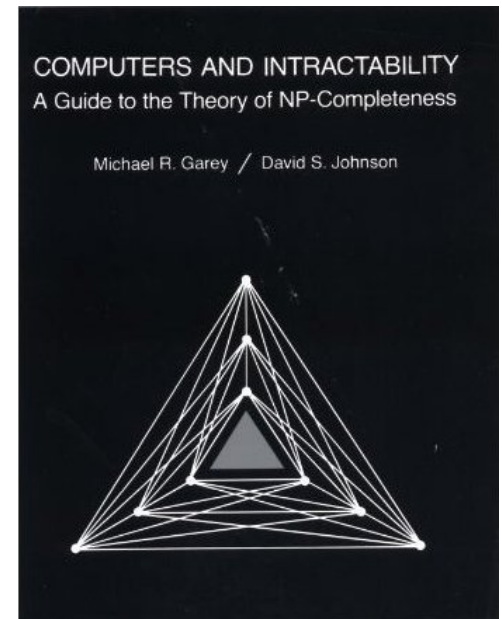
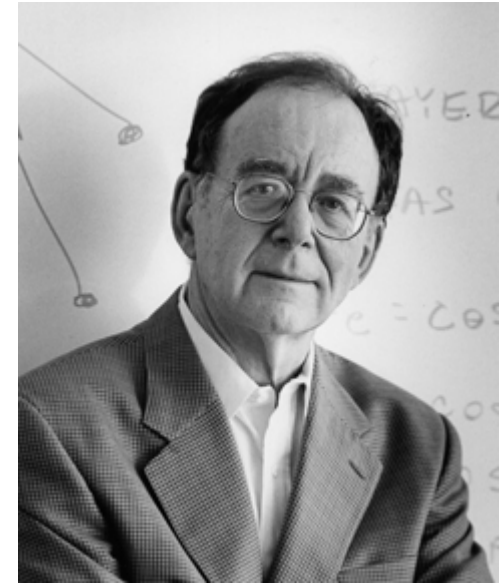
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

- approximierbar:

Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.

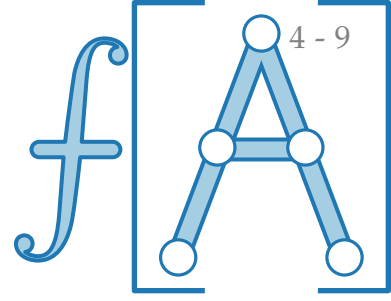
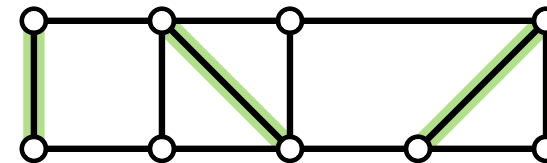


Richard M. Karp
*1935 Boston, MA

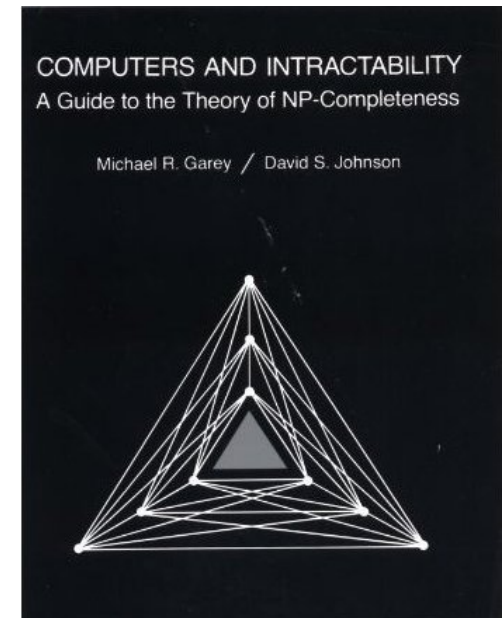
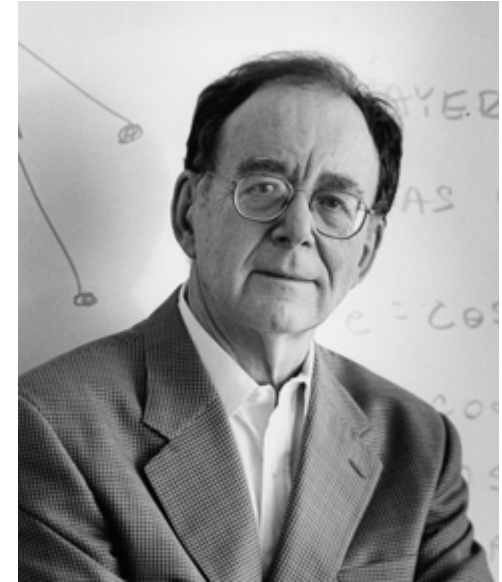


Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]
- approximierbar:
Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.



Richard M. Karp
*1935 Boston, MA



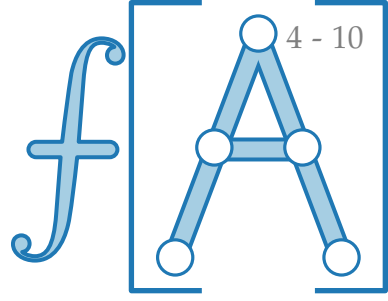
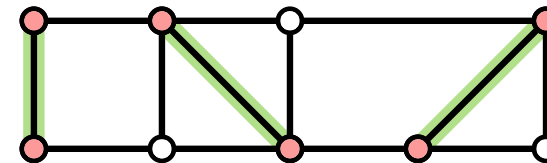
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

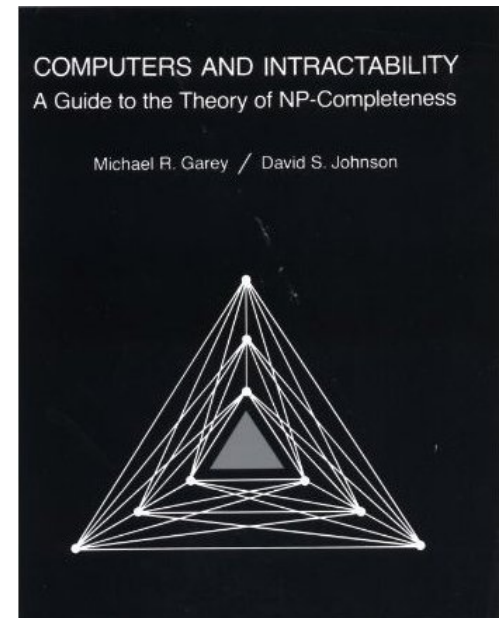
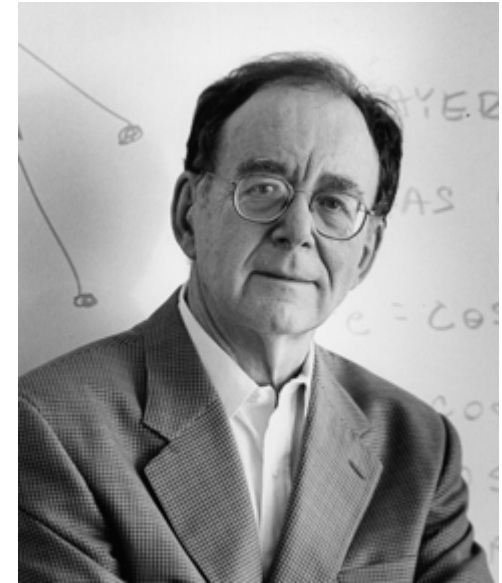
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

- approximierbar:

Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.



Richard M. Karp
*1935 Boston, MA



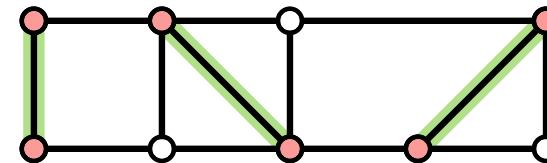
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

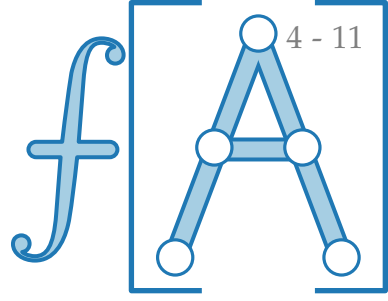
- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

- approximierbar:

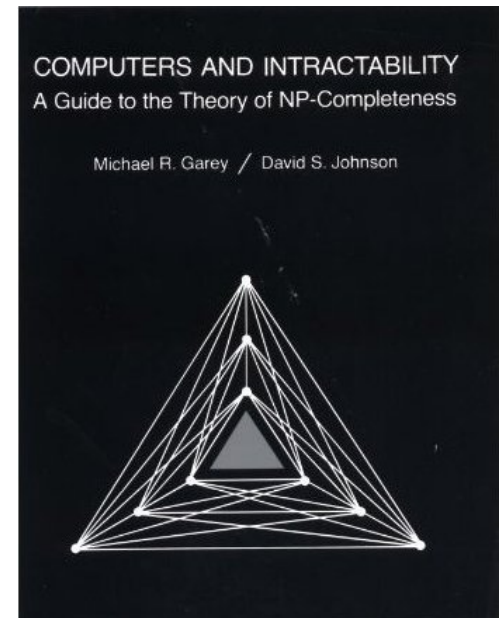
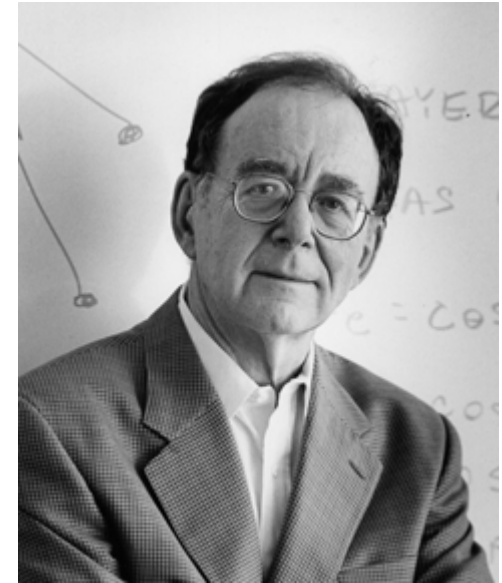
Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.



- ..., aber nicht beliebig gut:



Richard M. Karp
*1935 Boston, MA



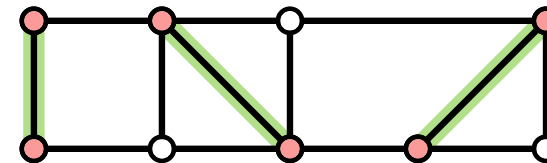
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

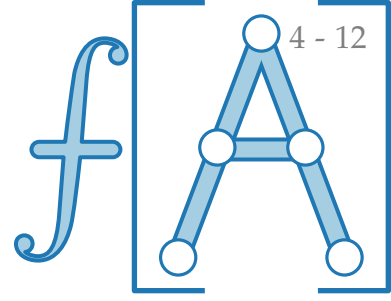
- approximierbar:

Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.

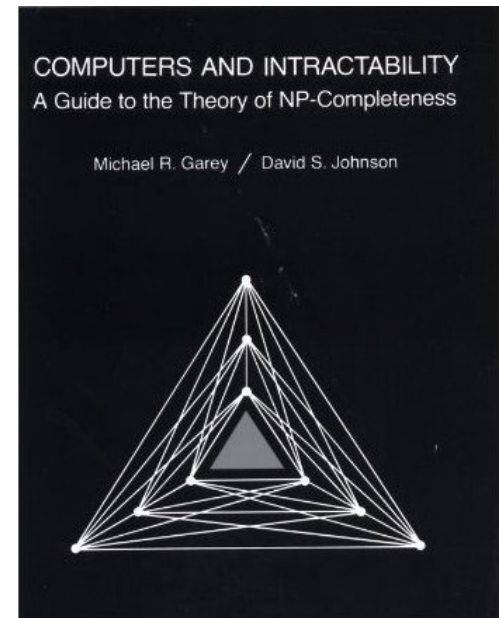
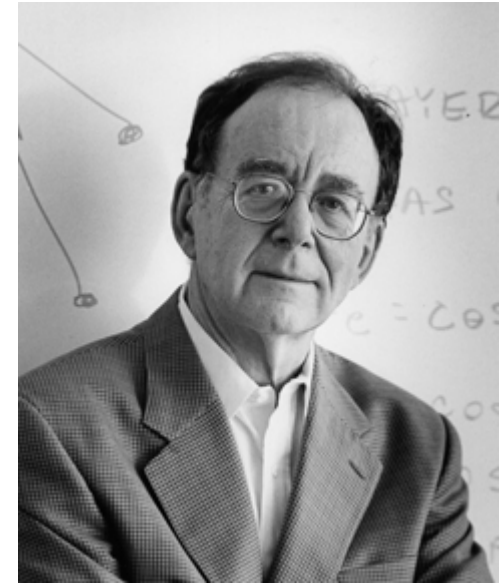


- ..., aber nicht beliebig gut:

Es gibt keine Faktor-1,3606-Approximation für VC



Richard M. Karp
*1935 Boston, MA



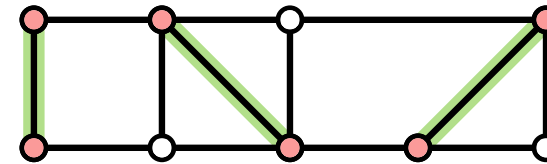
Resultate

- eines der ersten Probleme, dessen NP-Schwere gezeigt wurde
($\text{SAT} \preceq_p \text{CLIQUE} \preceq_p \text{VC} \preceq_p \dots$) [Karp, 1972]

- eines der sechs grundlegenden NP-vollständigen Probleme
in dem Klassiker: [Garey & Johnson, 1979]

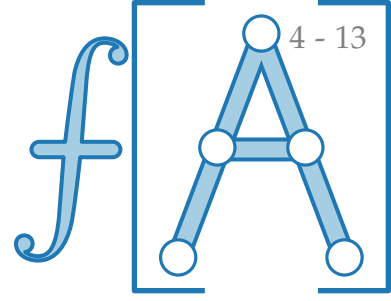
- approximierbar:

Nicht-erweiterbares **Matching** „liefert“
Faktor-2-Approximation für **VC**.

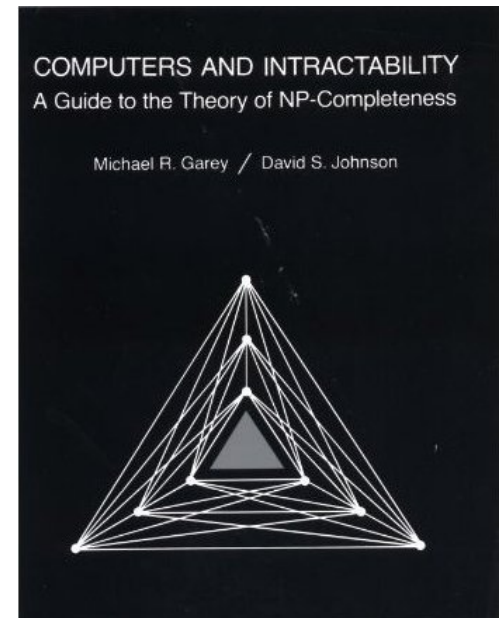
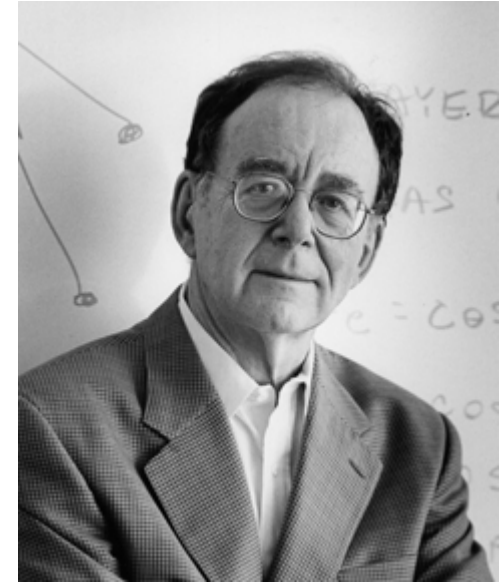


- ..., aber nicht beliebig gut:

Es gibt keine Faktor-1,3606-Approximation für VC,
falls $\mathcal{P} \neq \mathcal{NP}$. [Dinur & Safra, 2004]

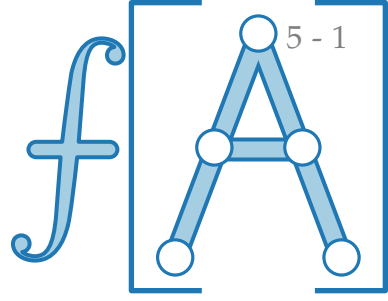


Richard M. Karp
*1935 Boston, MA



Ein exakter Algorithmus für k -VC

BRUTEFORCEVC(Graph G , Integer k)

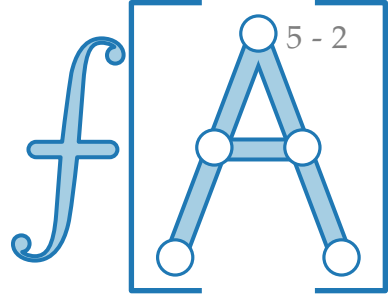


Ein exakter Algorithmus für k -VC

```
BRUTEFORCEVC(Graph  $G$ , Integer  $k$ )
```

```
  foreach  $C \in \binom{V}{k}$  do  
    // teste, ob  $C$  VC
```

```
  return false
```



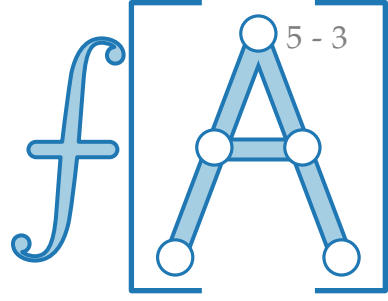
Ein exakter Algorithmus für k -VC

```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do  
    // teste, ob  $C$  VC  
    vc = true
```

```
    if vc then  
      return  $C$ 
```

```
  return false
```



Ein exakter Algorithmus für k -VC

BRUTEFORCEVC(Graph G , Integer k)

foreach $C \in \binom{V}{k}$ **do**
 // teste, ob C VC

$vc = \text{true}$

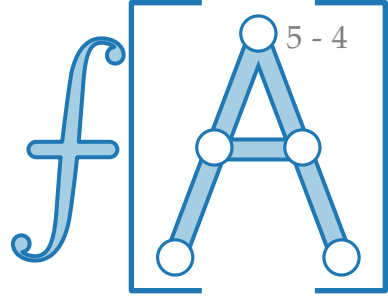
foreach $uv \in E$ **do**

 |

if vc **then**

 | **return** C

return false



Ein exakter Algorithmus für k -VC

BRUTEFORCEVC(Graph G , Integer k)

foreach $C \in \binom{V}{k}$ **do**

 // teste, ob C VC

$vc = \text{true}$

foreach $uv \in E$ **do**

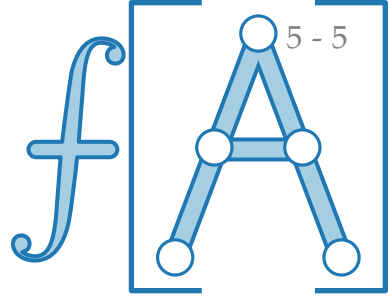
if $\{u, v\} \cap C = \emptyset$ **then**

$vc = \text{false}$

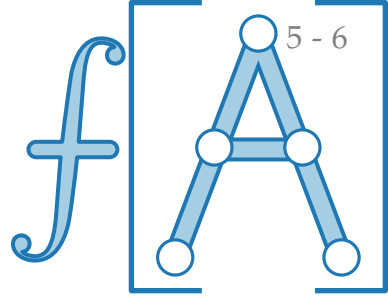
if vc **then**

return C

return false



Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
         $\perp$  vc = false
```

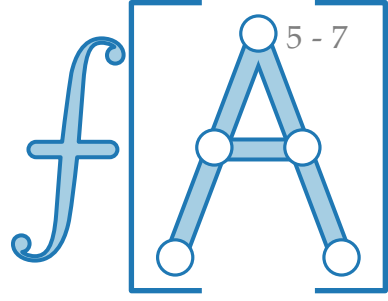
```
    if vc then
```

```
       $\perp$  return  $C$ 
```

```
  return false
```

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
         $\perp$  vc = false
```

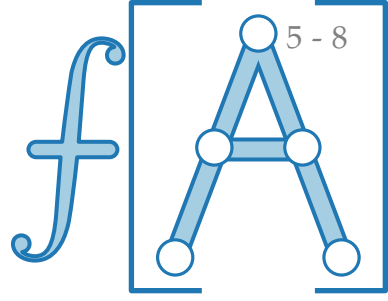
```
    if vc then
```

```
       $\perp$  return  $C$ 
```

```
  return false
```

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then  
        └ vc = false
```

```
    if vc then
```

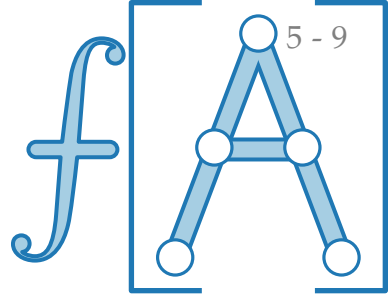
```
      └ return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(\quad)$$

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        └ vc = false
```

```
    if vc then
```

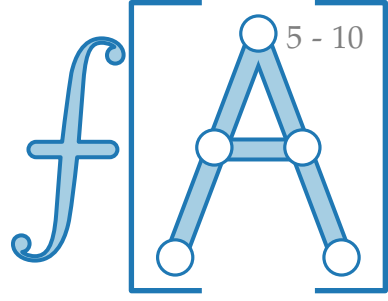
```
      └ return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then  
        └ vc = false
```

```
    if vc then
```

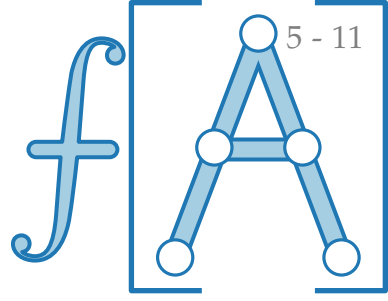
```
      └ return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        | vc = false
```

```
    if vc then
```

```
      | return  $C$ 
```

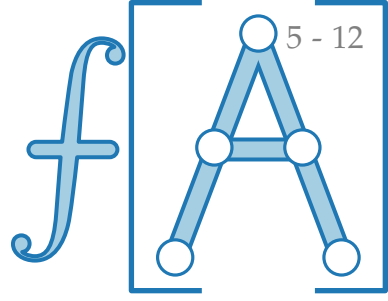
```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

$$\mathcal{O}(m)$$

Laufzeit.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        vc = false
```

```
    if vc then
```

```
      return  $C$ 
```

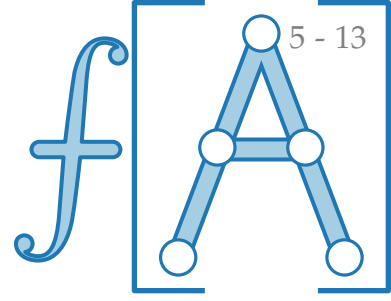
```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

$$\mathcal{O}(m)$$

Laufzeit. $\mathcal{O}(n^k m)$

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        | vc = false
```

```
    if vc then
```

```
      | return  $C$ 
```

```
  return false
```

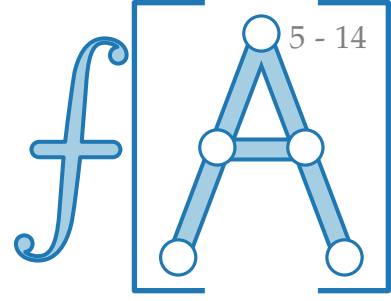
$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

$\mathcal{O}(m)$

Dies ist *nicht* polynomiell in der Größe der Eingabe, da k keine Konstante, sondern Teil der Eingabe.

Laufzeit. $\mathcal{O}(n^k m)$

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        | vc = false
```

```
    if vc then
```

```
      | return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

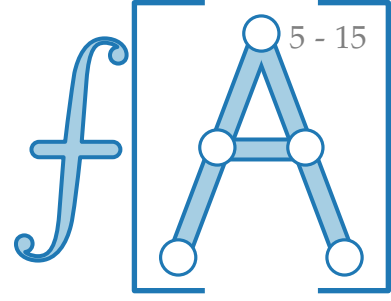
$\mathcal{O}(m)$

Dies ist *nicht* polynomiell in der Größe der Eingabe, da k keine Konstante, sondern Teil der Eingabe.

Laufzeit. $\mathcal{O}(n^k m)$

Def. Ein Problem, das in $\mathcal{O}(n^{f(k)})$ Zeit gelöst werden kann, heißt **schichtweise polynomiell** (**slice-wise polynomial**) bzgl. k .

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        vc = false
```

```
    if vc then
```

```
      return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

$\mathcal{O}(m)$

Dies ist *nicht* polynomiell in der Größe der Eingabe, da k keine Konstante, sondern Teil der Eingabe.

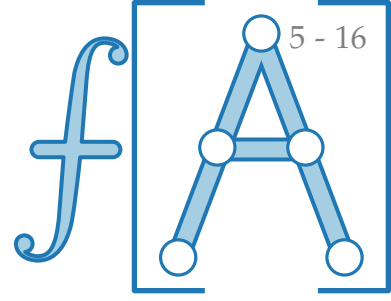
Laufzeit. $\mathcal{O}(n^k m)$

Def.

Ein Problem, das in $\mathcal{O}(n^{f(k)})$ Zeit gelöst werden kann, heißt **schichtweise polynomiell** (**slice-wise polynomial**) bzgl. k .

Die Klasse der schichtweise polynomiellen Probleme heißt $\mathcal{XP}$.

Ein exakter Algorithmus für k -VC



```
BRUTEFORCEVC(Graph G, Integer k)
```

```
  foreach  $C \in \binom{V}{k}$  do
```

```
    // teste, ob  $C$  VC
```

```
    vc = true
```

```
    foreach  $uv \in E$  do
```

```
      if  $\{u, v\} \cap C = \emptyset$  then
```

```
        | vc = false
```

```
    if vc then
```

```
      | return  $C$ 
```

```
  return false
```

$$\left| \binom{V}{k} \right| = \binom{|V|}{k} = \binom{n}{k} = \mathcal{O}(n^k)$$

$\mathcal{O}(m)$

Dies ist *nicht* polynomiell in der Größe der Eingabe, da k keine Konstante, sondern Teil der Eingabe.

Laufzeit. $\mathcal{O}(n^k m)$

Def.

Ein Problem, das in $\mathcal{O}(n^{f(k)})$ Zeit gelöst werden kann, heißt **schichtweise polynomiell** (**slice-wise polynomial**) bzgl. k .

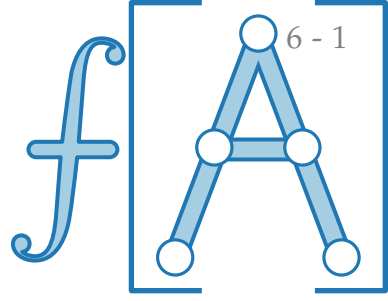
Die Klasse der schichtweise polynomiellen Probleme heißt $\mathcal{XP}$.

Ein Algorithmus mit Laufzeit $\mathcal{O}(n^{f(k)})$ heißt **XP-Algorithmus**.

Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

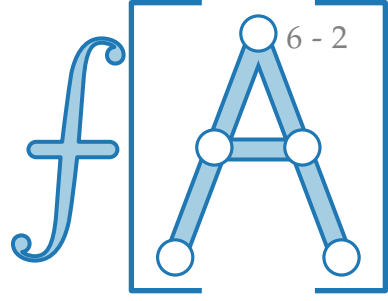


Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz

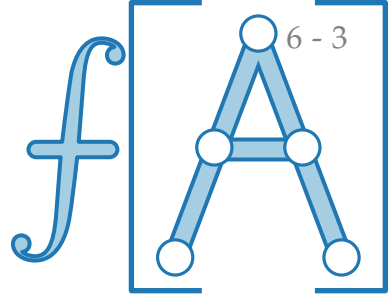


Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)

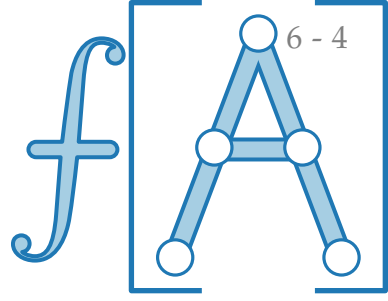


Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)



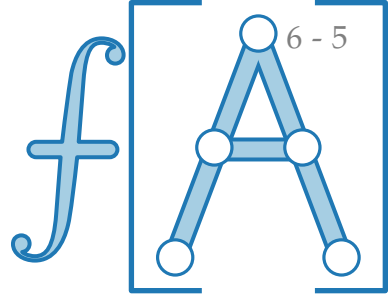
Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen



Ein neues Ziel

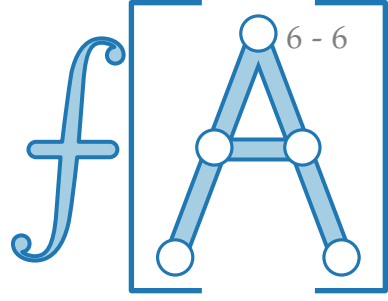
Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k



Ein neues Ziel

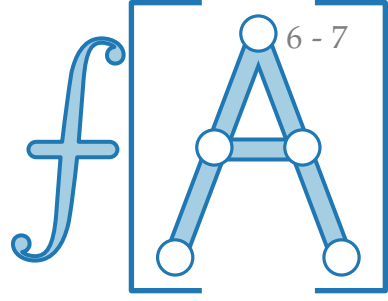
Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz



Ein neues Ziel

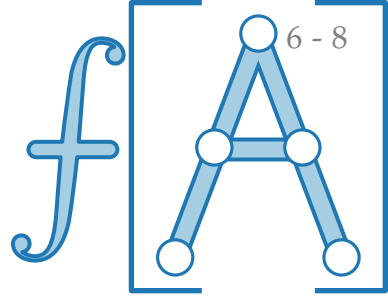
Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.



Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

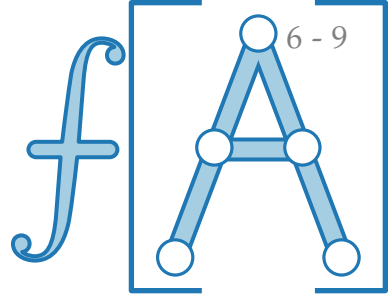
$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.

Def. Ein Problem, das in $\mathcal{O}(f(k) \cdot n^c)$ Zeit gelöst werden kann, heißt **festparameterberechenbar** (**fixed-parameter tractable**) bzgl. k .



Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

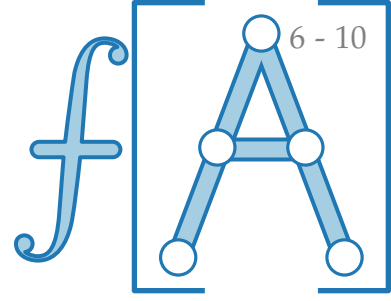
$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.

Def. Ein Problem, das in $\mathcal{O}(f(k) \cdot n^c)$ Zeit gelöst werden kann, heißt **festparameterberechenbar** (**fixed-parameter tractable**) bzgl. k .
Die Klasse der festparameterberechenbaren Probleme heißt $\mathcal{FPT}$.



Ein neues Ziel

Finde einen Algorithmus für k -VC mit Laufzeit

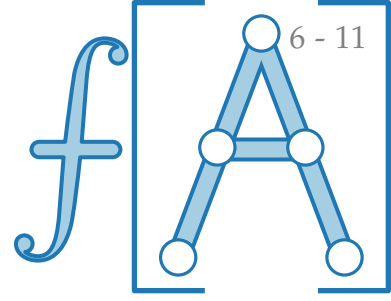
$$\mathcal{O}(f(k) \cdot n^c)$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

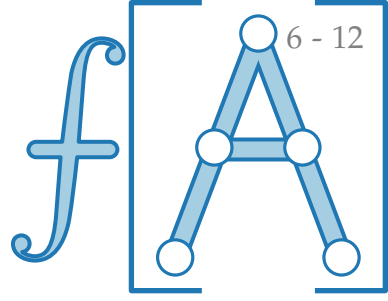
D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.

Def. Ein Problem, das in $\mathcal{O}(f(k) \cdot n^c)$ Zeit gelöst werden kann, heißt **festparameterberechenbar** (**fixed-parameter tractable**) bzgl. k .
Die Klasse der festparameterberechenbaren Probleme heißt $\mathcal{FPT}$.
Ein Algorithmus mit Laufzeit $\mathcal{O}(f(k) \cdot n^c)$ heißt **FPT-Algorithmus**.



Ein neues Ziel



Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c) = \mathcal{O}^*(f(k))$$

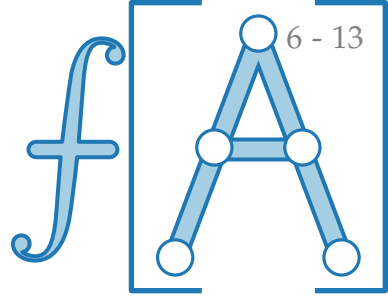
- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.

Def. Ein Problem, das in $\mathcal{O}(f(k) \cdot n^c)$ Zeit gelöst werden kann, heißt **festparameterberechenbar** (**fixed-parameter tractable**) bzgl. k .
Die Klasse der festparameterberechenbaren Probleme heißt $\mathcal{FPT}$.
Ein Algorithmus mit Laufzeit $\mathcal{O}(f(k) \cdot n^c)$ heißt **FPT-Algorithmus**.

Ein neues Ziel



Finde einen Algorithmus für k -VC mit Laufzeit

$$\mathcal{O}(f(k) \cdot n^c) = \mathcal{O}^*(f(k))$$

- n Größe der gegebenen Instanz
- c Konstante (unabh. von k und n)
- $f: \mathbb{N} \rightarrow \mathbb{N}$ berechenbare Funktion (unabh. von n)

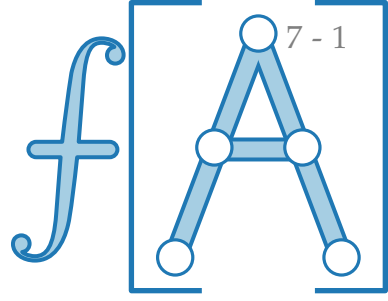
D.h. die Laufzeit soll abhängen

- beliebig vom Parameter k Schwierigkeit der Instanz
- polynomiell von der Größe n der Instanz.

BRUTEFORCEVC hat *nicht* die gewünschte Laufzeit.

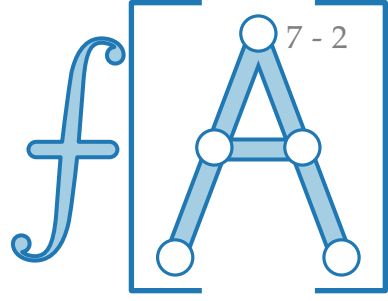
Def. Ein Problem, das in $\mathcal{O}(f(k) \cdot n^c)$ Zeit gelöst werden kann, heißt **festparameterberechenbar** (**fixed-parameter tractable**) bzgl. k .
Die Klasse der festparameterberechenbaren Probleme heißt $\mathcal{FPT}$.
Ein Algorithmus mit Laufzeit $\mathcal{O}(f(k) \cdot n^c)$ heißt **FPT-Algorithmus**.

Zwei Grundtechniken



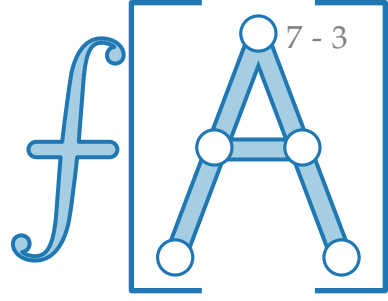
Zwei Grundtechniken

1. Kernbildung:

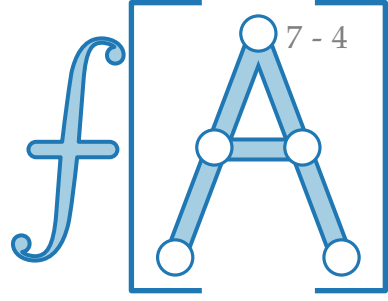


Zwei Grundtechniken

- 1. Kernbildung:** Reduziere die Instanz durch Anwendung verschiedener Regeln auf einen (schweren) Problemkern.



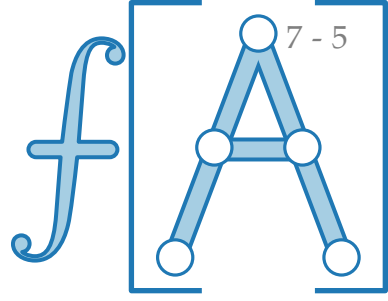
Zwei Grundtechniken



1. Kernbildung: Reduziere die Instanz durch Anwendung verschiedener Regeln auf einen (schweren) Problemkern.

2. Tiefenbeschränkte Suchbäume:

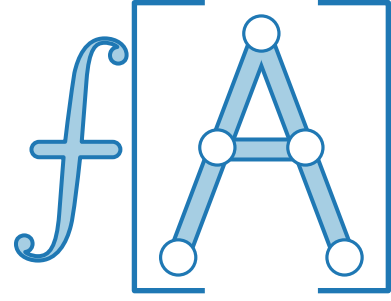
Zwei Grundtechniken



1. Kernbildung: Reduziere die Instanz durch Anwendung verschiedener Regeln auf einen (schweren) Problemerkern.

2. Tiefenbeschränkte Suchbäume:

Erschöpfende Suche in einem geeigneten Suchbaum mit beschränkter Tiefe.

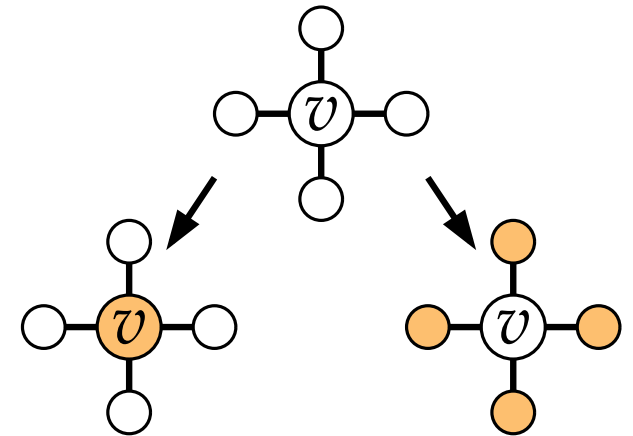
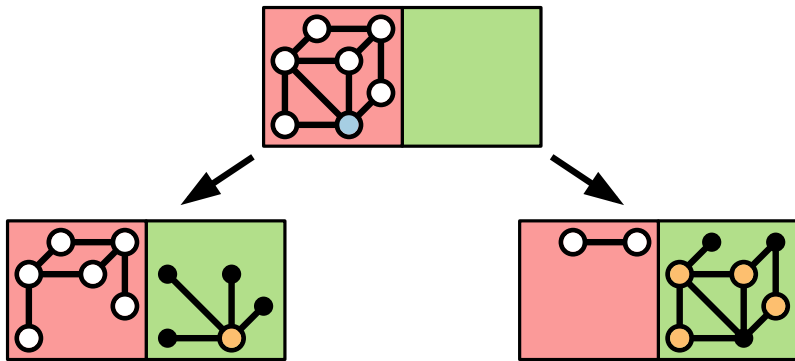


Fortgeschrittene Algorithmen

20. Vorlesung

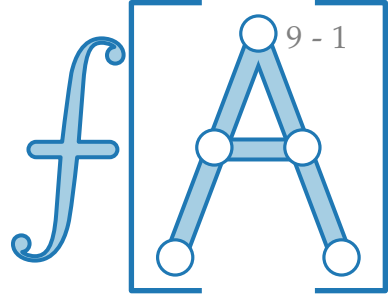
Parametrisierte Algorithmen

Teil II: Kernbildung



Ein paar einfache Beobachtungen...

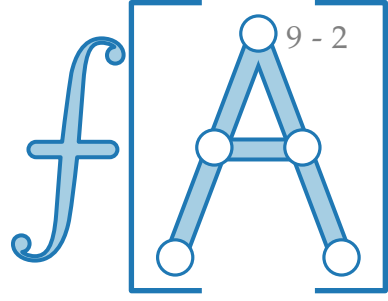
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?



Ein paar einfache Beobachtungen...

Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

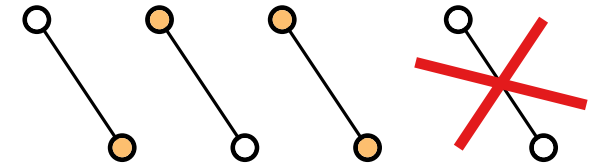
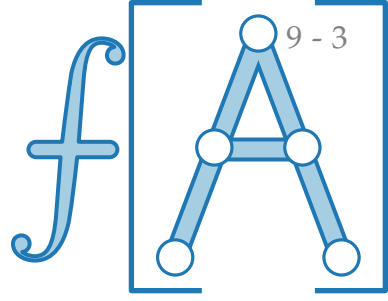
Beob. 1. Für jede Kante $uv \in E$ gilt:



Ein paar einfache Beobachtungen...

Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

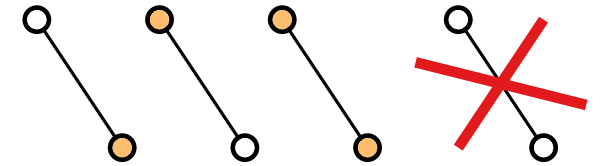
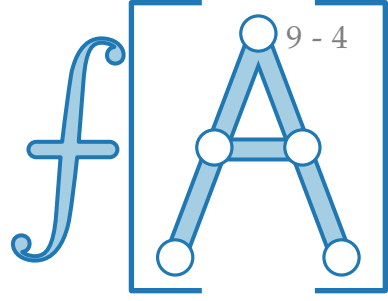


Ein paar einfache Beobachtungen...

Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

Betrachte Entscheidungsproblem k -VC.

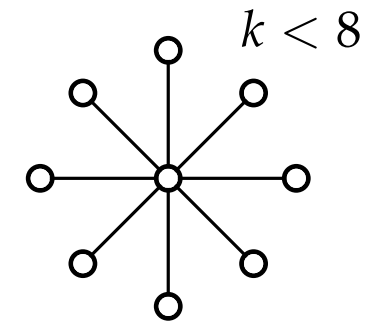
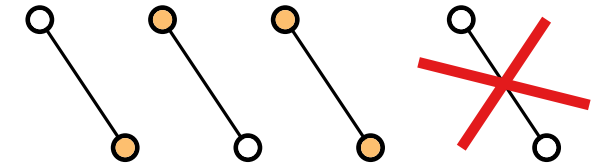
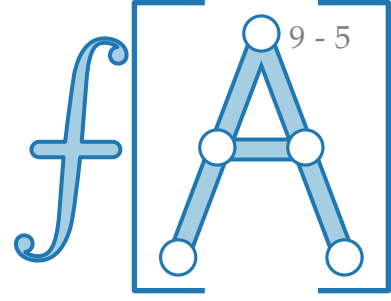


Ein paar einfache Beobachtungen...

Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

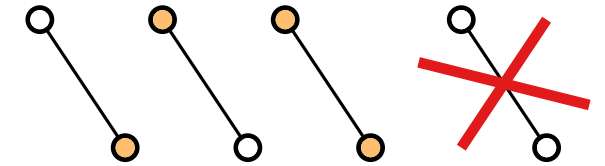
Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?



Ein paar einfache Beobachtungen...

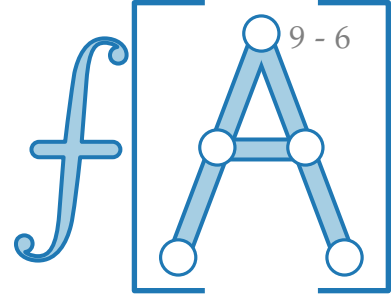
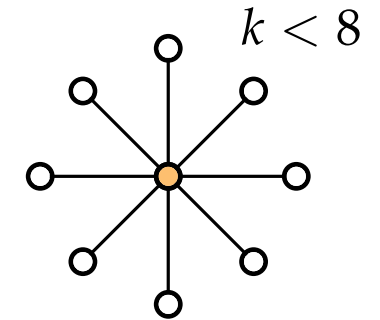
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

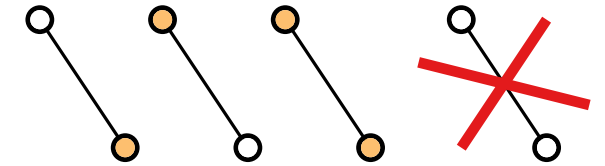
Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.



Ein paar einfache Beobachtungen...

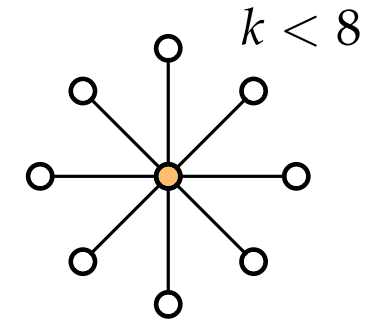
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

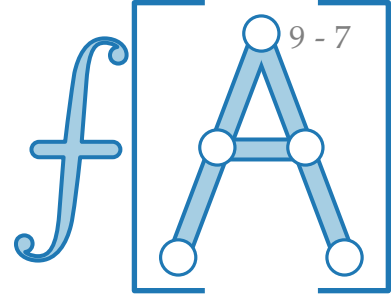
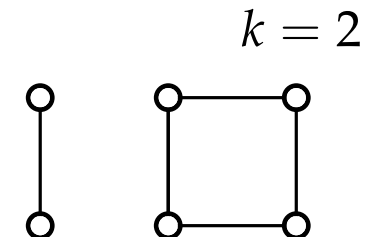


Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

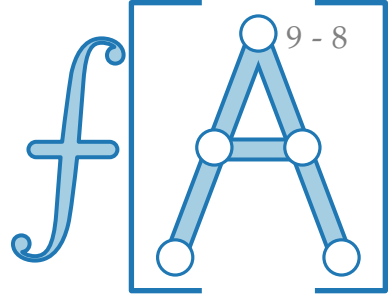
Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.



Was gilt, falls $m > k^2$ und alle Knoten $\text{Grad} \leq k$ haben?

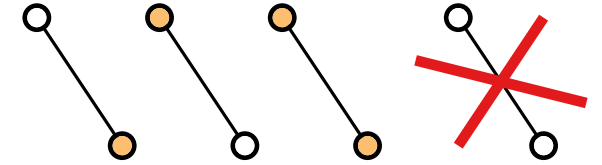


Ein paar einfache Beobachtungen...



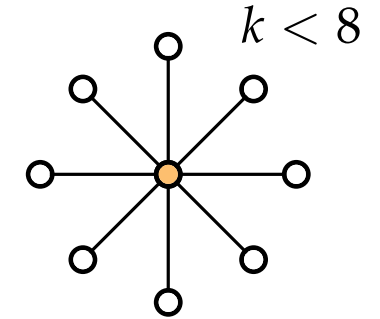
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



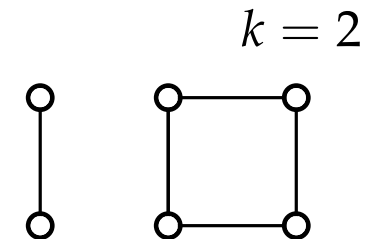
Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.



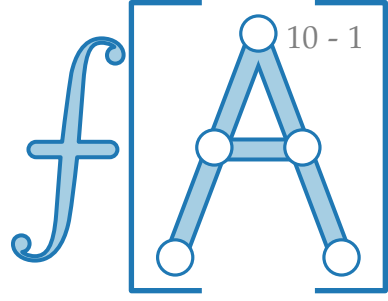
Was gilt, falls $m > k^2$ und alle Knoten $\text{Grad} \leq k$ haben?

Beob. 3. Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Algorithmus von Buss

BussVC(Graph G , Integer k)



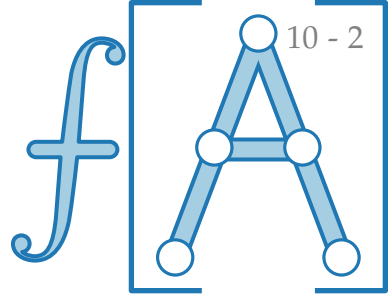
Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')



Jonathan F. Buss

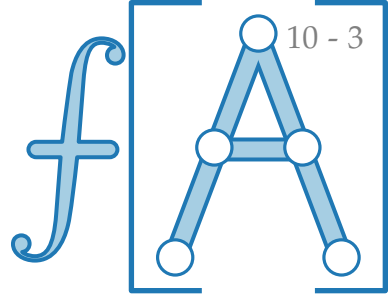


Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

II) Lösung der Restinstanz mit roher Gewalt



Jonathan F. Buss



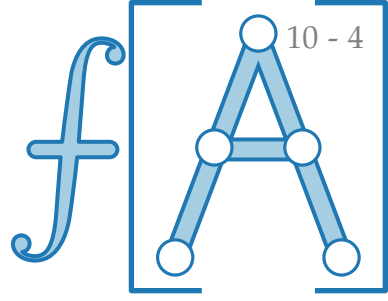
Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$



Jonathan F. Buss



Algorithmus von Buss

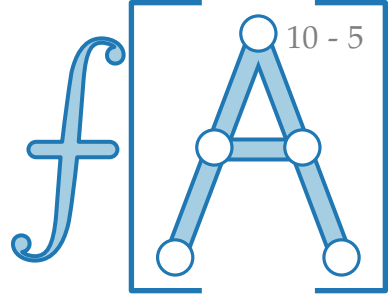
BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

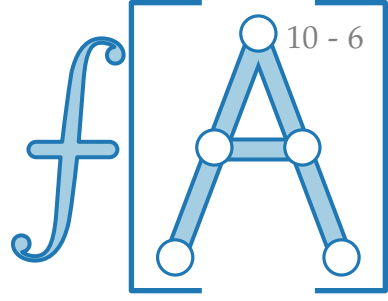
Benutze Beobachtungen 2 & 3, um eine Instanz (G, k) in eine Instanz (G', k') zu verwandeln, so dass $|G'| \in \mathcal{O}(k^2)$!

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

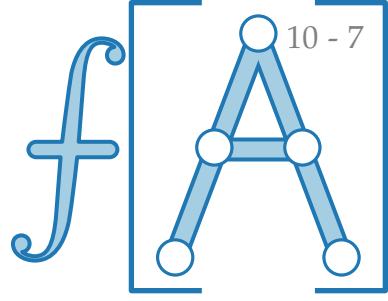
$$C = \{v \in V \mid \deg v > k\}$$

II) Lösung der Restinstanz mit roher Gewalt

$$C' = \text{BRUTEFORCEVC}(G', k')$$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

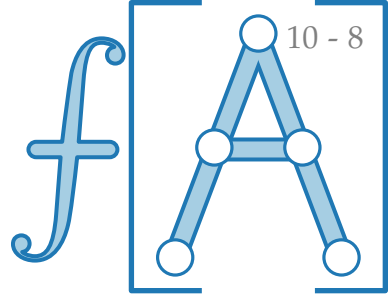
if $|C| > k$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

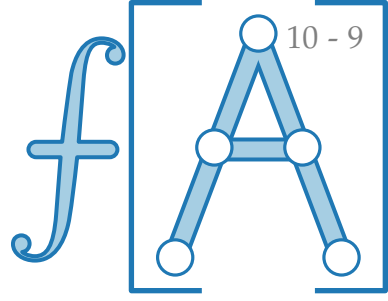
$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

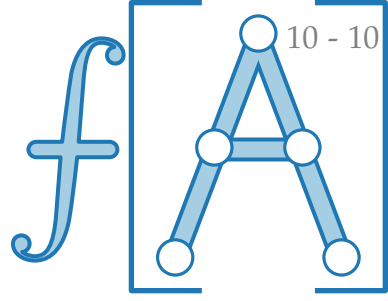
$k' = k - |C|$

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

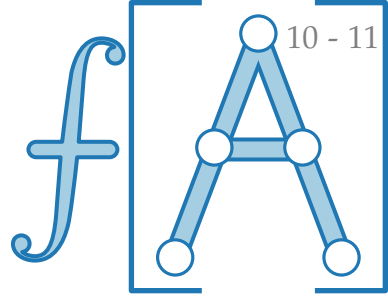
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

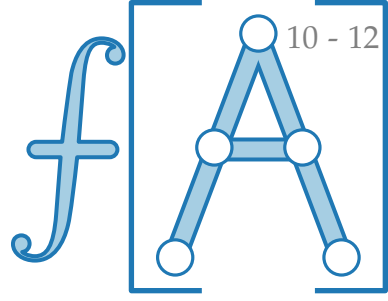
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

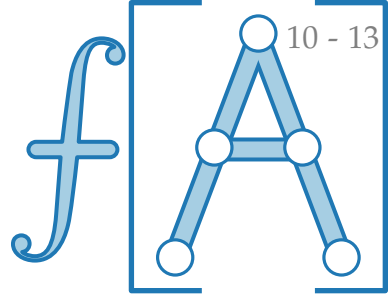
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

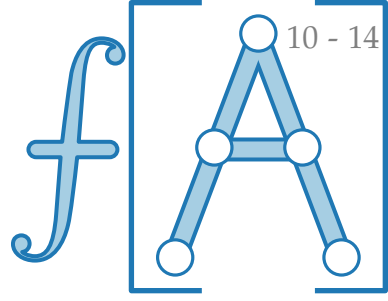
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

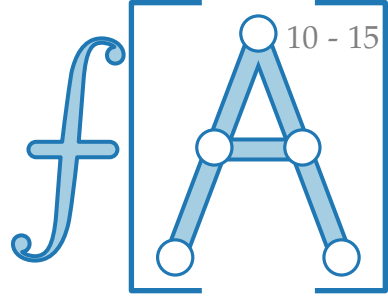
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

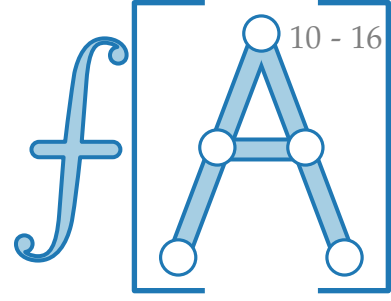
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit

Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

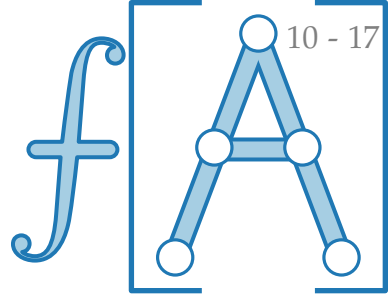
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$

Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

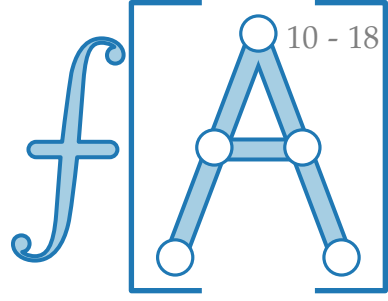
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Laufzeit.

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

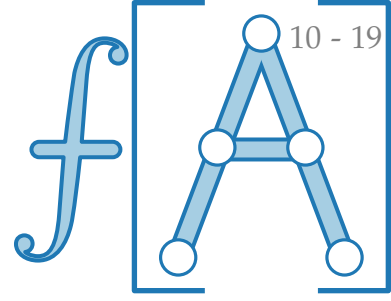
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Laufzeit. $\mathcal{O}(n + m + k^2 \cdot (2k^2)^k)$

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



$\mathcal{O}(n + m)$
Zeit

$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

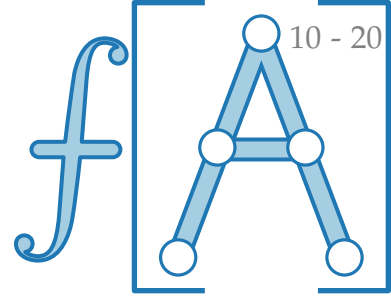
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(n + m + 2^k k^{2k+2}\right)$

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



$\mathcal{O}(n + m)$
Zeit

$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

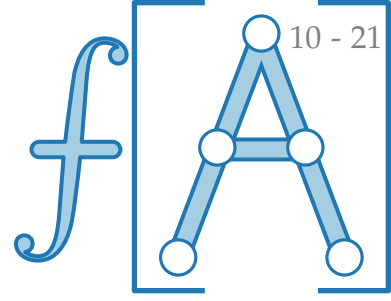
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



$\mathcal{O}(n + m)$
Zeit

$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(n + m + 2^k k^{2k+2}\right) = \mathcal{O}\left(n + m + 2^{k+(2k+2)\log_2 k}\right)$

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

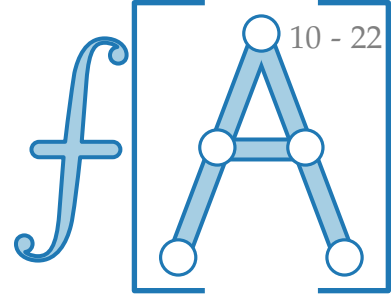
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\mathcal{O}(n + m)$
Zeit

Jonathan F. Buss



$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(n + m + 2^k k^{2k+2}\right) = \mathcal{O}\left(n + m + 2^{k+(2k+2)\log_2 k}\right)$

Also:

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$C = \{v \in V \mid \deg v > k\}$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$k' = k - |C|$

if $|E'| > k \cdot k'$ then return false

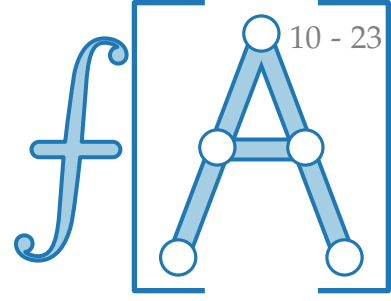
II) Lösung der Restinstanz mit roher Gewalt

$C' = \text{BRUTEFORCEVC}(G', k')$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.

Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



$\left. \begin{array}{l} \mathcal{O}(n + m) \\ \text{Zeit} \end{array} \right\}$

Jonathan F. Buss



$\left. \begin{array}{l} \mathcal{O}(m' \cdot (n')^{k'}) \text{ Zeit} \\ \text{wobei } m' = |E'| \leq k \cdot k' \leq k^2 \\ \Rightarrow n' = |V'| \leq 2k^2 \end{array} \right\}$

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(n + m + 2^k k^{2k+2}\right) = \mathcal{O}\left(n + m + 2^{k+(2k+2)\log_2 k}\right)$

Also: $k\text{-VC} \in \mathcal{FPT}$

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$$C = \{v \in V \mid \deg v > k\}$$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$$k' = k - |C|$$

if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

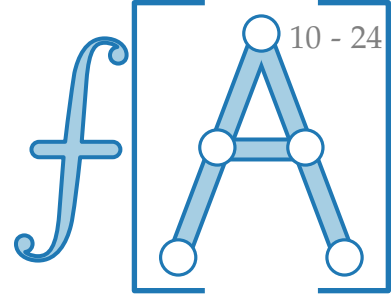
$$C' = \text{BRUTEFORCEVC}(G', k')$$

if C' then return $C \cup C'$ else return false

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(\underbrace{n + m}_{\mathcal{O}(n^2)} + 2^k k^{2k+2}\right) = \mathcal{O}\left(n + m + 2^{k+(2k+2)\log_2 k}\right)$

Also: $k\text{-VC} \in \mathcal{FPT}$

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss



$\mathcal{O}(n + m)$
Zeit

$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Algorithmus von Buss

BussVC(Graph G , Integer k)

I) Reduktion der Instanz auf ihren harten Kern (G', k')

$$C = \{v \in V \mid \deg v > k\}$$

if $|C| > k$ then return false

$G' = (V', E') := G[V \setminus C]$ (ohne isolierte Knoten)

$$k' = k - |C|$$

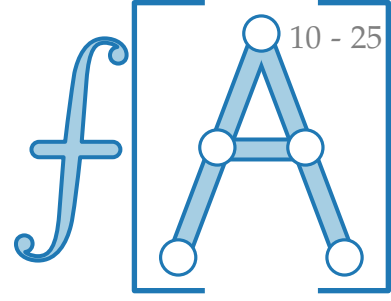
if $|E'| > k \cdot k'$ then return false

II) Lösung der Restinstanz mit roher Gewalt

$$C' = \text{BRUTEFORCEVC}(G', k')$$

if C' then return $C \cup C'$ else return false

Jeder Knoten mit Grad $> k$ ist in jedem k -VC enthalten.
Falls $m > \Delta \cdot k$ mit
 $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Jonathan F. Buss

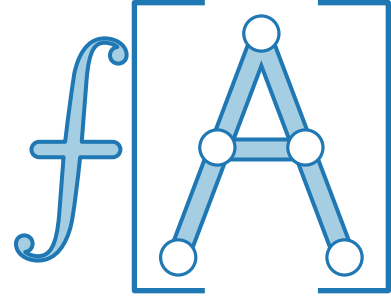


$\mathcal{O}(n + m)$
Zeit

$\mathcal{O}(m' \cdot (n')^{k'})$ Zeit
wobei $m' = |E'| \leq k \cdot k' \leq k^2$
 $\Rightarrow n' = |V'| \leq 2k^2$

Laufzeit. $\mathcal{O}\left(n + m + k^2 \cdot (2k^2)^k\right) = \mathcal{O}\left(\underbrace{n + m}_{\mathcal{O}(n^2)} + \underbrace{2^k k^{2k+2}}_{f(k)}\right) = \mathcal{O}\left(n + m + 2^{k + (2k+2) \log_2 k}\right)$

Also: $k\text{-VC} \in \mathcal{FPT}$

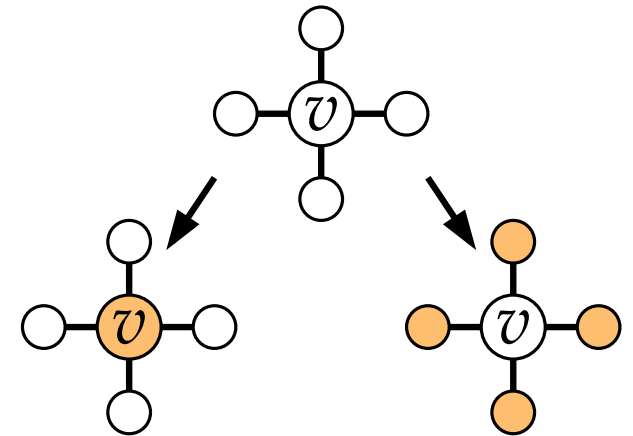
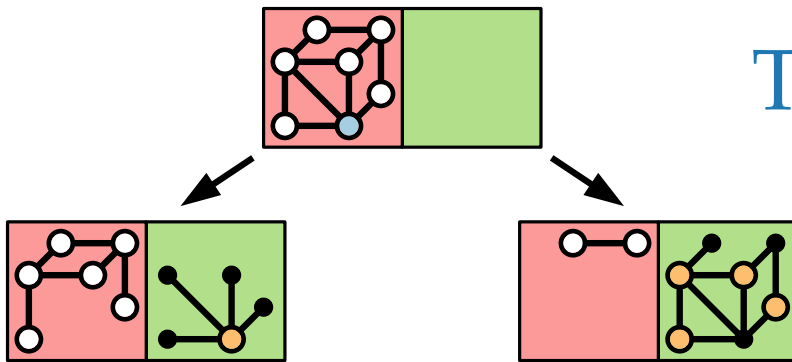


Fortgeschrittene Algorithmen

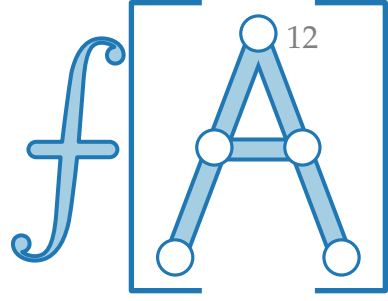
20. Vorlesung

Parametrisierte Algorithmen

Teil III: Tiefenbeschränkte Suchbäume

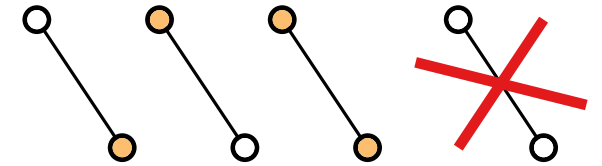


Ein paar einfache Beobachtungen...



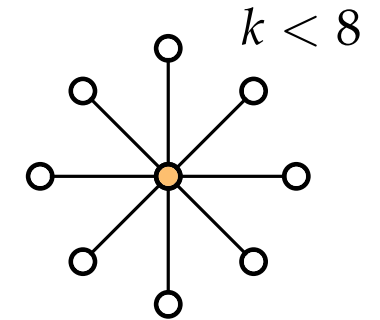
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



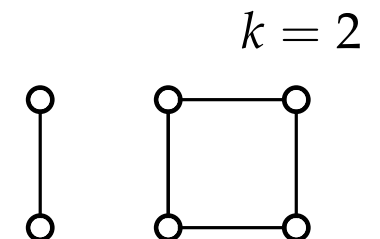
Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.



Was gilt, falls $m > k^2$ und alle Knoten $\text{Grad} \leq k$ haben?

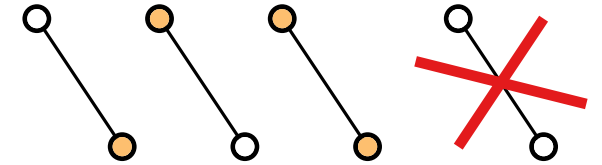
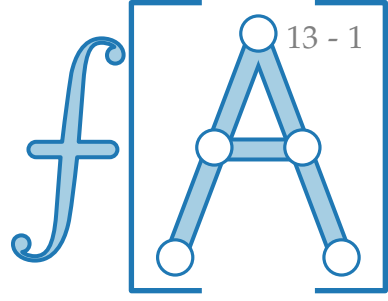
Beob. 3. Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

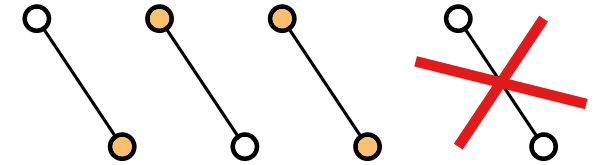
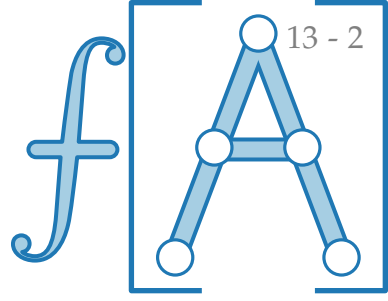
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



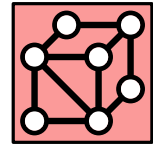
Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



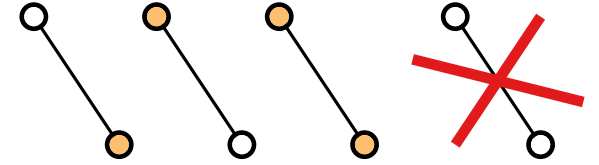
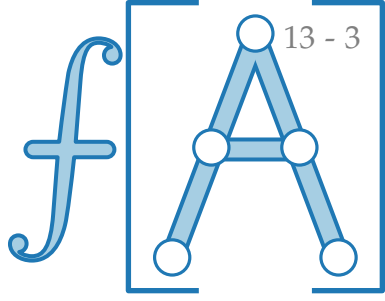
noch zu überdeckender Teilgraph



Algorithmus für VERTEXCOVER

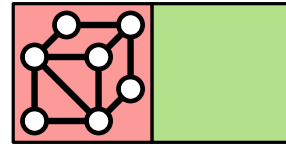
Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

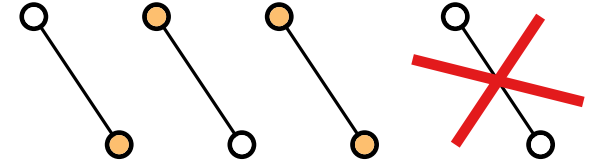
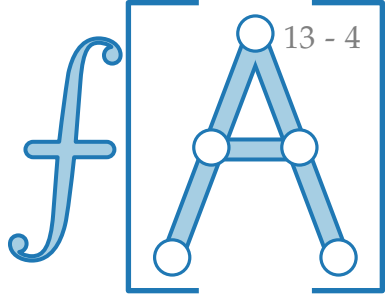
ausgewählte Knoten & überdeckte Kanten



Algorithmus für VERTEXCOVER

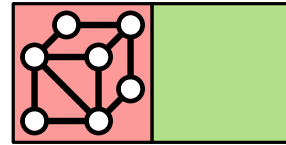
Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

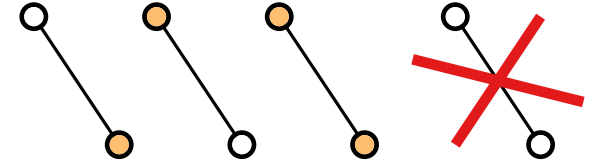
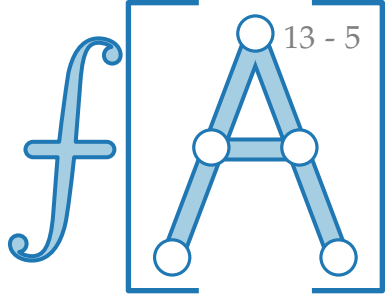


$$k = 3$$

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

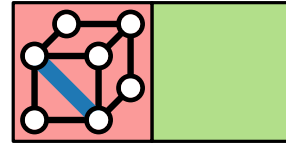
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

Jede Kante muss noch überdeckt werden. ➡ wähle eine beliebige.

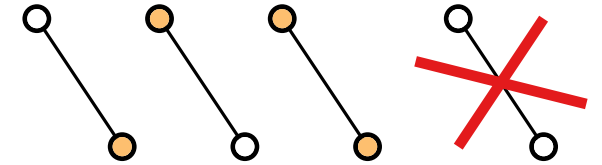
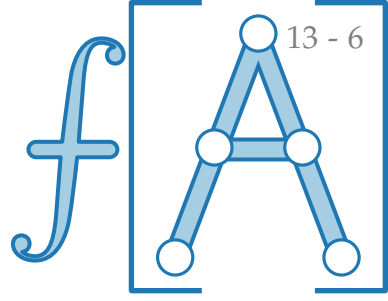


$k = 3$

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

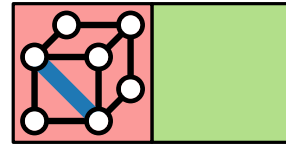
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$$k = 3$$

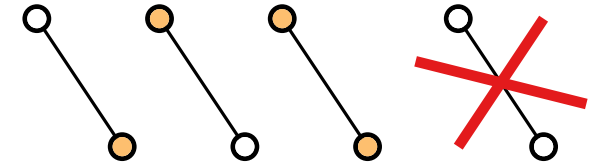
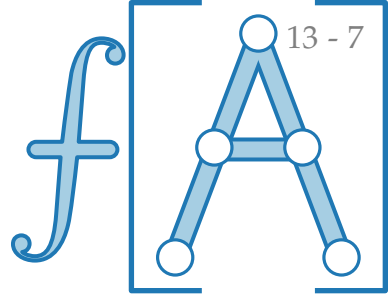
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

→ binärer Entscheidungsbaum

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

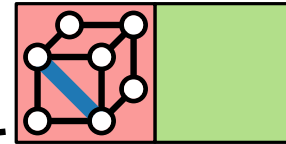
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



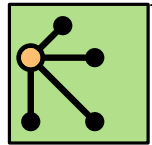
noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$



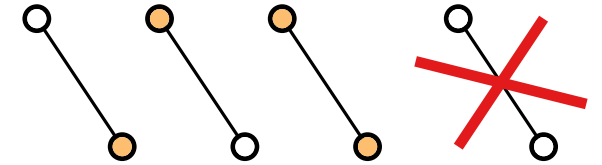
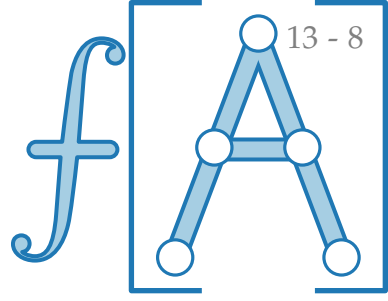
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

→ binärer Entscheidungsbaum

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

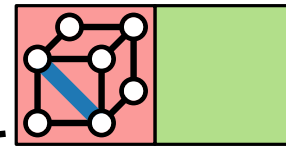
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



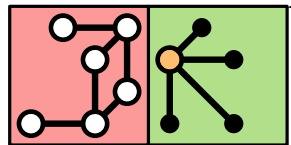
noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$



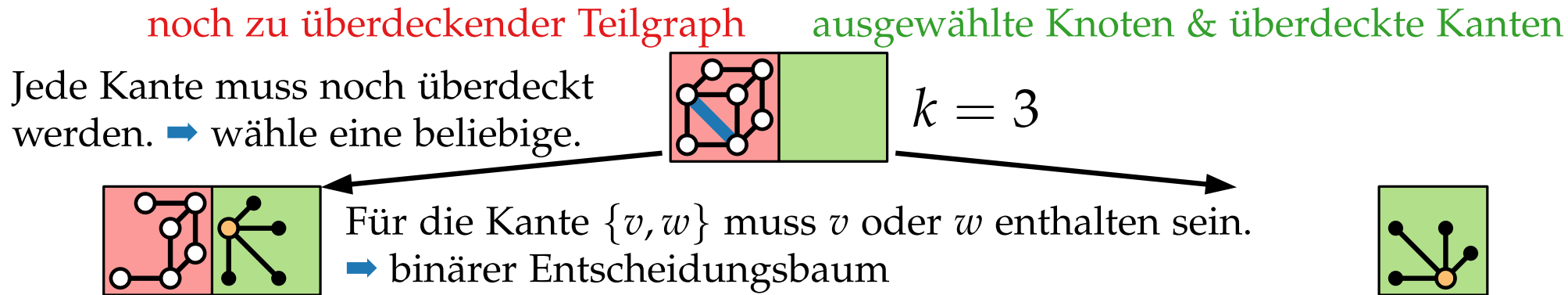
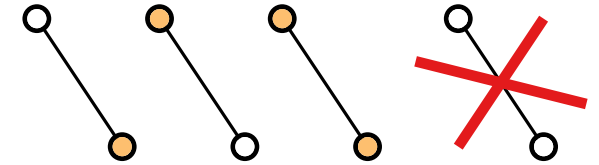
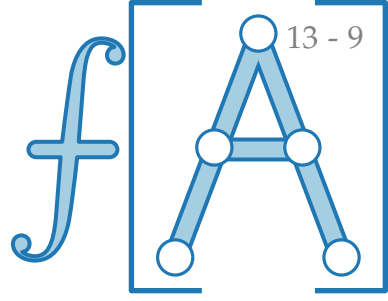
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

→ binärer Entscheidungsbaum

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

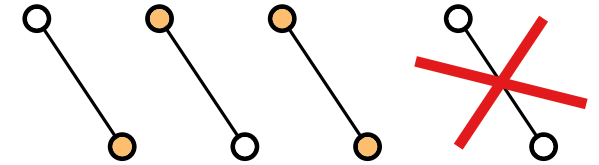
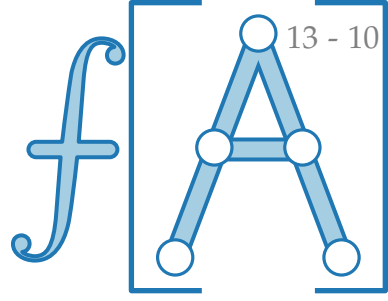
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

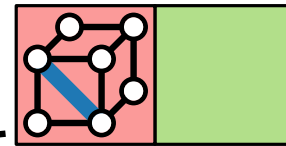
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



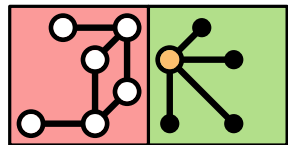
noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

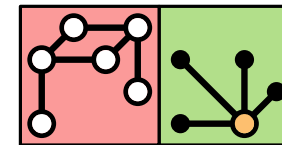
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$



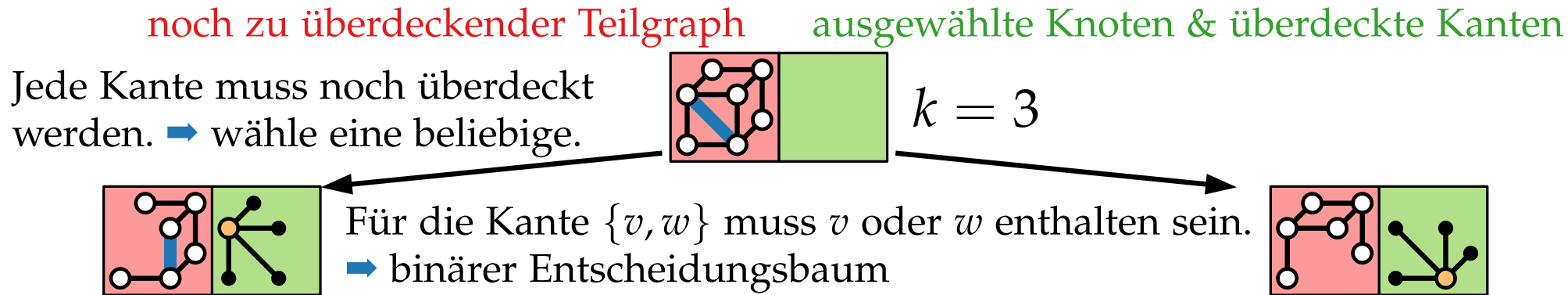
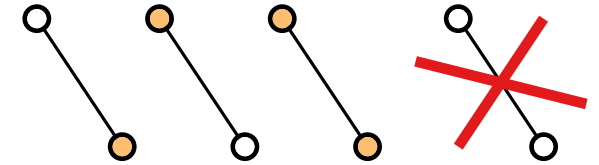
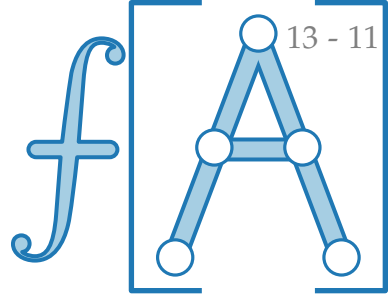
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

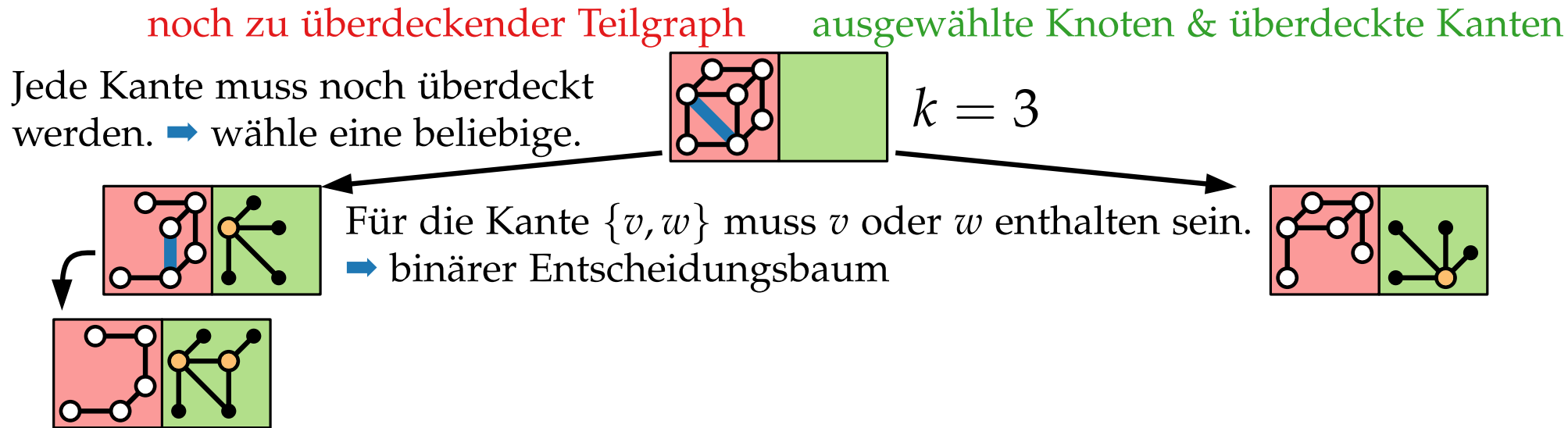
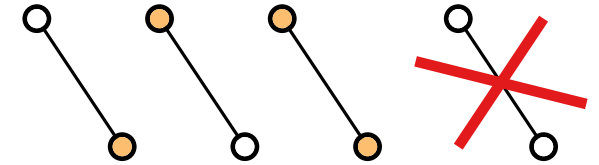
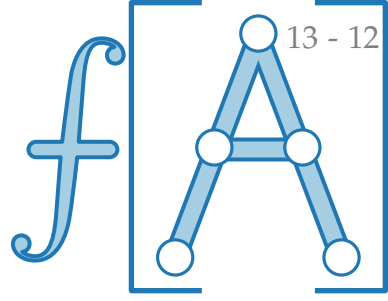
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

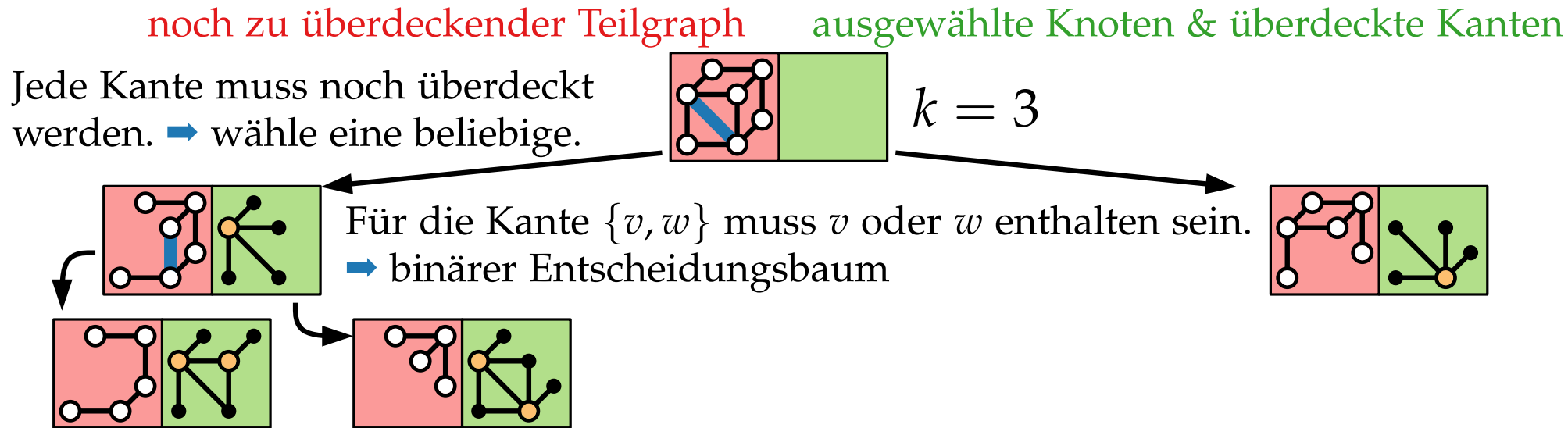
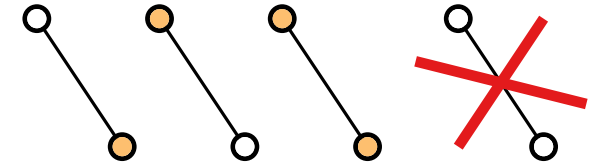
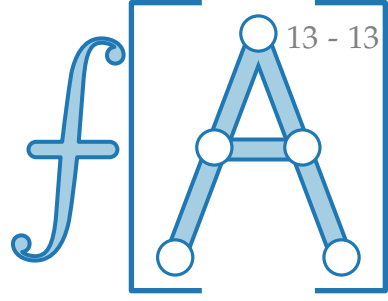
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

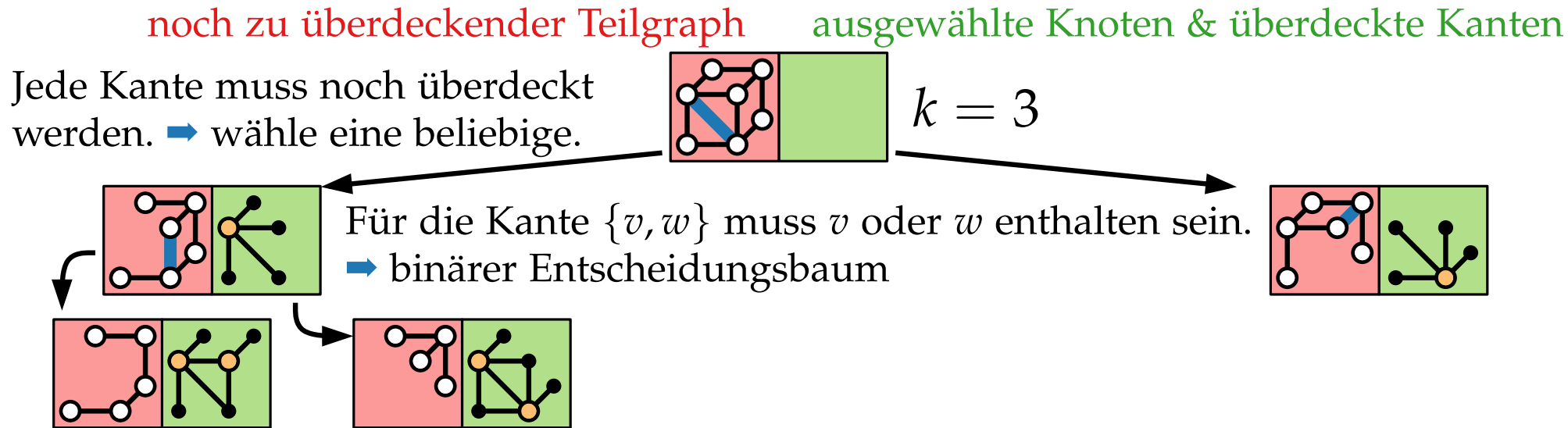
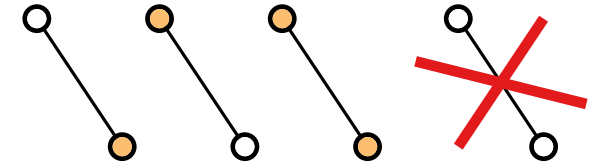
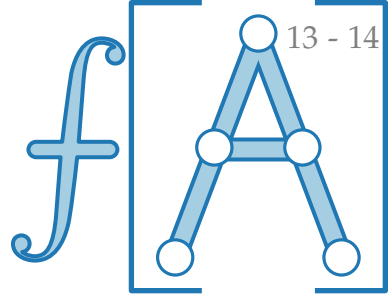
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

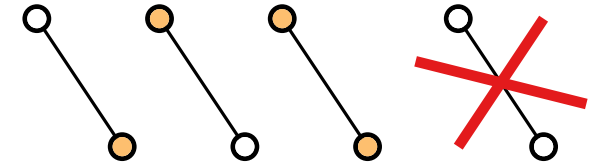
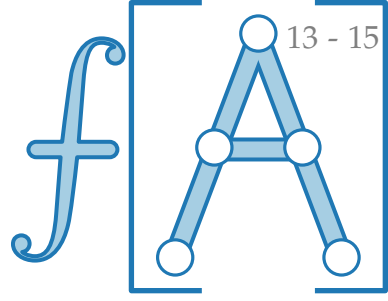
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

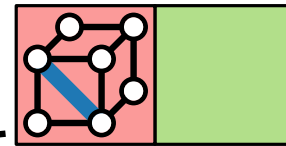
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



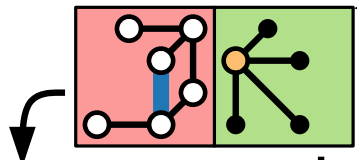
noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

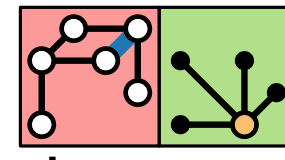
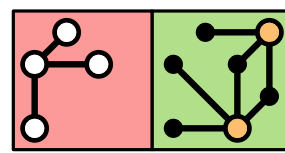
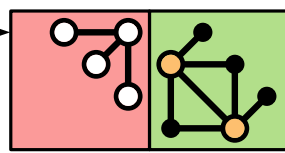
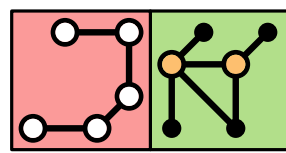
Jede Kante muss noch überdeckt werden. $\rightarrow$ wähle eine beliebige.



$k = 3$



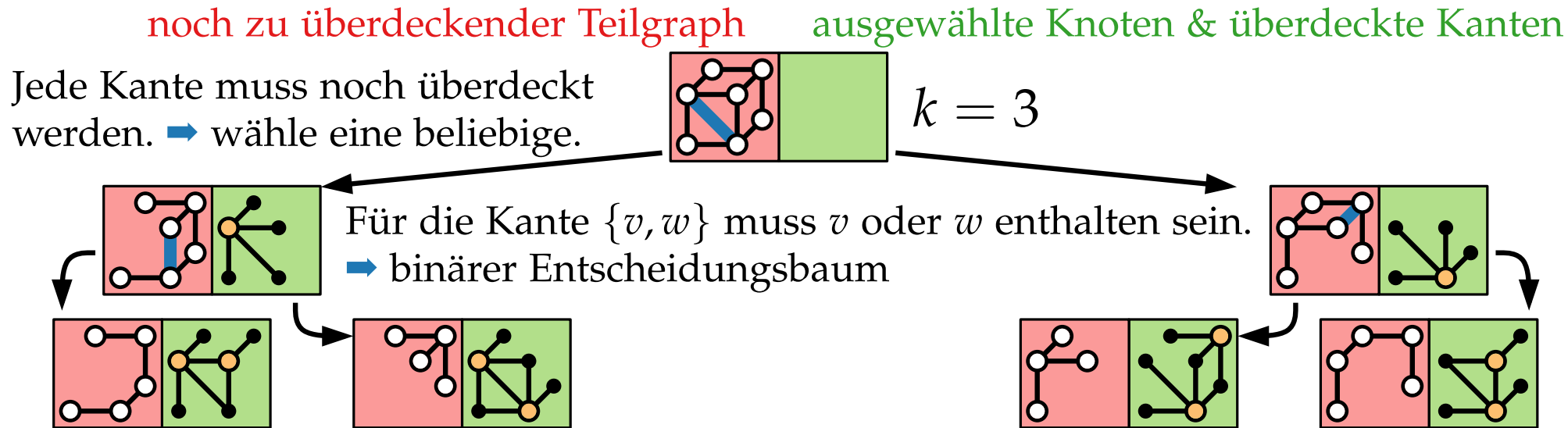
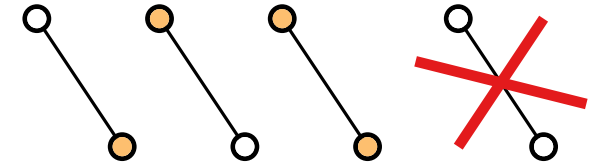
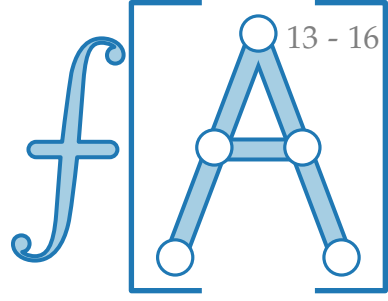
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.
 $\rightarrow$ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

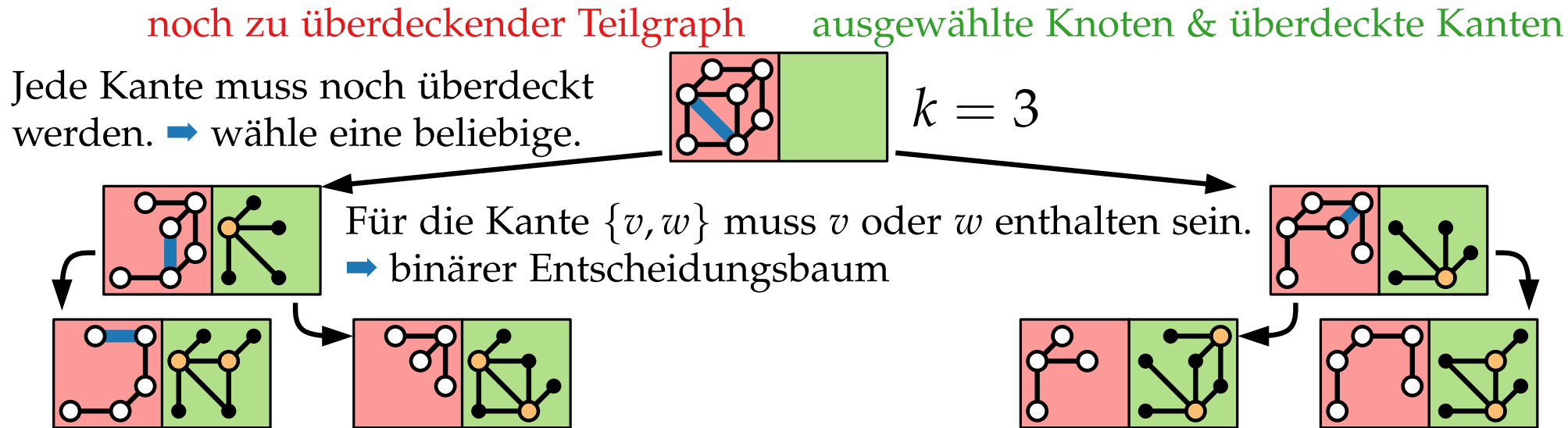
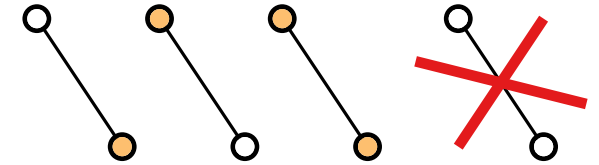
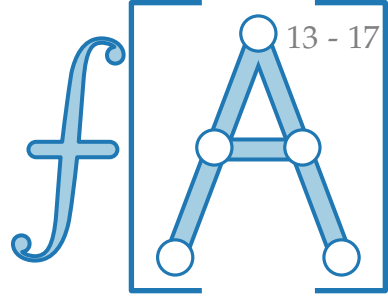
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

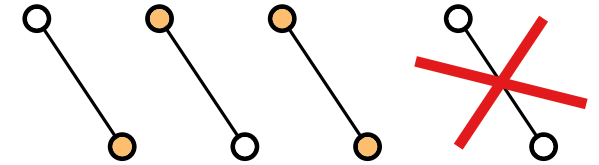
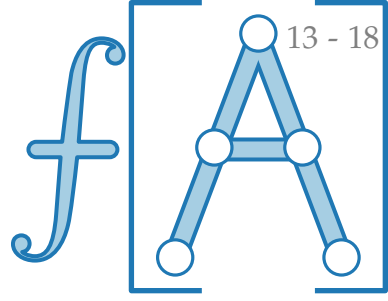
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

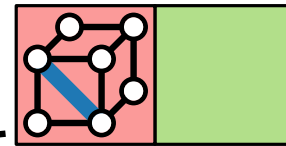
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

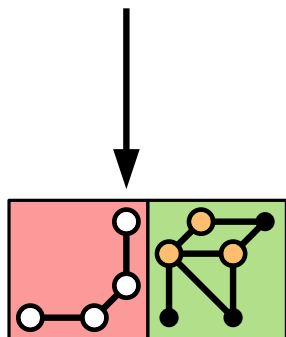
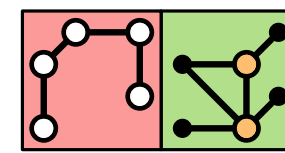
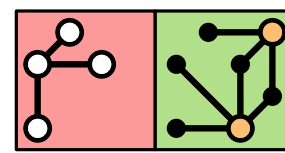
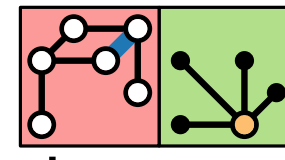
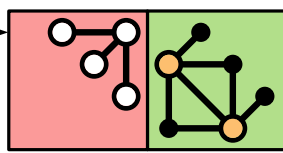
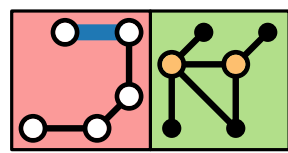
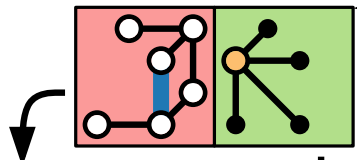
Jede Kante muss noch überdeckt werden. $\rightarrow$ wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

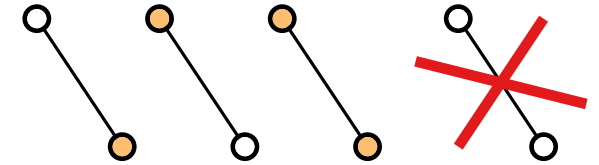
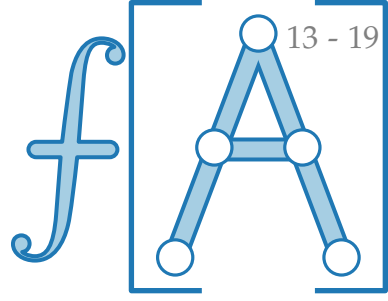
$\rightarrow$ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

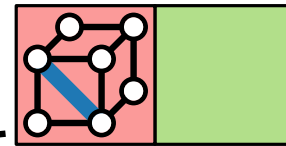
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

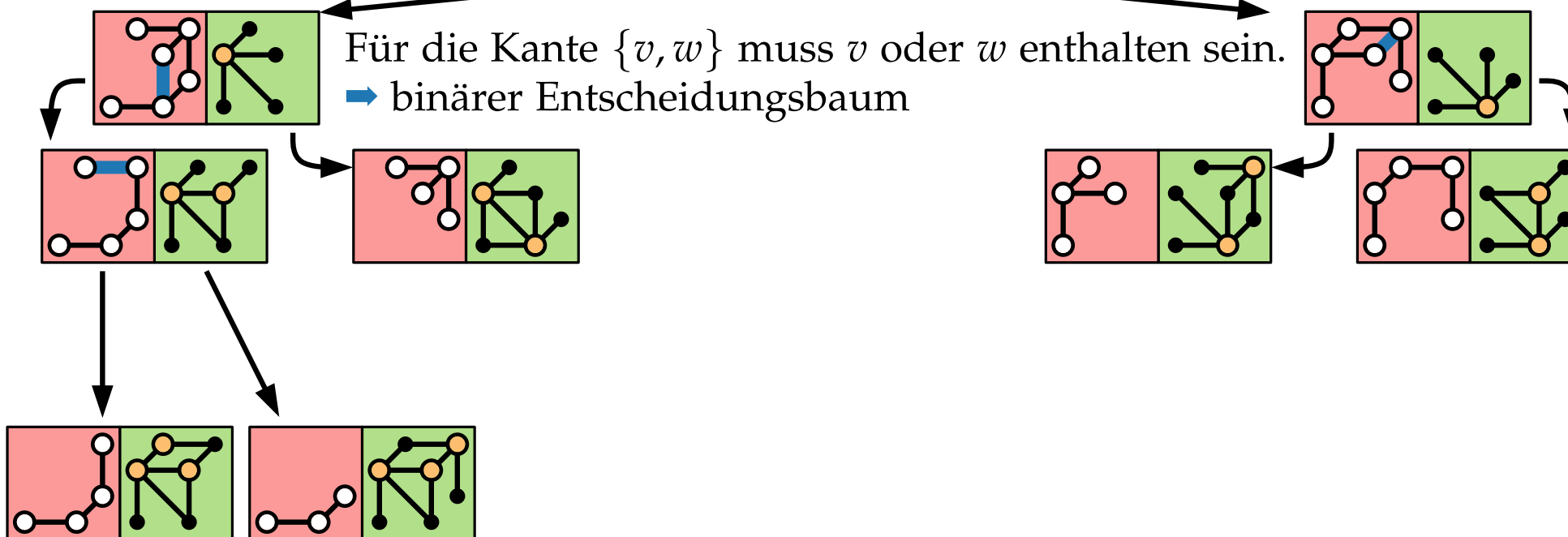
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

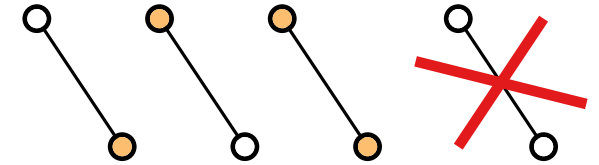
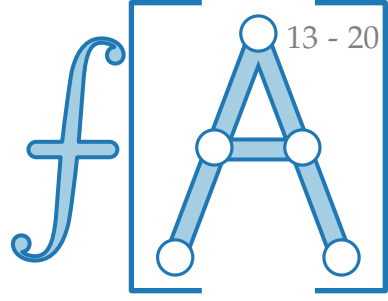
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

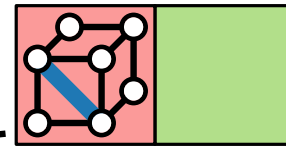
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

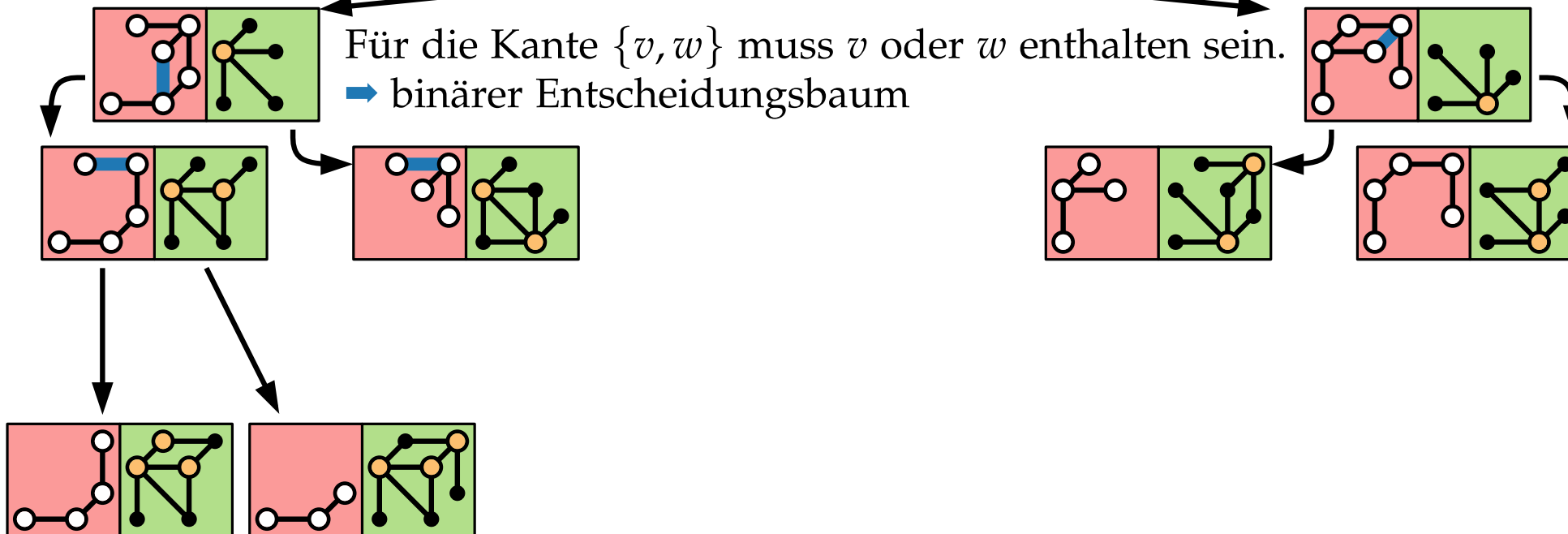
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

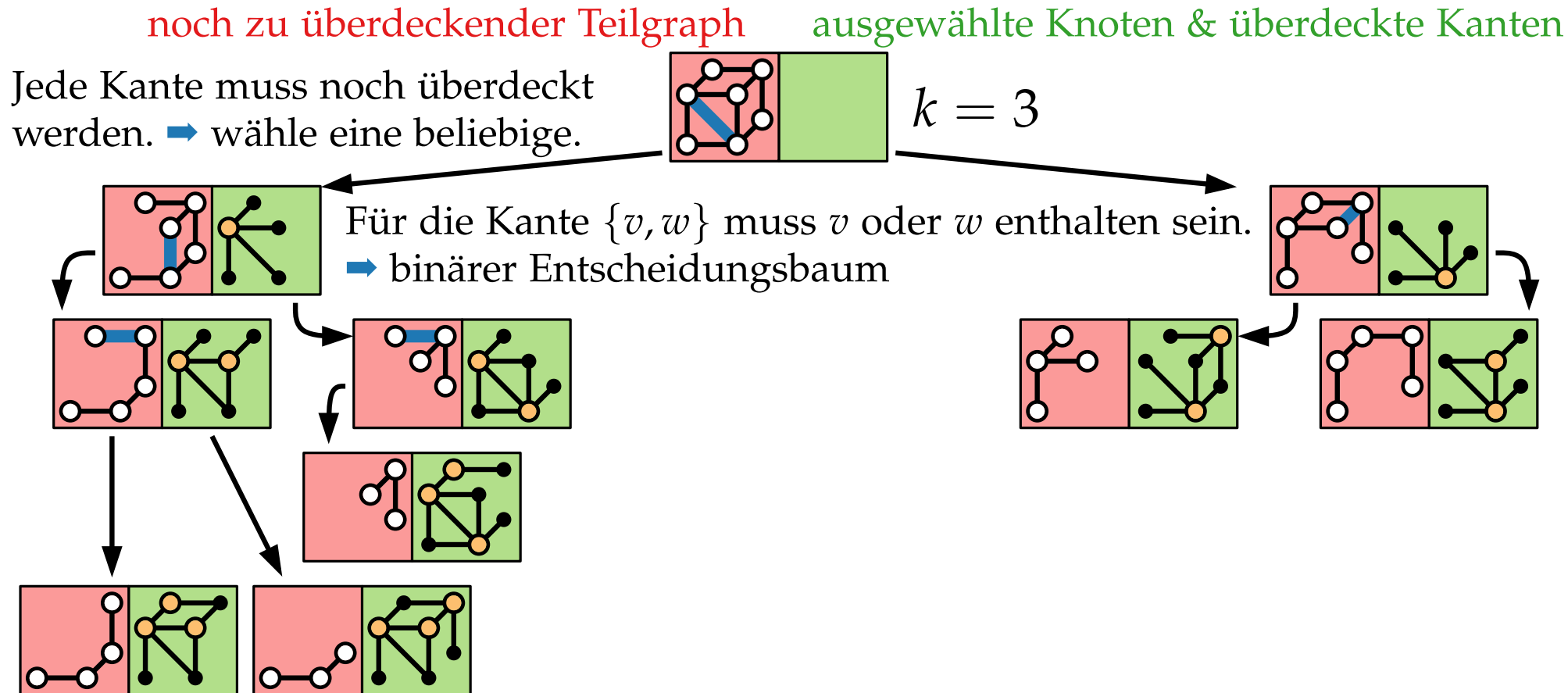
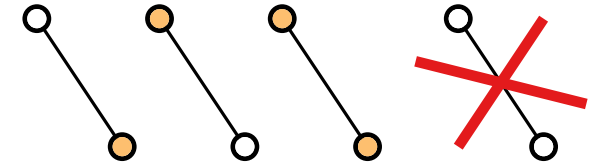
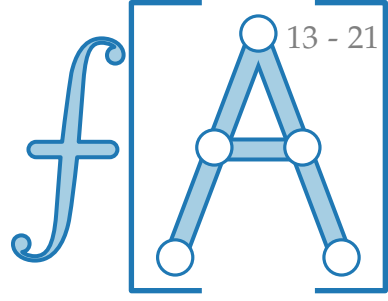
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

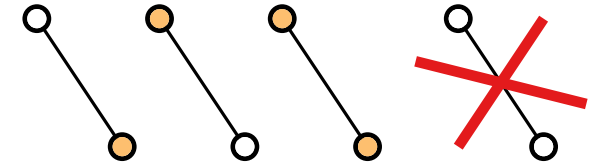
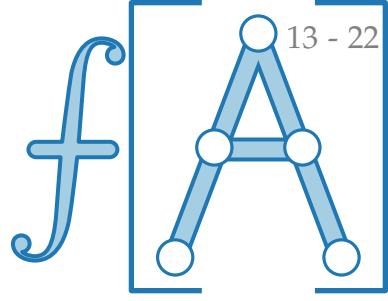
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

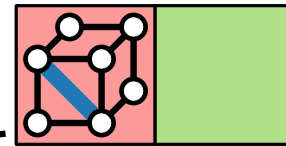
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

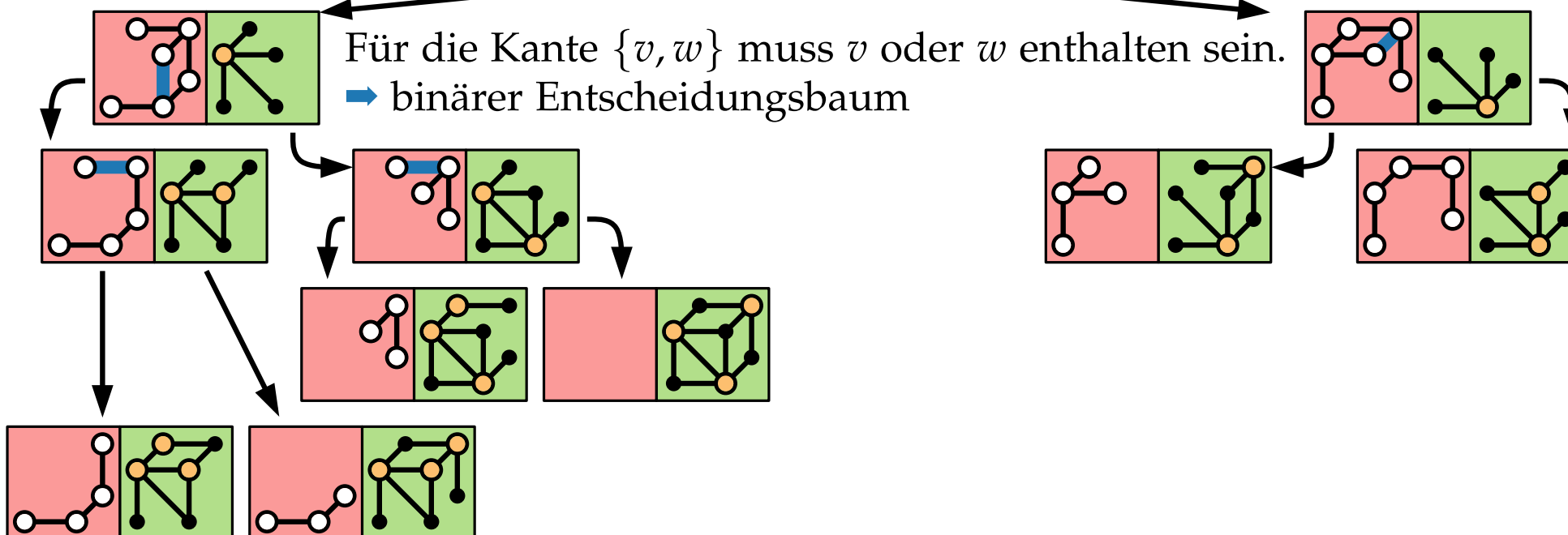
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

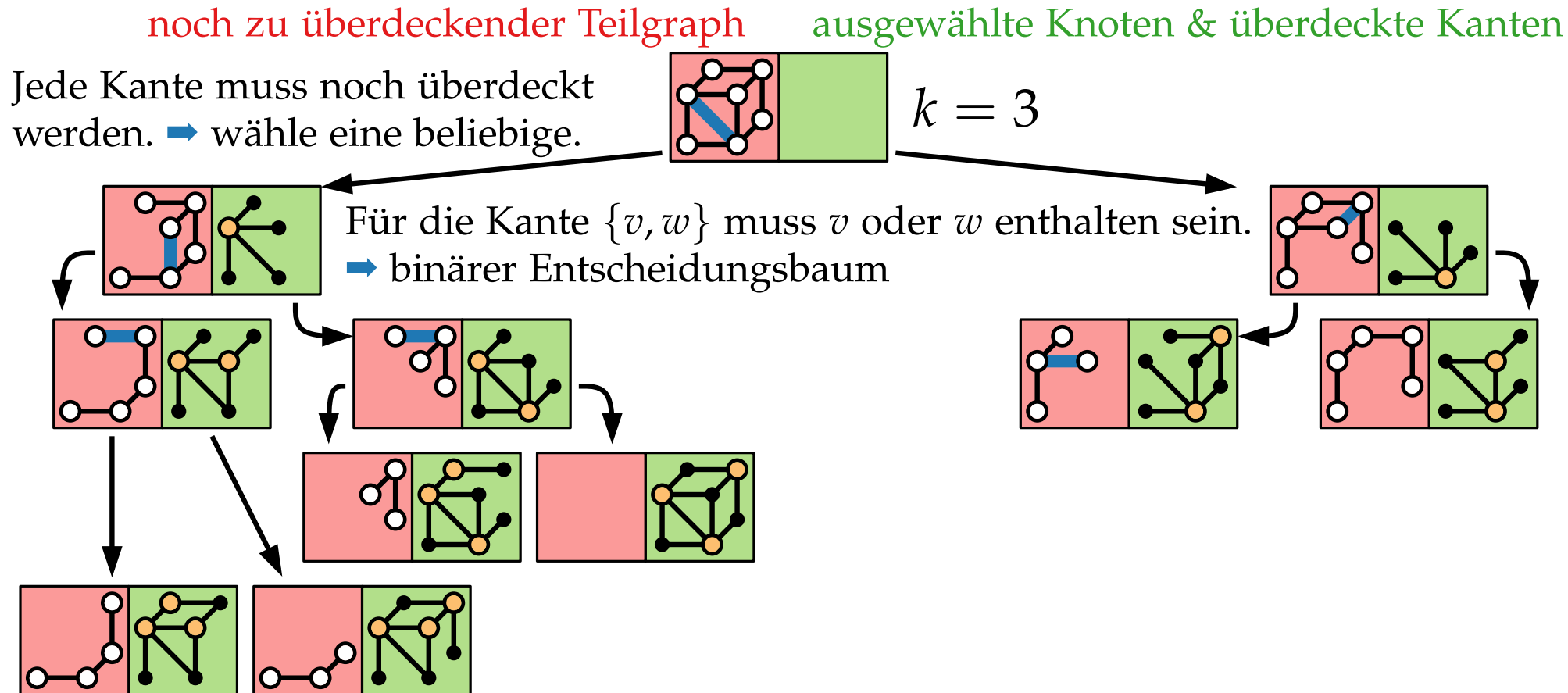
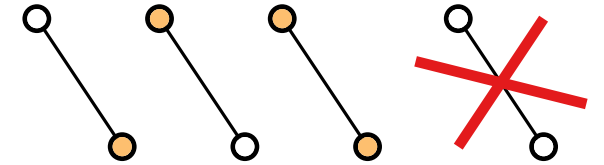
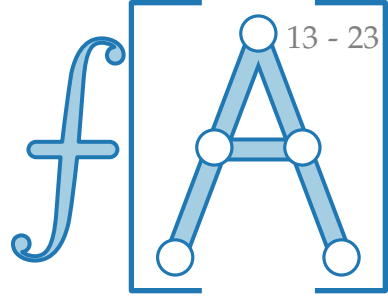
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

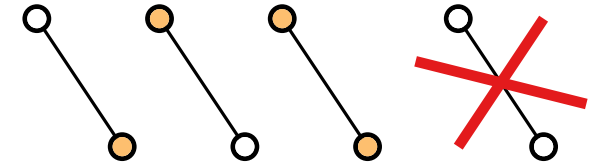
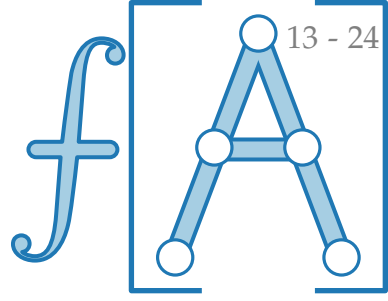
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

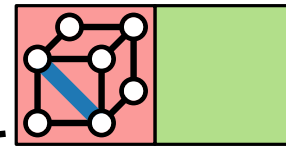
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

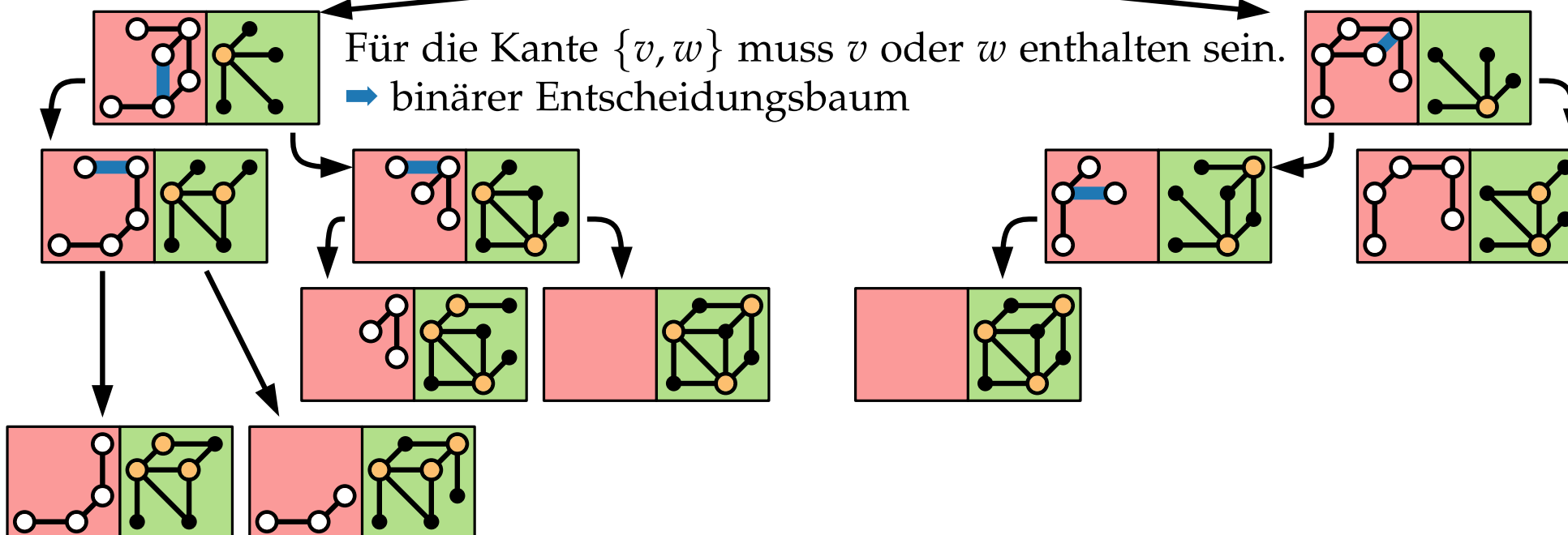
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

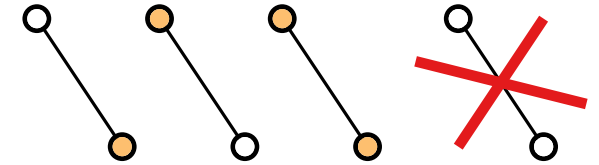
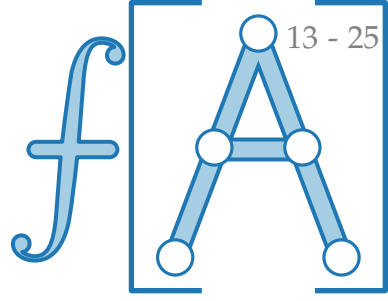
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

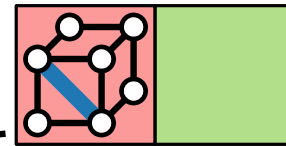
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

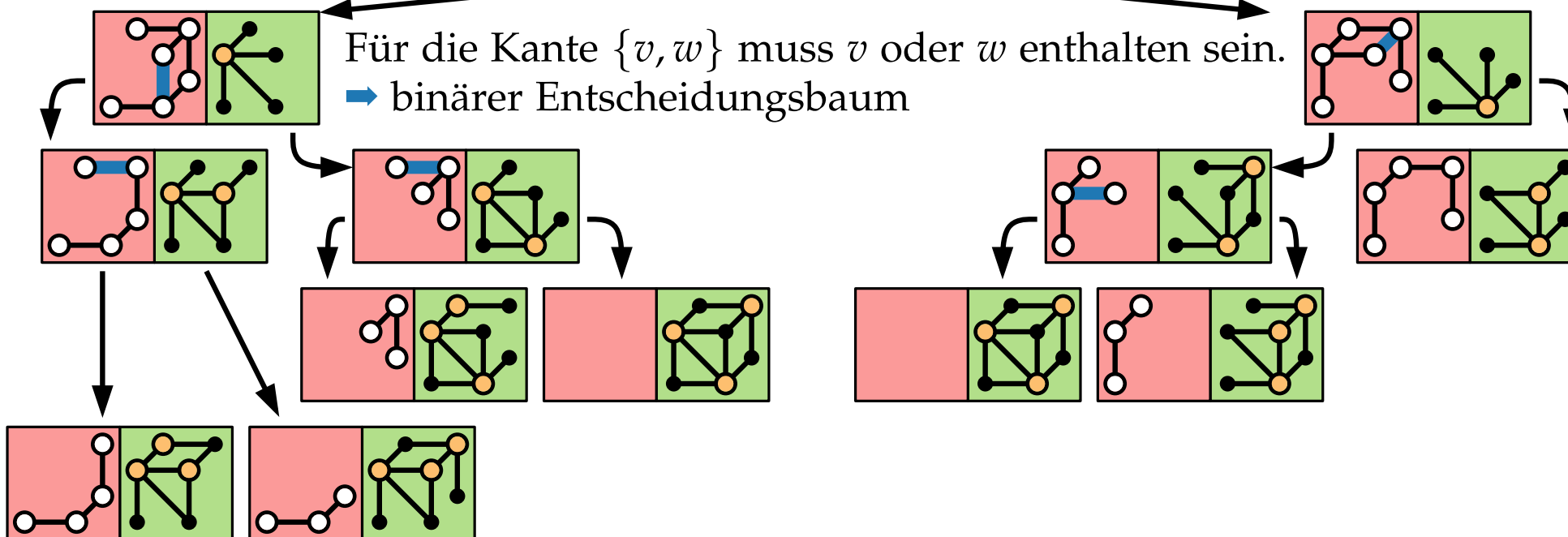
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

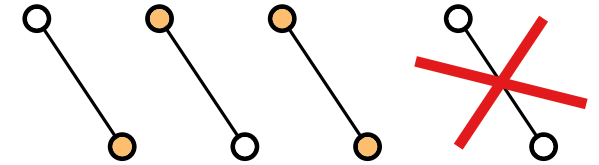
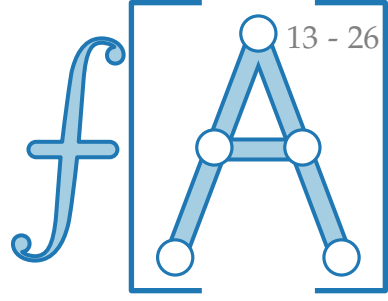
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

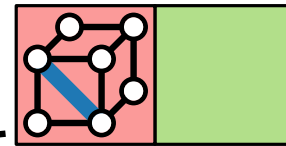
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

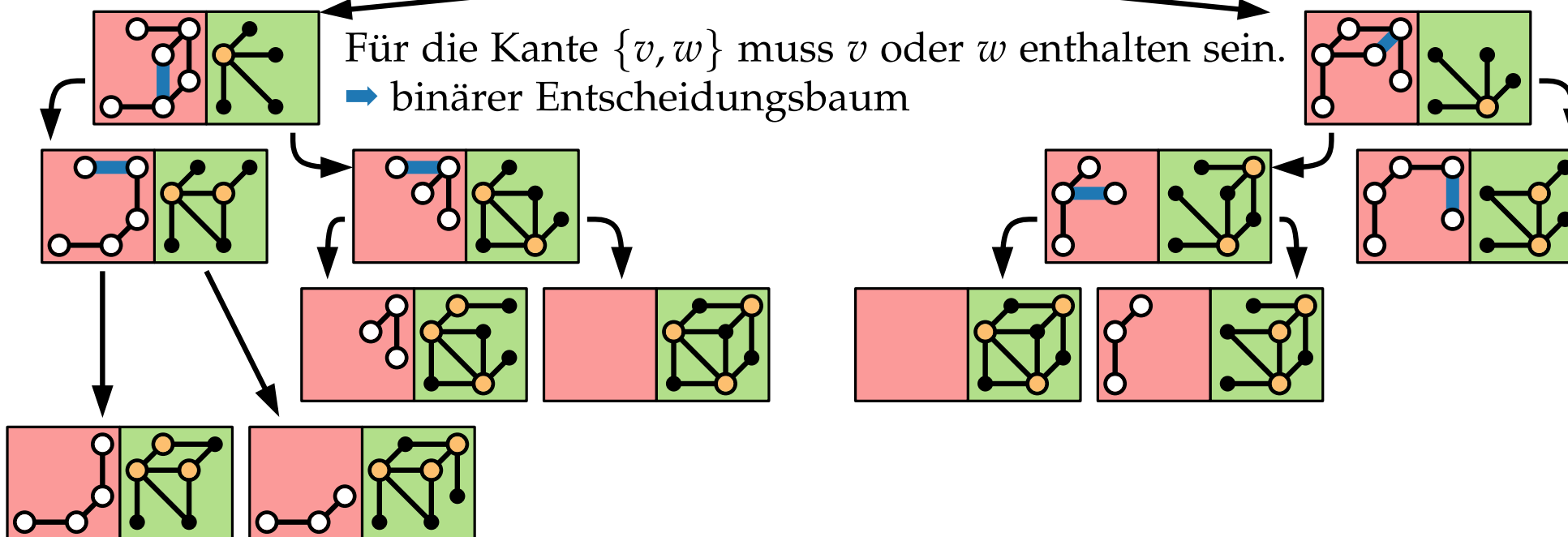
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

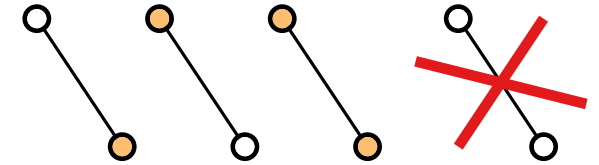
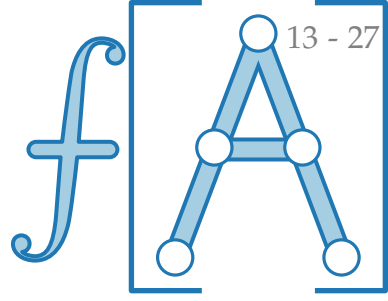
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

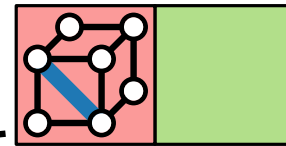
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

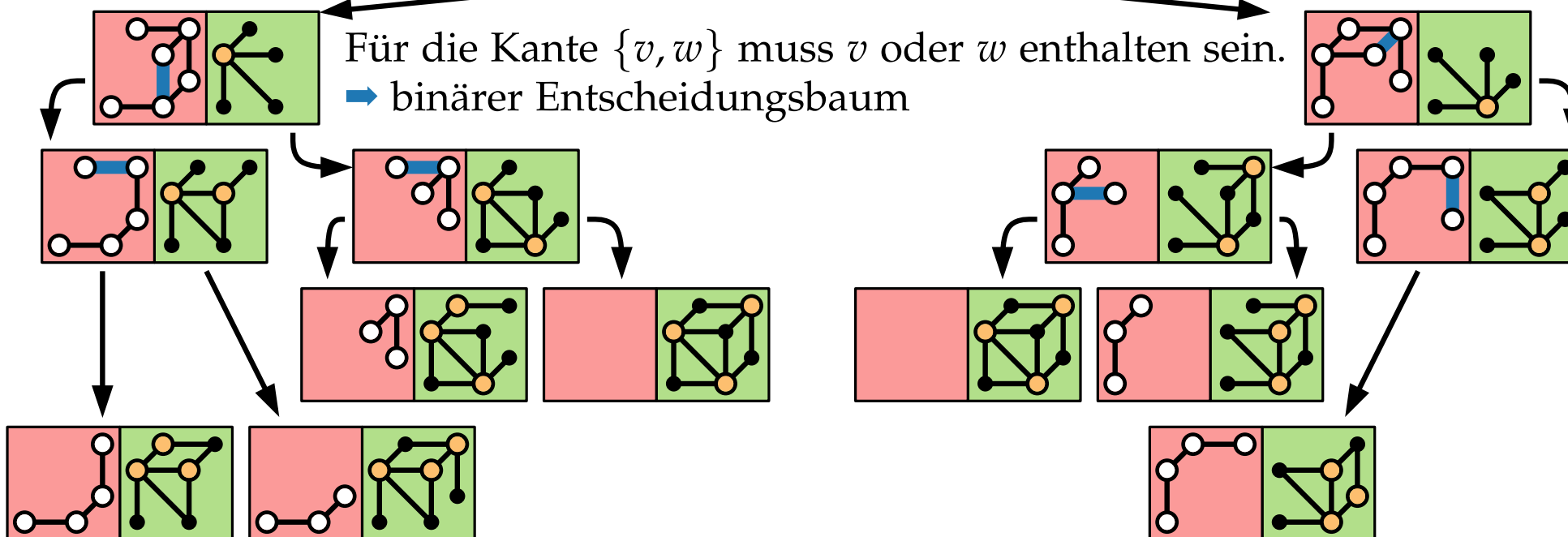
Jede Kante muss noch überdeckt werden. $\rightarrow$ wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

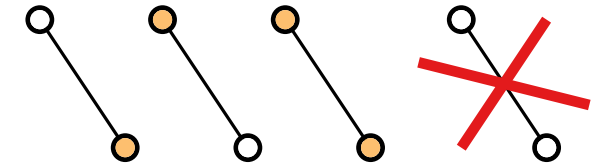
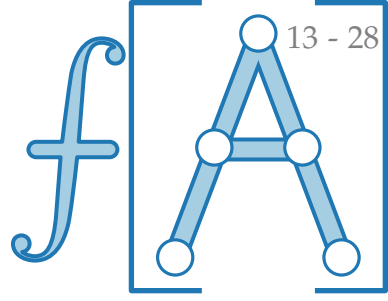
$\rightarrow$ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

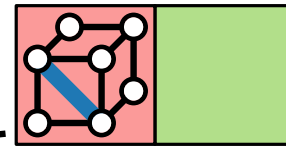
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

ausgewählte Knoten & überdeckte Kanten

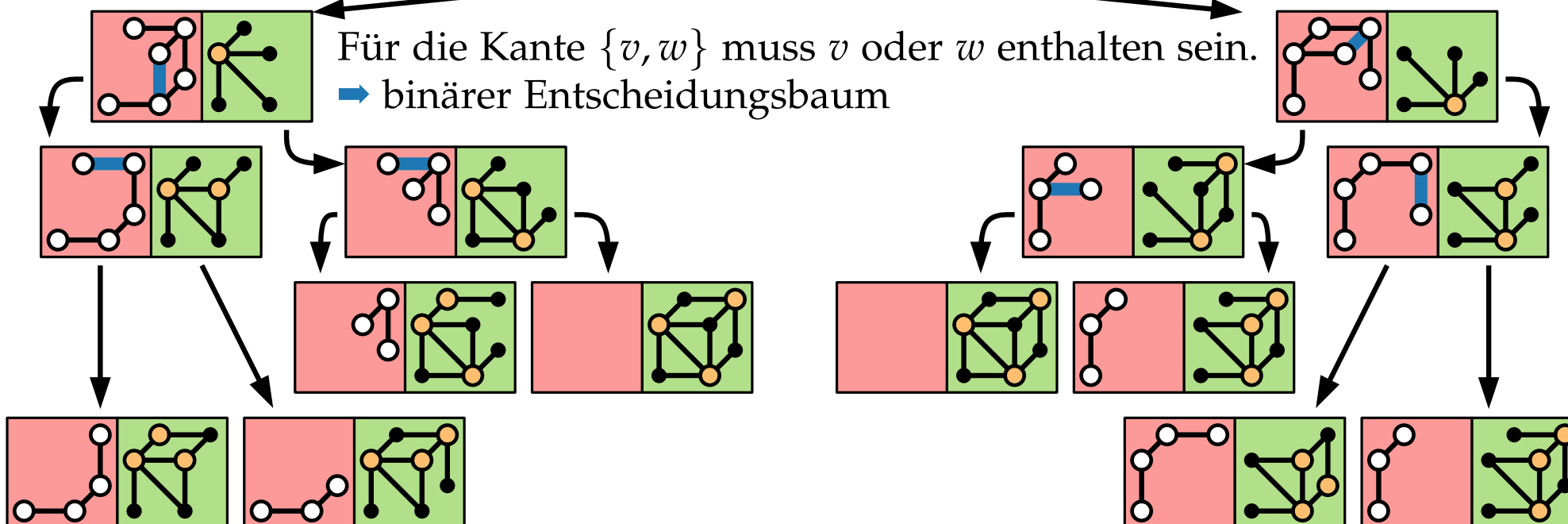
Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

Für die Kante $\{v, w\}$ muss v oder w enthalten sein.

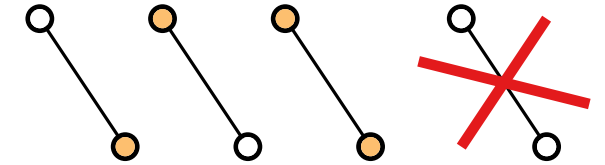
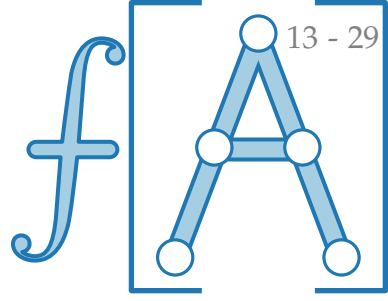
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

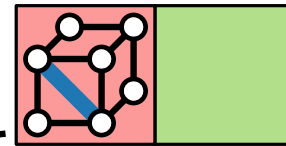
Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



noch zu überdeckender Teilgraph

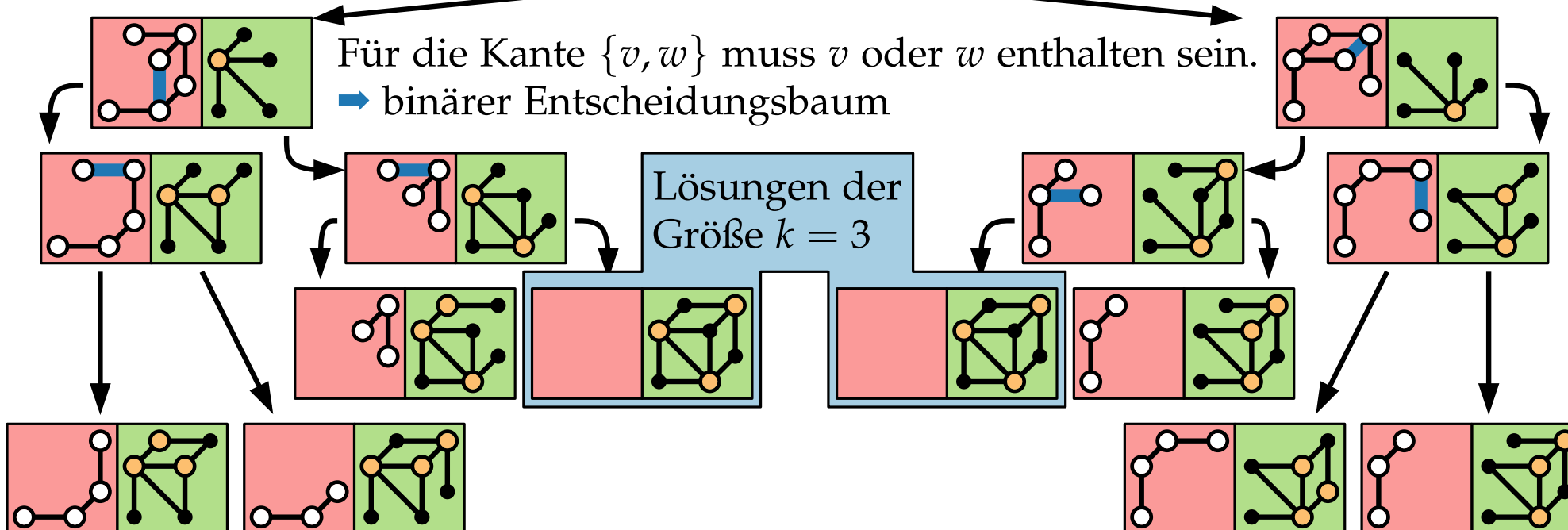
ausgewählte Knoten & überdeckte Kanten

Jede Kante muss noch überdeckt werden. → wähle eine beliebige.



$k = 3$

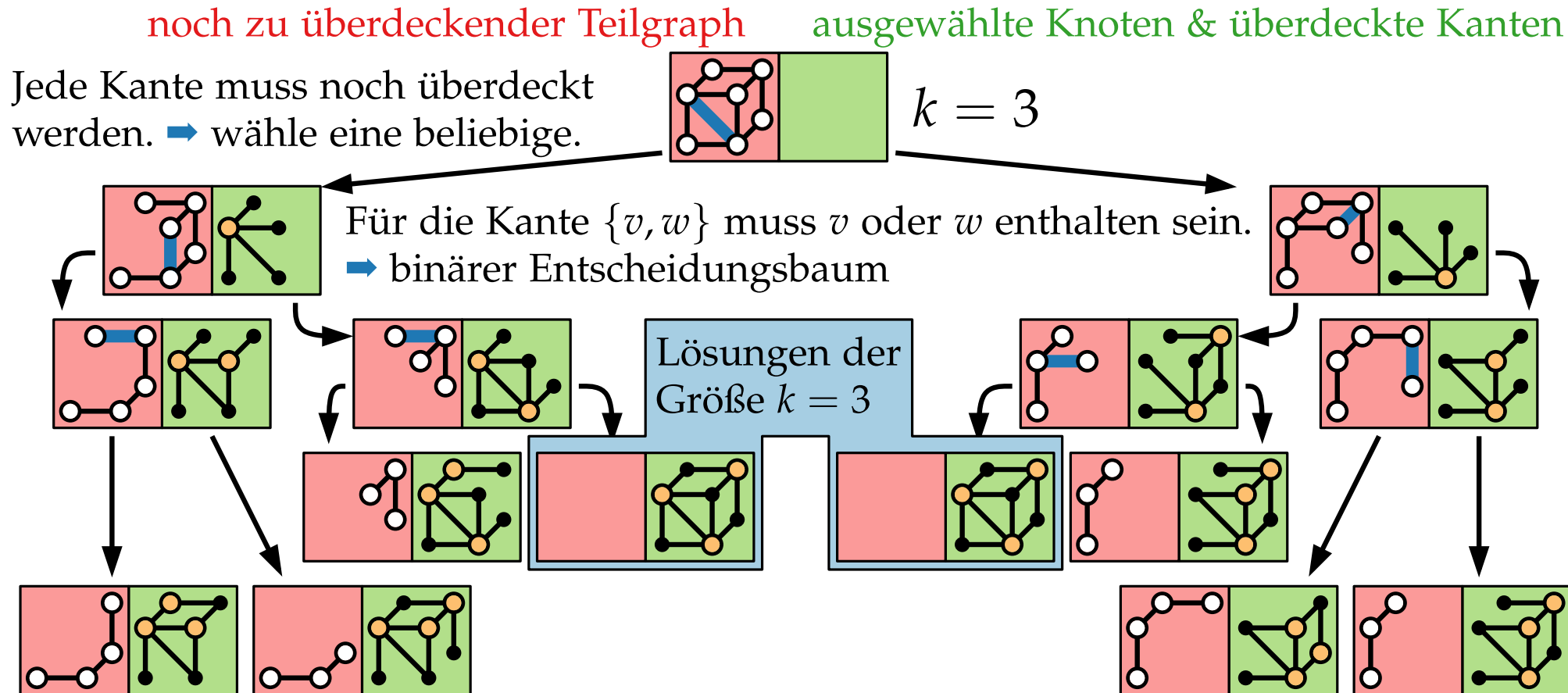
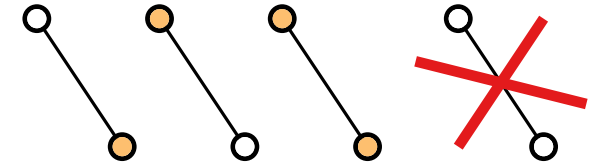
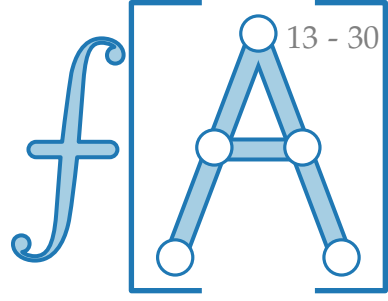
Für die Kante $\{v, w\}$ muss v oder w enthalten sein.
→ binärer Entscheidungsbaum



Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

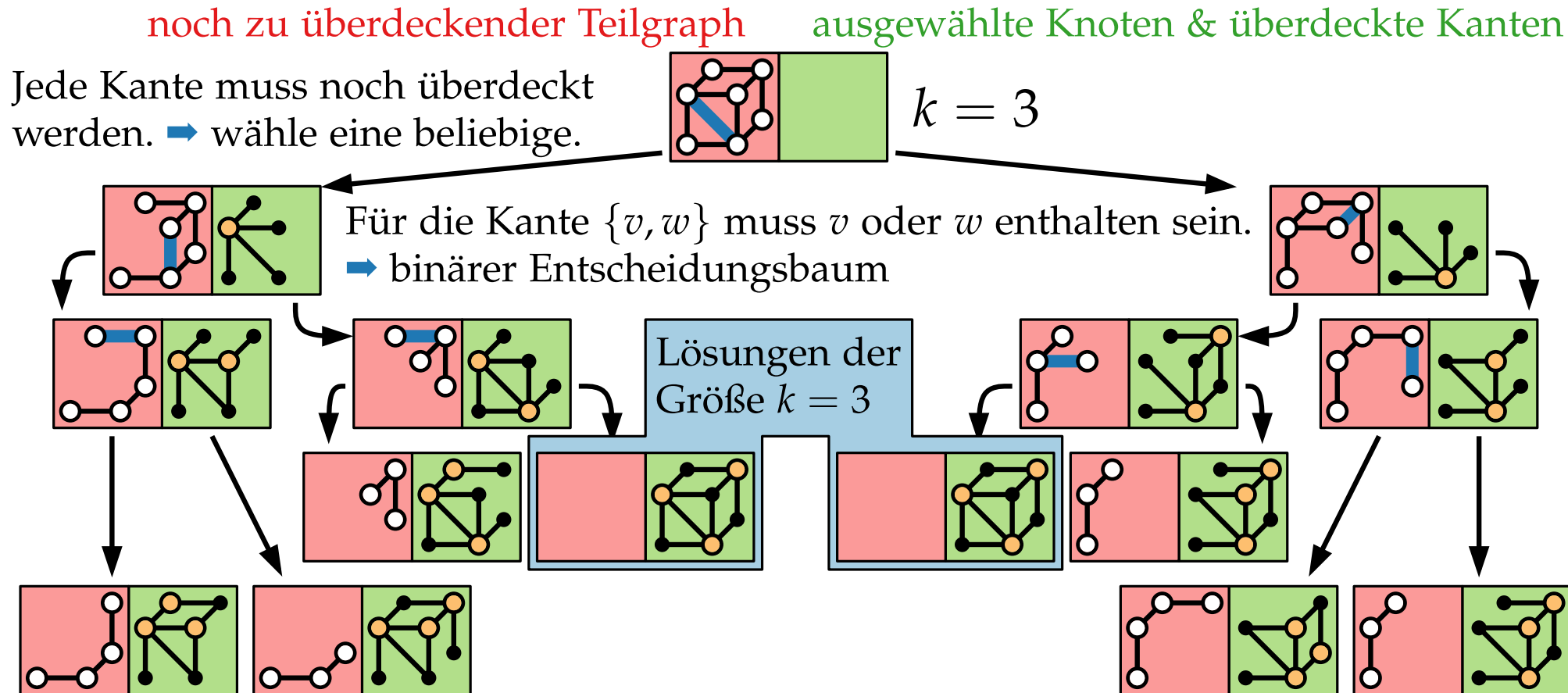
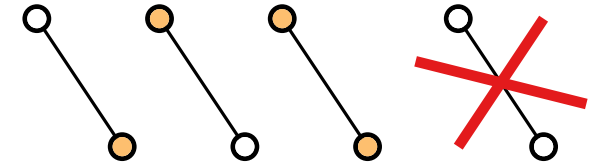
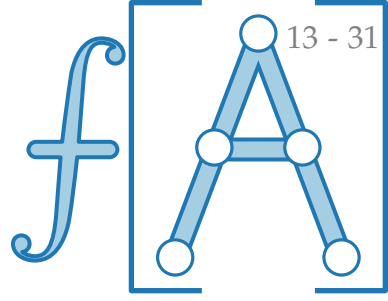


Laufzeit.

Algorithmus für VERTEXCOVER

Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.

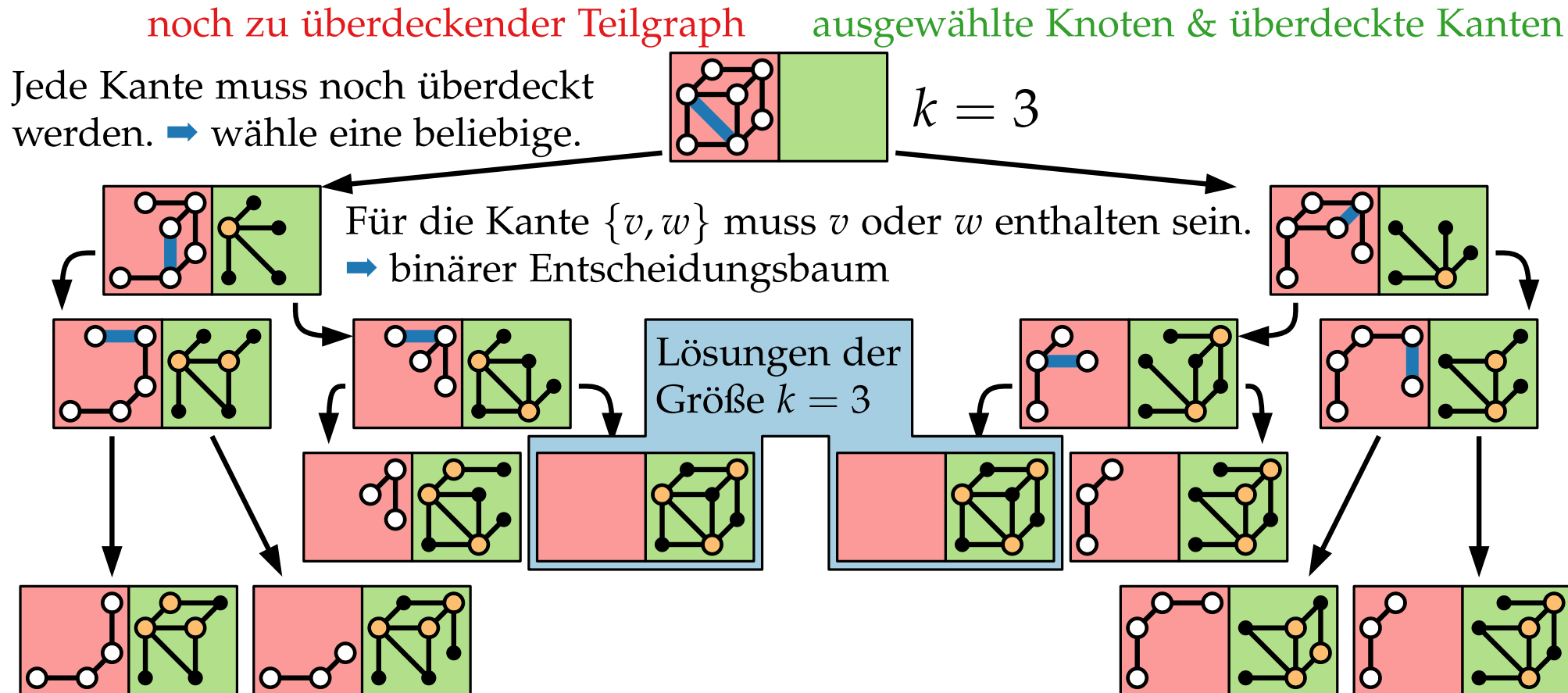
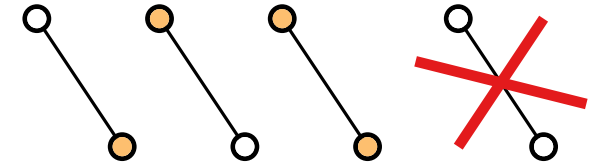
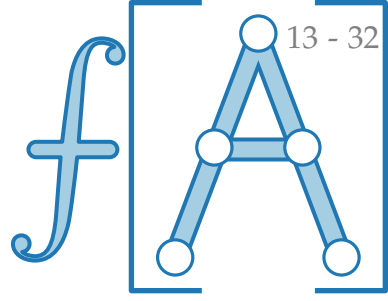


Laufzeit.
 $\mathcal{O}(2^k \cdot (n + m))$

Algorithmus für VERTEXCOVER

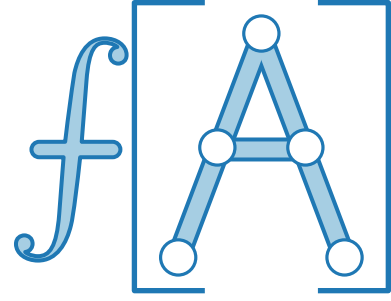
Idee. Verbessere Phase II durch Aufbau eines Suchbaums.

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



Laufzeit.

$$\mathcal{O}(2^k \cdot (n + m)) \\ = \mathcal{O}^*(2^k)$$

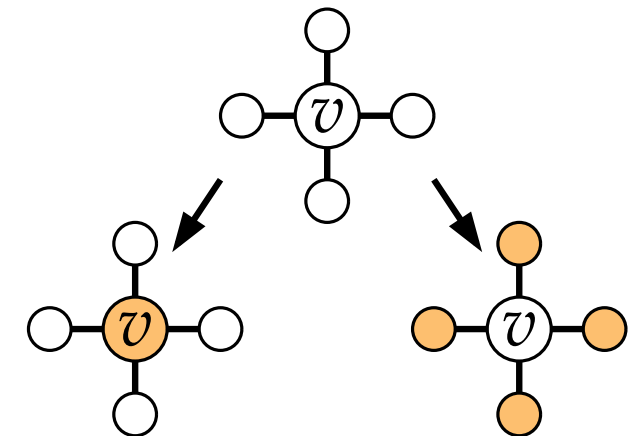
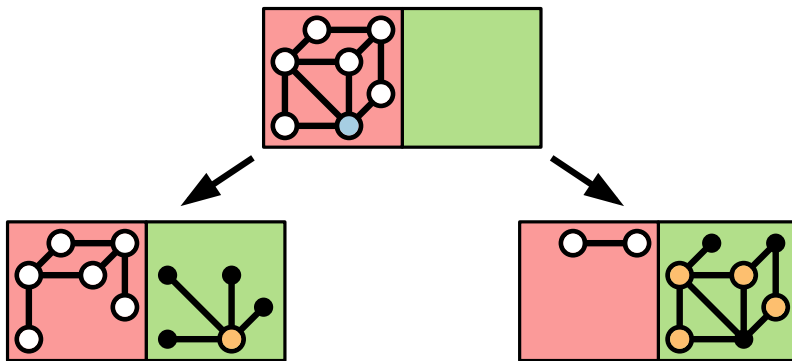


Fortgeschrittene Algorithmen

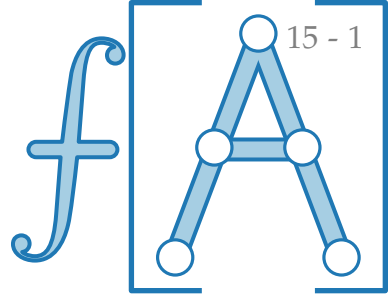
20. Vorlesung

Parametrisierte Algorithmen

Teil IV: Der Grad-4-Algorithmus

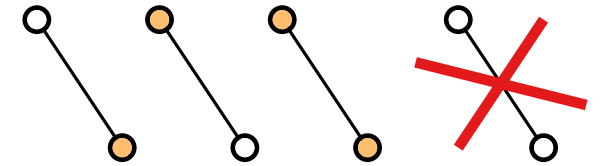


Ein paar einfache Beobachtungen...



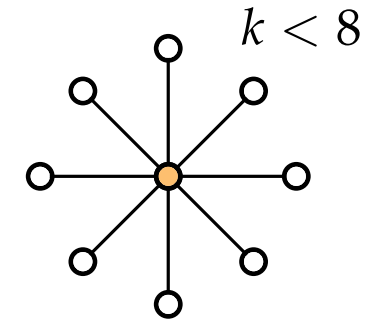
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1. Für jede Kante $uv \in E$ gilt: $u \in C$ oder $v \in C$.



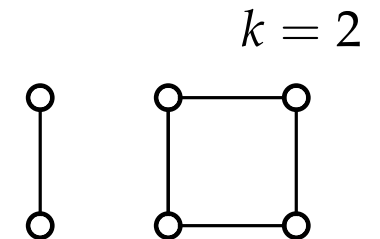
Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.

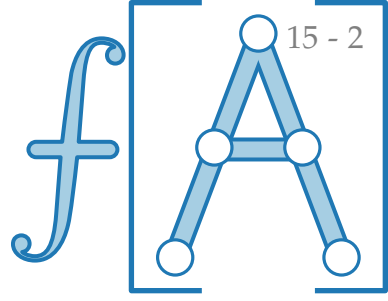


Was gilt, falls $m > k^2$ und alle Knoten $\text{Grad} \leq k$ haben?

Beob. 3. Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.

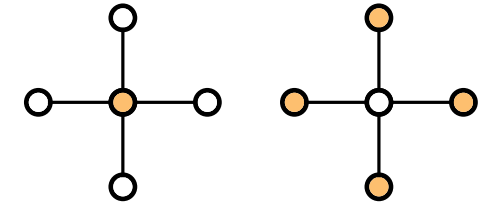


Ein paar einfache Beobachtungen...



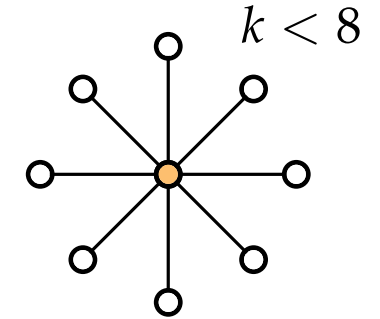
Sei G Graph, C ein VC für G , v Knoten, der nicht in C liegt.
Welche Knoten liegen dann sicher in C ?

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



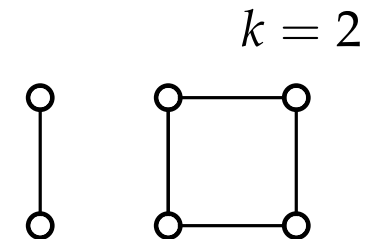
Betrachte Entscheidungsproblem k -VC.
Was gilt für Knoten mit $\text{Grad} > k$?

Beob. 2. Jeder Knoten mit $\text{Grad} > k$ ist in jedem k -VC enthalten.



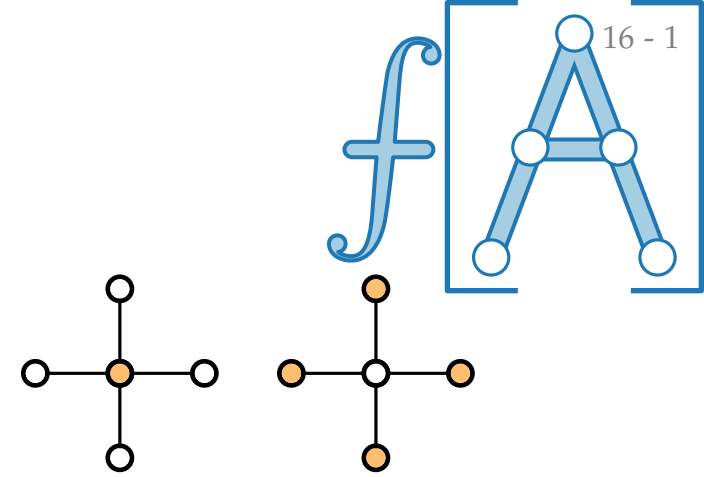
Was gilt, falls $m > k^2$ und alle Knoten $\text{Grad} \leq k$ haben?

Beob. 3. Falls $m > \Delta \cdot k$ mit $\Delta = \max_{v \in V} \deg v$, so hat G kein k -VC.



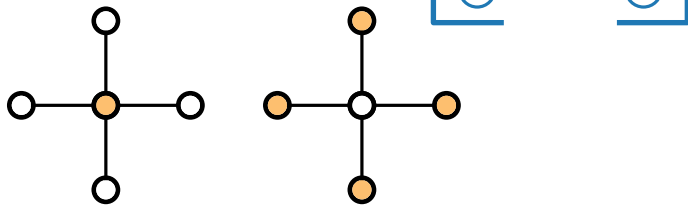
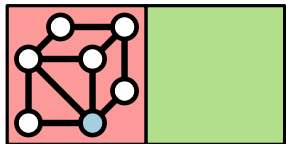
Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



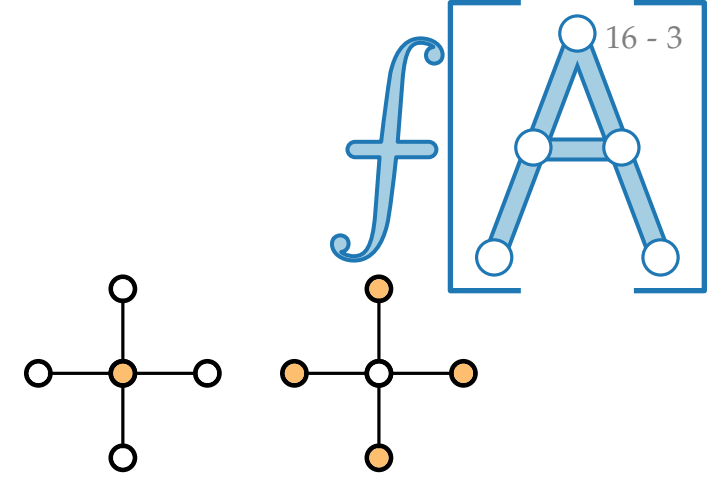
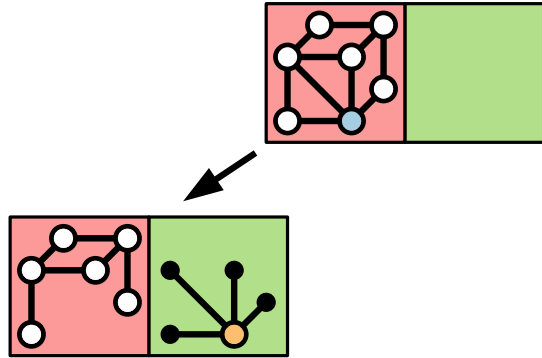
Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



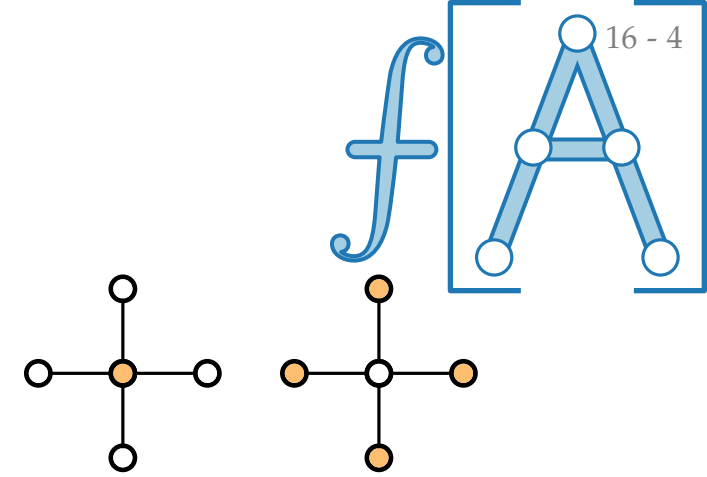
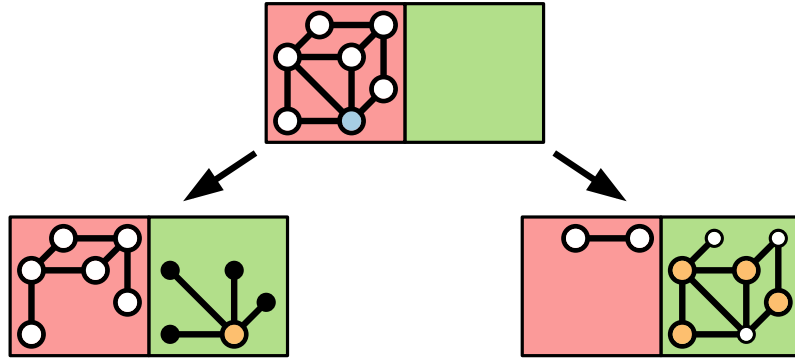
Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



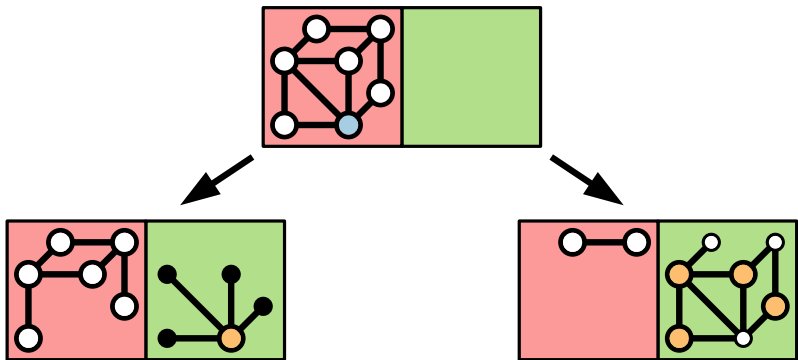
Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

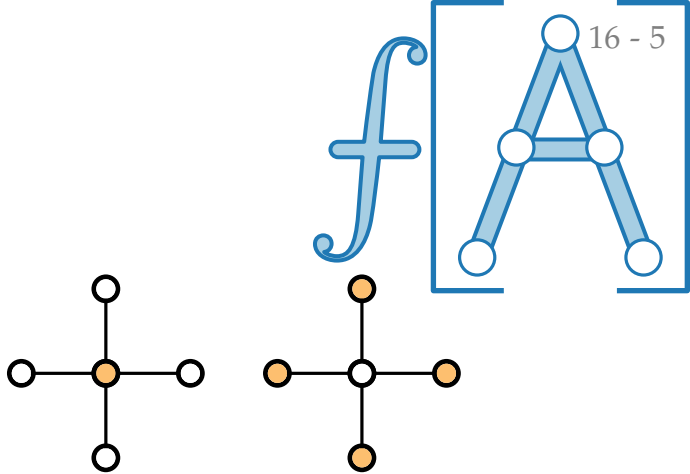


Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

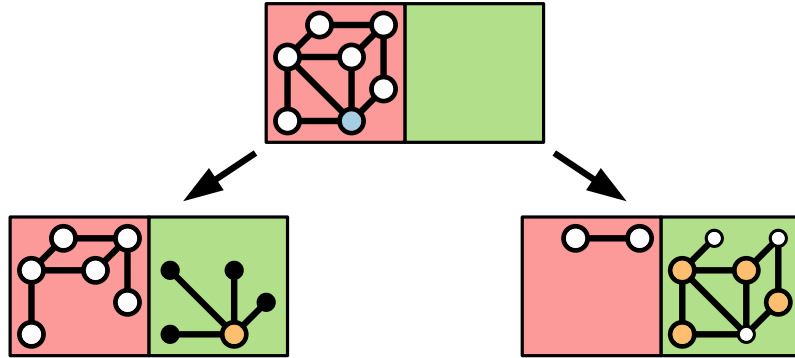


→ $T(k) =$

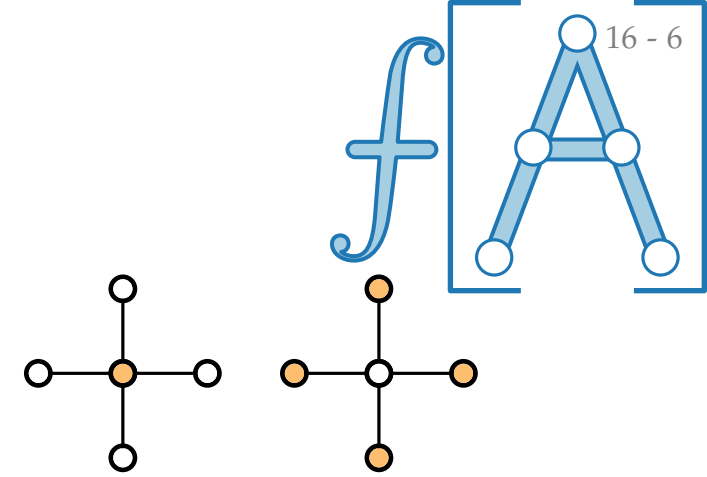


Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

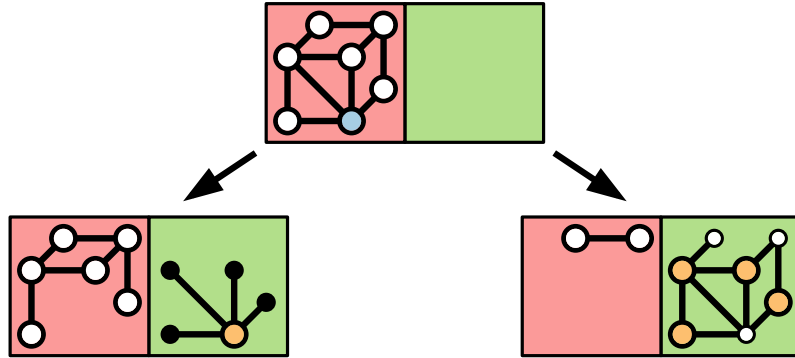


$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1$$

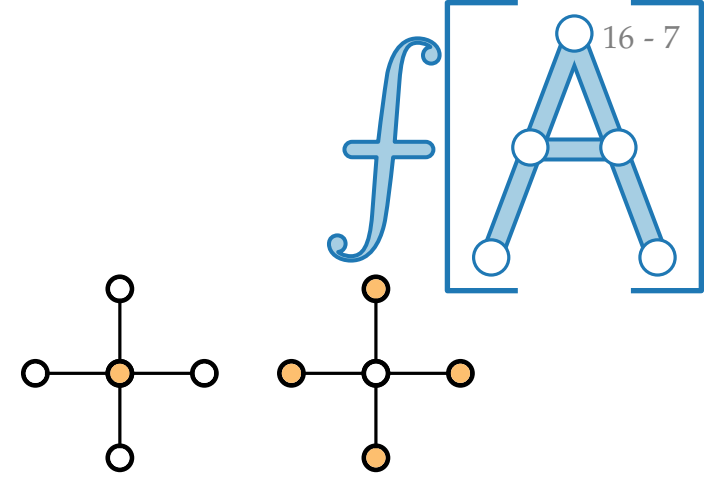


Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

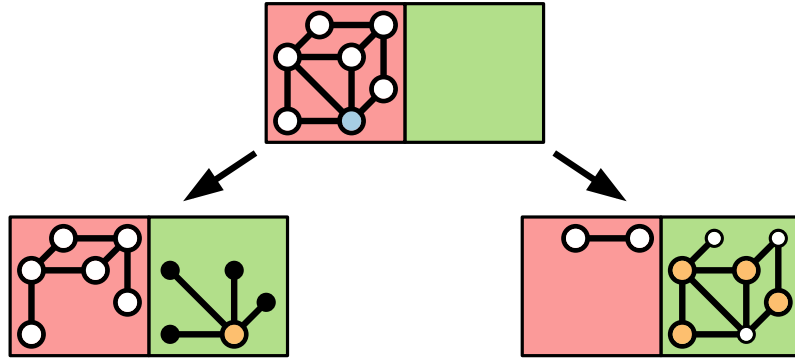


$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1$$

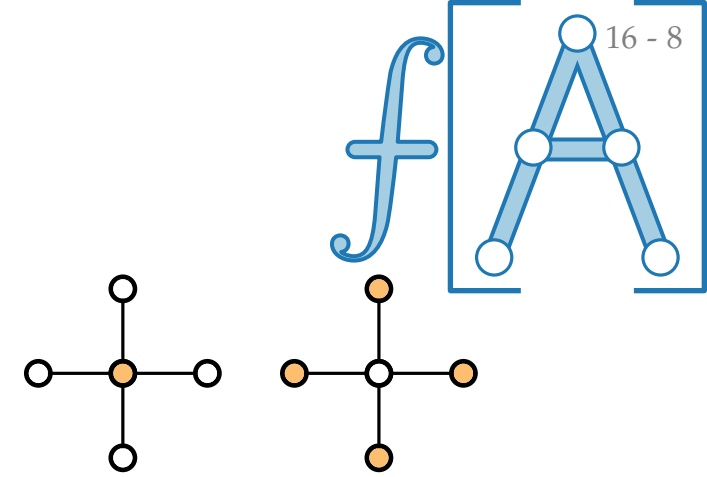


Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

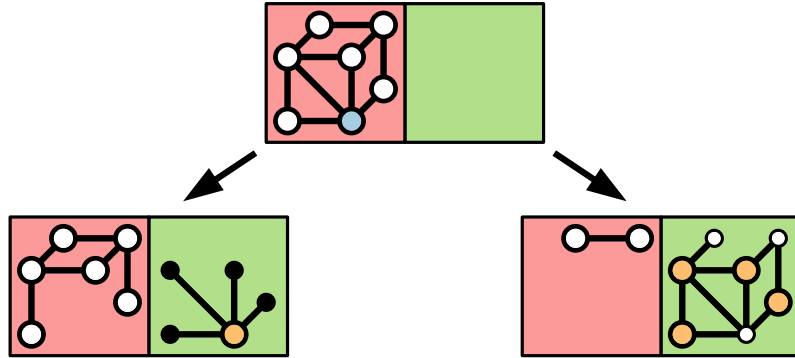


$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k)$$

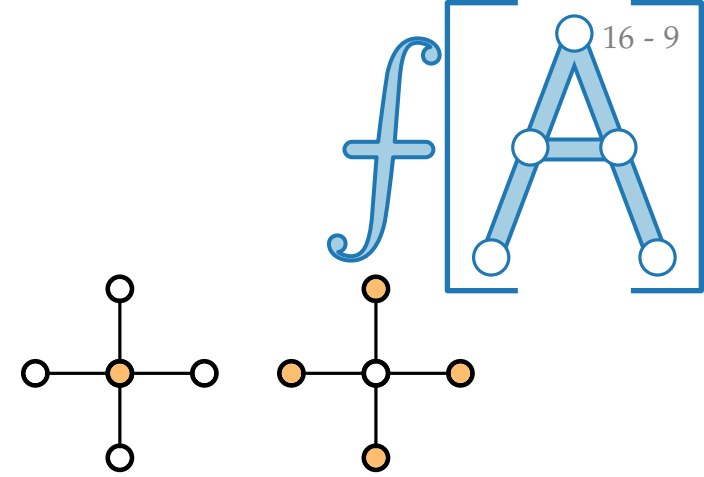


Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

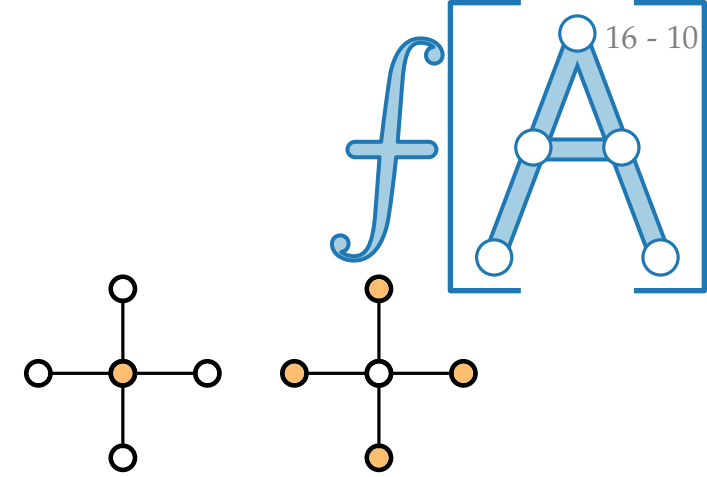
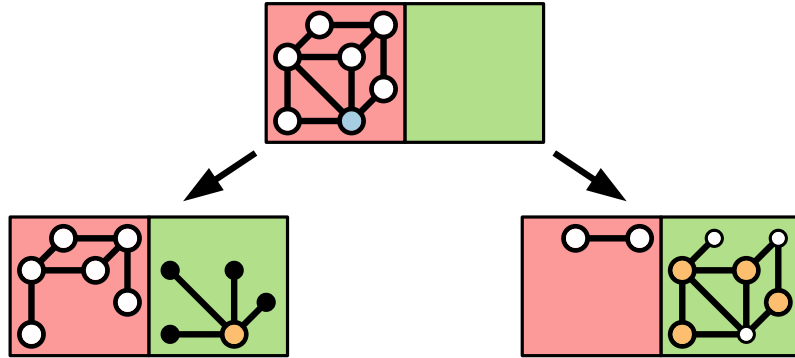


$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$



Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.

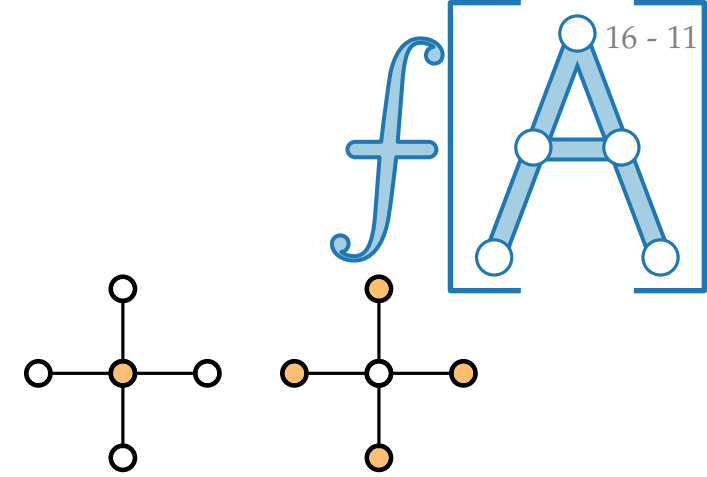
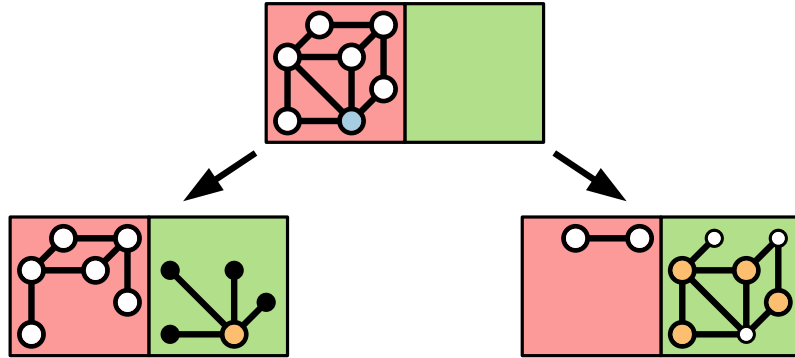


$$\Rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{😞}$$

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



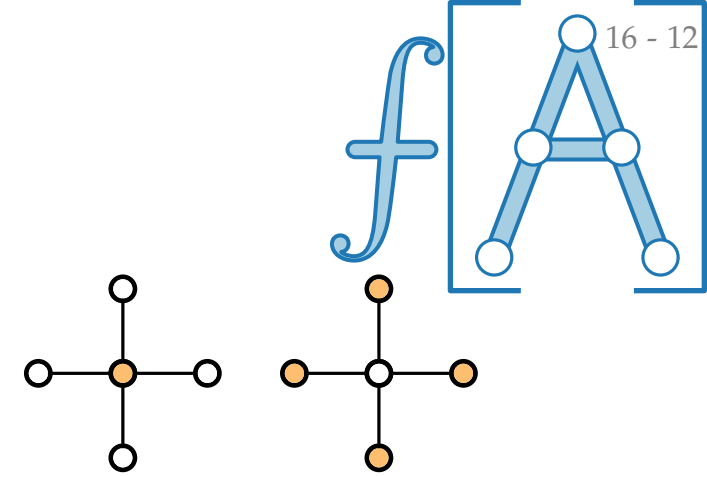
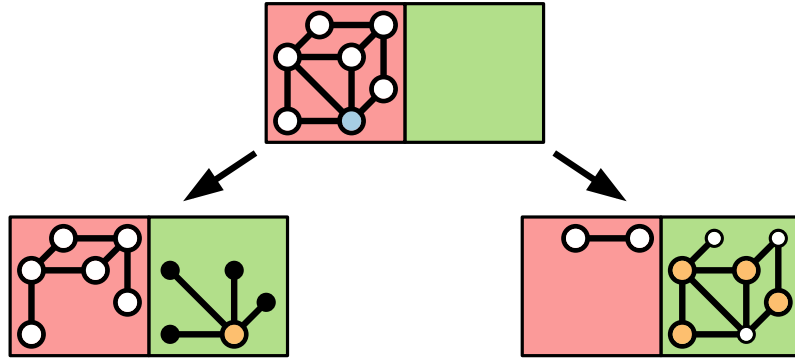
$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

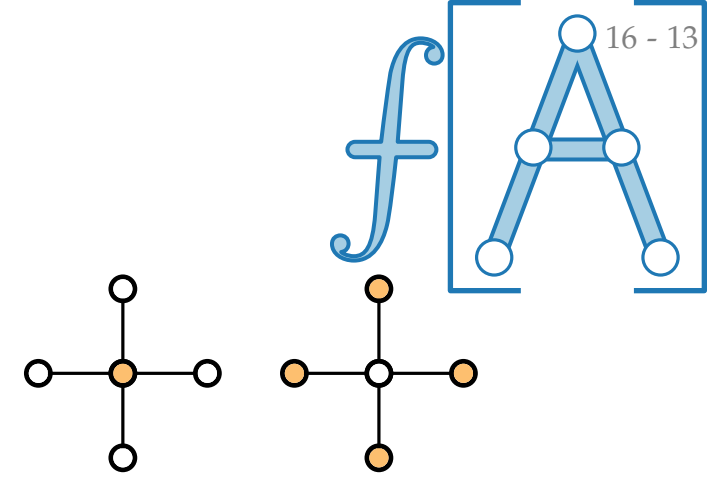
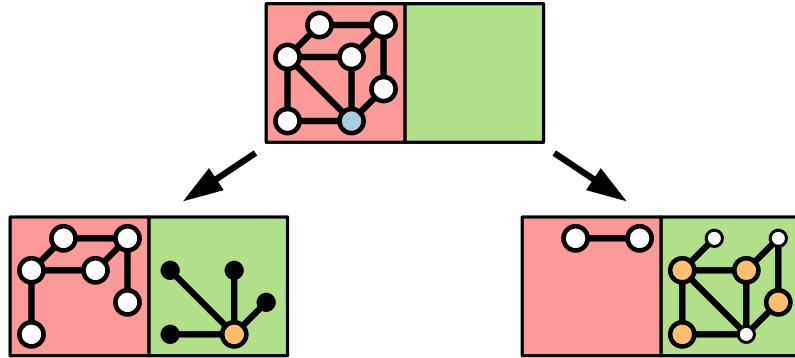
Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

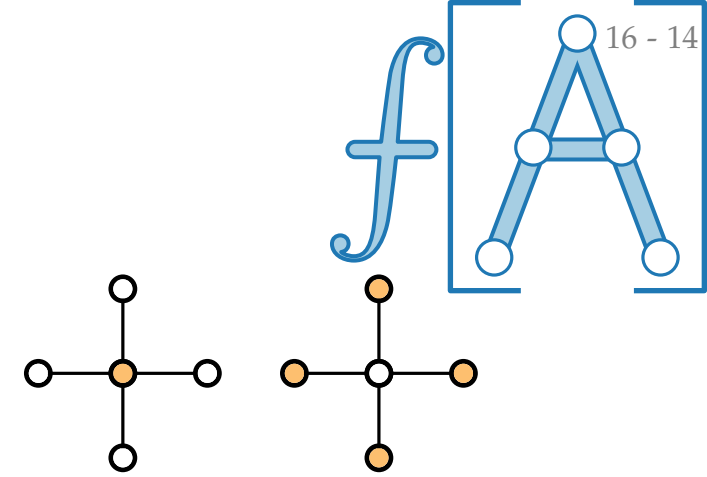
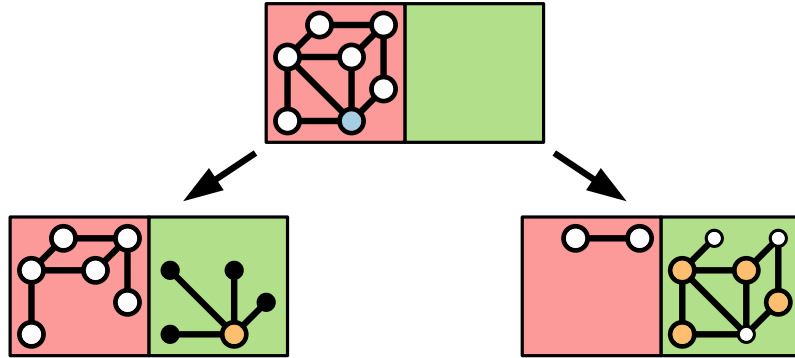
Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k =$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

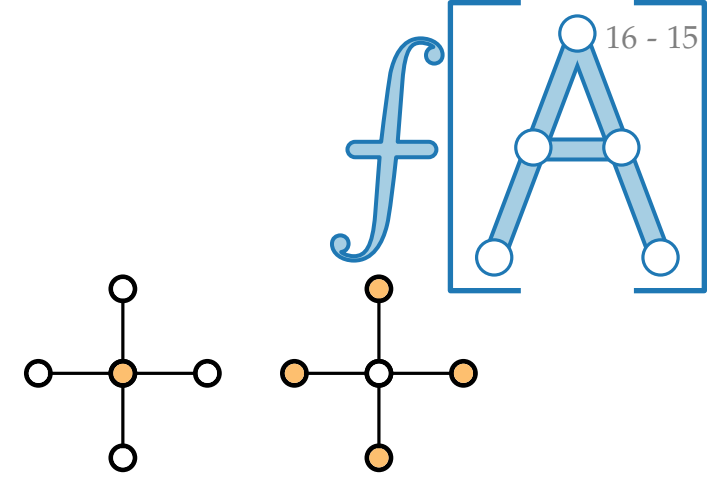
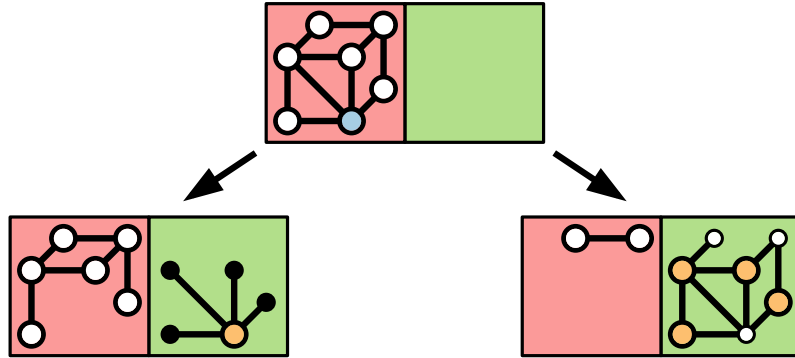
Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1}$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

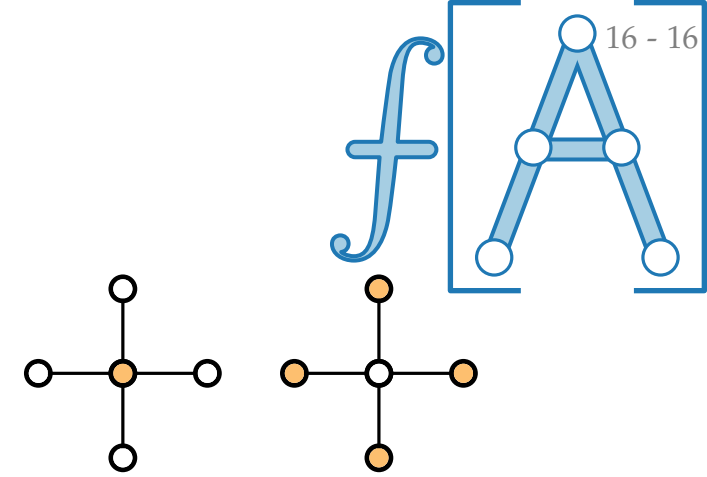
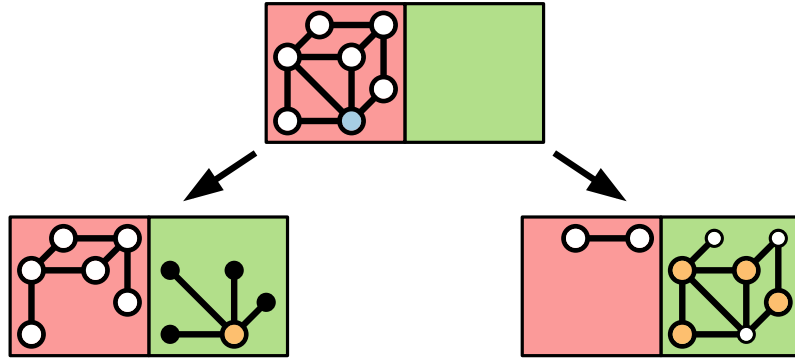
Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1}$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

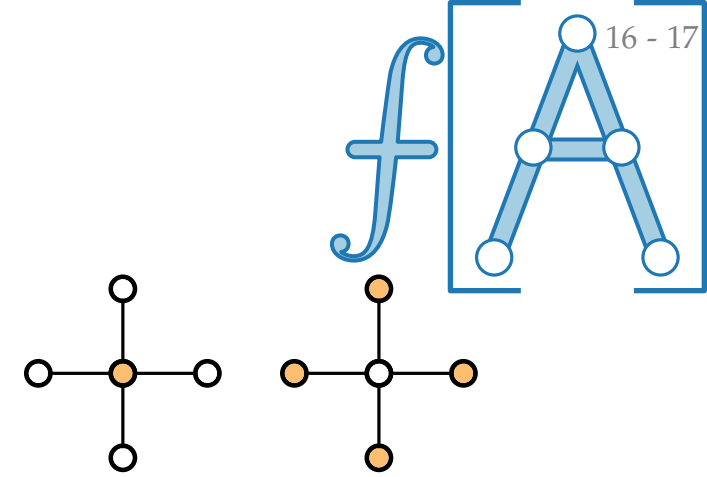
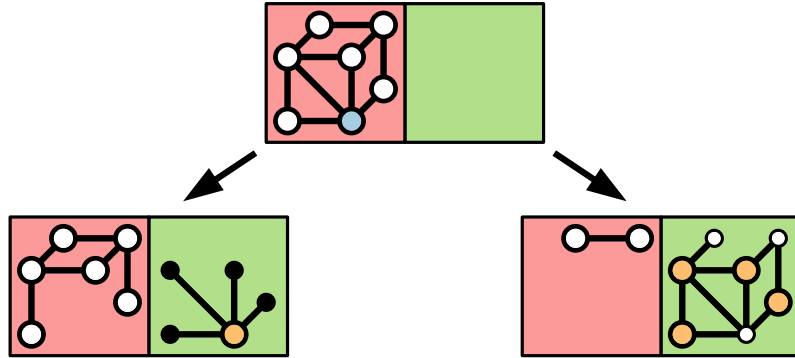
Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow \quad z^k = z^{k-4} + z^{k-1} \quad \cdot \frac{1}{z^{k-4}}$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

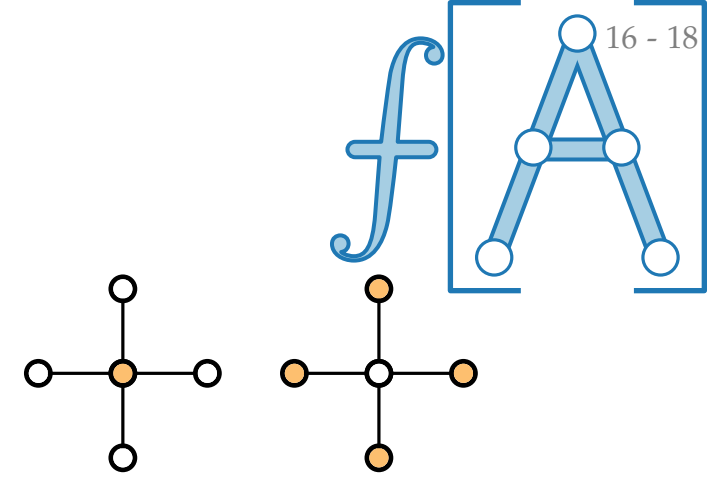
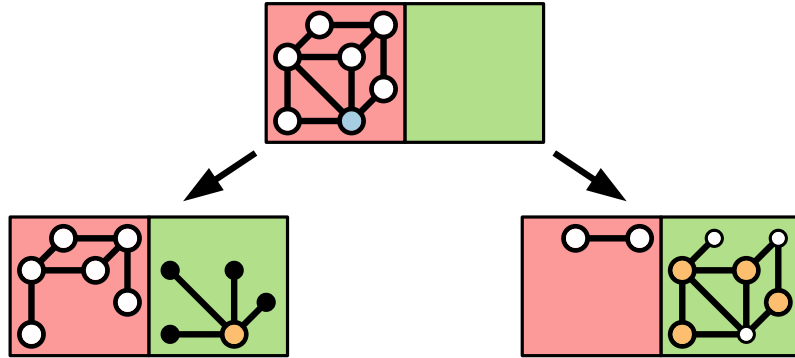
$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1} \quad \cdot \frac{1}{z^{k-4}}$$

$$\rightarrow \text{Charakteristisches Polynom: } z^4 = 1 + z^3$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



→ $T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k)$ 😞

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

→ $T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$

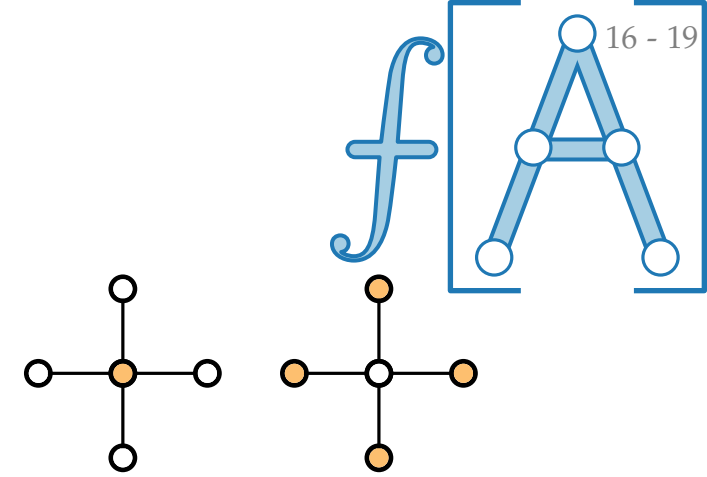
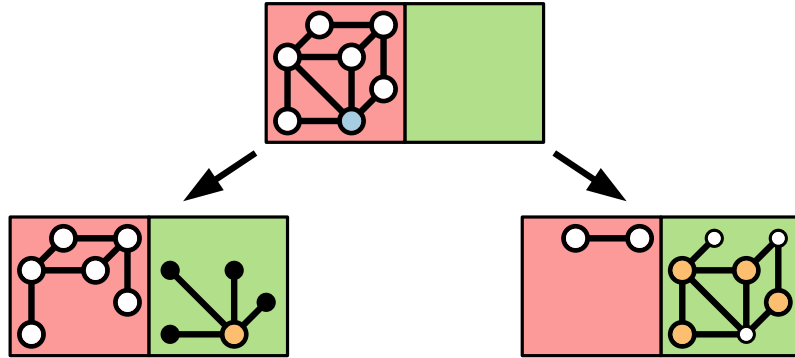
Teste $T(k) = z^k - 1$ → $z^k = z^{k-4} + z^{k-1}$ $\cdot \frac{1}{z^{k-4}}$

→ Charakteristisches Polynom: $z^4 = 1 + z^3$

→ Größte positive Lösung:

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{😞}$$

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

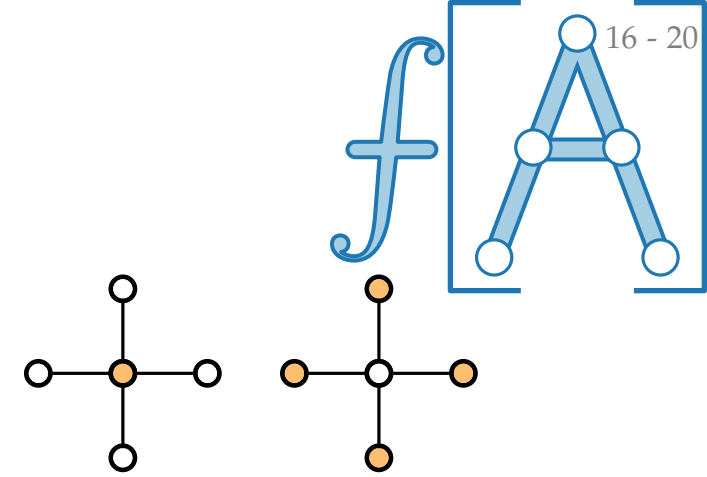
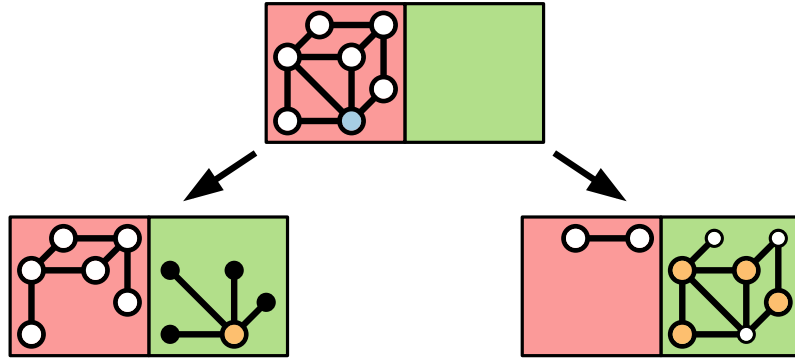
$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1} \quad \cdot \frac{1}{z^{k-4}}$$

$$\rightarrow \text{Charakteristisches Polynom: } z^4 = 1 + z^3$$

$$\rightarrow \text{Größte positive Lösung: } z \approx 1.38$$

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k)$


Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$

Teste $T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1} \quad \cdot \frac{1}{z^{k-4}}$

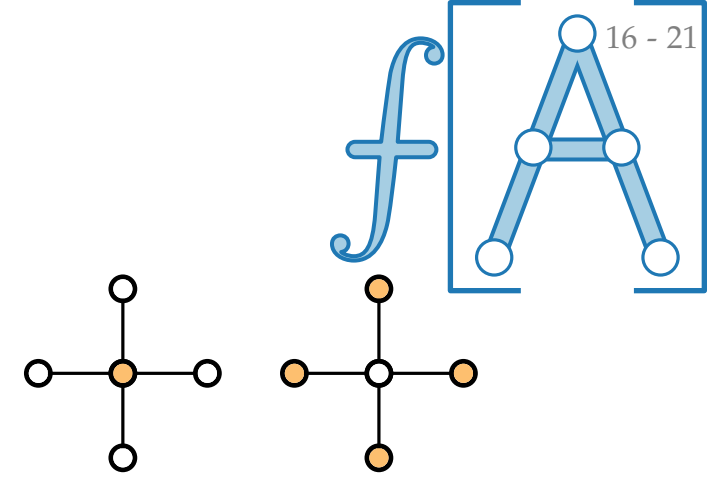
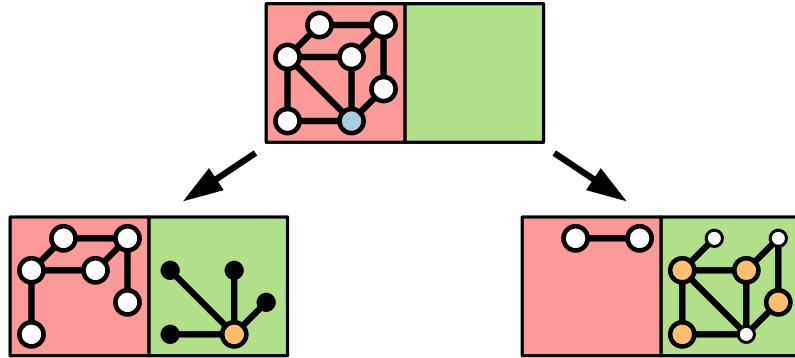
$\rightarrow$ Charakteristisches Polynom: $z^4 = 1 + z^3$

$\rightarrow$ Größte positive Lösung: $z \approx 1.38$

$\rightarrow T(k) \in \mathcal{O}(1.38^k)$.

Der Grad-4-Algorithmus

Beob. 1'. Für jeden Knoten v gilt: $v \in C$ oder $N(v) \subseteq C$.



$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-1) + 1 \in \mathcal{O}(2^k) \quad \text{☹️}$$

Was wäre, wenn wir garantieren könnten, dass wir immer mit einem Knoten v verzweigen, dessen Grad mindestens 4 ist?

$$\rightarrow T(k) = T(k-1) + T(k - \deg v) + 1 \leq T(k-1) + T(k-4) + 1$$

$$\text{Teste } T(k) = z^k - 1 \quad \rightarrow z^k = z^{k-4} + z^{k-1} \quad \cdot \frac{1}{z^{k-4}}$$

$$\rightarrow \text{Charakteristisches Polynom: } z^4 = 1 + z^3$$

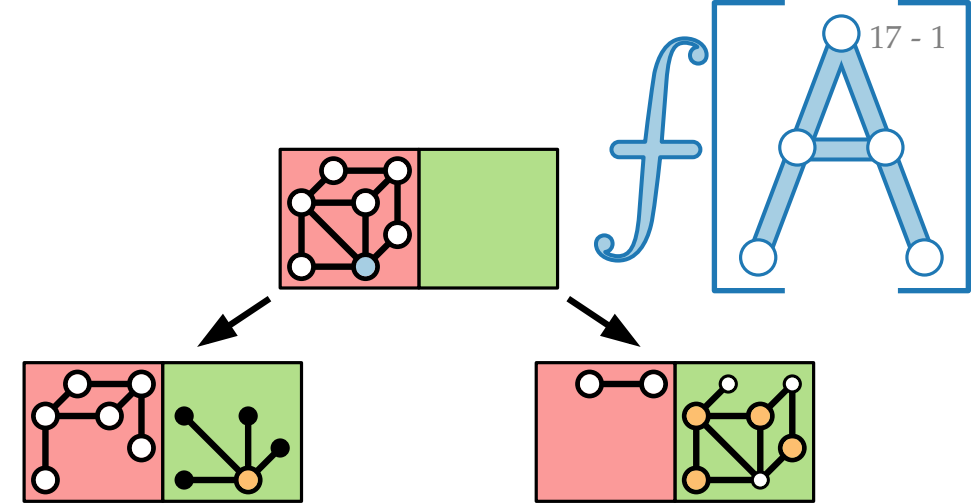
$$\rightarrow \text{Größte positive Lösung: } z \approx 1.38$$

$$\rightarrow T(k) \in \mathcal{O}(1.38^k). \quad \text{Aber wie stellen wir } \deg v \geq 4 \text{ sicher?}$$

Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

$$\Rightarrow T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$$

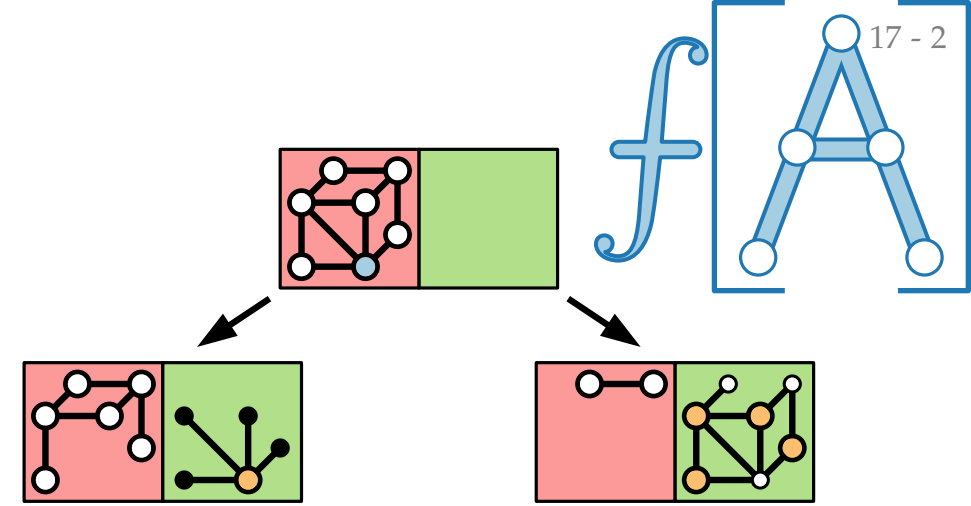


Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

$$\Rightarrow T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.



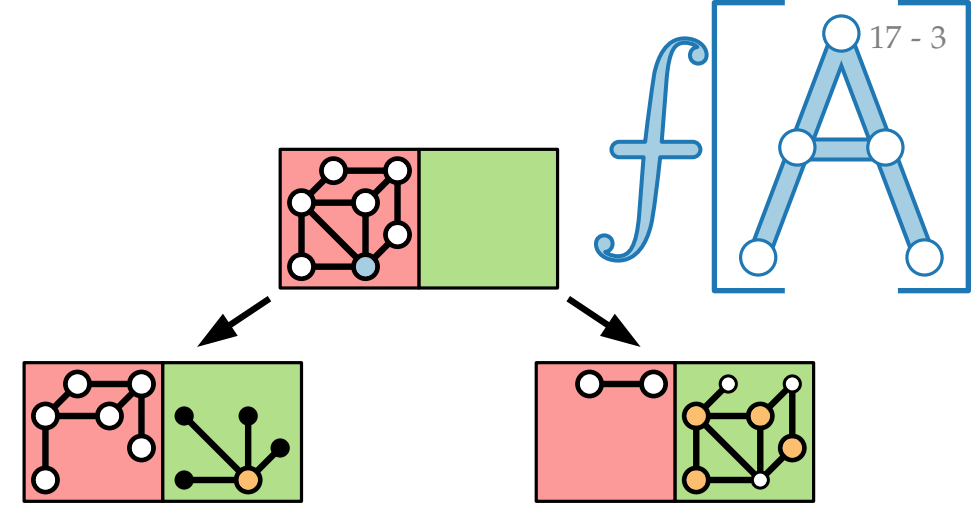
Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$



Verzweigungsvektor

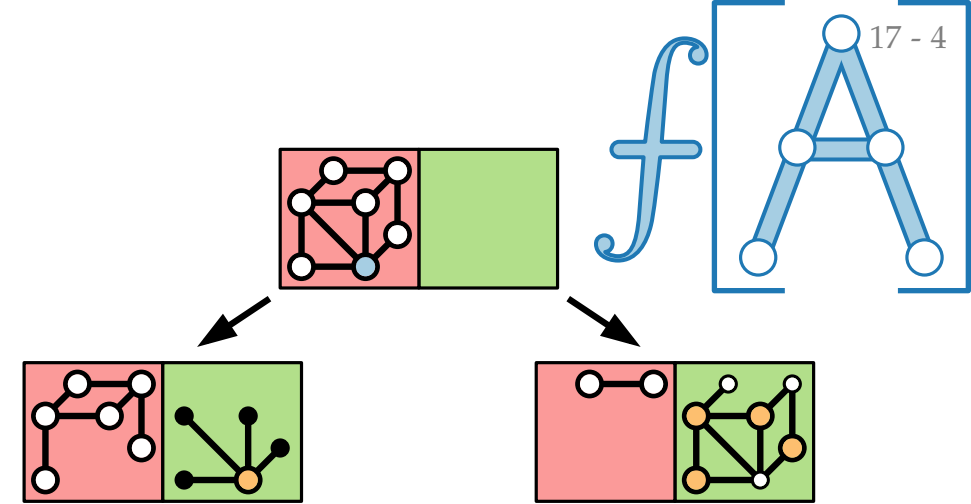
Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

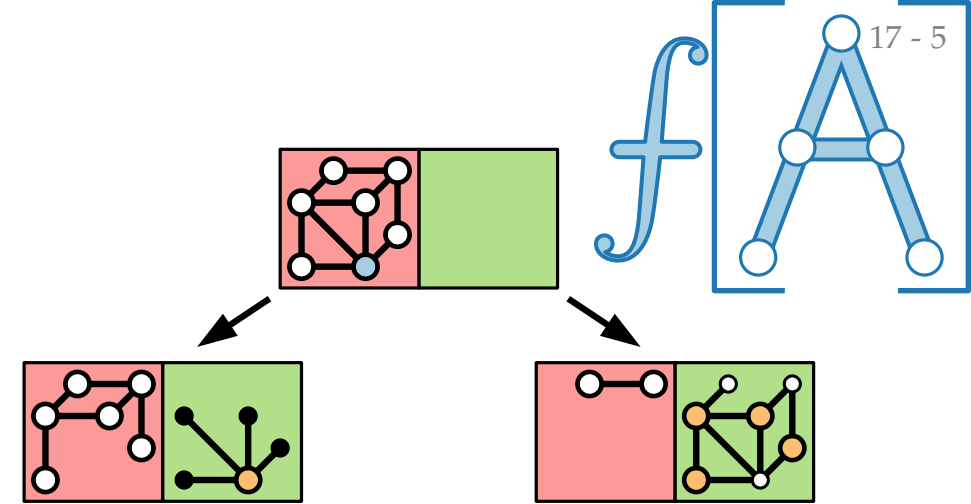
→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

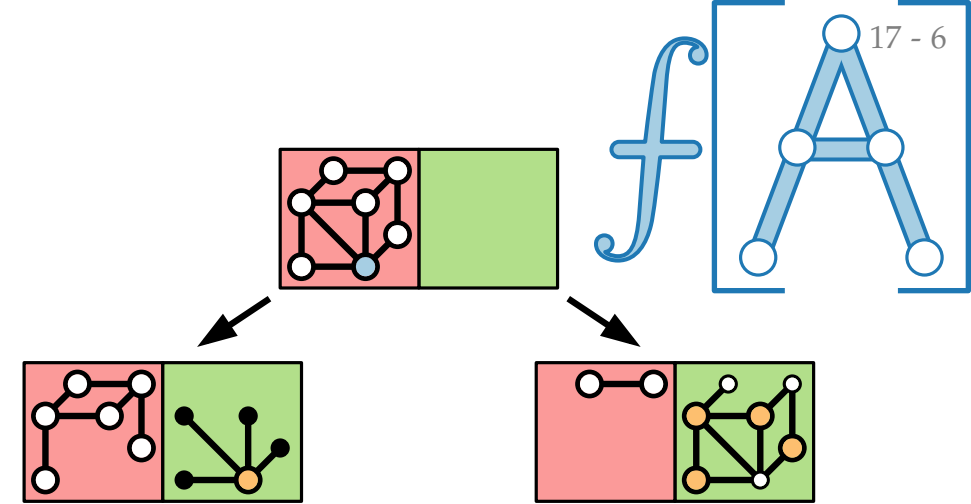
Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor

Basis b



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

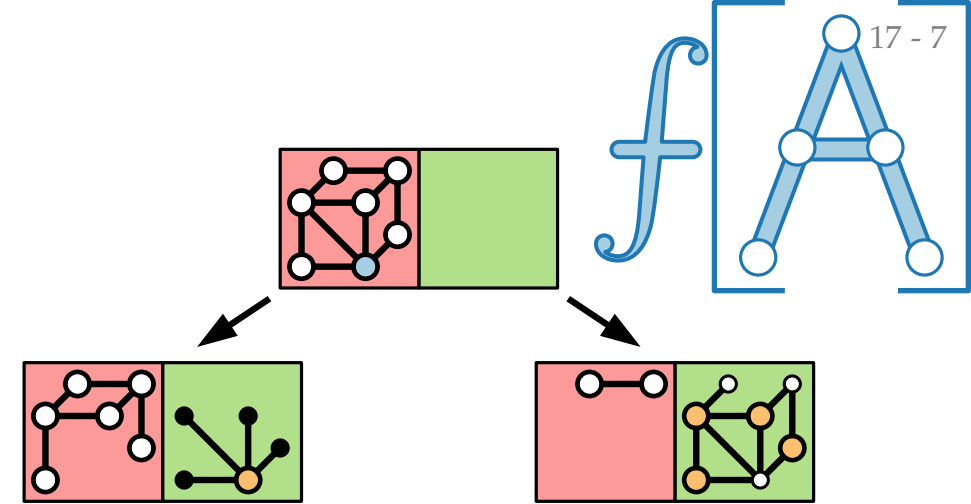
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k - t_1) + \dots + T(k - t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	$(1, 1)$
Basis b	2



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

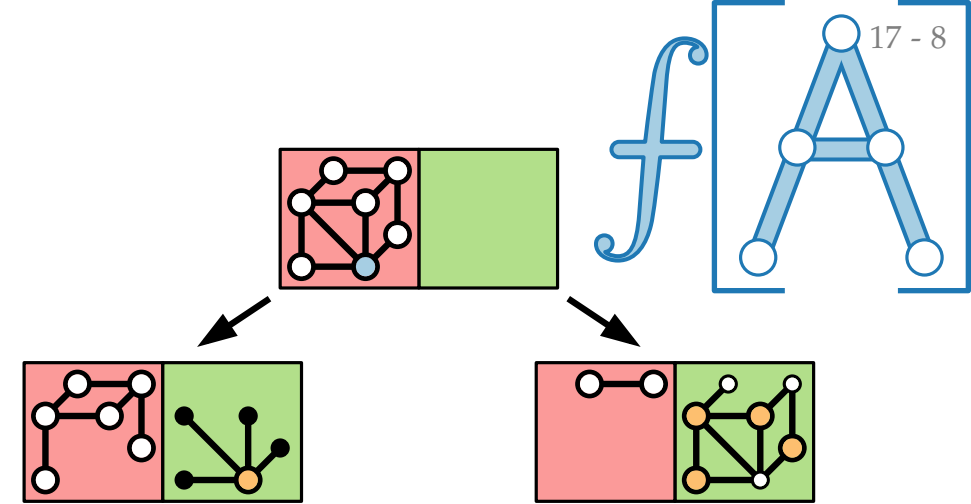
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	$(1, 1)$	$(1, 2)$
Basis b	2	1.618



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

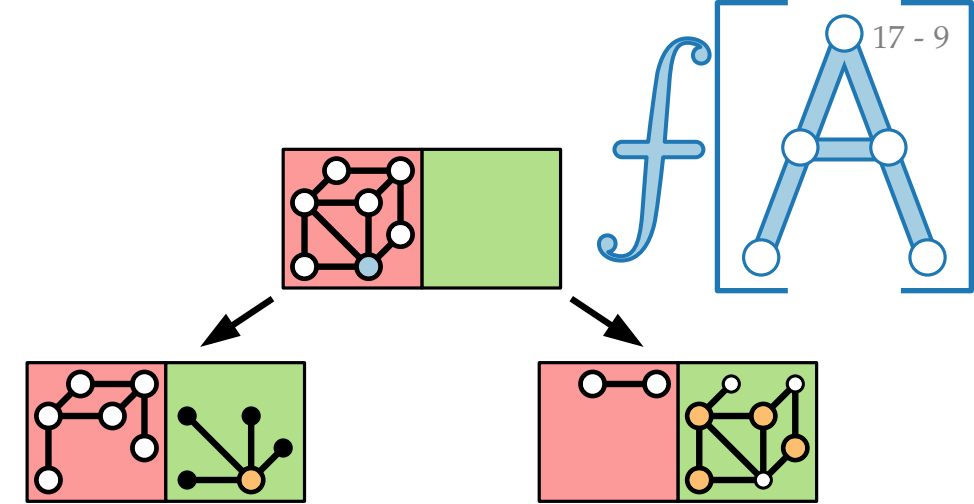
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)
Basis b	2	1.618	1.4656



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

➔ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

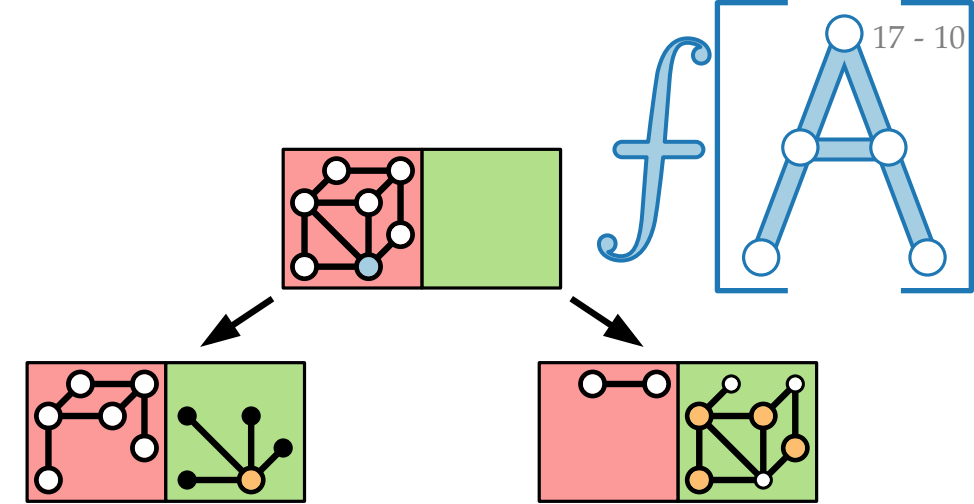
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

➔ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)
Basis b	2	1.618	1.4656	1.3803



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

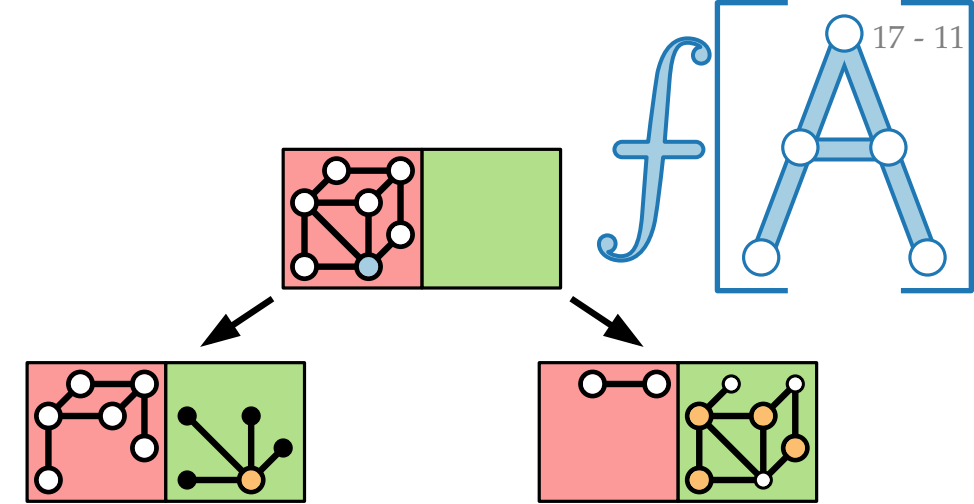
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)
Basis b	2	1.618	1.4656	1.3803	1.325



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

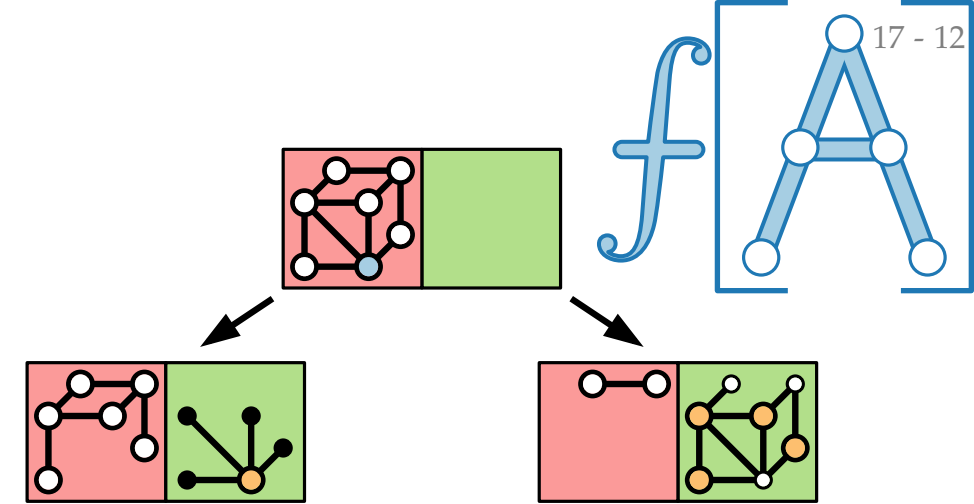
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(2, 3)
Basis b	2	1.618	1.4656	1.3803	1.325	1.325



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

➔ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

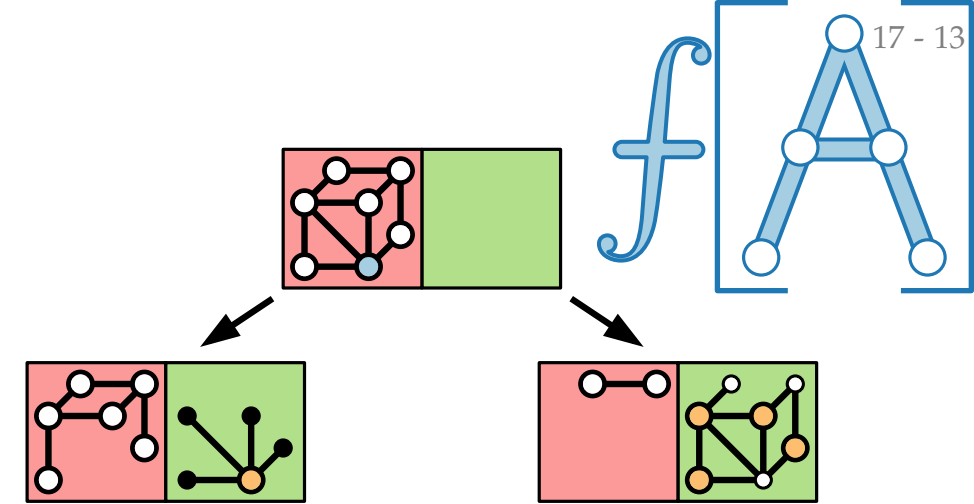
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

➔ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(2, 3)	(3, 3)
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

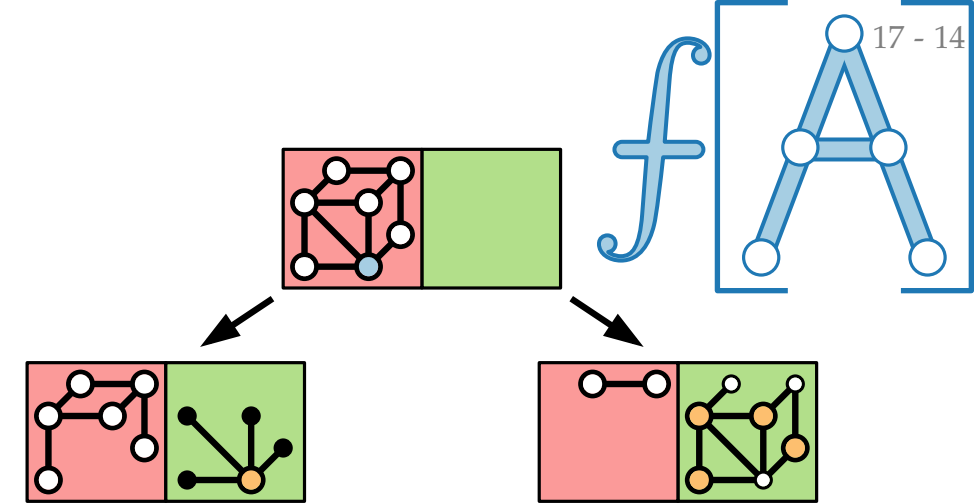
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(2, 3)	(3, 3)	(3, 3, 6)
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26	1.342



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

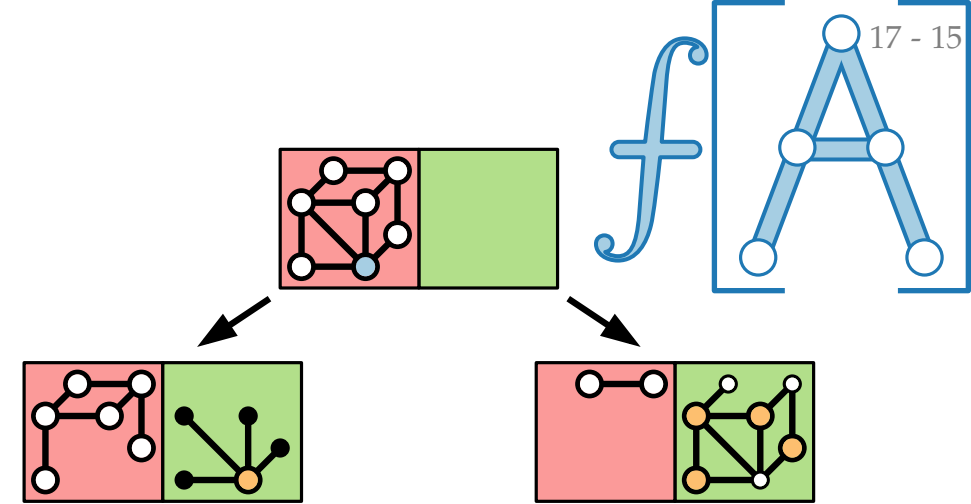
Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(2, 3)	(3, 3)	(3, 3, 6)	(3, 4, 6)
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26	1.342	1.305



Verzweigungsvektor

Annahme: Es gibt immer einen Knoten mit $\deg v \geq 4$

→ $T(k) \leq T(k-1) + T(k-4) + 1 \in \mathcal{O}(1.38^k)$

Problem: Man findet nicht immer einen Knoten mit Mindestgrad 4.

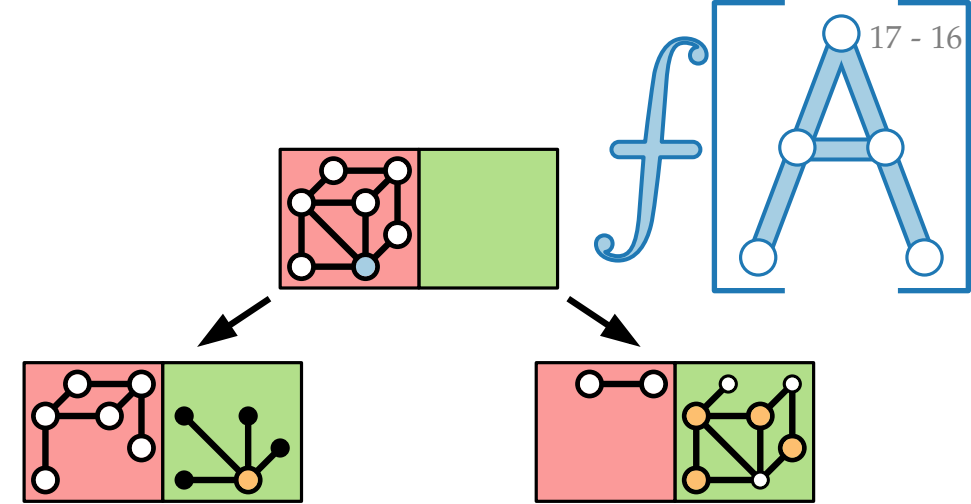
Allgemein: $T(k) \leq T(k-t_1) + \dots + T(k-t_s) + c$

→ Der Vektor $(t_1, \dots, t_s)$ heißt **Verzweigungsvektor**. (hier also $(1, 4)$)

Die Basis b in Laufzeit $\mathcal{O}(b^k)$ hängt vom Verzweigungsvektor ab.

Verzweigungsvektor	(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(2, 3)	(3, 3)	(3, 3, 6)	(3, 4, 6)
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26	1.342	1.305

Idee: Verzweige an jedem Knoten je nach eintretendem Fall mit Verzweigungsvektor $(1, 4)$, $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$ oder $(3, 3, 6)$

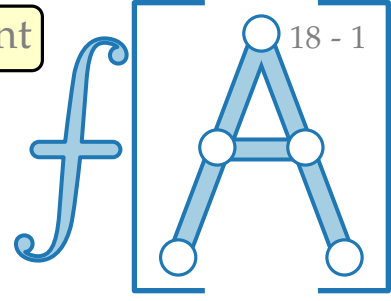


Regel 1

Bisherige Kernbildung:

Regel K: Eliminiere Knoten mit $\text{Grad} > k$

nicht prüfungsrelevant



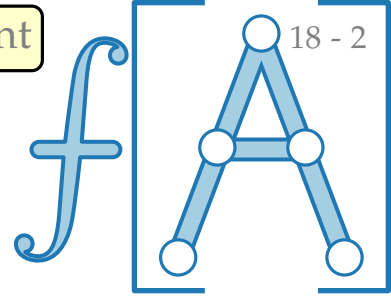
Regel 1

Bisherige Kernbildung:

Regel K: Eliminiere Knoten mit $\text{Grad} > k$

Regel 0: Eliminiere Knoten mit Grad 0

nicht prüfungsrelevant



Regel 1

Bisherige Kernbildung:

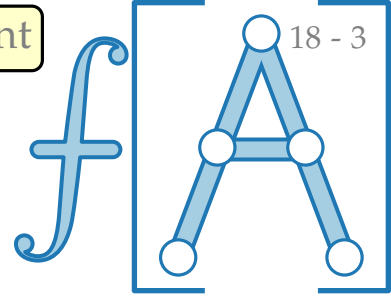
Regel K: Eliminiere Knoten mit $\text{Grad} > k$

Regel 0: Eliminiere Knoten mit Grad 0

Verbesserte Kernbildung:

Regel 1:

nicht prüfungsrelevant



Regel 1

Bisherige Kernbildung:

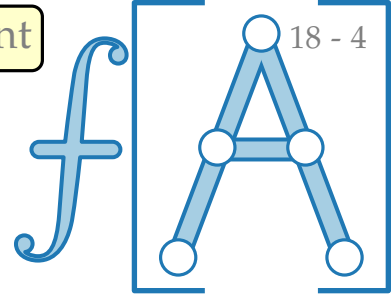
Regel K: Eliminiere Knoten mit $\text{Grad} > k$

Regel 0: Eliminiere Knoten mit Grad 0

Verbesserte Kernbildung:

Regel 1: Eliminiere Knoten v mit Grad 1

nicht prüfungsrelevant



Regel 1

Bisherige Kernbildung:

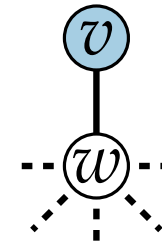
Regel K: Eliminiere Knoten mit $\text{Grad} > k$

Regel 0: Eliminiere Knoten mit $\text{Grad} 0$

Verbesserte Kernbildung:

Regel 1: Eliminiere Knoten v mit $\text{Grad} 1$

Sei w der Nachbar von v .



Regel 1

Bisherige Kernbildung:

Regel K: Eliminiere Knoten mit $\text{Grad} > k$

Regel 0: Eliminiere Knoten mit Grad 0

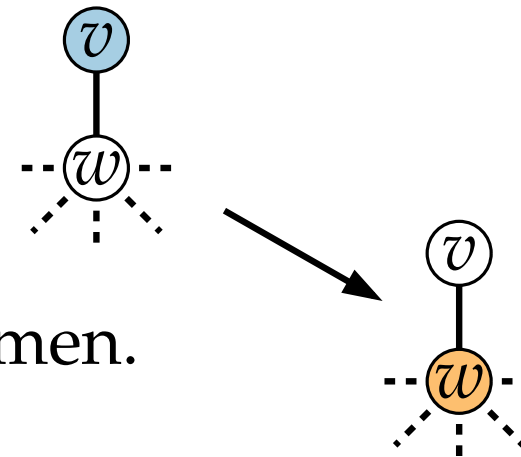
Verbesserte Kernbildung:

Regel 1: Eliminiere Knoten v mit Grad 1

Sei w der Nachbar von v .

Es kann nie besser sein, v statt w zu nehmen.

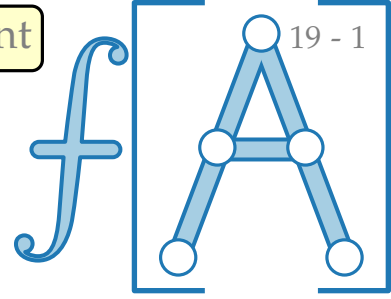
➡ keine Verzweigung nötig.



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

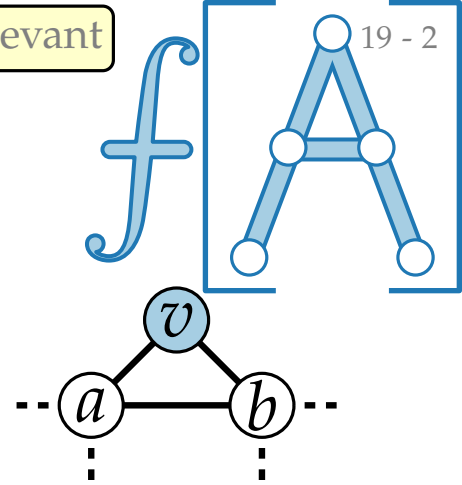
nicht prüfungsrelevant



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.



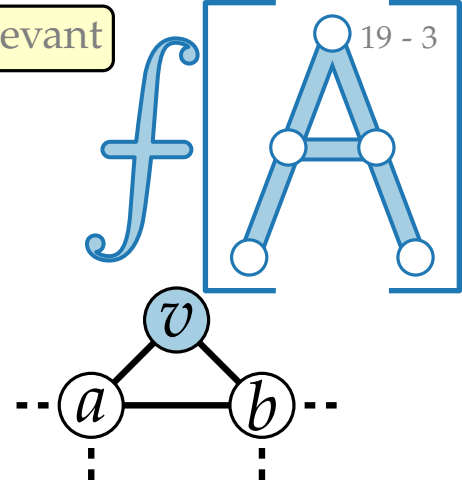
Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

➔ Zwei der Knoten v, a, b müssen gewählt werden.

nicht prüfungsrelevant

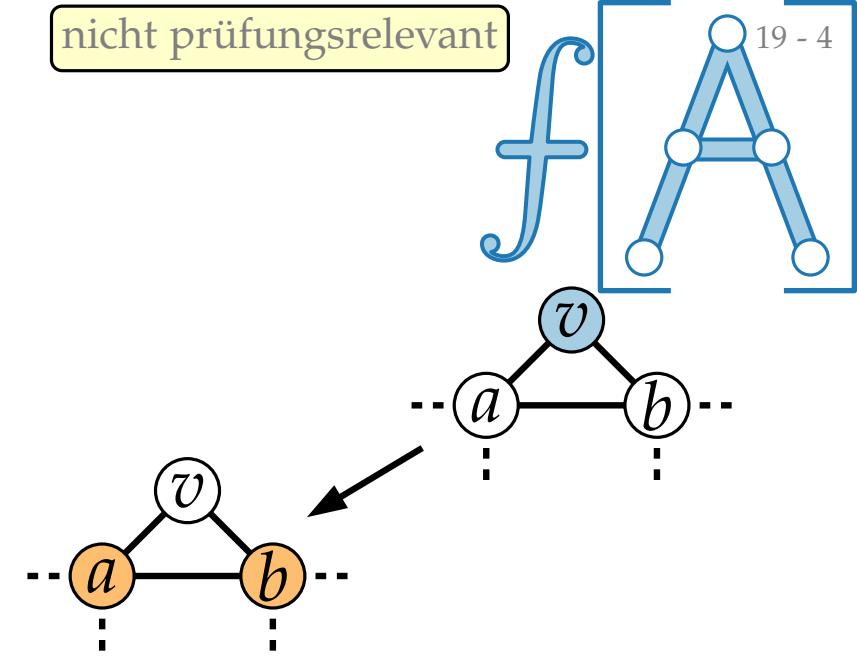


Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➡ Zwei der Knoten v, a, b müssen gewählt werden.
- ➡ Es ist nie besser v zu wählen.

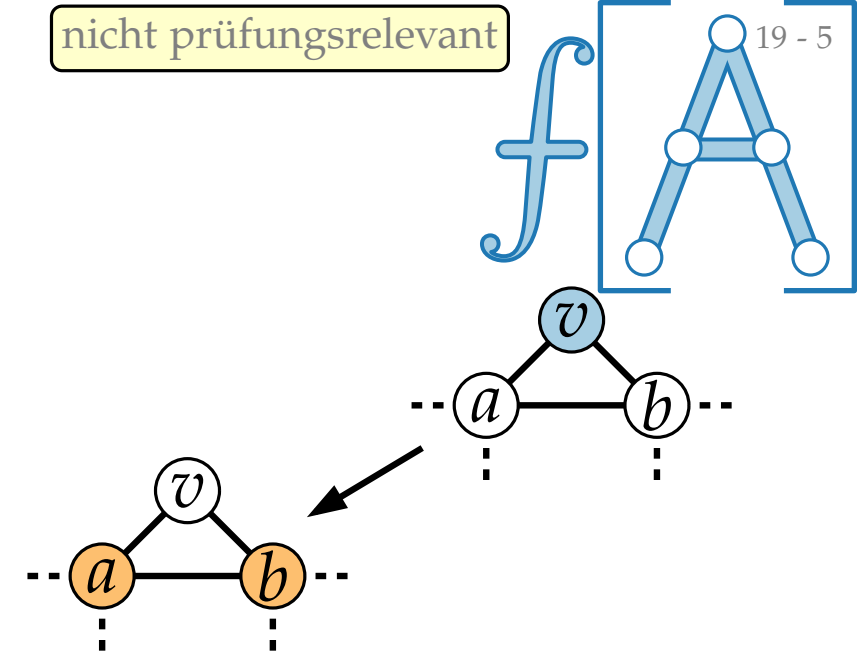


Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➡ Zwei der Knoten v, a, b müssen gewählt werden.
- ➡ Es ist nie besser v zu wählen.
- ➡ Keine Verzweigung nötig.



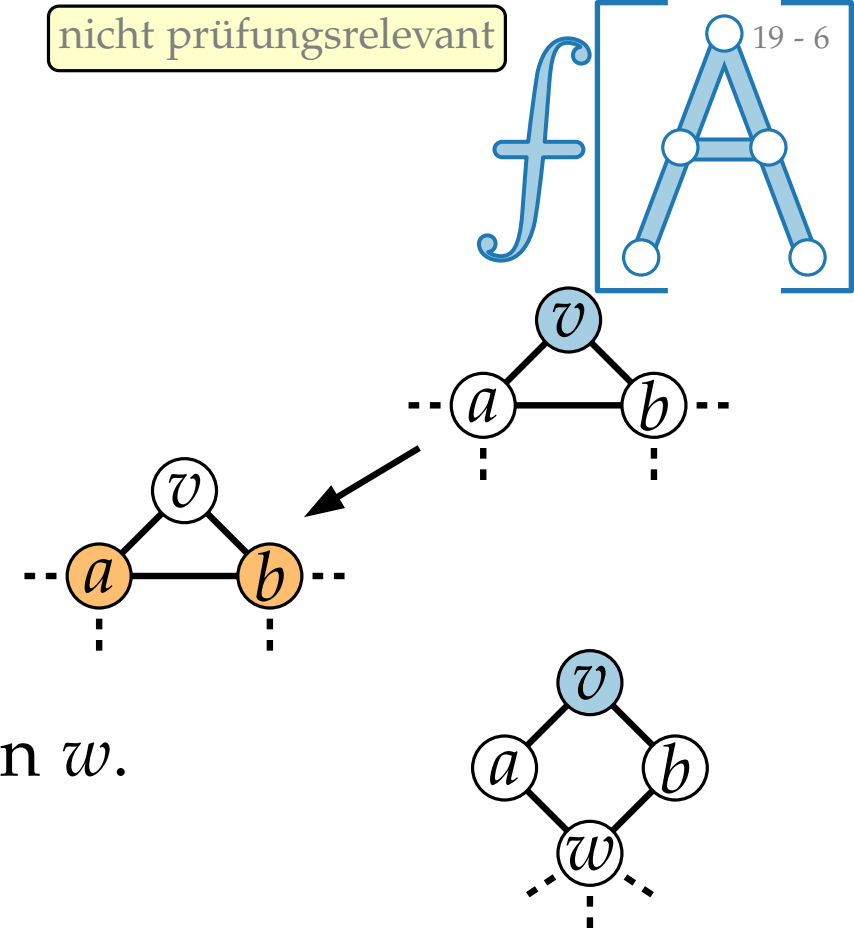
Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➡ Zwei der Knoten v, a, b müssen gewählt werden.
- ➡ Es ist nie besser v zu wählen.
- ➡ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .



Regel 2

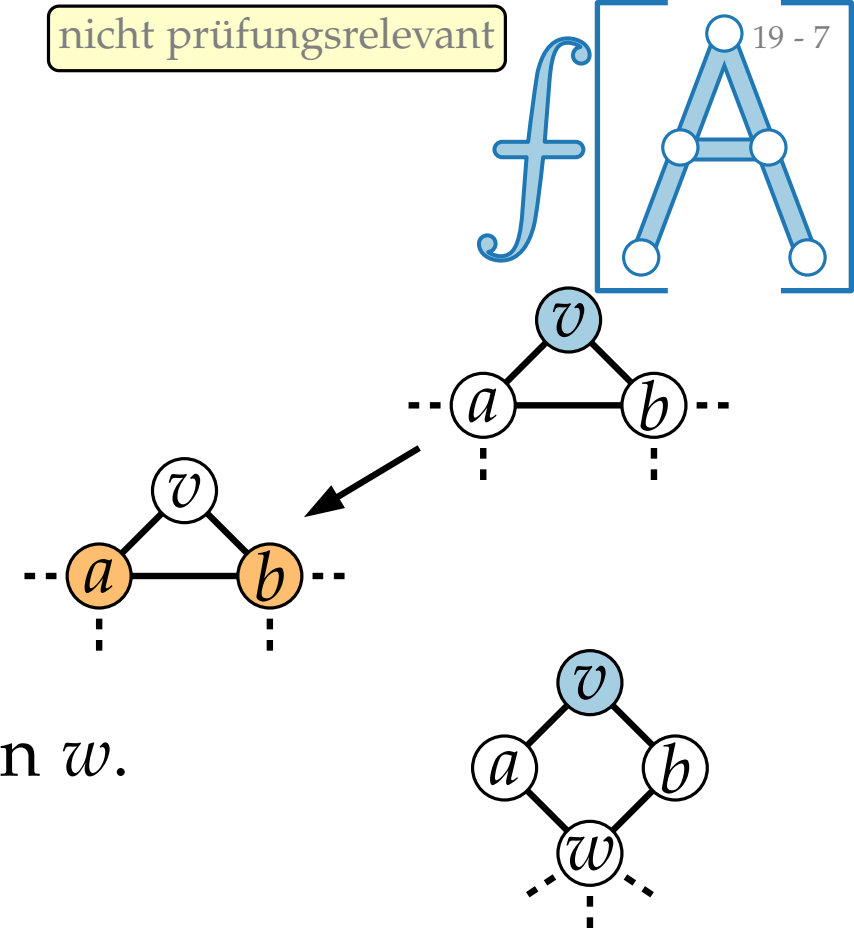
Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.



Regel 2

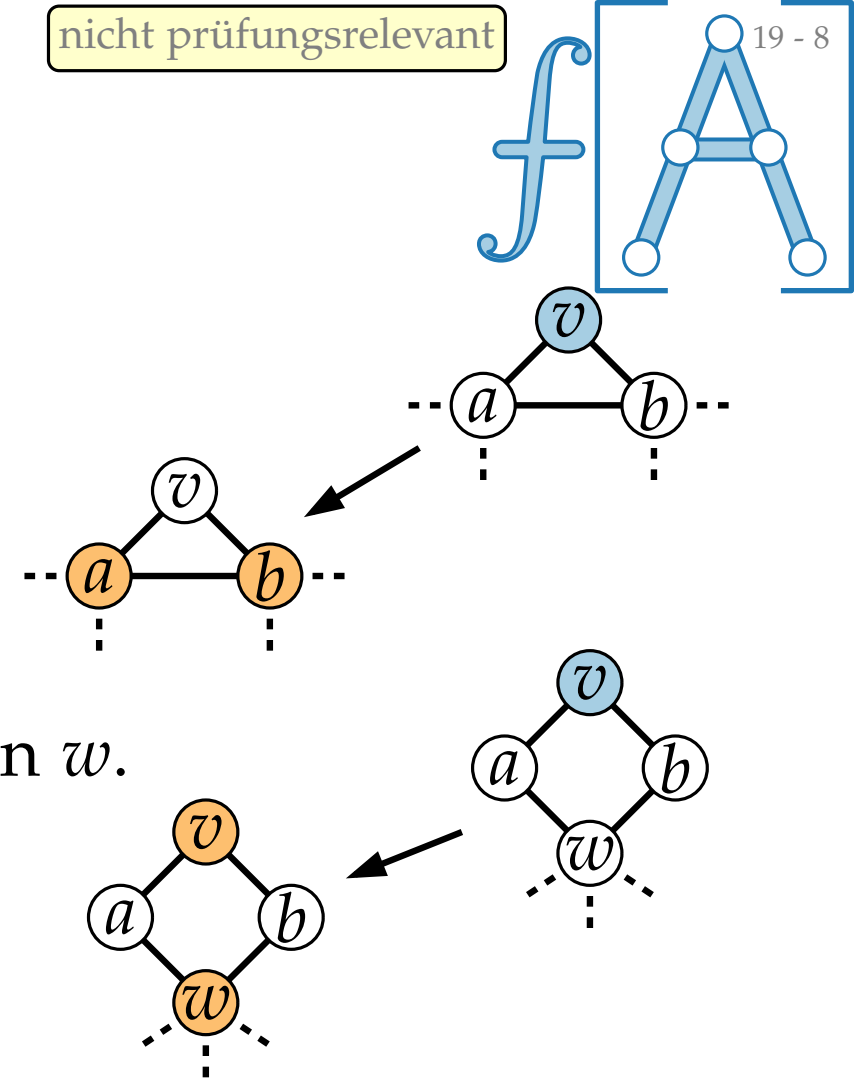
Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.



Regel 2

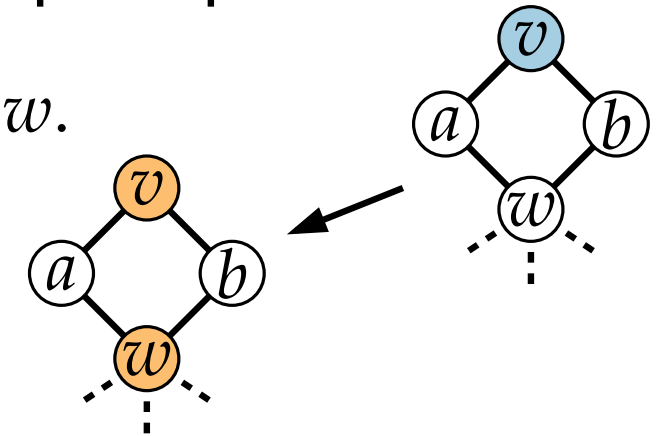
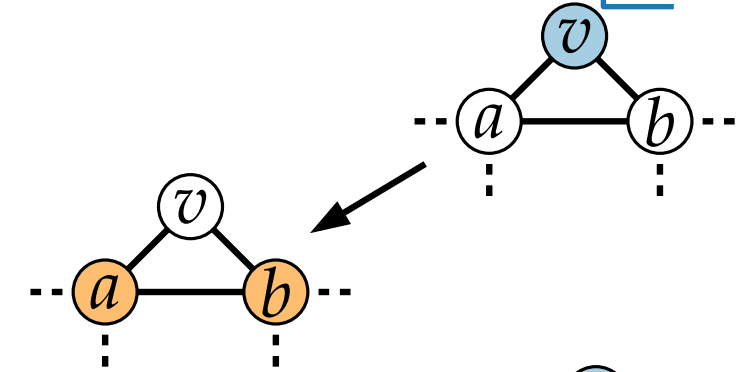
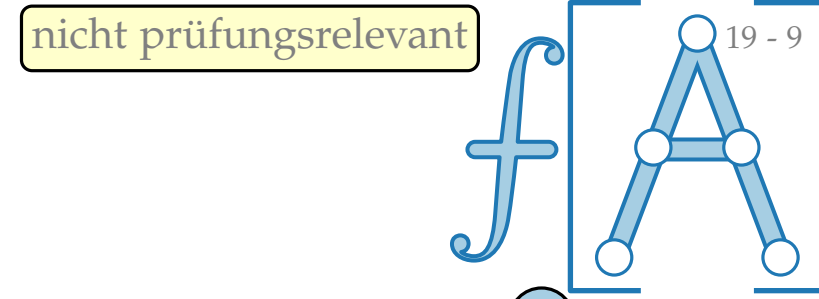
Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

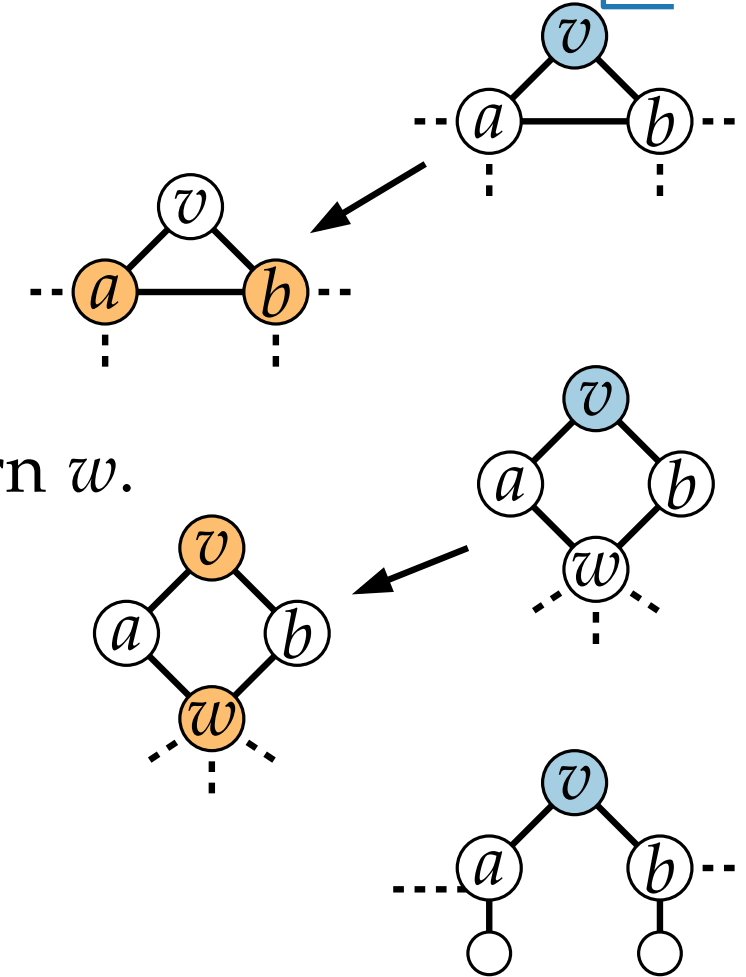
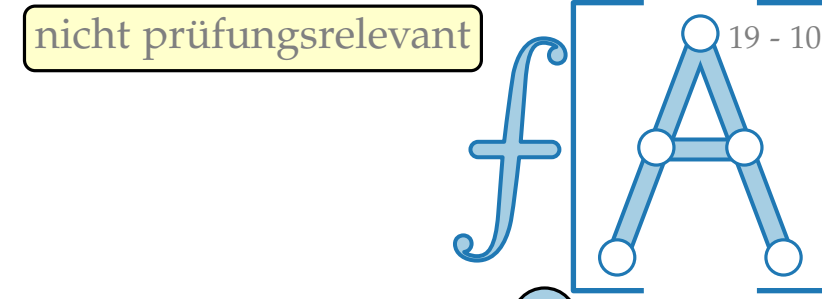
Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.3: sonst



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

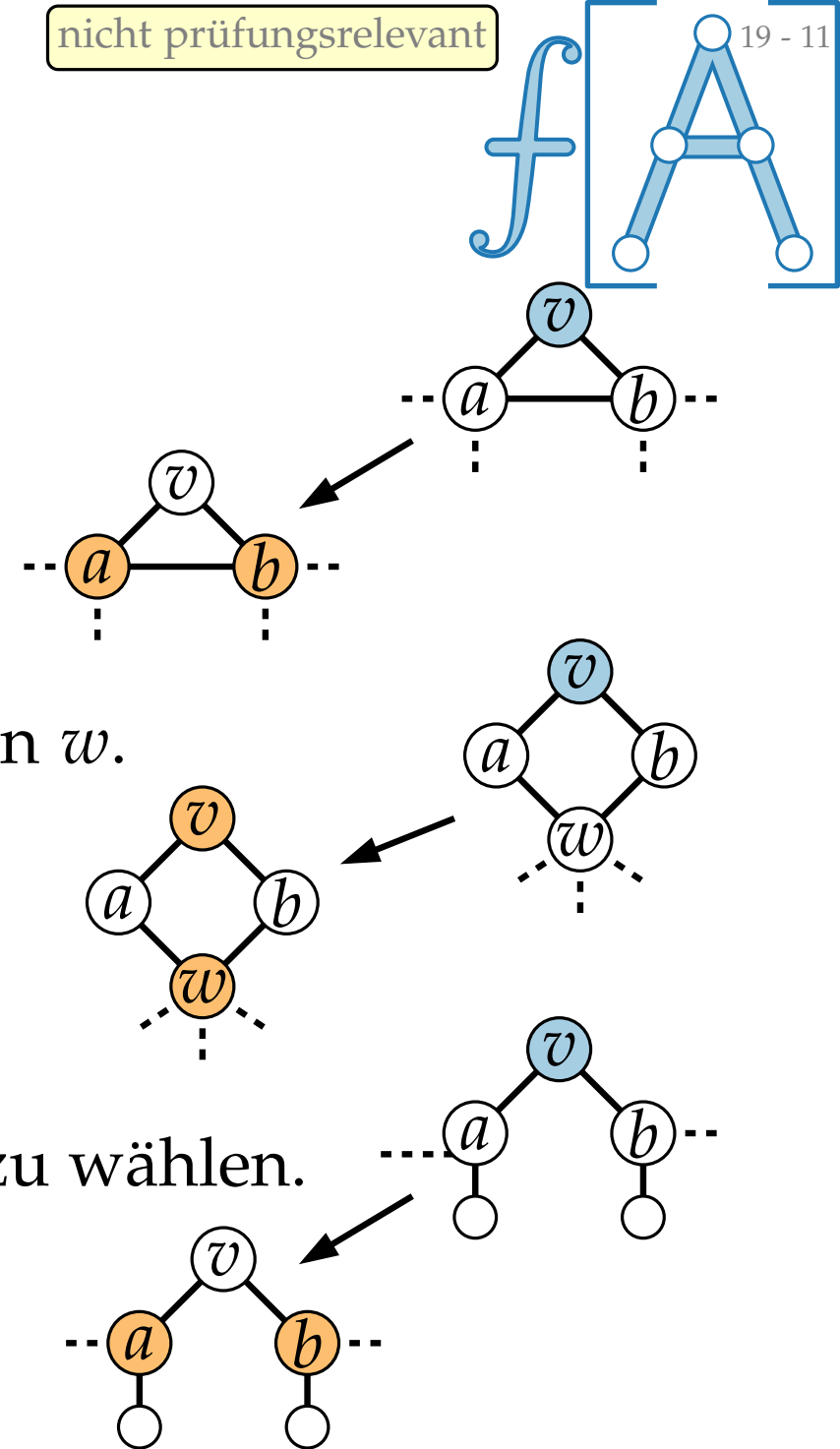
- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.3: sonst

Falls a oder b gewählt wurden, macht es keinen Sinn mehr, v zu wählen.



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

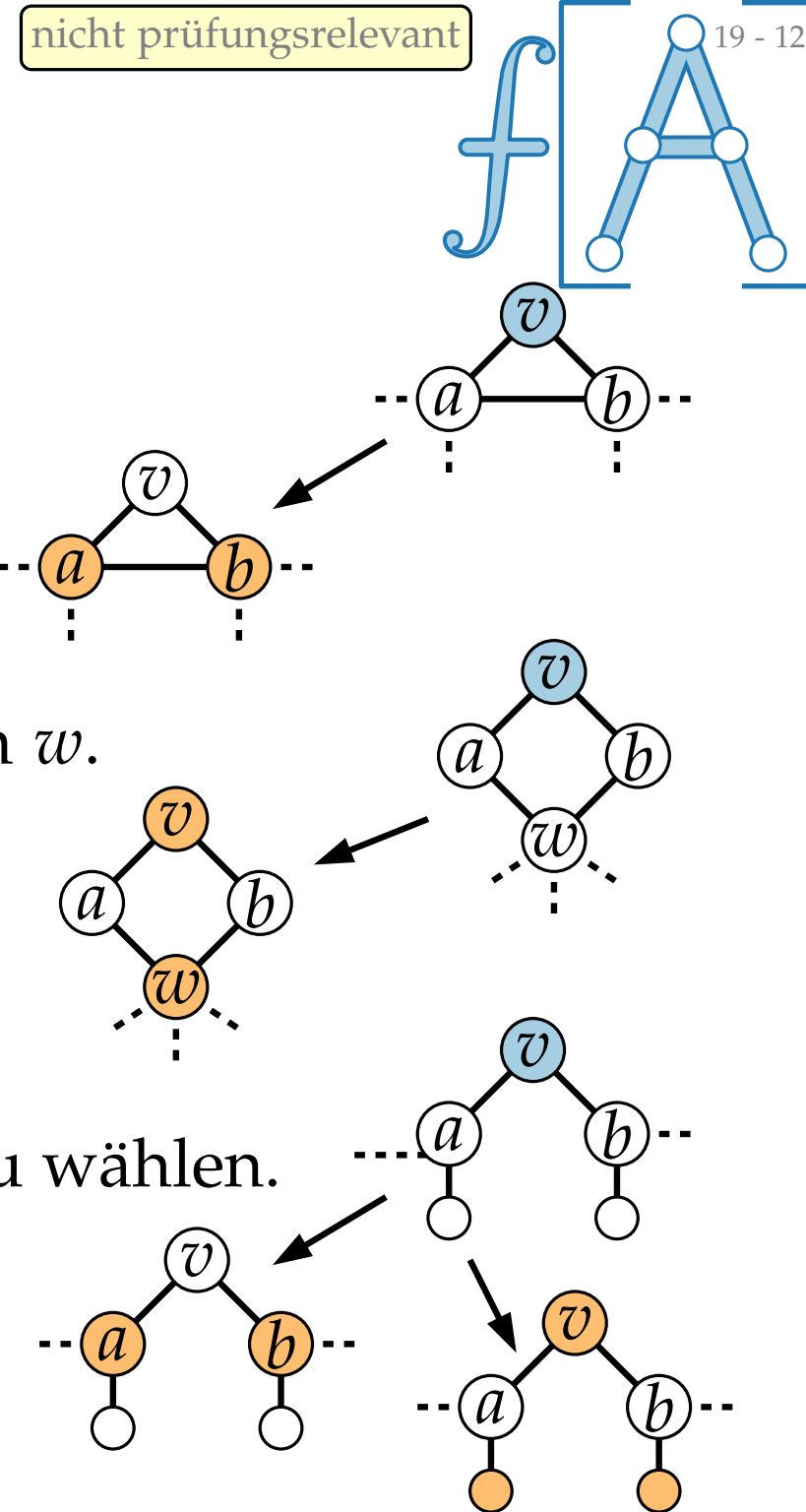
Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.3: sonst

Falls a oder b gewählt wurden, macht es keinen Sinn mehr, v zu wählen.

- ➔ Wähle entweder a und b oder $N(a) \cup N(b)$.



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

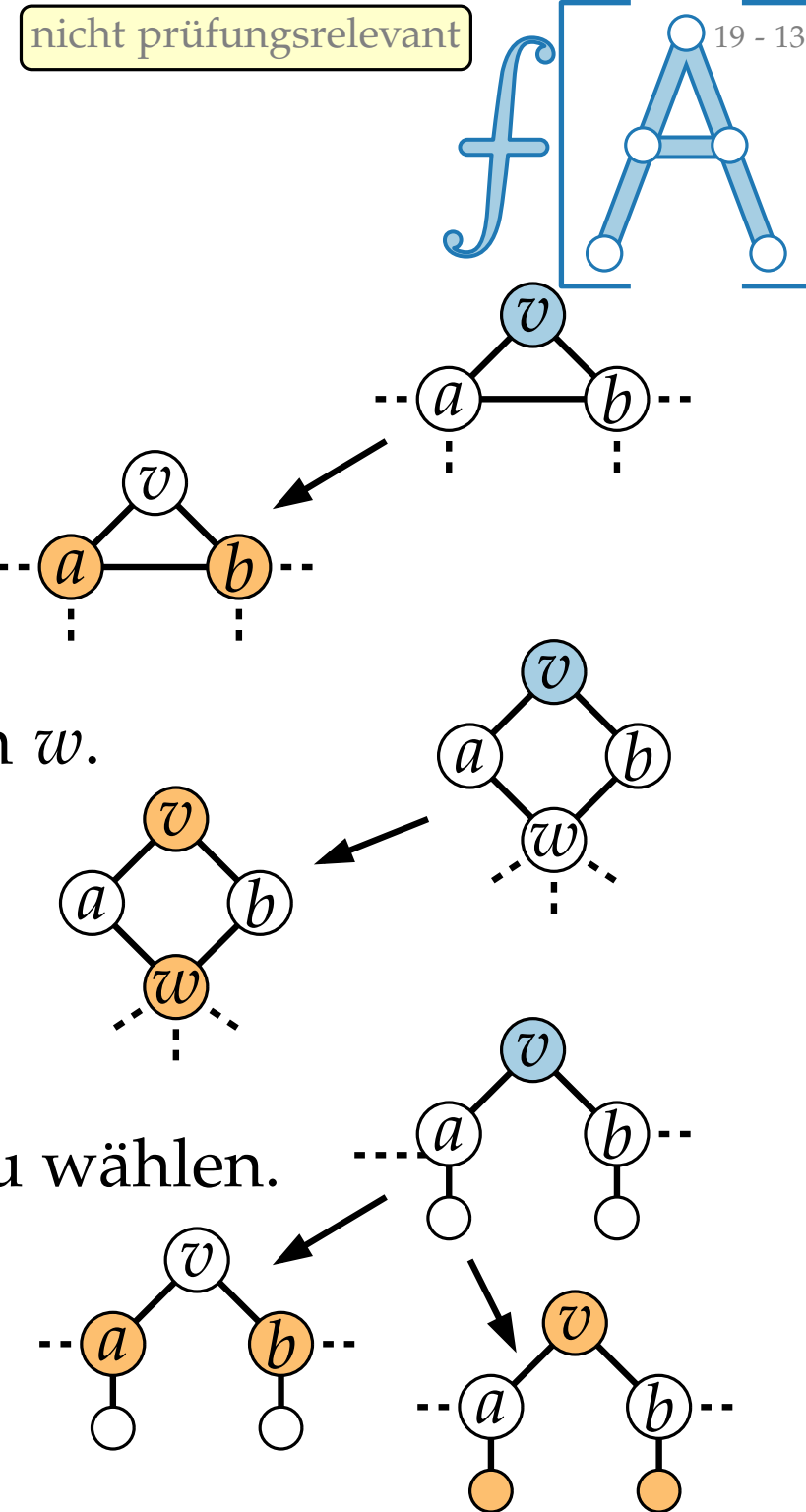
- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.3: sonst

Falls a oder b gewählt wurden, macht es keinen Sinn mehr, v zu wählen.

- ➔ Wähle entweder a und b oder $N(a) \cup N(b)$.

Es gilt $|N(a) \cup N(b)| \geq 3$



Regel 2

Regel 2: Eliminiere Knoten v mit Grad 2

Fall 2.1: Die Nachbarn a und b von v sind adjazent.

- ➔ Zwei der Knoten v, a, b müssen gewählt werden.
- ➔ Es ist nie besser v zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.2: a und b haben Grad zwei und gemeinsamen Nachbarn w .

- ➔ Zwei der Knoten v, a, b, w müssen gewählt werden.
- ➔ Es ist nie besser a und b zu wählen.
- ➔ Keine Verzweigung nötig.

Fall 2.3: sonst

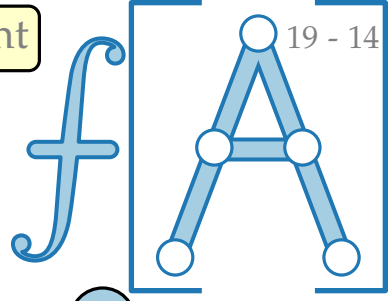
Falls a oder b gewählt wurden, macht es keinen Sinn mehr, v zu wählen.

- ➔ Wähle entweder a und b oder $N(a) \cup N(b)$.

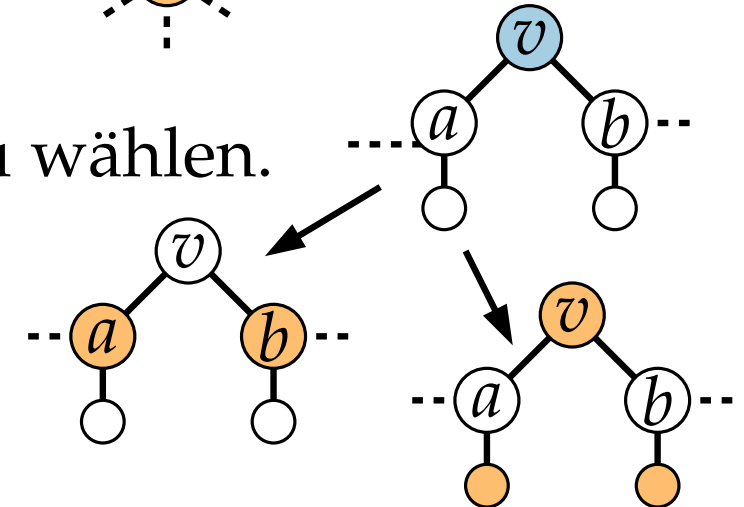
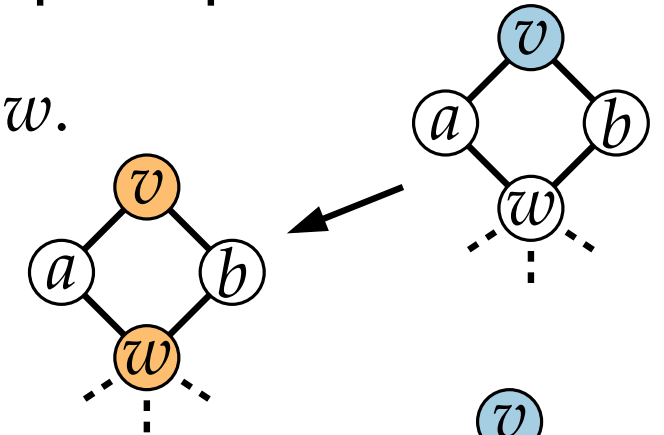
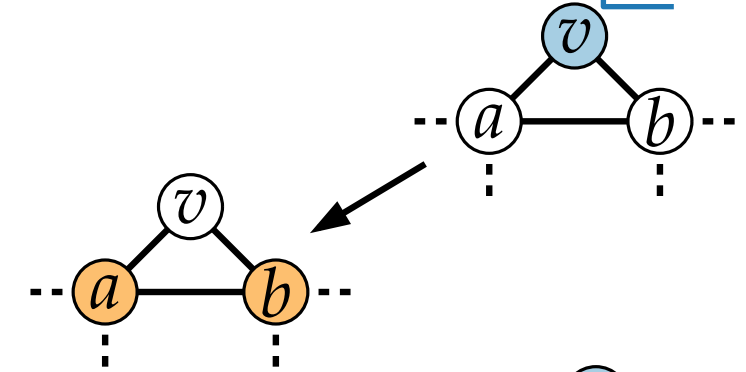
Es gilt $|N(a) \cup N(b)| \geq 3$

- ➔ Verzweigungsvektor $(2, 3)$

nicht prüfungsrelevant



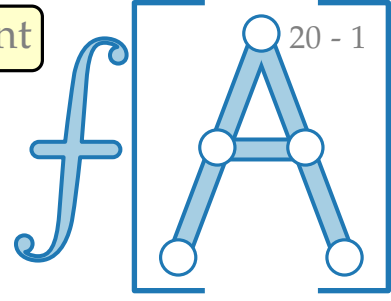
19 - 14



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

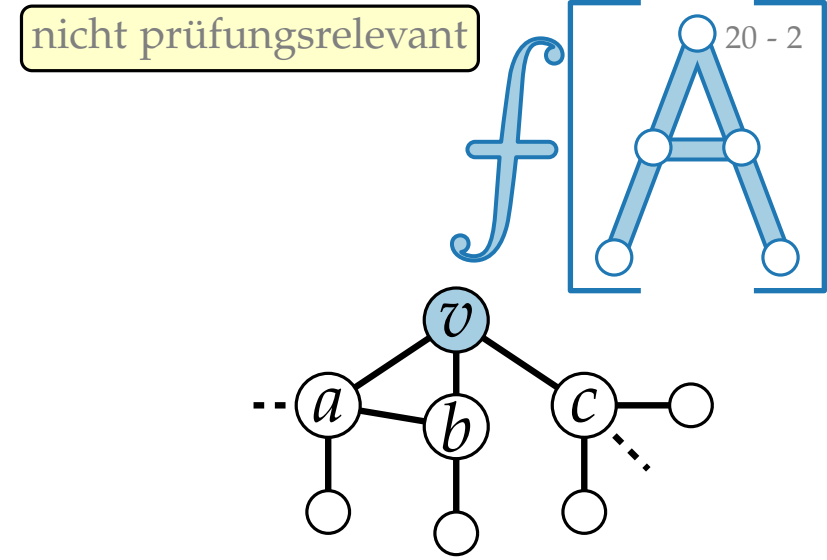
nicht prüfungsrelevant



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

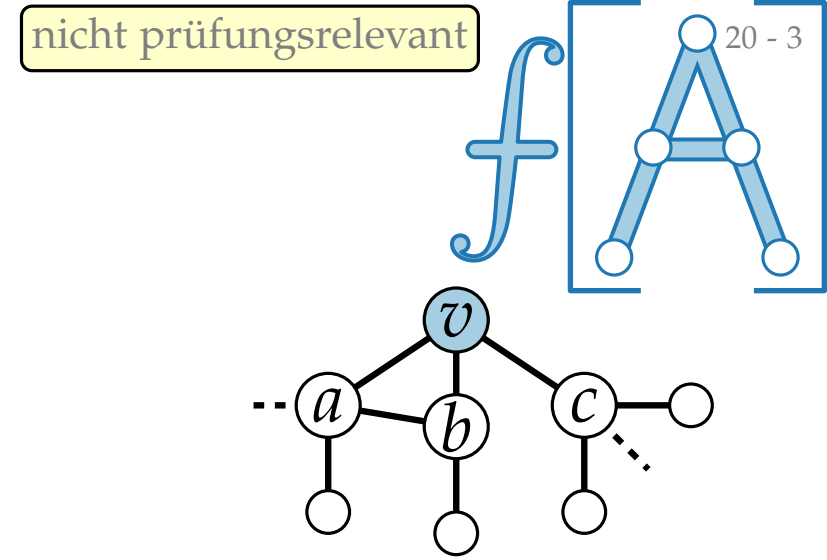


Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.



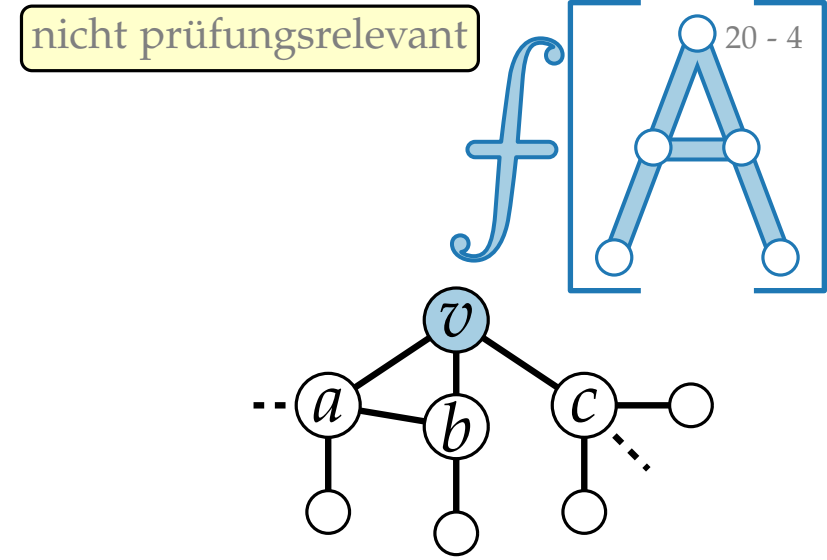
Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

➡ Füge entweder a, b, c oder $N(c)$ hinzu.



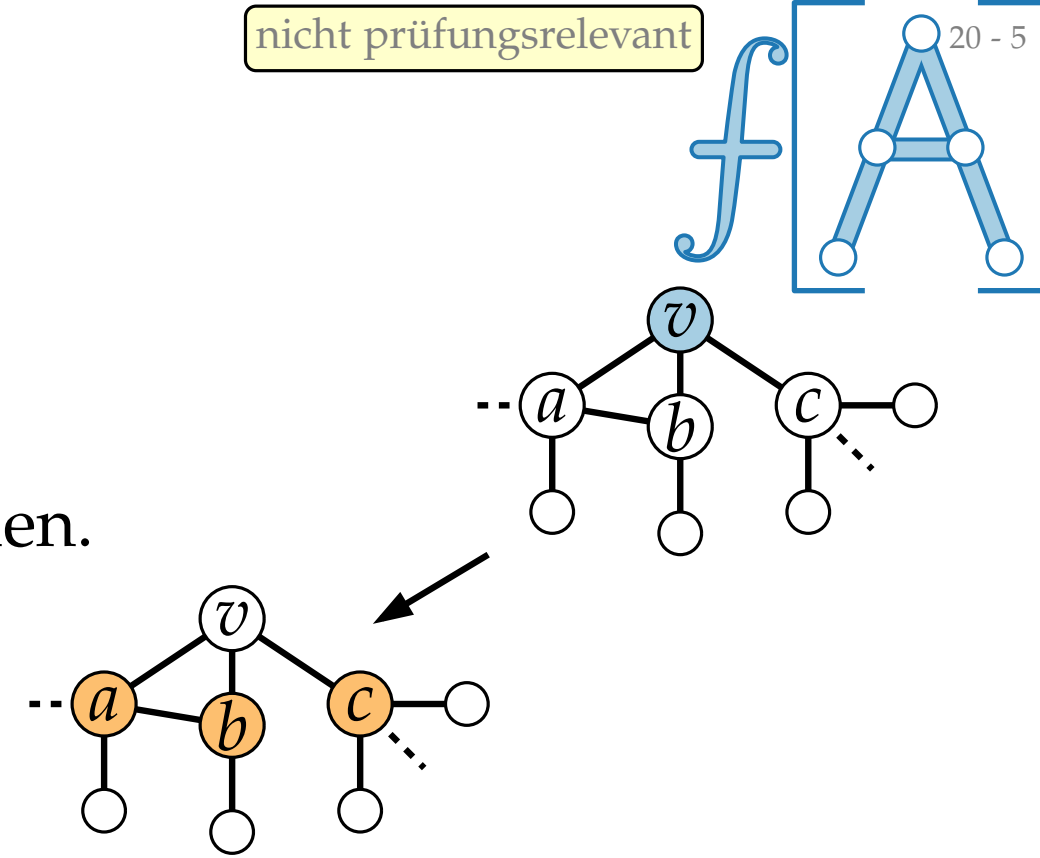
Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

➡ Füge entweder a, b, c oder $N(c)$ hinzu.



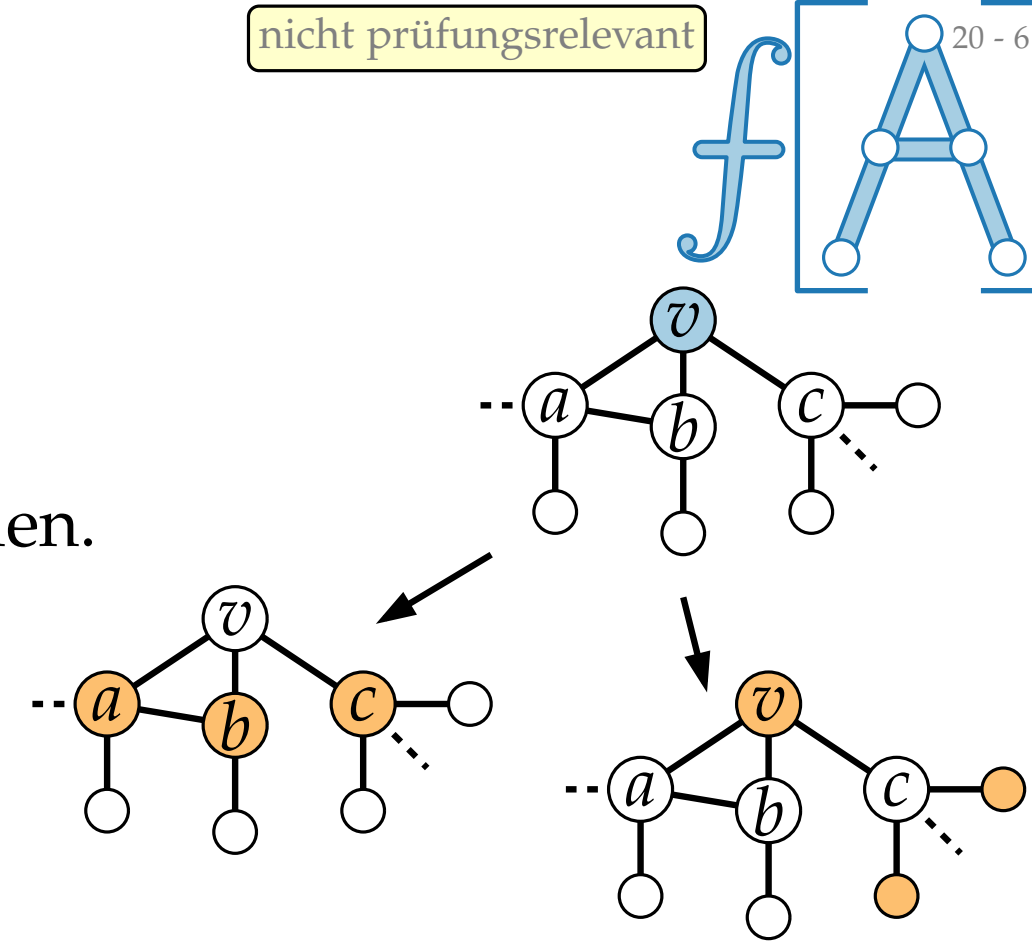
Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

➡ Füge entweder a, b, c oder $N(c)$ hinzu.



Regel 3

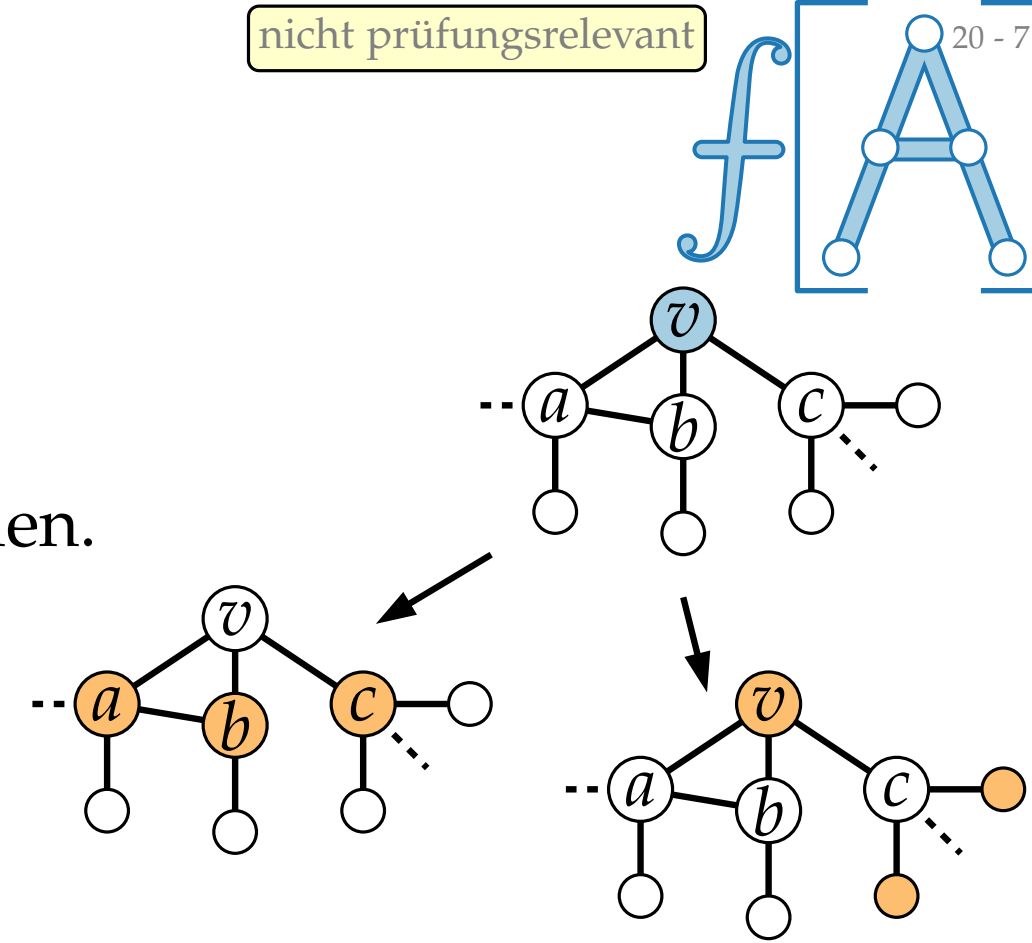
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

➡ Füge entweder a, b, c oder $N(c)$ hinzu.

➡ Verzweigungsvektor $(3, 3)$



Regel 3

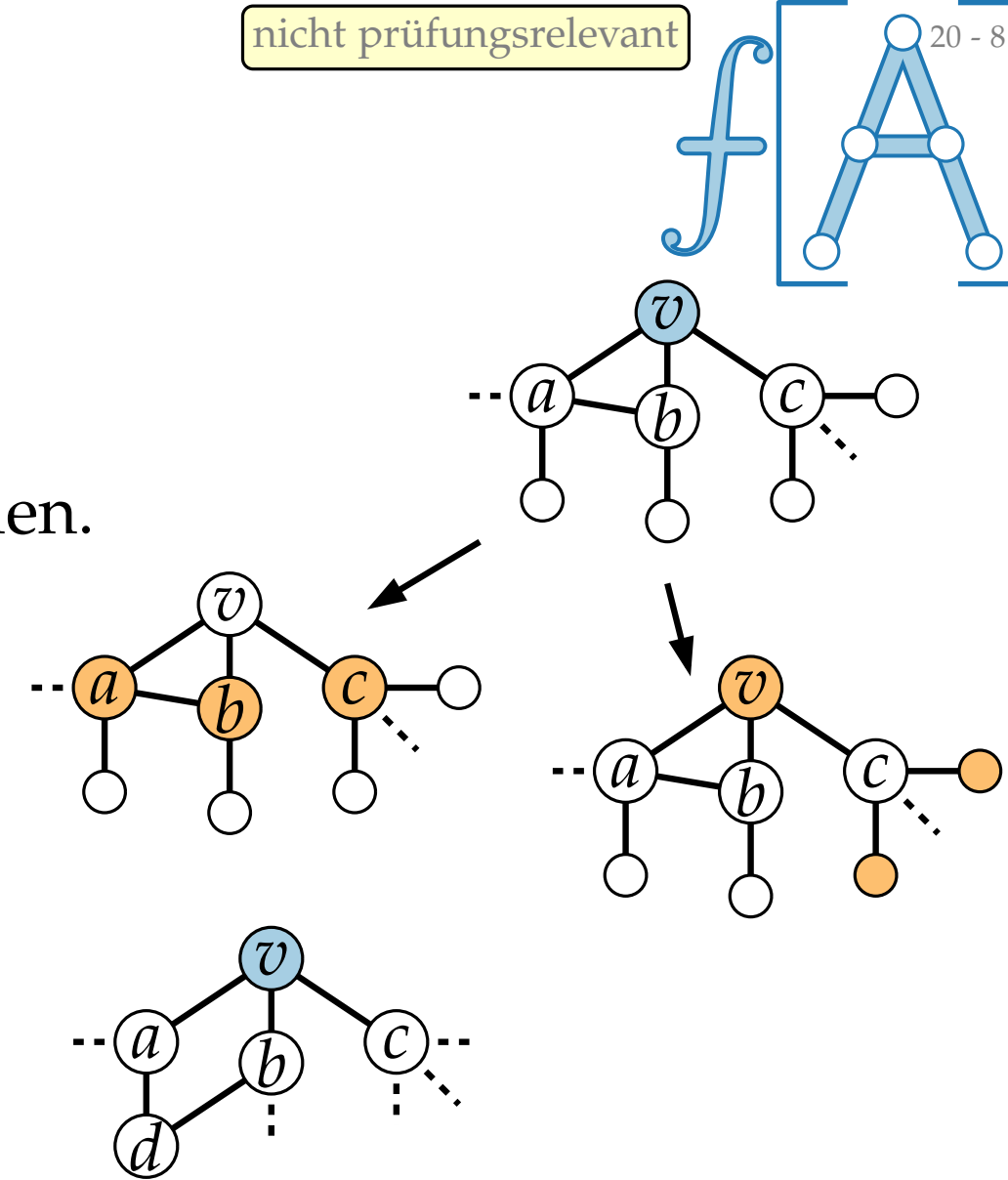
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➡ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➡ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

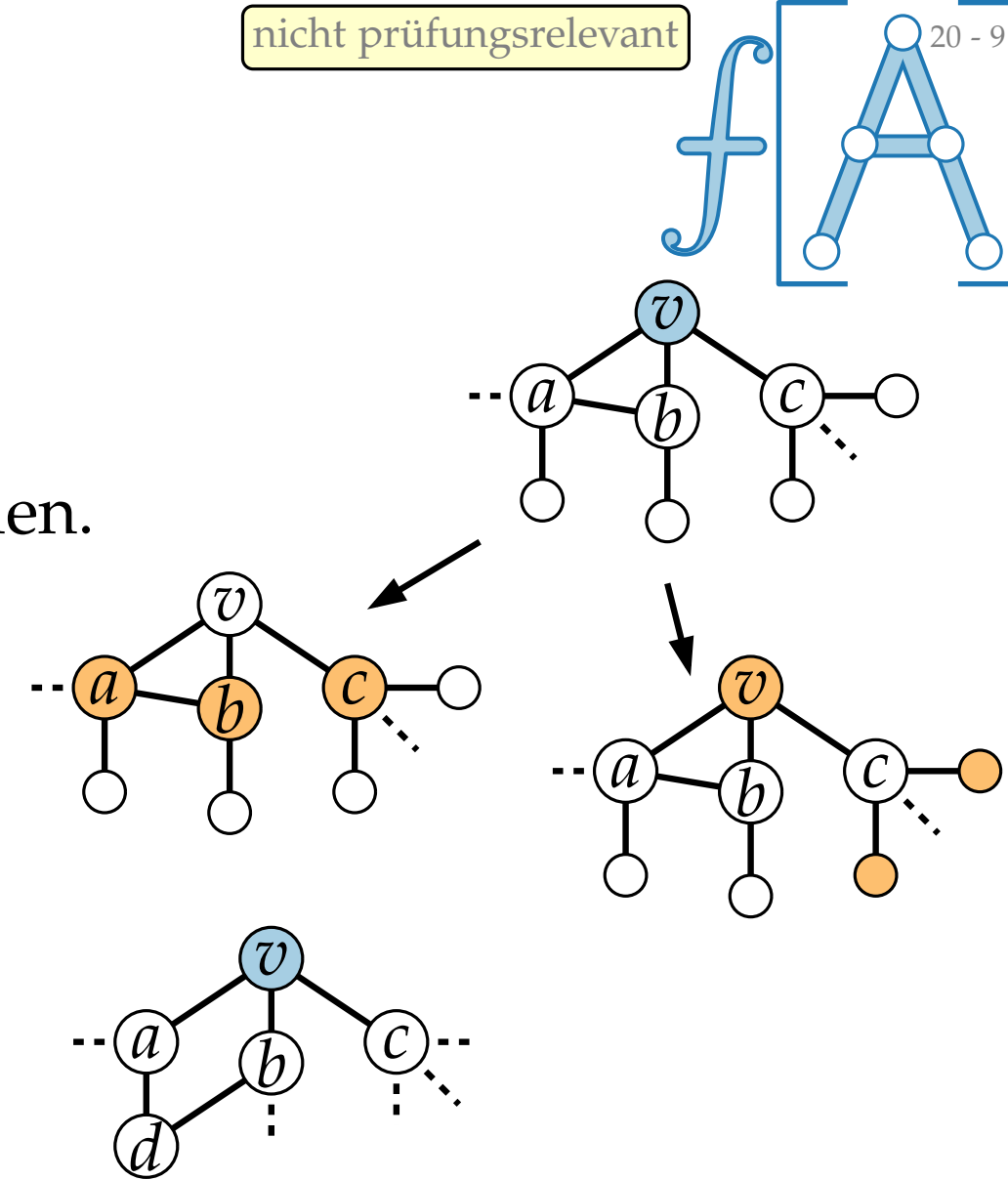
Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➔ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten.



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

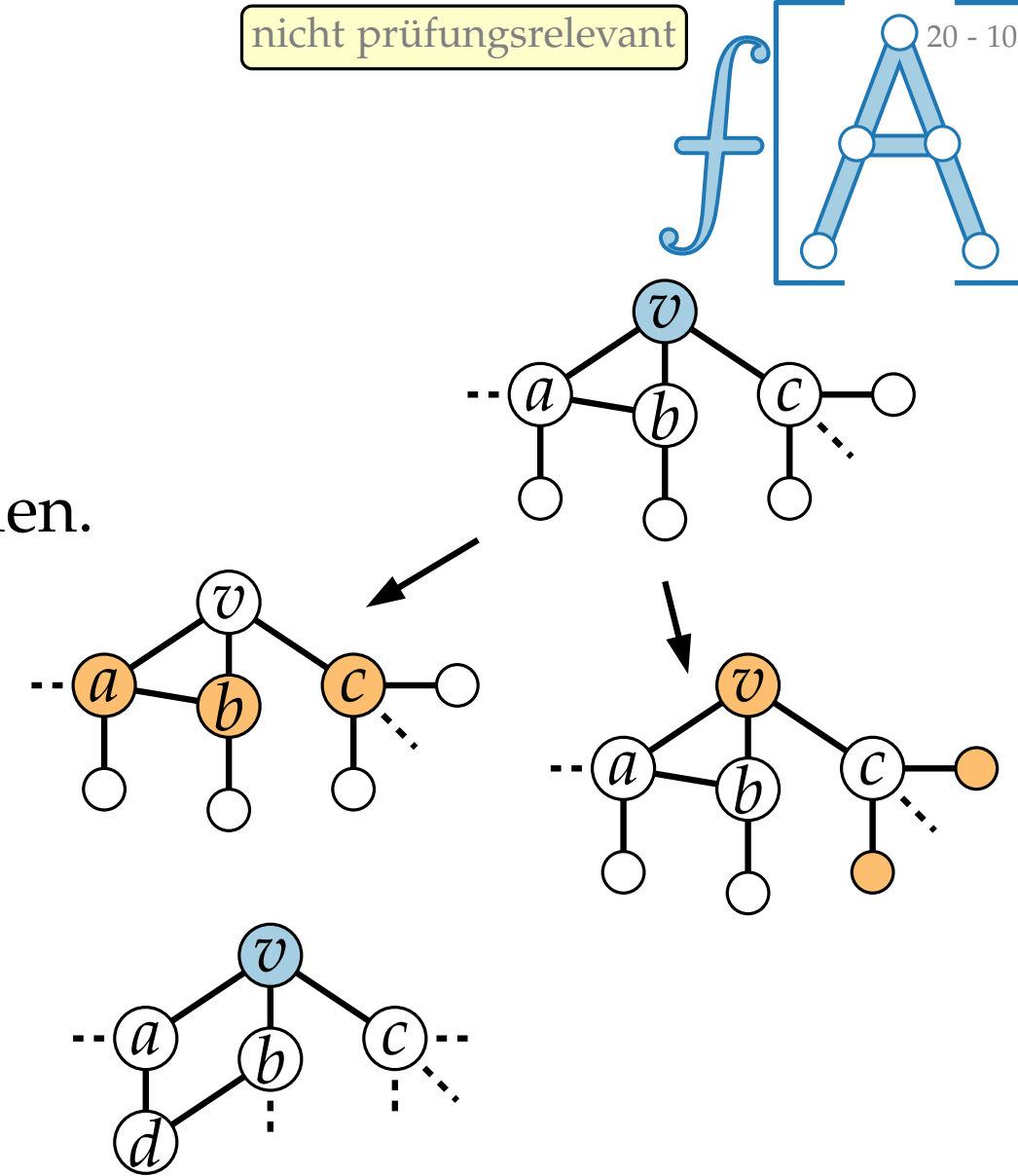
Fall 3.1: v ist Teil eines Dreiecks v, a, b .

Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➡ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➡ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten. Wenn a und b enthalten sind macht es keinen Sinn v zu wählen.



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

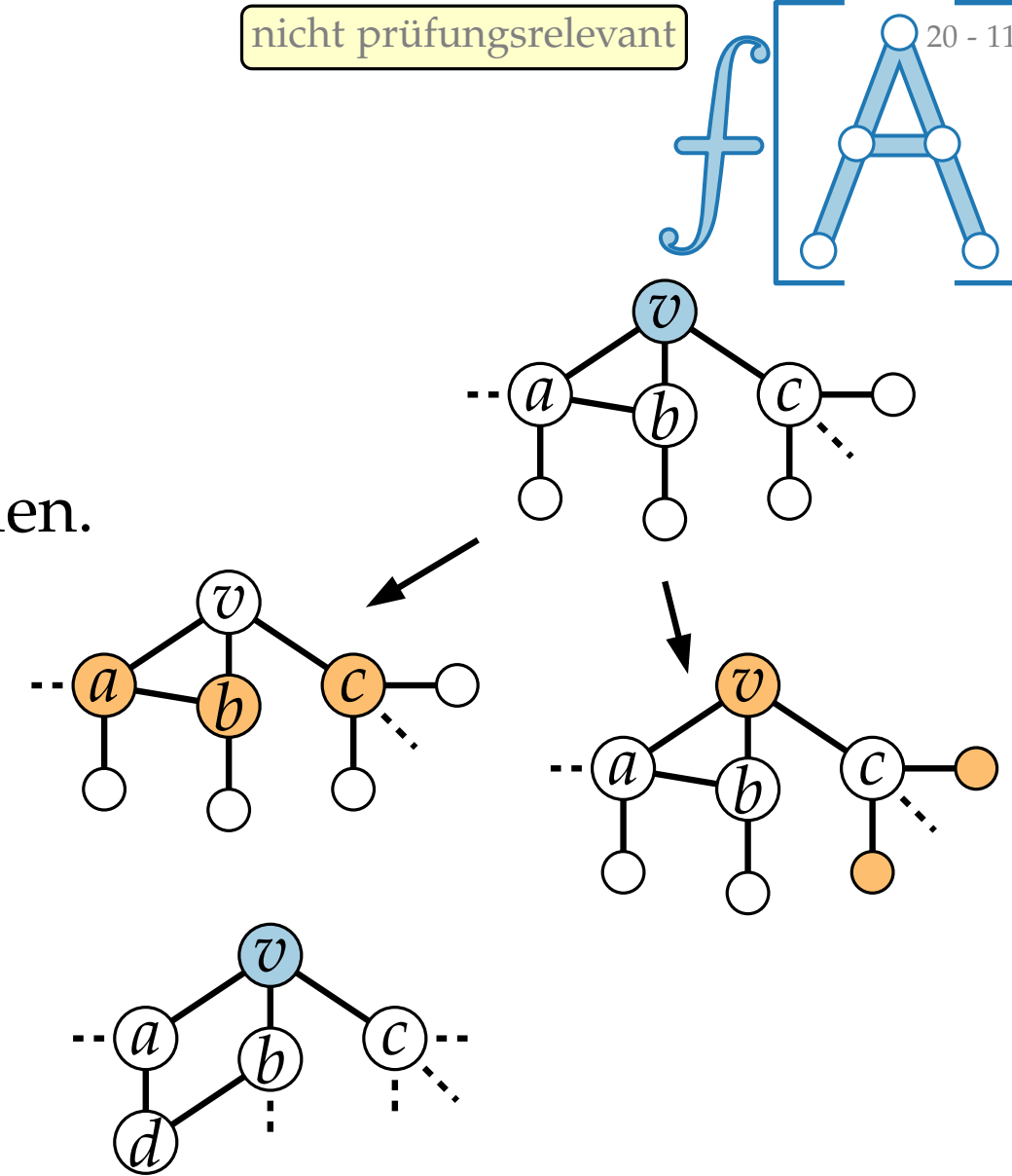
Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➔ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten. Wenn a und b enthalten sind macht es keinen Sinn v zu wählen.

- ➔ Füge entweder a, b, c , oder v, d hinzu.



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

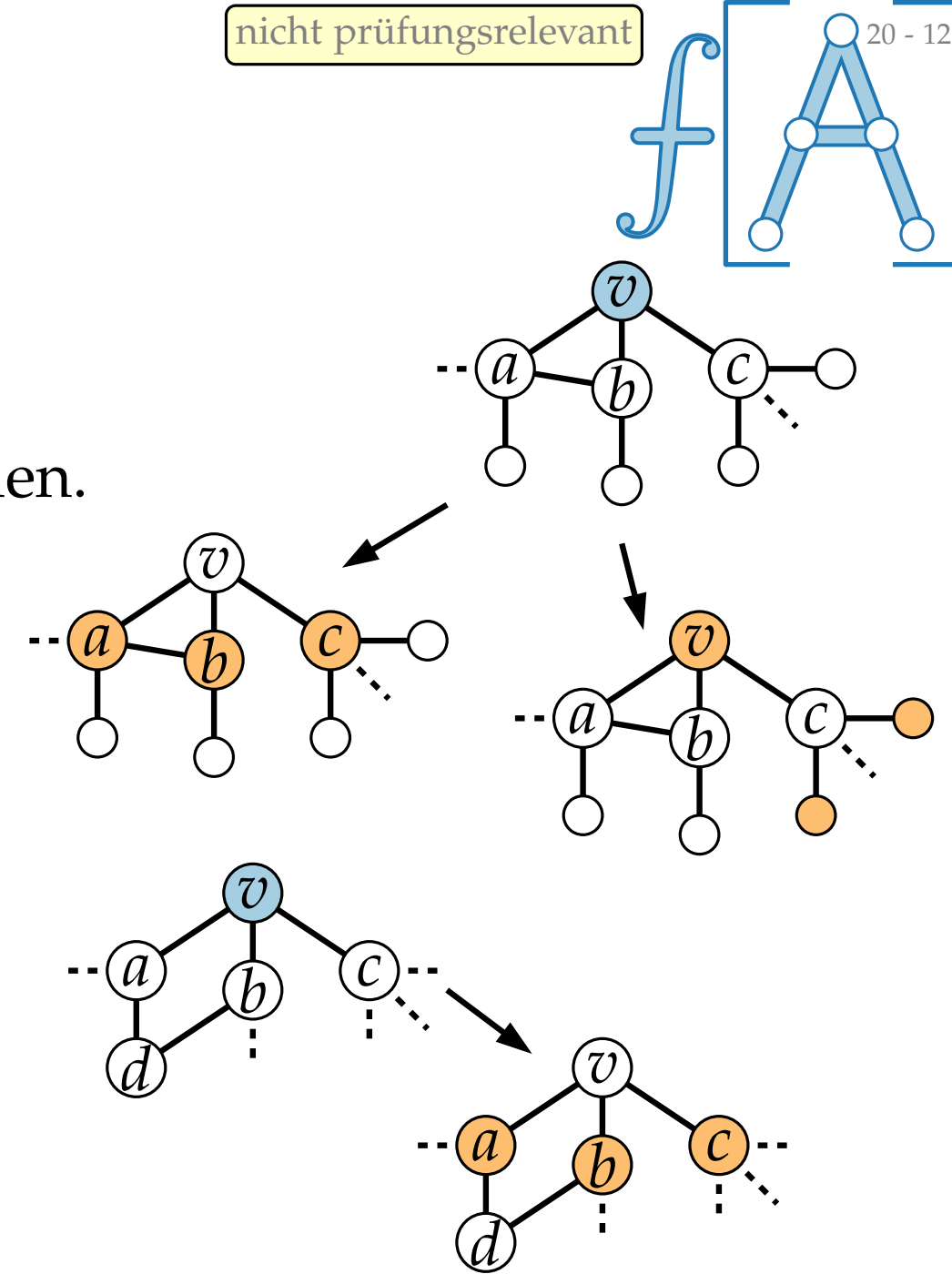
Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➔ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten. Wenn a und b enthalten sind macht es keinen Sinn v zu wählen.

- ➔ Füge entweder a, b, c , oder v, d hinzu.



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

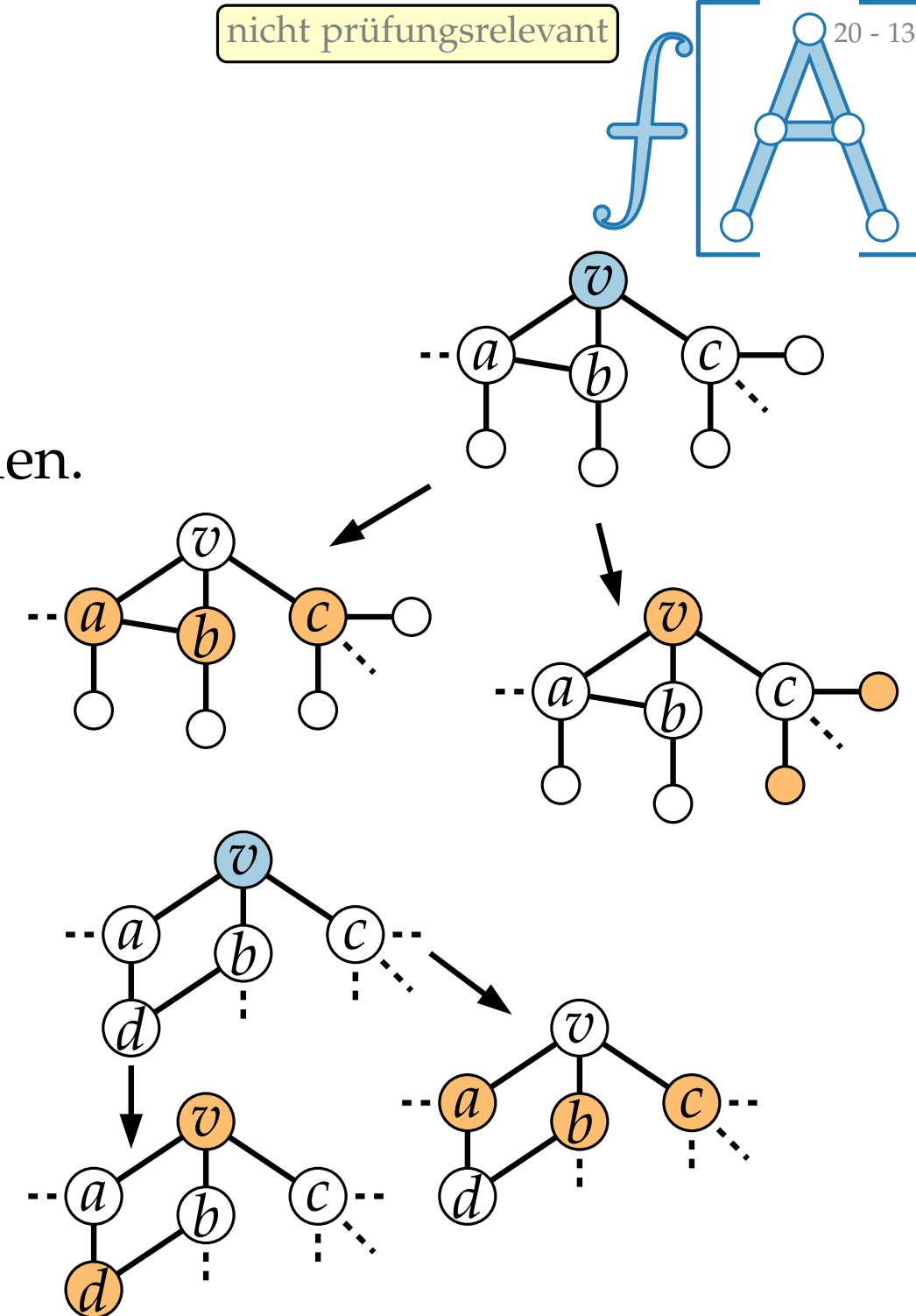
Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➔ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten. Wenn a und b enthalten sind macht es keinen Sinn v zu wählen.

- ➔ Füge entweder a, b, c , oder v, d hinzu.



Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.1: v ist Teil eines Dreiecks v, a, b .

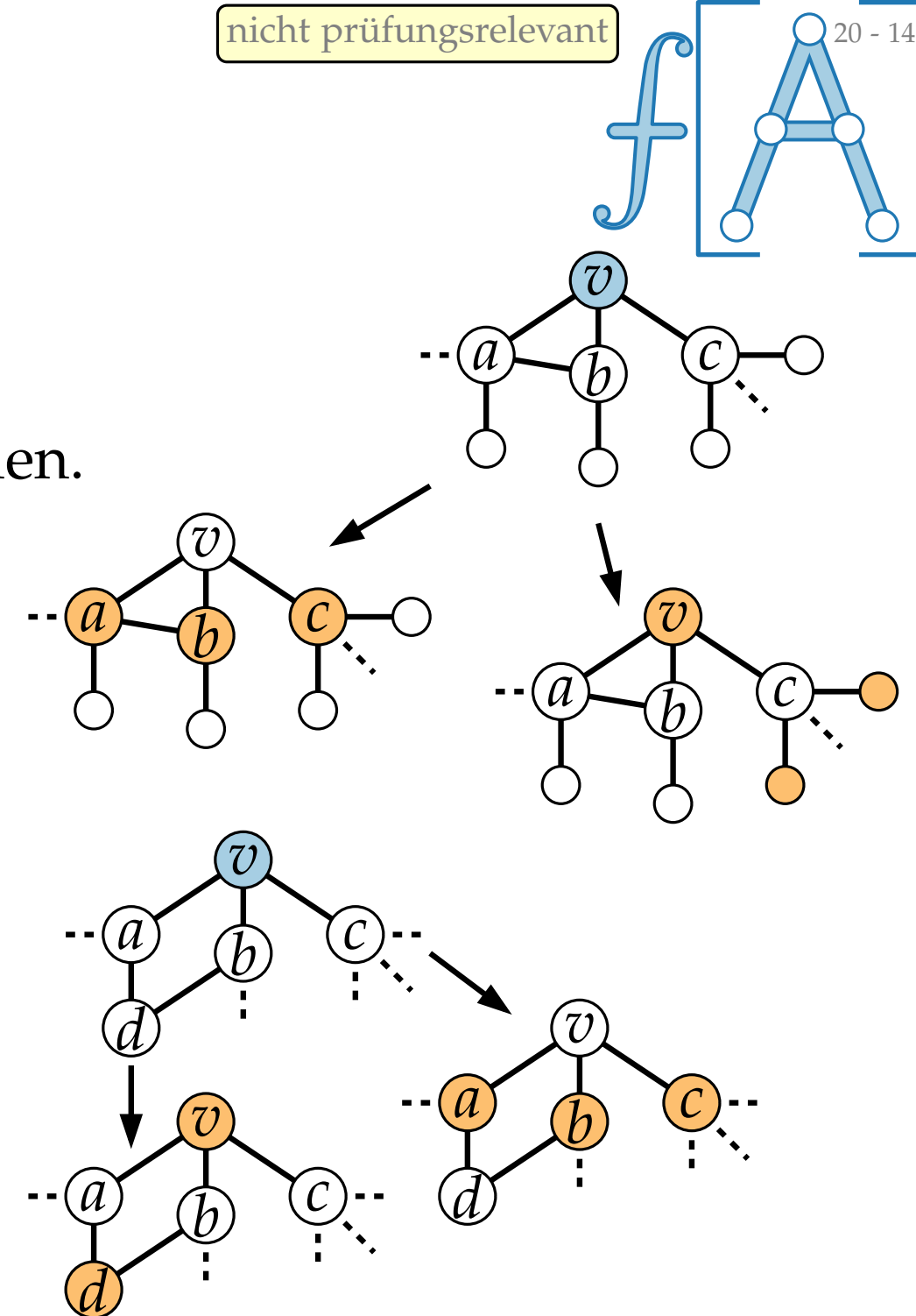
Falls c gewählt wurde, macht es keinen Sinn, v zu wählen.

- ➔ Füge entweder a, b, c oder $N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3)$

Fall 3.2: v ist Teil eines Vierecks v, a, d, b .

In jedem Vertex Cover sind zumindest v und d oder a und b enthalten. Wenn a und b enthalten sind macht es keinen Sinn v zu wählen.

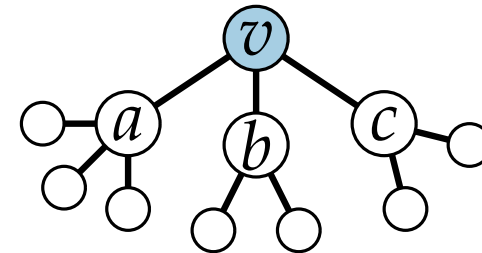
- ➔ Füge entweder a, b, c , oder v, d hinzu.
- ➔ Verzweigungsvektor $(2, 3)$



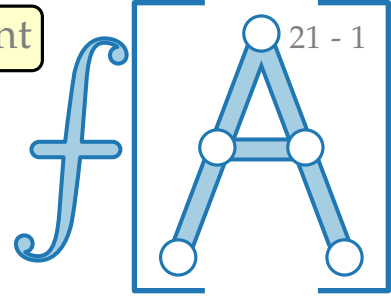
Regel 3

Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$

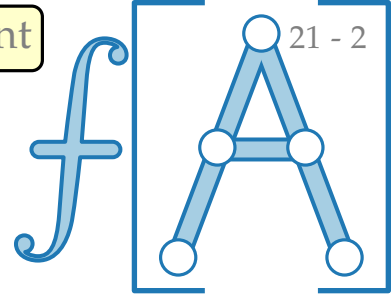


nicht prüfungsrelevant



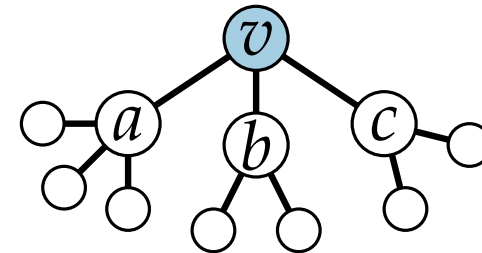
Regel 3

nicht prüfungsrelevant



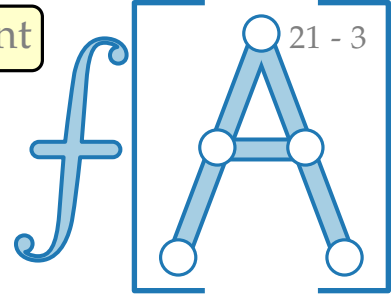
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.



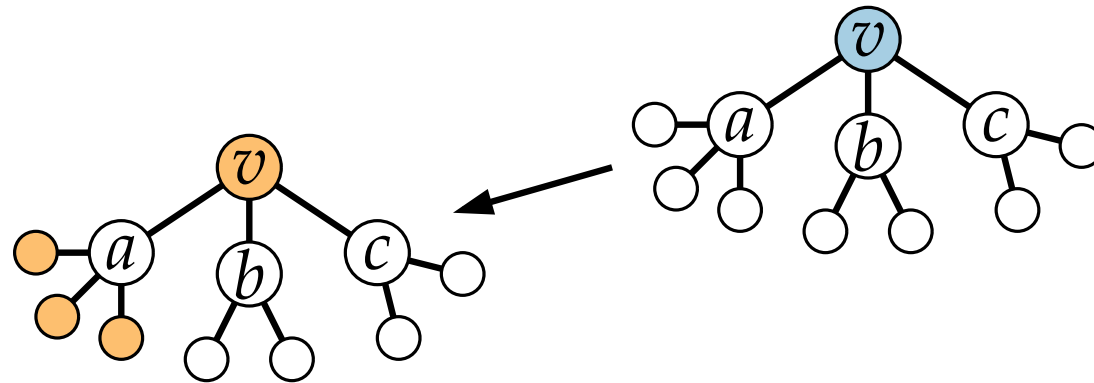
Regel 3

nicht prüfungsrelevant



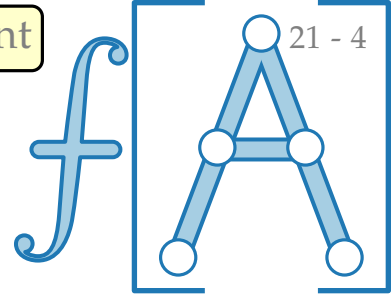
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.



Regel 3

nicht prüfungsrelevant

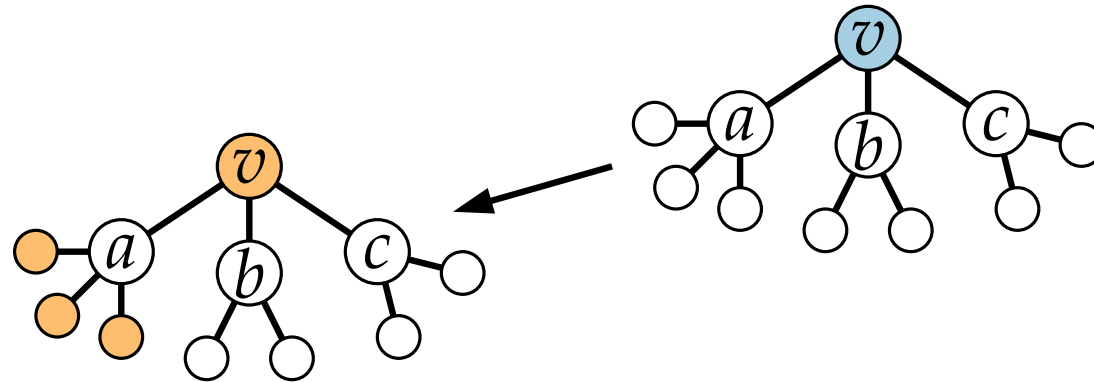


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$

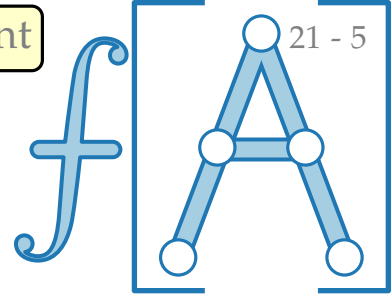
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.



Regel 3

nicht prüfungsrelevant

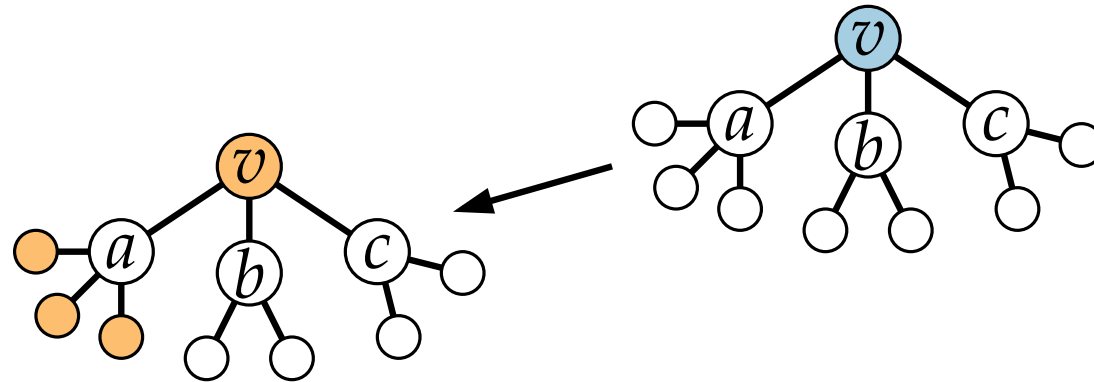


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

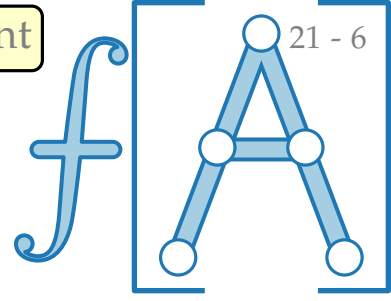
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

➡ Wenn a enthalten ist, sind entweder b und c enthalten oder beide nicht.



Regel 3

nicht prüfungsrelevant

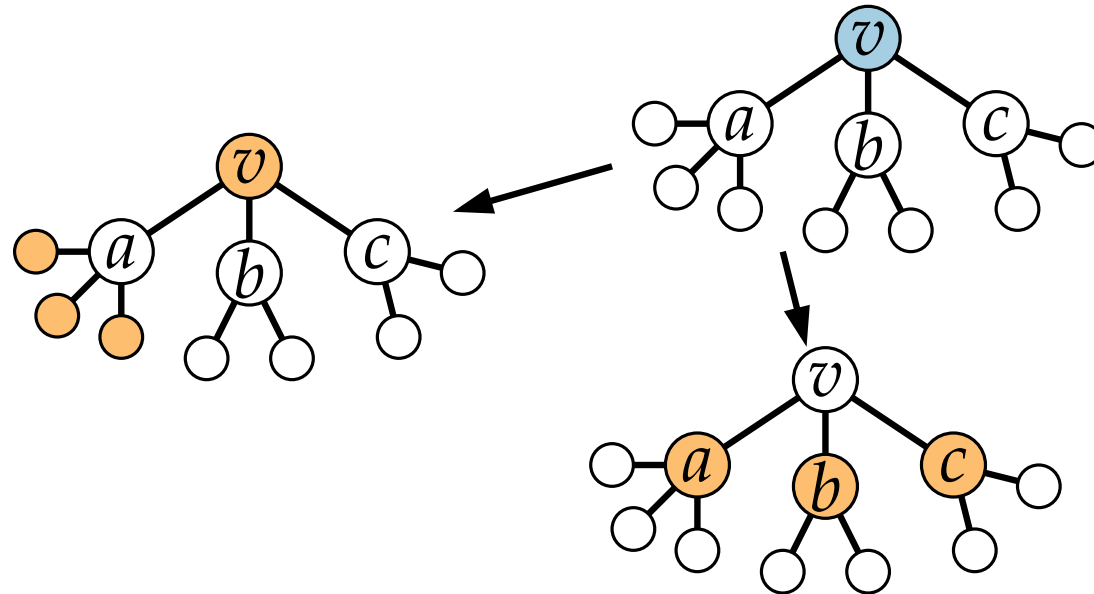


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

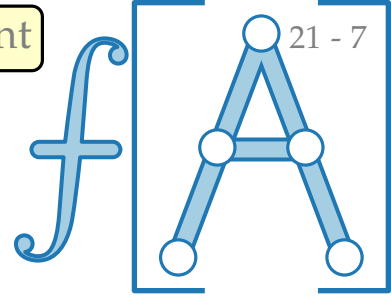
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

➡ Wenn a enthalten ist, sind entweder b und c enthalten oder beide nicht.



Regel 3

nicht prüfungsrelevant

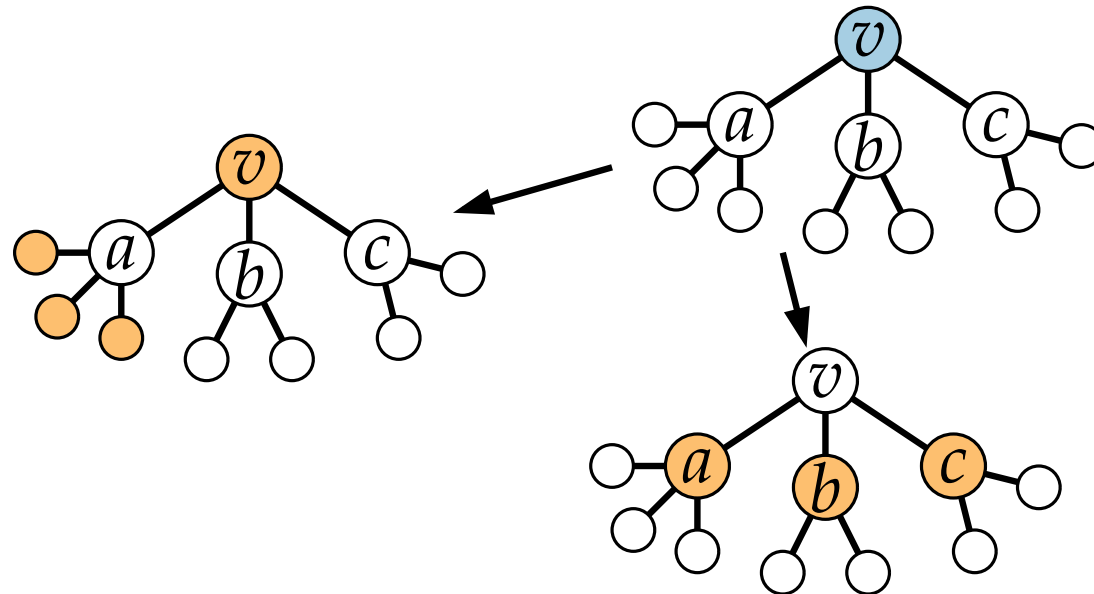


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

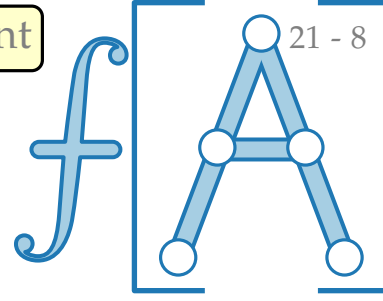
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

- ➔ Wenn a enthalten ist, sind entweder b und c enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.



Regel 3

nicht prüfungsrelevant

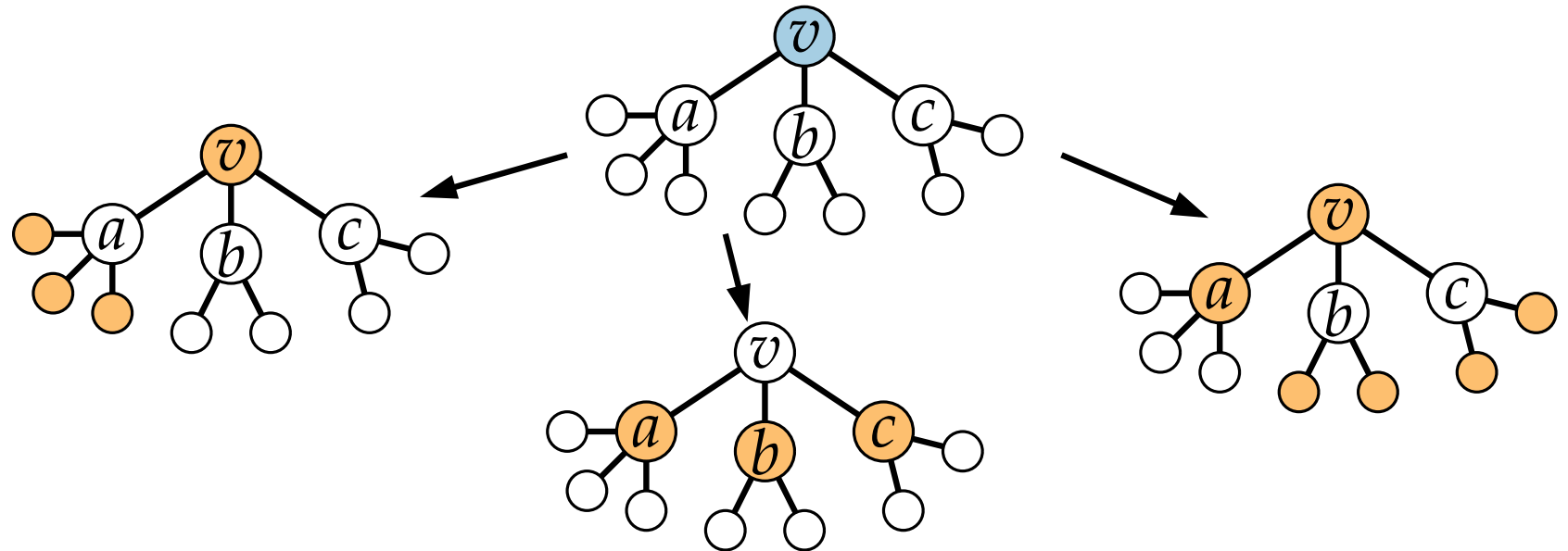


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

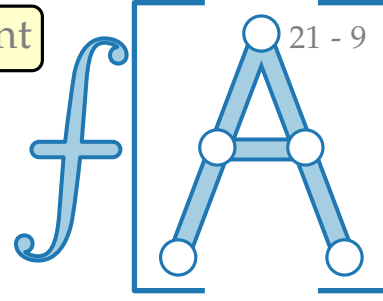
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

- ➔ Wenn a enthalten ist, sind entweder b und c enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.



Regel 3

nicht prüfungsrelevant

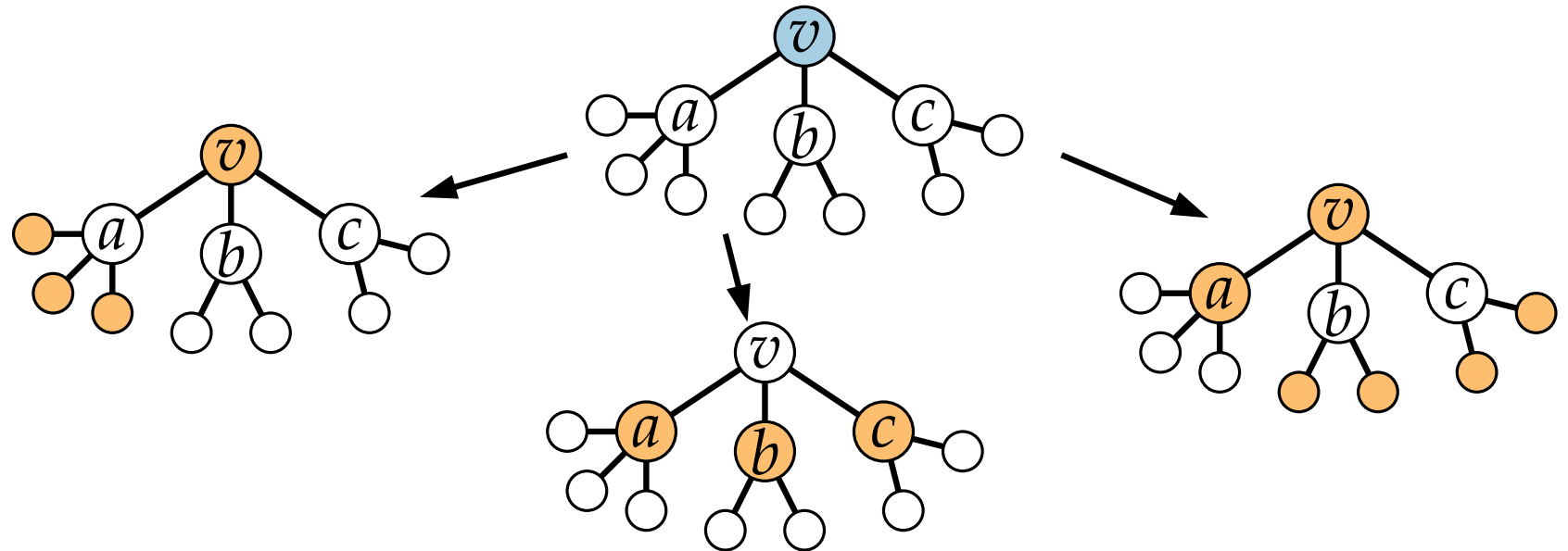


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.3: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) \geq 4$
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

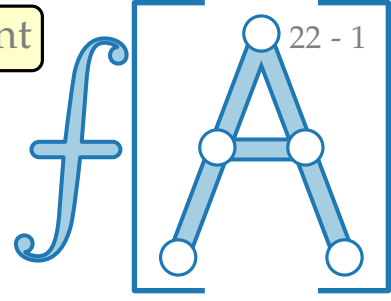
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

- ➔ Wenn a enthalten ist, sind entweder b und c enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 4, 6)$



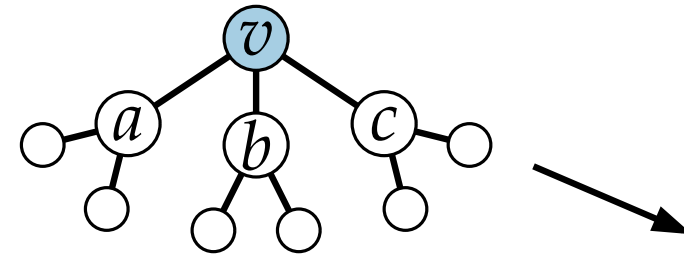
Regel 3

nicht prüfungsrelevant



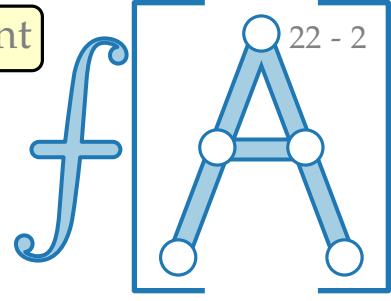
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$



Regel 3

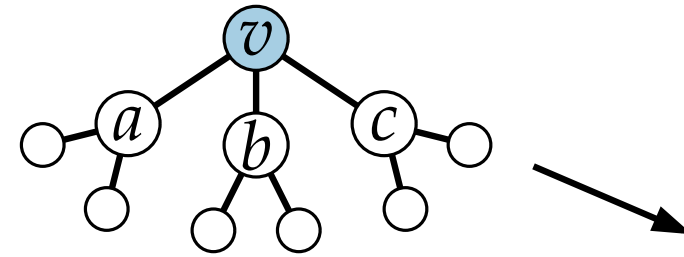
nicht prüfungsrelevant



Regel 3: Eliminiere Knoten v mit Grad 3

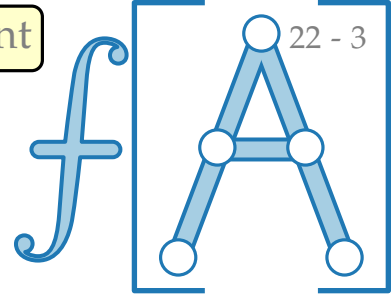
Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.



Regel 3

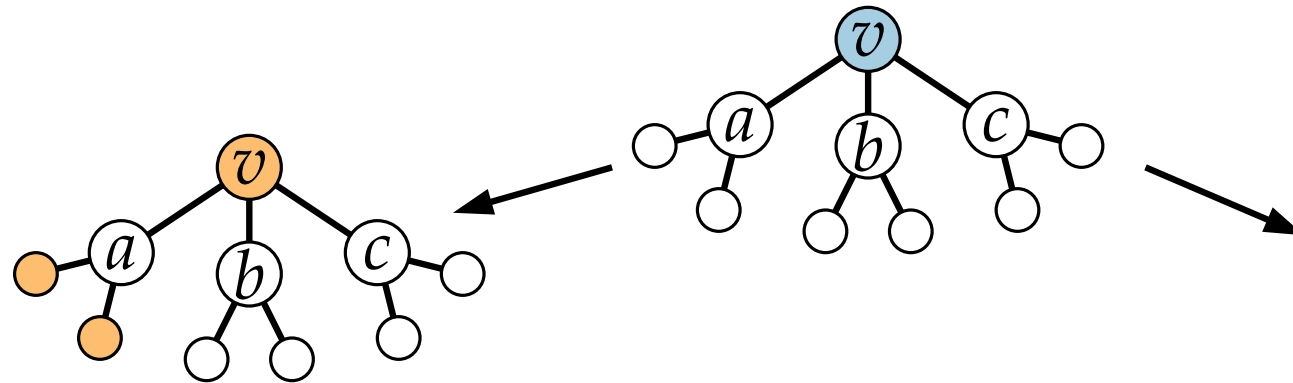
nicht prüfungsrelevant



Regel 3: Eliminiere Knoten v mit Grad 3

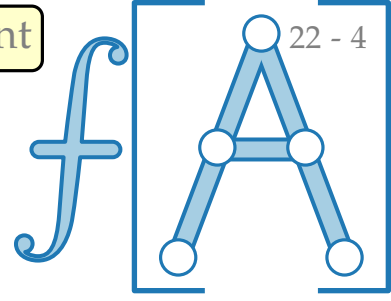
Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.



Regel 3

nicht prüfungsrelevant

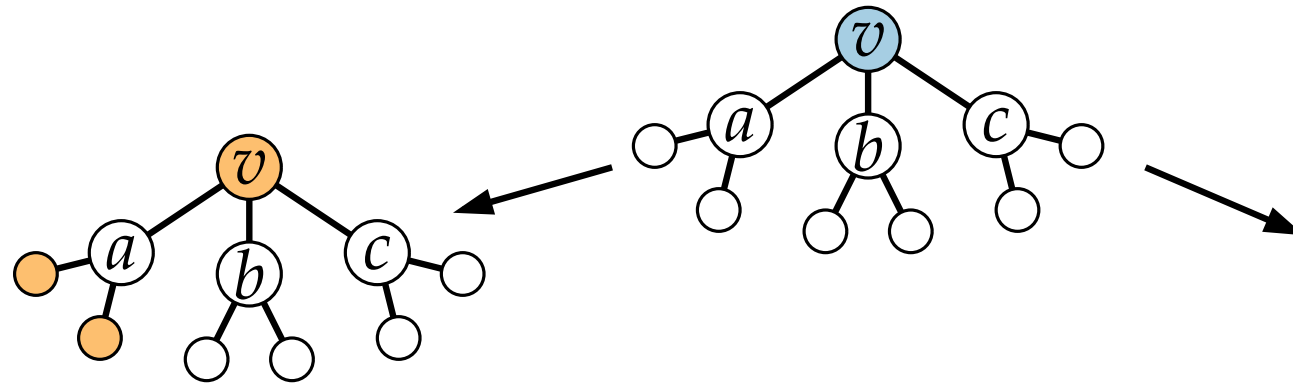


Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

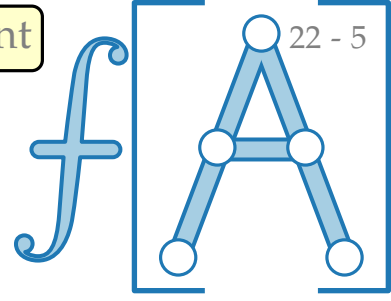
Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.



Regel 3

nicht prüfungsrelevant



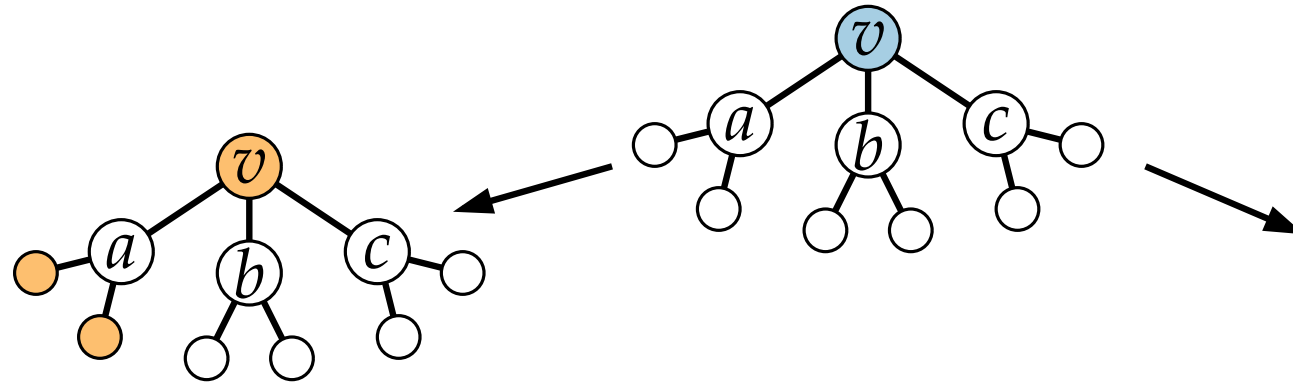
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

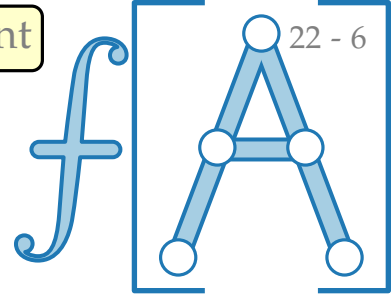
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

➡ Wenn a enthalten ist sind entweder b und c beide enthalten oder beide nicht.



Regel 3

nicht prüfungsrelevant



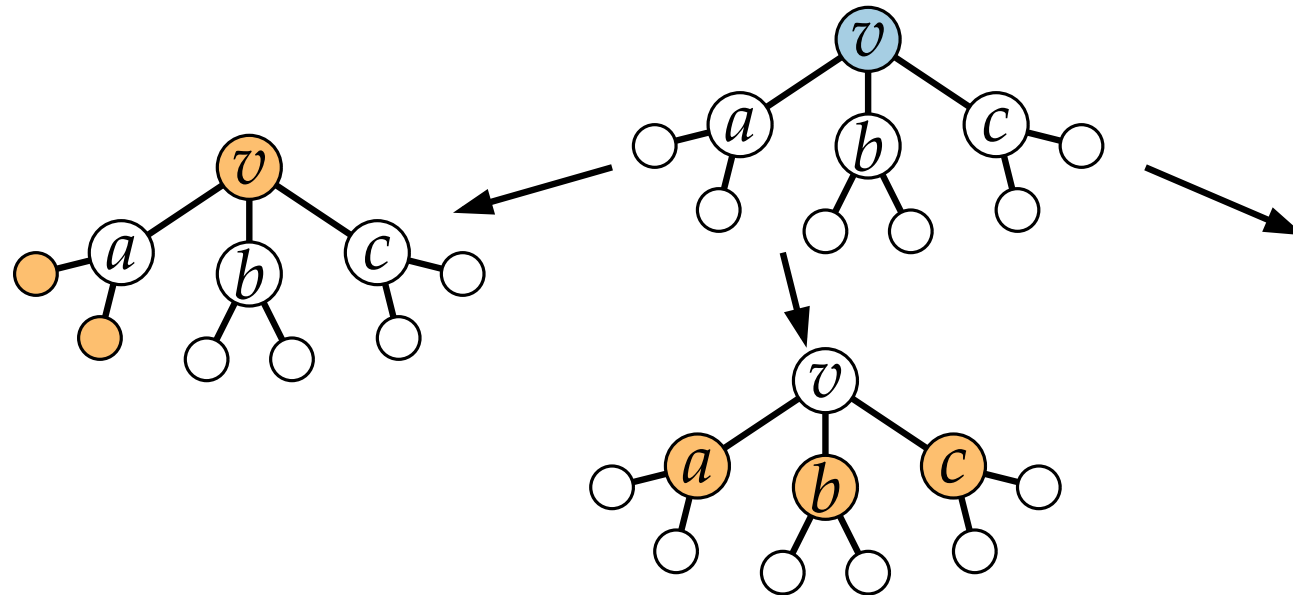
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

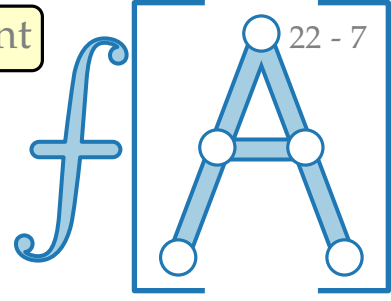
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

➔ Wenn a enthalten ist sind entweder b und c beide enthalten oder beide nicht.



Regel 3

nicht prüfungsrelevant



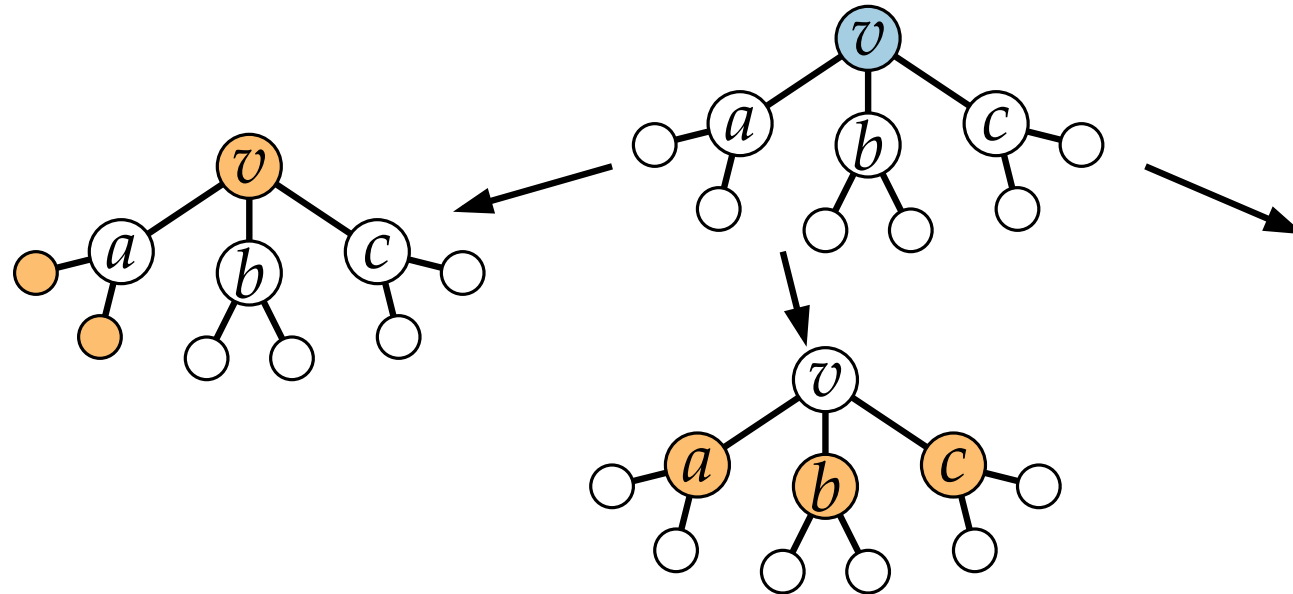
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

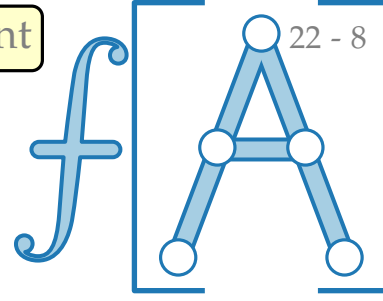
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

- ➔ Wenn a enthalten ist sind entweder b und c beide enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.



Regel 3

nicht prüfungsrelevant



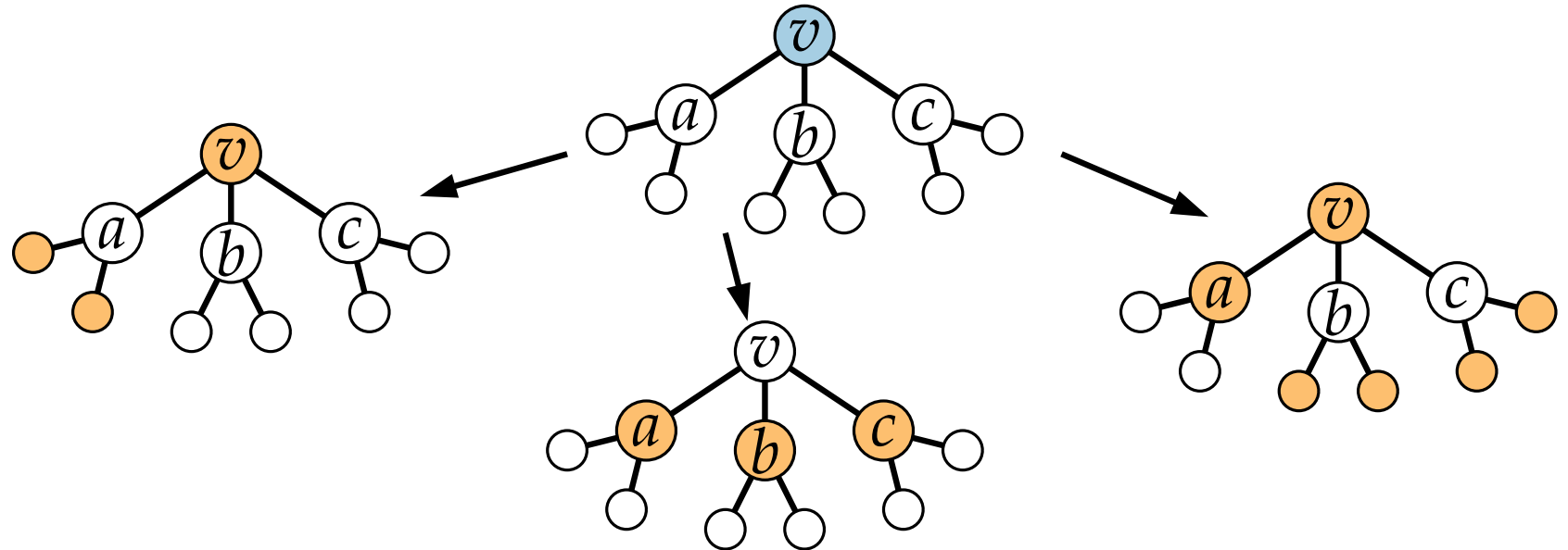
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

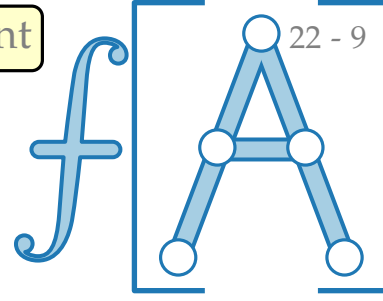
Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

- ➔ Wenn a enthalten ist sind entweder b und c beide enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.



Regel 3

nicht prüfungsrelevant



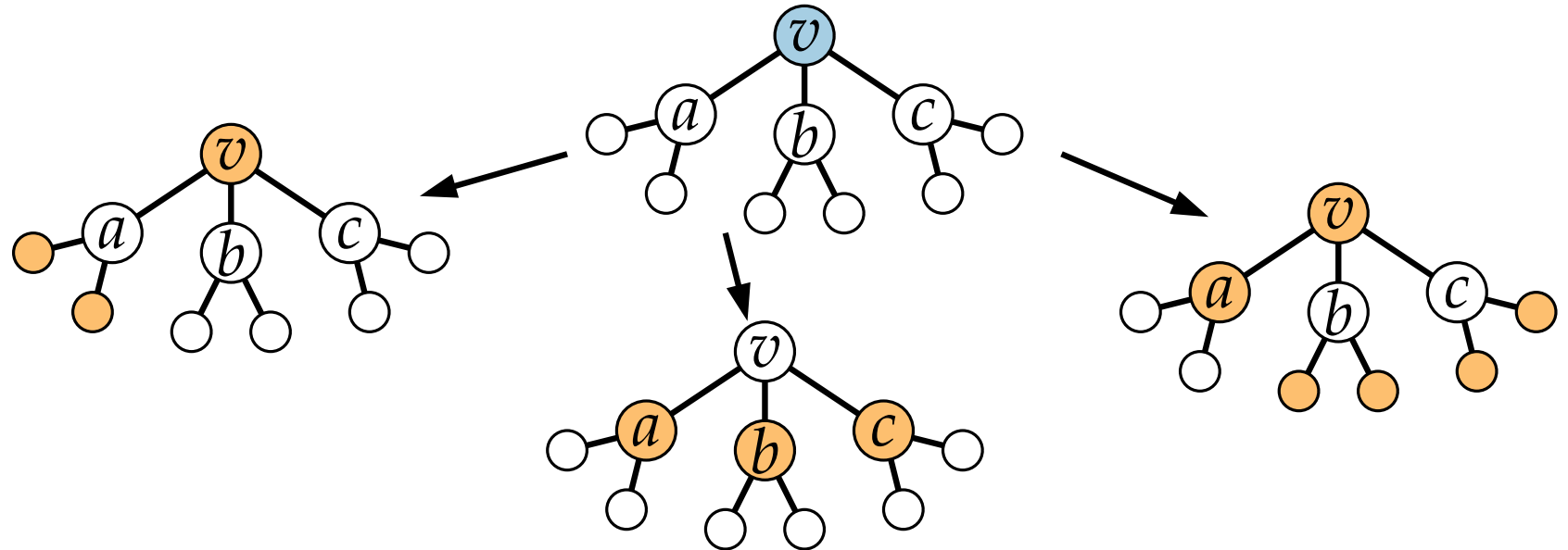
Regel 3: Eliminiere Knoten v mit Grad 3

Fall 3.4: v ist in keinem Drei- oder Viereck enthalten und es gilt $\deg(a) = \deg(b) = \deg(c) = 3$

Ein Vertex Cover, das a nicht enthält, enthält $N(a)$.

Ist a sowie einer von b oder c enthalten, so macht es keinen Sinn v zu wählen.

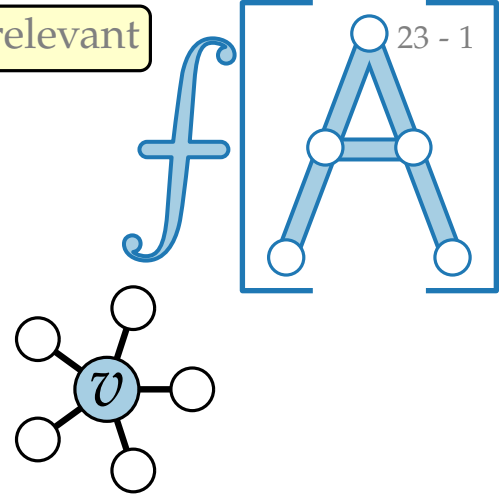
- ➔ Wenn a enthalten ist sind entweder b und c beide enthalten oder beide nicht.
- ➔ Füge entweder $N(a)$ oder a, b, c oder $a, N(b), N(c)$ hinzu.
- ➔ Verzweigungsvektor $(3, 3, 6)$



Regel 5

Regel 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

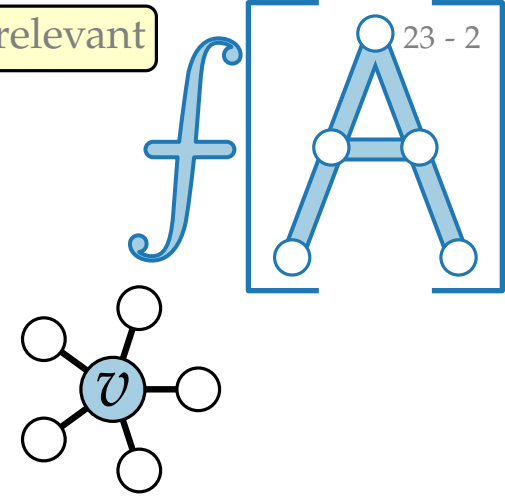
nicht prüfungsrelevant



Regel 5

Regel 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

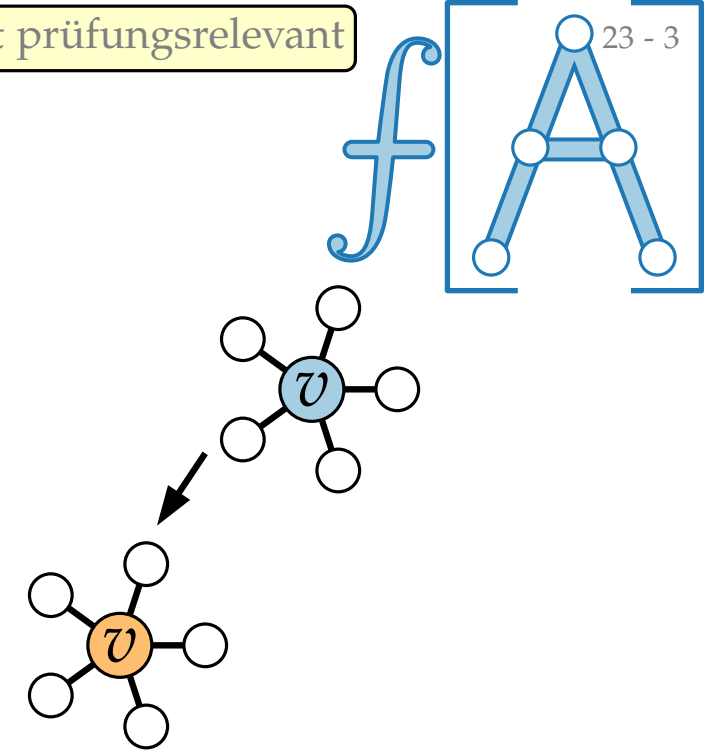
Wähle einen Knoten v oder alle seine Nachbarn.



Regel 5

Regel 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

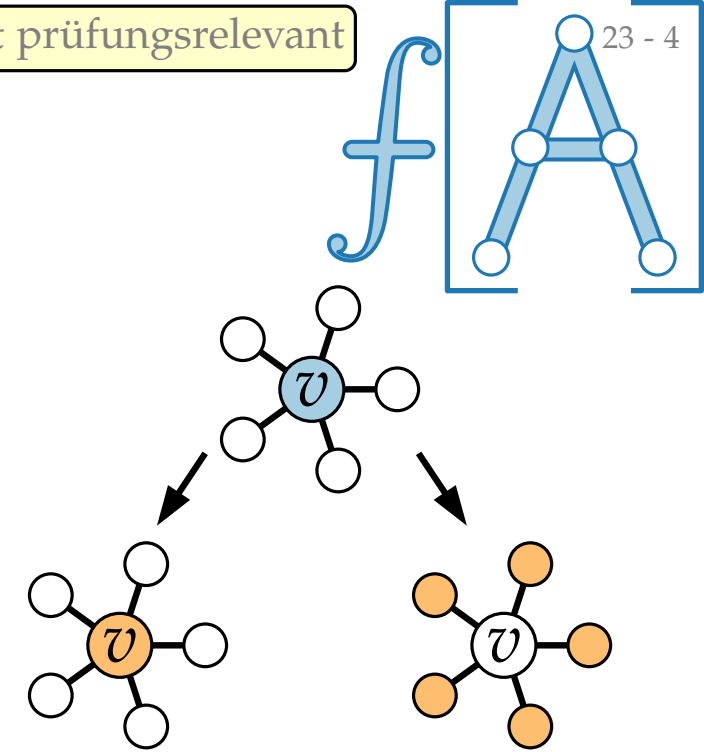
Wähle einen Knoten v oder alle seine Nachbarn.



Regel 5

Regel 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

Wähle einen Knoten v oder alle seine Nachbarn.

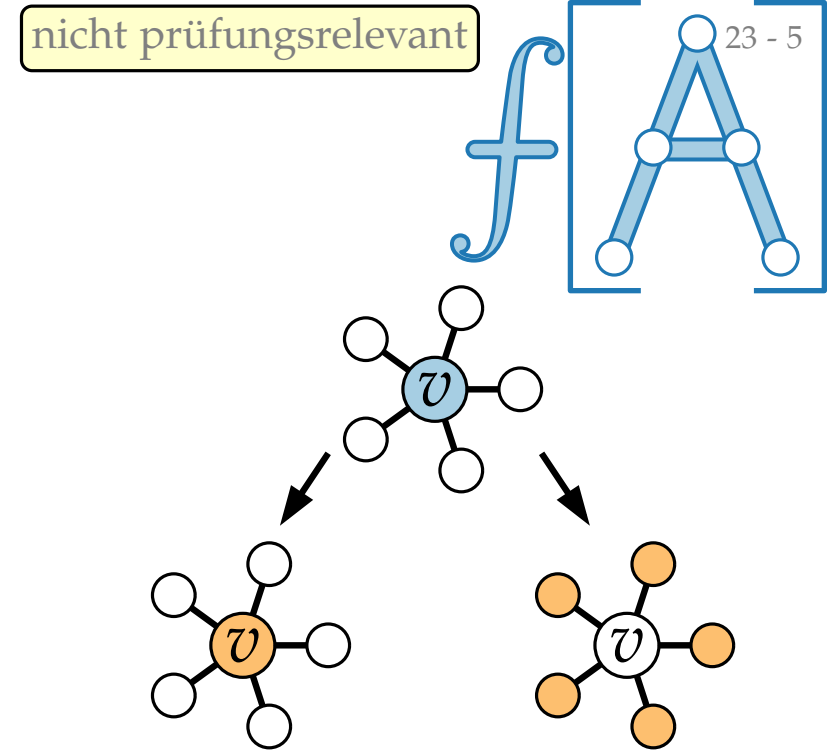


Regel 5

Regel 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

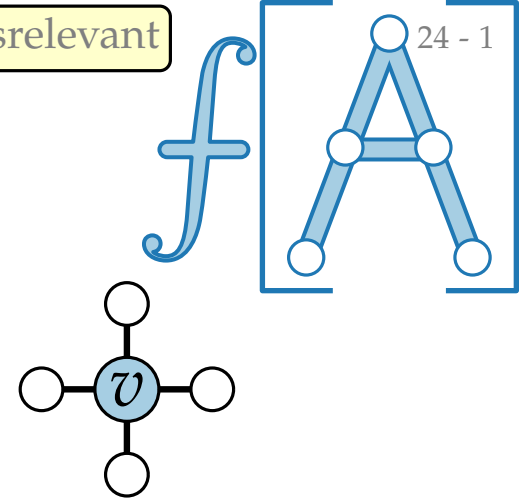
Wähle einen Knoten v oder alle seine Nachbarn.

➔ Verzweigungsvektor $(1, 5)$



Regel 4

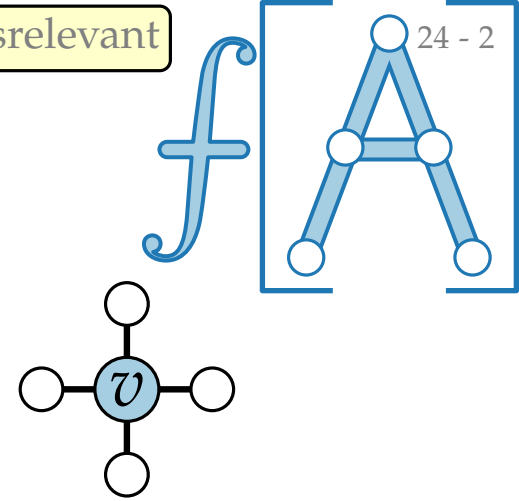
Regel 4: Alle Knoten haben Grad 4



Regel 4

Regel 4: Alle Knoten haben Grad 4

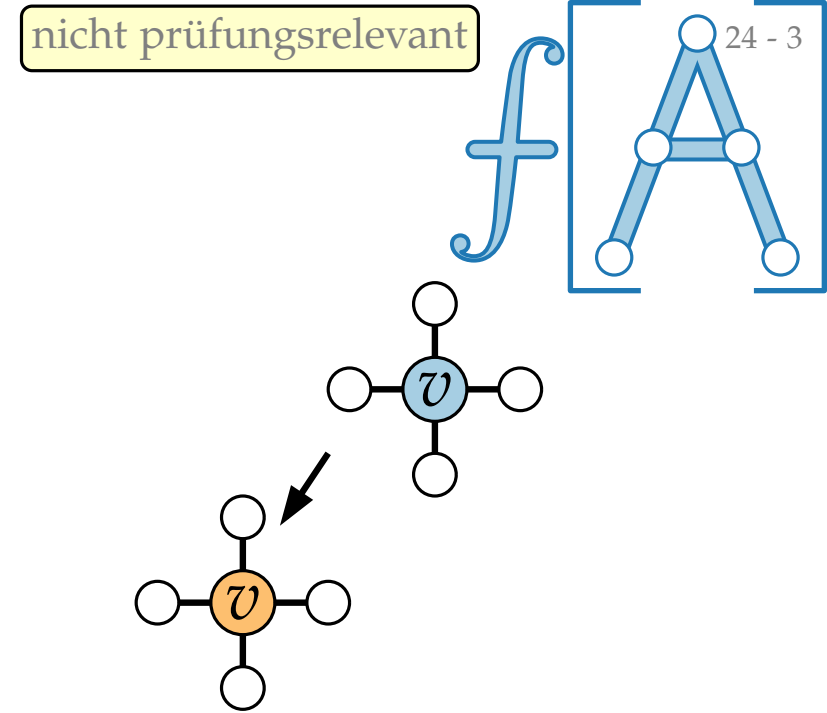
Wähle einen Knoten v oder alle seine Nachbarn.



Regel 4

Regel 4: Alle Knoten haben Grad 4

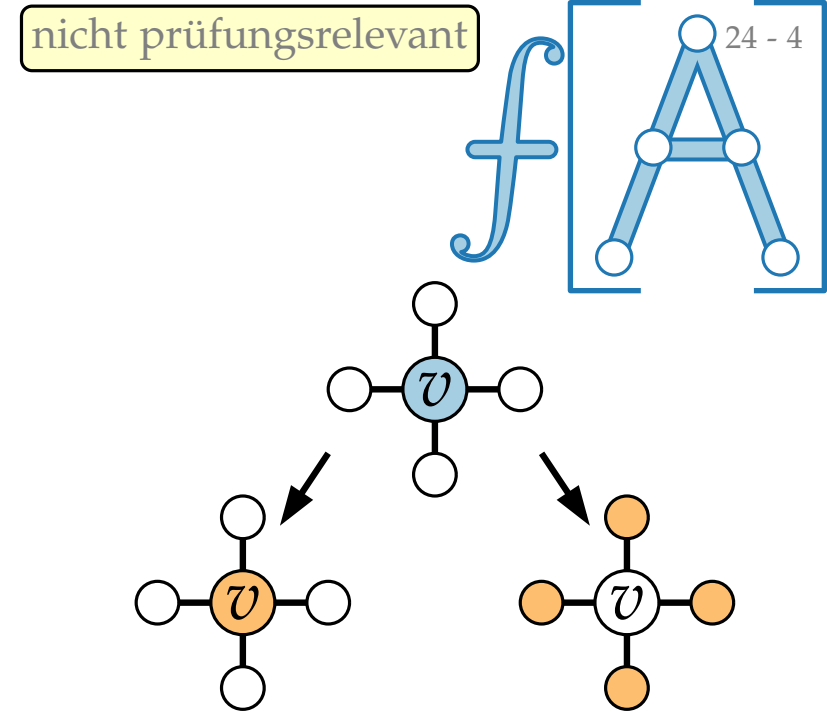
Wähle einen Knoten v oder alle seine Nachbarn.



Regel 4

Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

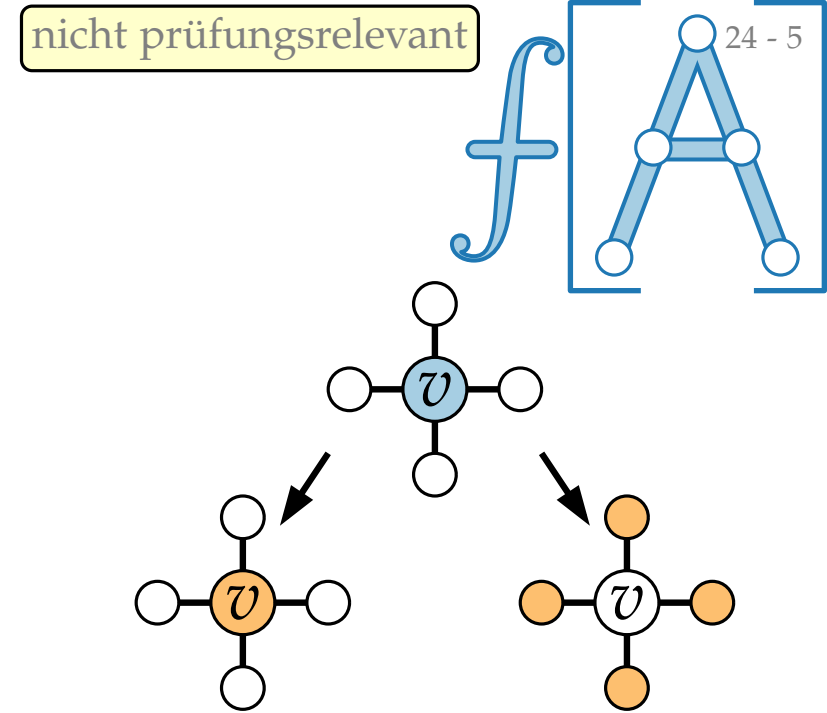


Regel 4

Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

➡ Verzweigungsvektor $(1, 4)$



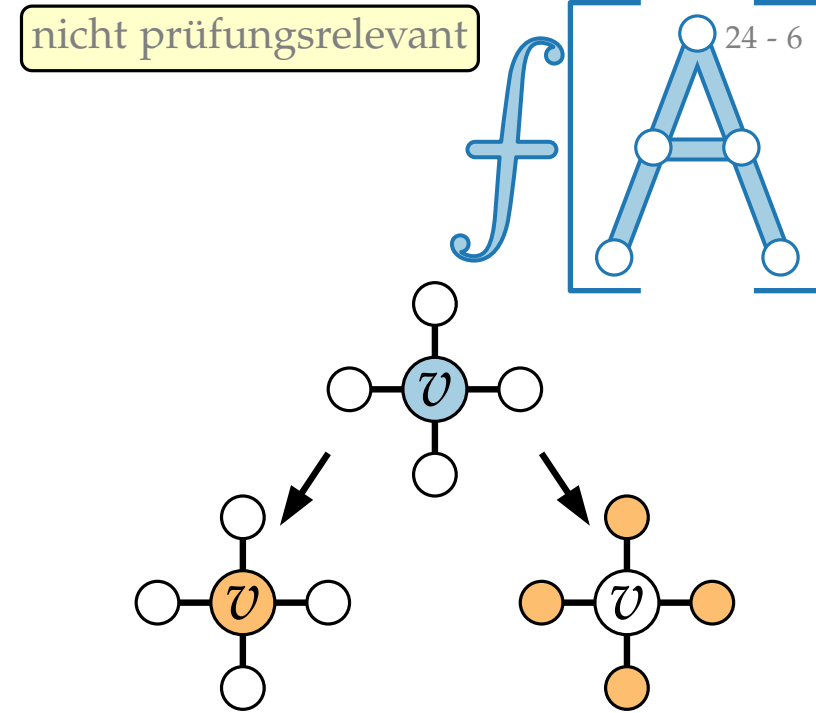
Regel 4

Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

➡ Verzweigungsvektor $(1, 4)$

Beobachtung.



Regel 4

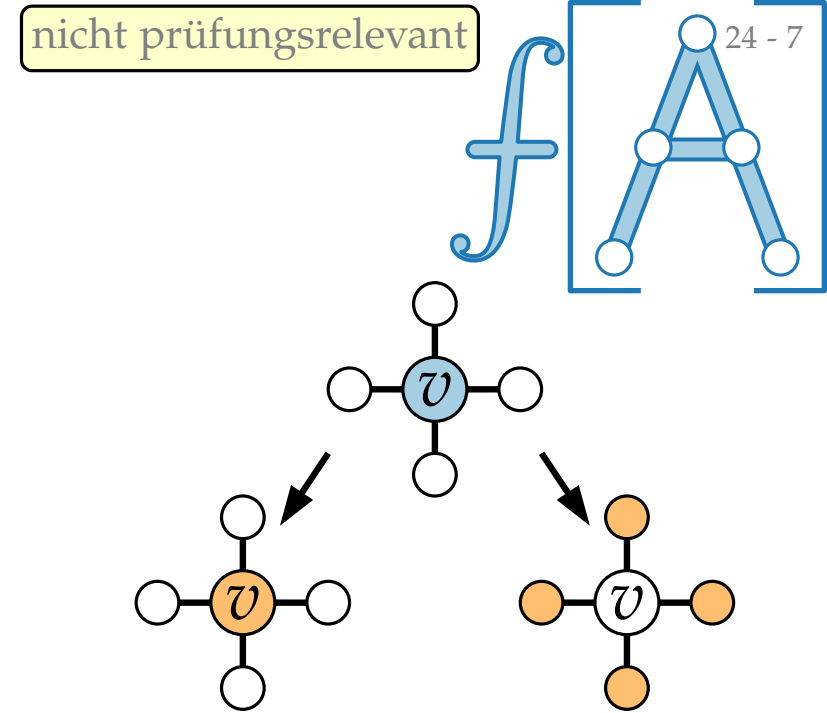
Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

➔ Verzweigungsvektor $(1, 4)$

Beobachtung.

- Die Regel wird nur angewendet, wenn jeder Knoten Grad 4 hat.



Regel 4

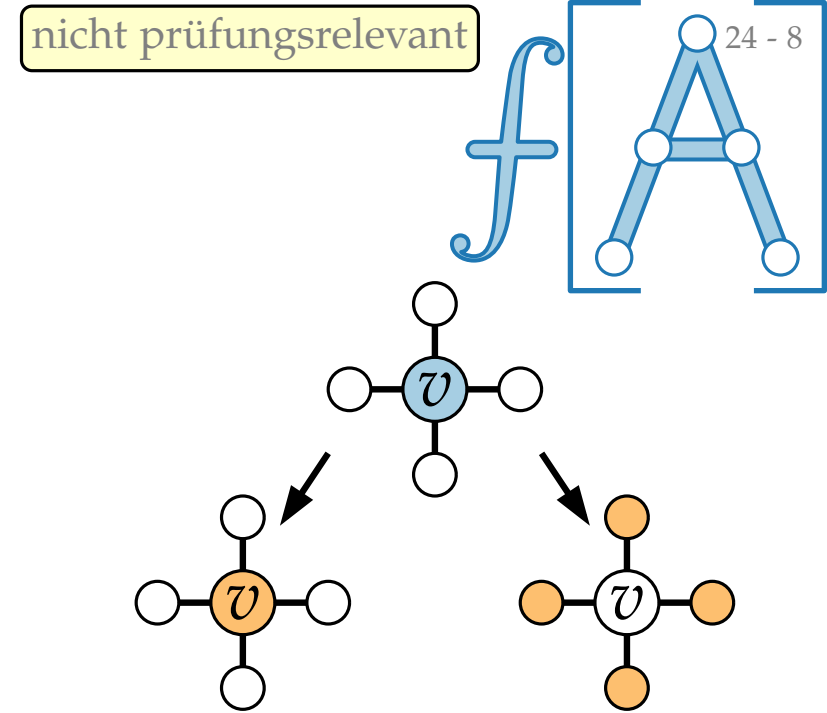
Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

➔ Verzweigungsvektor $(1, 4)$

Beobachtung.

- Die Regel wird nur angewendet, wenn jeder Knoten Grad 4 hat.
- Jeder Subgraph enthält Knoten mit kleinerem Grad.



Regel 4

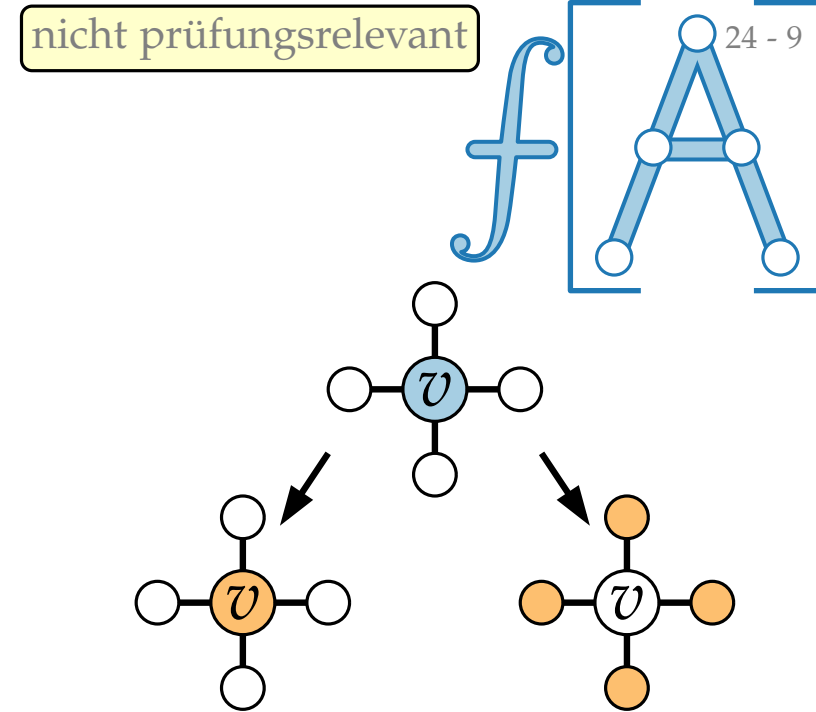
Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

➔ Verzweigungsvektor $(1, 4)$

Beobachtung.

- Die Regel wird nur angewendet, wenn jeder Knoten Grad 4 hat.
- Jeder Subgraph enthält Knoten mit kleinerem Grad.
 - ➔ Regel 4 wird pro Pfad von der Wurzel zu einem Blatt nur einmal ausgeführt.



Regel 4

Regel 4: Alle Knoten haben Grad 4

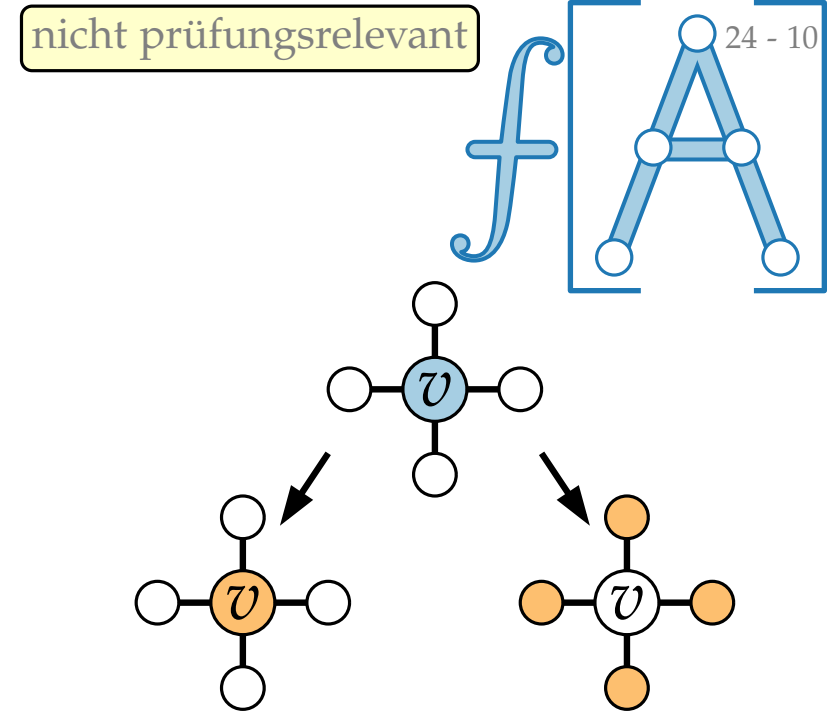
Wähle einen Knoten v oder alle seine Nachbarn.

➔ Verzweigungsvektor $(1, 4)$

Beobachtung.

- Die Regel wird nur angewendet, wenn jeder Knoten Grad 4 hat.
- Jeder Subgraph enthält Knoten mit kleinerem Grad.
 - ➔ Regel 4 wird pro Pfad von der Wurzel zu einem Blatt nur einmal ausgeführt.

Verzweigungsvektor	$(1, 1)$	$(1, 2)$	$(1, 3)$	$(1, 4)$	$(1, 5)$	$(2, 3)$	$(3, 3)$	$(3, 3, 6)$	$(3, 4, 6)$
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26	1.342	1.305



Regel 4

Regel 4: Alle Knoten haben Grad 4

Wähle einen Knoten v oder alle seine Nachbarn.

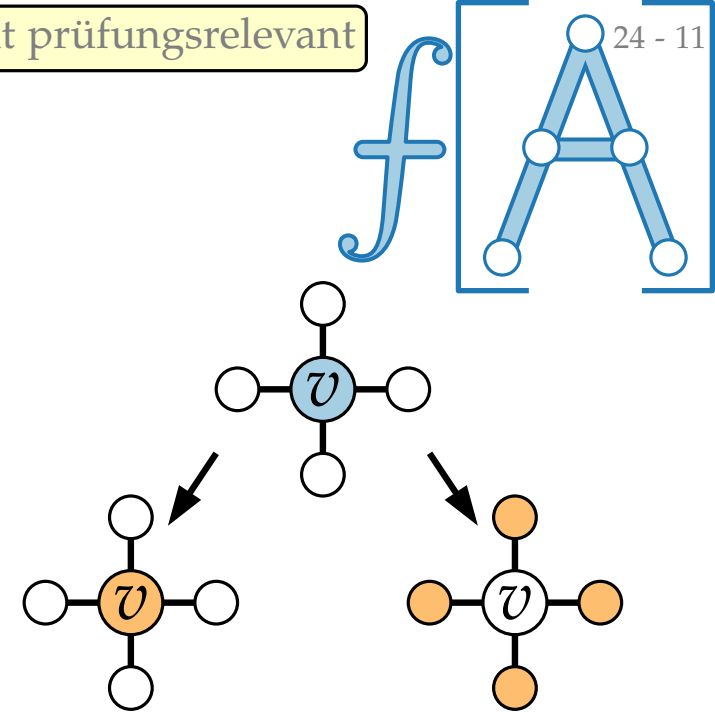
➔ Verzweigungsvektor $(1, 4)$

Beobachtung.

- Die Regel wird nur angewendet, wenn jeder Knoten Grad 4 hat.
- Jeder Subgraph enthält Knoten mit kleinerem Grad.
 - ➔ Regel 4 wird pro Pfad von der Wurzel zu einem Blatt nur einmal ausgeführt.

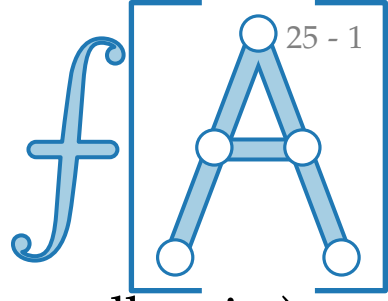
Verzweigungsvektor	$(1, 1)$	$(1, 2)$	$(1, 3)$	$(1, 4)$	$(1, 5)$	$(2, 3)$	$(3, 3)$	$(3, 3, 6)$	$(3, 4, 6)$
Basis b	2	1.618	1.4656	1.3803	1.325	1.325	1.26	1.342	1.305

➔ Laufzeit wird dominiert durch $(3, 3, 6)$ ➔ Gesamtlaufzeit $\mathcal{O}^*(1.342^k)$.



Zusammenfassung

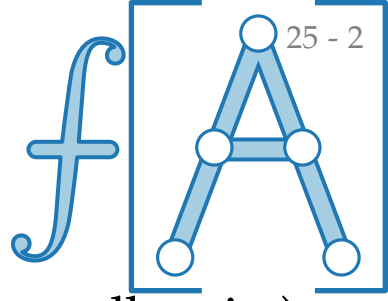
■ Menge von Regeln zur Verzweigung



(sodass immer eine der Regeln anwendbar ist).

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

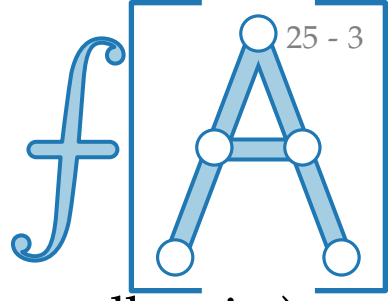
Zusammenfassung



- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$

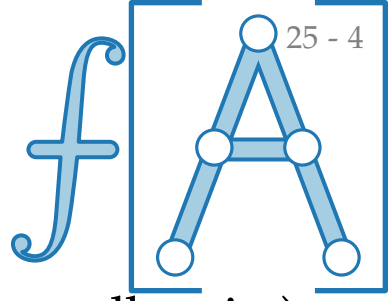
- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

Zusammenfassung



- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.
 - 0: Eliminiere Knoten mit Grad 0
 - 1: Eliminiere Knoten mit Grad 1
 - 2: Eliminiere Knoten mit Grad 2
 - 3: Eliminiere Knoten mit Grad 3
 - 4: Alle Knoten haben Grad 4
 - 5: Eliminiere Knoten mit Grad ≥ 5

Zusammenfassung

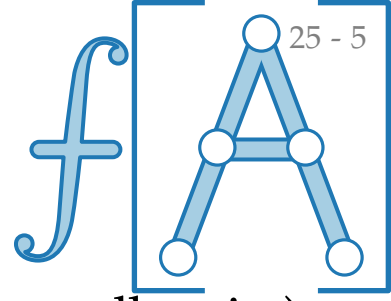


- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1,5)$, $(2,3)$, $(3,3)$, $(3,4,6)$, $(3,3,6)$, höchstens einmal $(1,4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

Laufzeit. $\mathcal{O}^*(1.342^k)$

Zusammenfassung



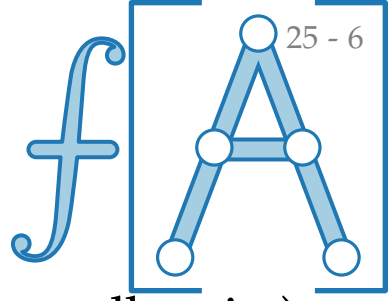
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5



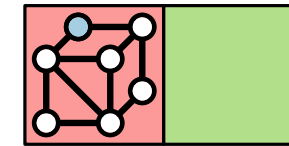
Laufzeit. $\mathcal{O}^*(1.342^k)$

Zusammenfassung



- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

Regel 2

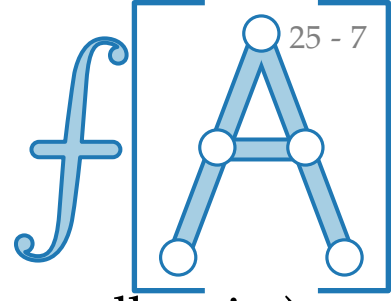


$$k = 3$$

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

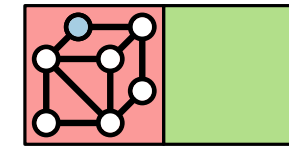
Laufzeit. $\mathcal{O}^*(1.342^k)$

Zusammenfassung



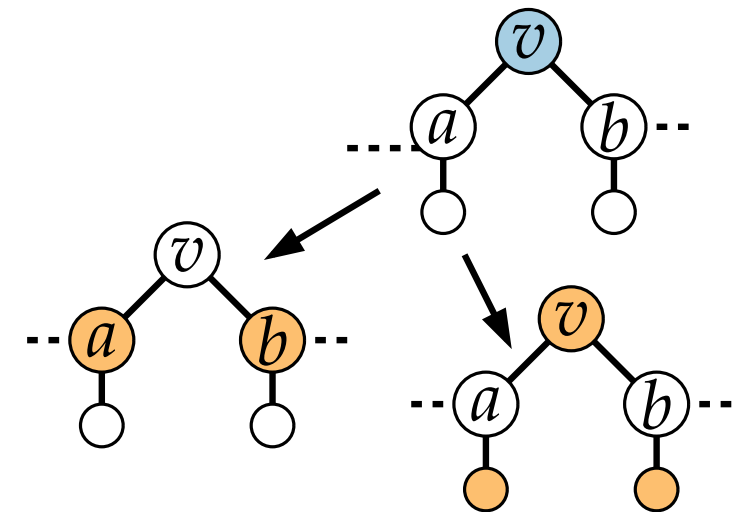
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

Regel 2



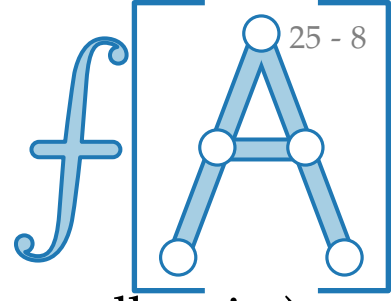
$$k = 3$$

Fall 2.3:



Laufzeit. $\mathcal{O}^*(1.342^k)$

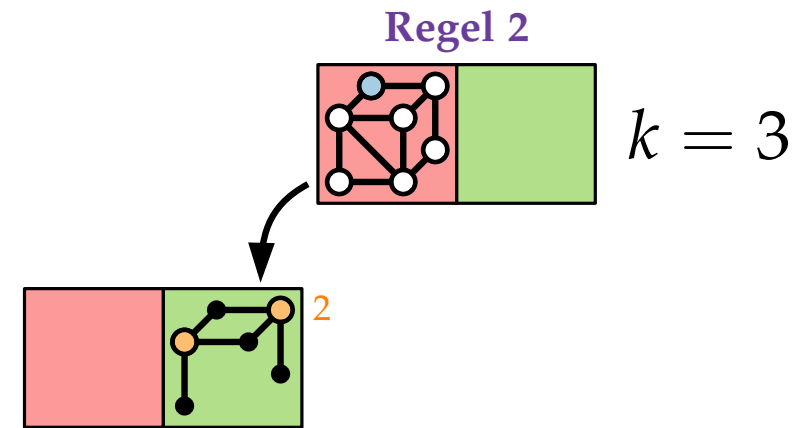
Zusammenfassung



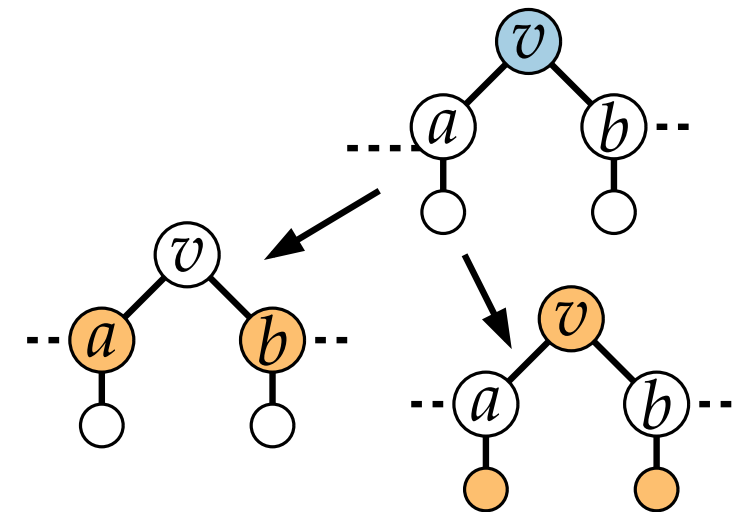
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

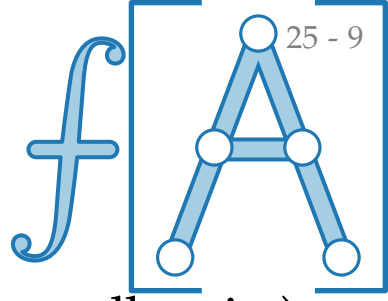
Laufzeit. $\mathcal{O}^*(1.342^k)$



Fall 2.3:



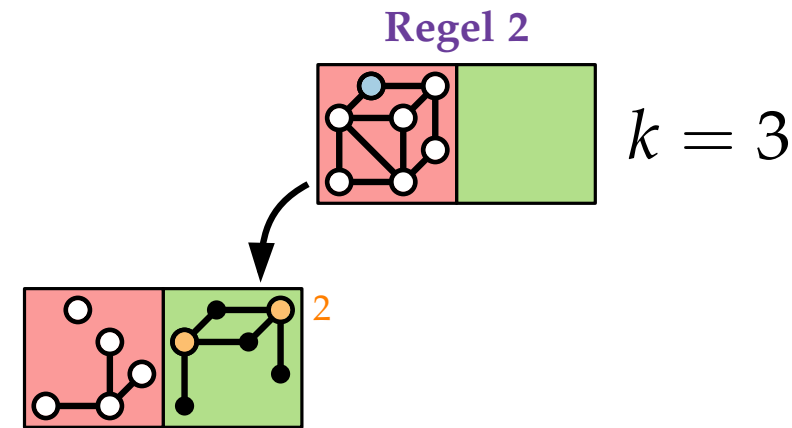
Zusammenfassung



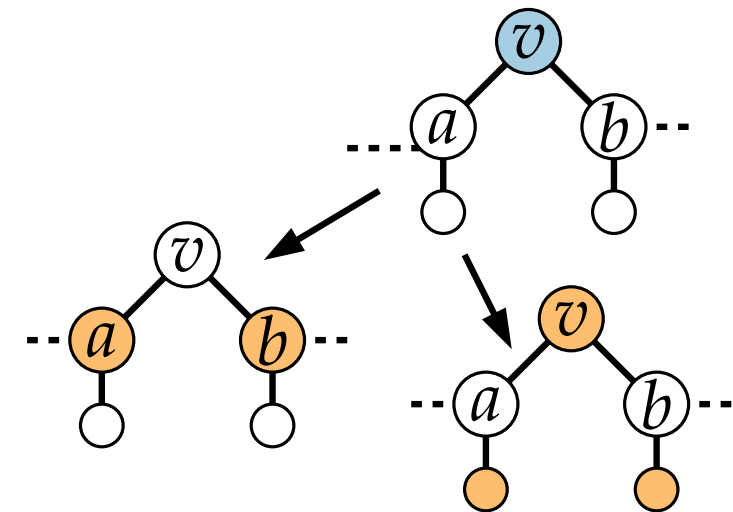
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

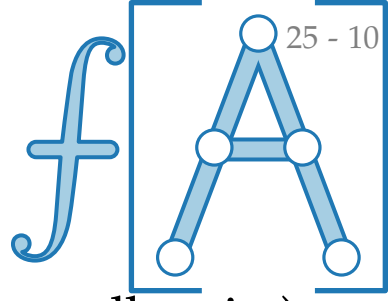
Laufzeit. $\mathcal{O}^*(1.342^k)$



Fall 2.3:



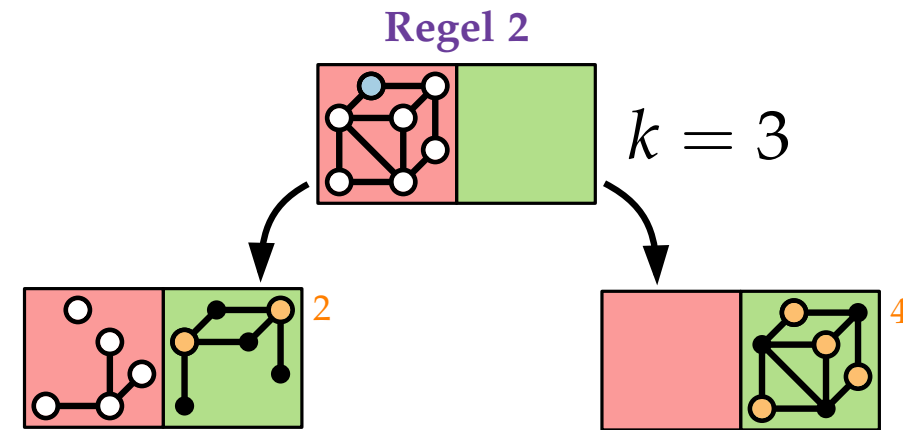
Zusammenfassung



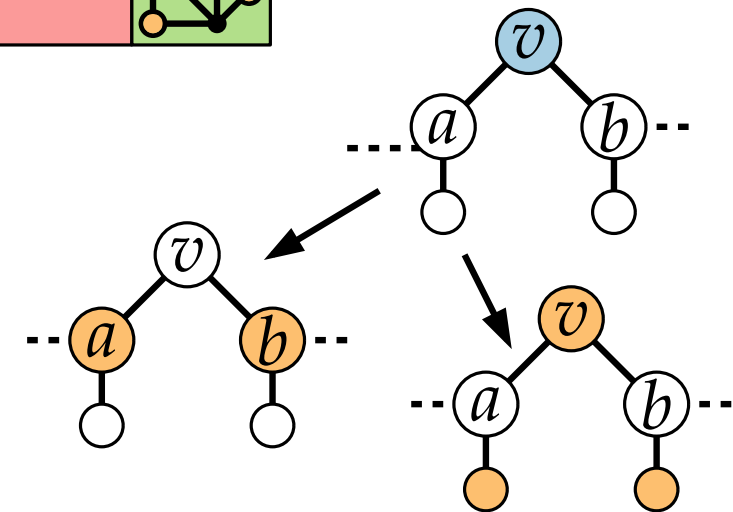
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1,5)$, $(2,3)$, $(3,3)$, $(3,4,6)$, $(3,3,6)$, höchstens einmal $(1,4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

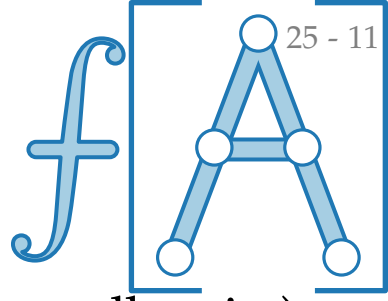
Laufzeit. $\mathcal{O}^*(1.342^k)$



Fall 2.3:



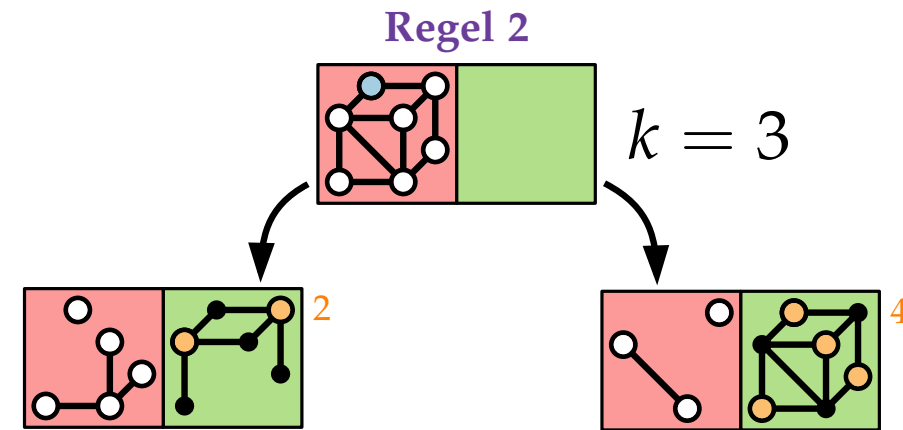
Zusammenfassung



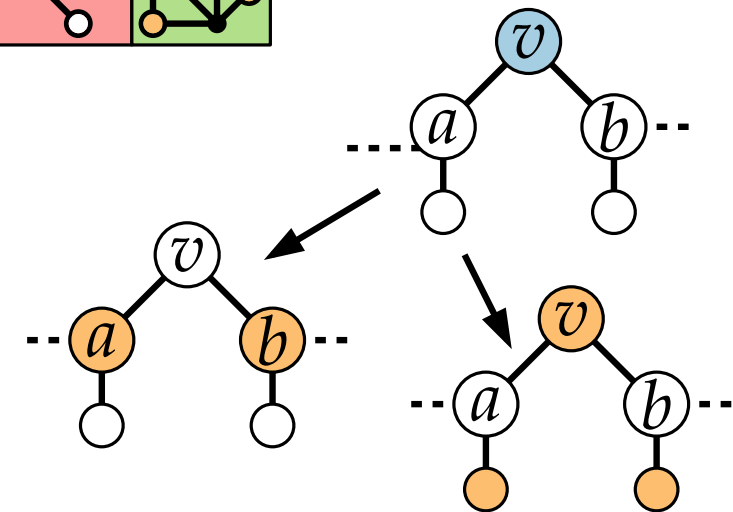
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1,5)$, $(2,3)$, $(3,3)$, $(3,4,6)$, $(3,3,6)$, höchstens einmal $(1,4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

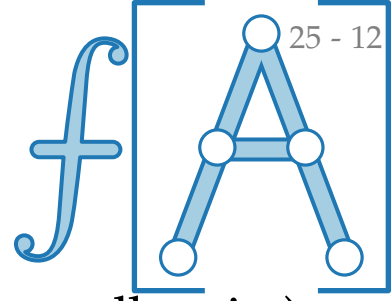
Laufzeit. $\mathcal{O}^*(1.342^k)$



Fall 2.3:



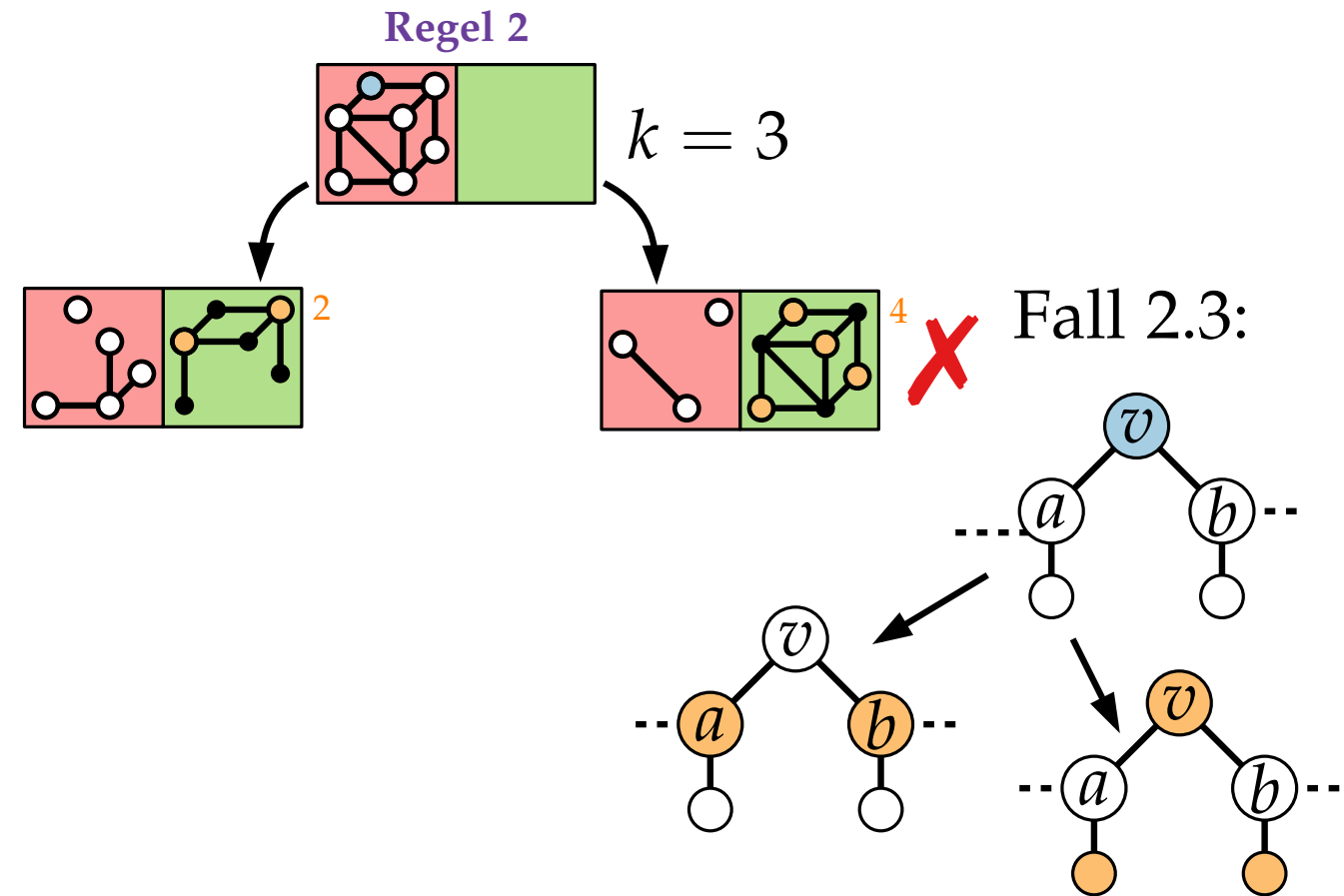
Zusammenfassung



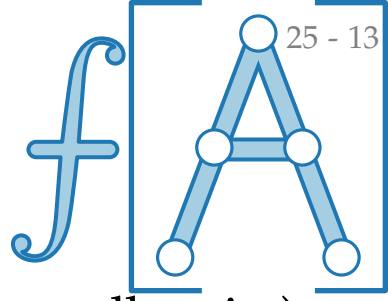
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

Laufzeit. $\mathcal{O}^*(1.342^k)$

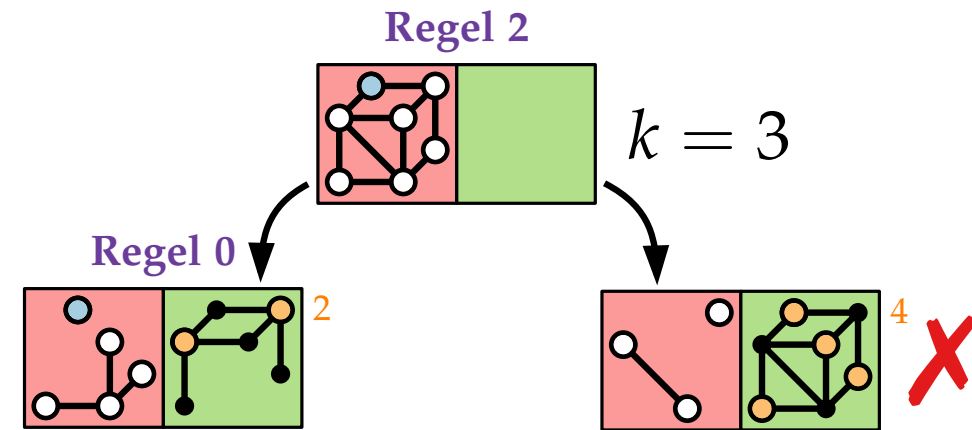


Zusammenfassung



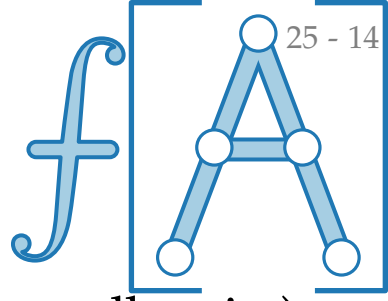
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5



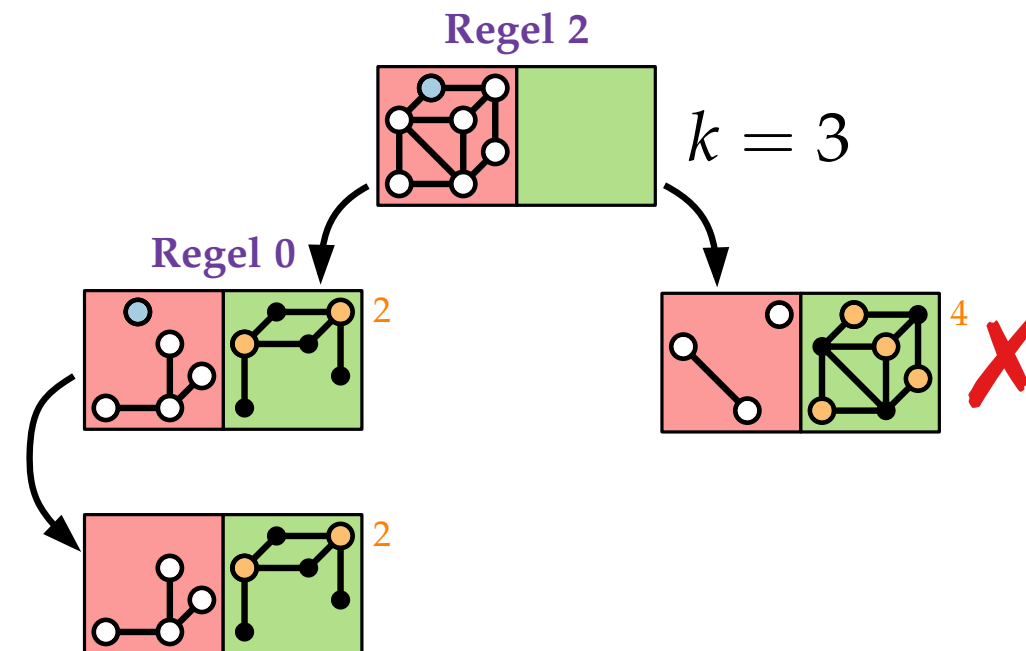
Laufzeit. $\mathcal{O}^*(1.342^k)$

Zusammenfassung



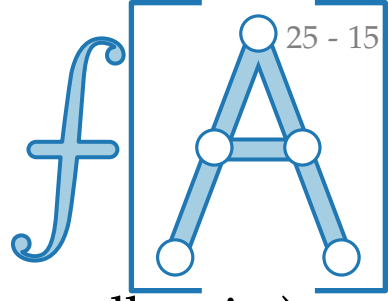
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5



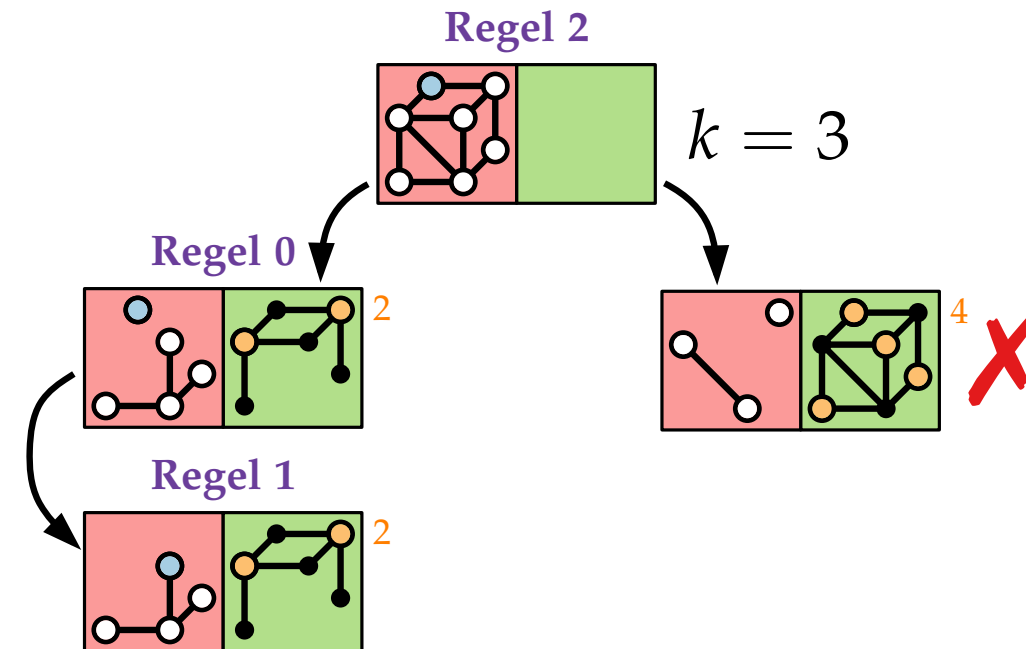
Laufzeit. $\mathcal{O}^*(1.342^k)$

Zusammenfassung



- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit $\text{Grad} \geq 5$

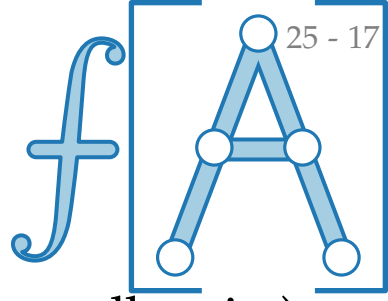


Laufzeit. $\mathcal{O}^*(1.342^k)$

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

The diagram illustrates the application of rules to a graph structure. At the top, a box labeled "Regel 2" contains a graph with 6 nodes and 9 edges, with a green box next to it. Below this box, the text $k = 3$ is shown. Two arrows point from the "Regel 2" box to two different configurations. The left configuration is labeled "Regel 0" and "Regel 1" and is marked with a "2". The right configuration is labeled "Regel 1" and is marked with a "4" and a red "X", indicating it is invalid. Below the configurations, a graph structure is shown with nodes v and w , where w is connected to v and has dashed lines indicating further connections.

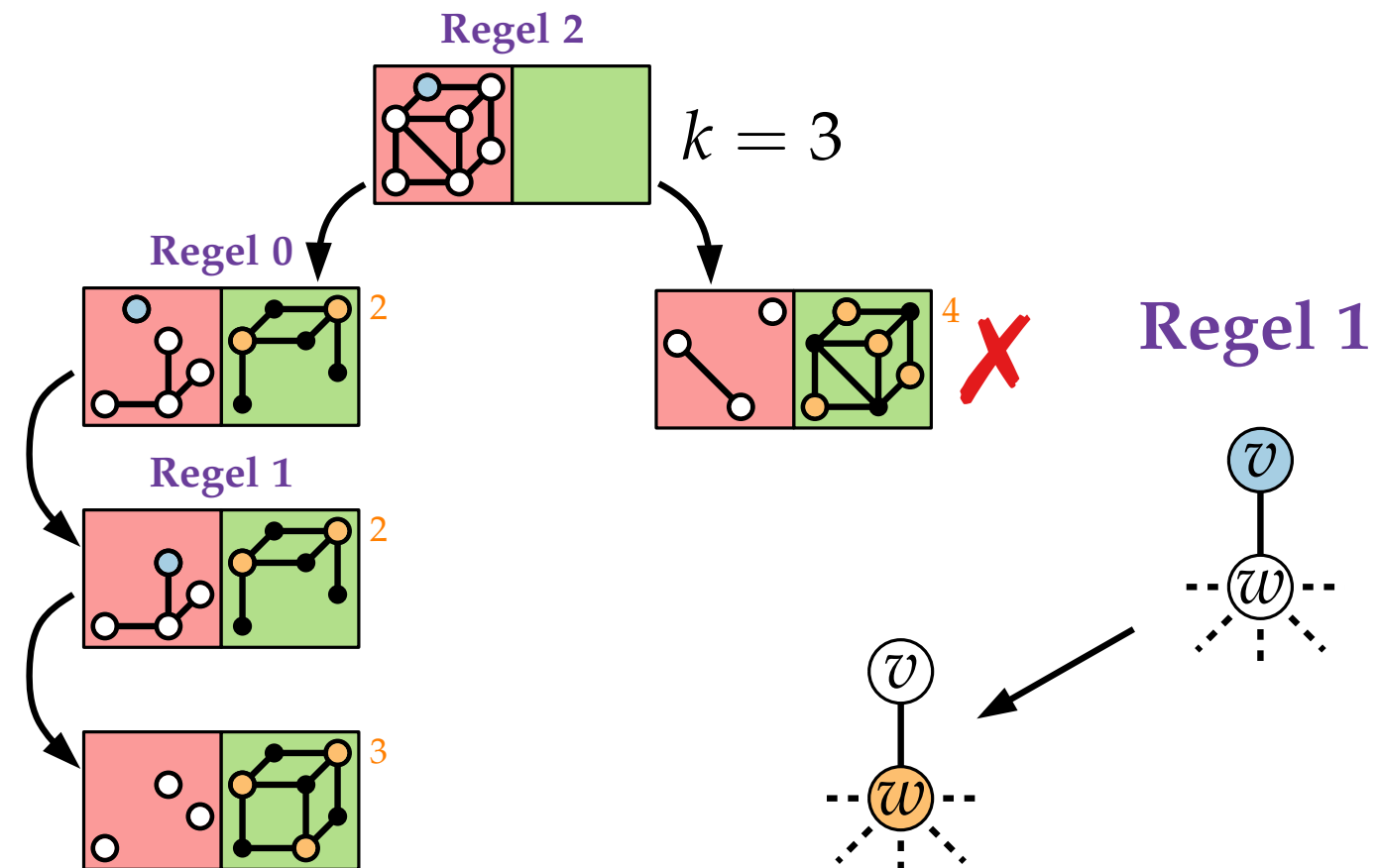
Zusammenfassung



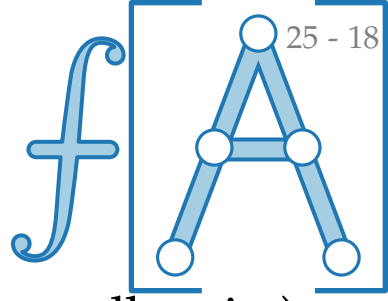
- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1,5)$, $(2,3)$, $(3,3)$, $(3,4,6)$, $(3,3,6)$, höchstens einmal $(1,4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

Laufzeit. $\mathcal{O}^*(1.342^k)$



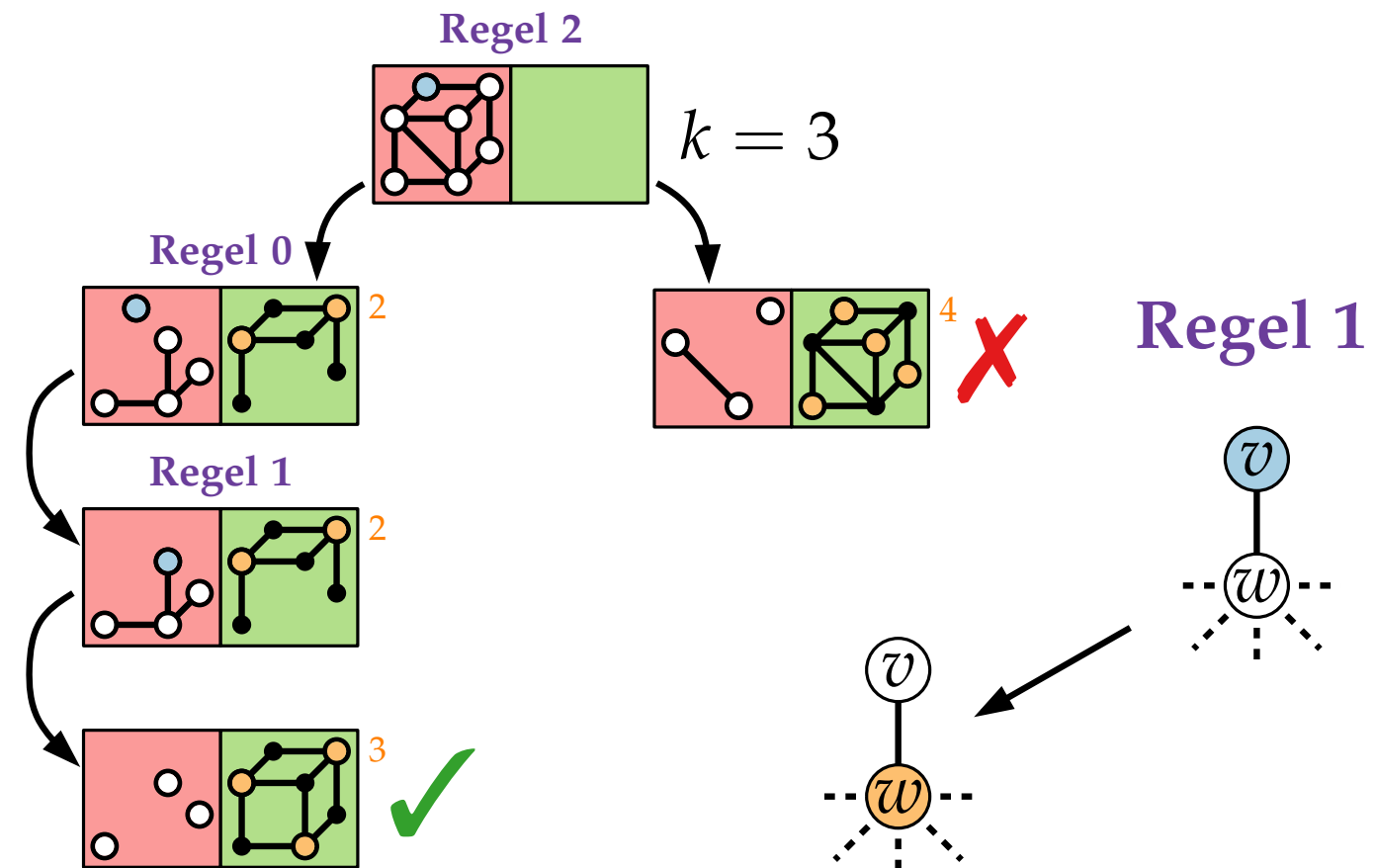
Zusammenfassung



- Menge von Regeln zur Verzweigung (sodass immer eine der Regeln anwendbar ist).
- Erlaubte Verzweigungsvektoren: $(1, 5)$, $(2, 3)$, $(3, 3)$, $(3, 4, 6)$, $(3, 3, 6)$, höchstens einmal $(1, 4)$
- Wende bei jeder Verzweigung die Regel mit der kleinsten Nummer an.

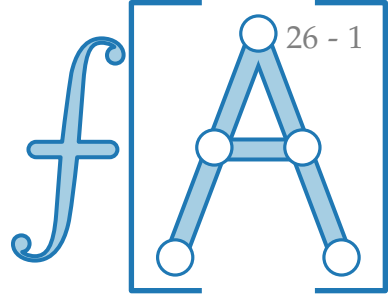
- 0: Eliminiere Knoten mit Grad 0
- 1: Eliminiere Knoten mit Grad 1
- 2: Eliminiere Knoten mit Grad 2
- 3: Eliminiere Knoten mit Grad 3
- 4: Alle Knoten haben Grad 4
- 5: Eliminiere Knoten mit Grad ≥ 5

Laufzeit. $\mathcal{O}^*(1.342^k)$



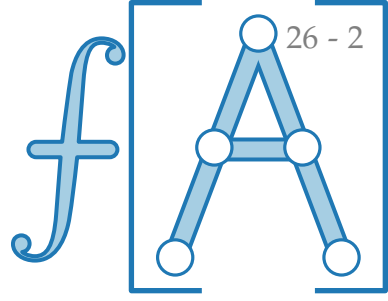
Fazit

- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.



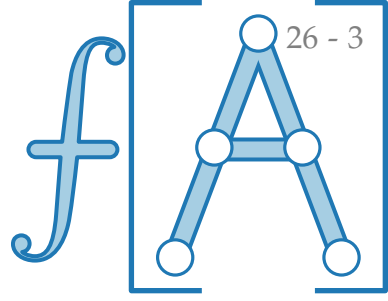
Fazit

- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.
- Der beste bekannte FPT-Algorithmus hat Laufzeit $\mathcal{O}^*(1.2738^k)$.
[Chen, Kanj & Xia '06].

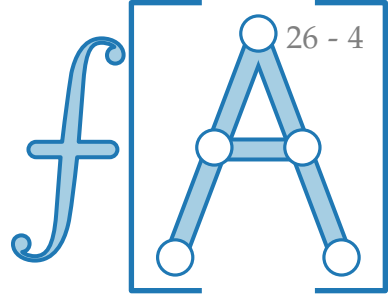


Fazit

- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.
- Der beste bekannte FPT-Algorithmus hat Laufzeit $\mathcal{O}^*(1.2738^k)$.
[Chen, Kanj & Xia '06].
- Dieser löst Instanzen bis ca. $k = 190$ in annehmbarer Zeit
(**Klamwert** $20 / \log_{10} c \approx 190$ für $c = 1.2738$)

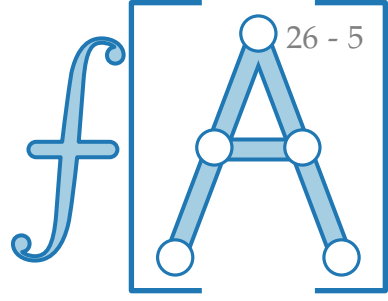


Fazit



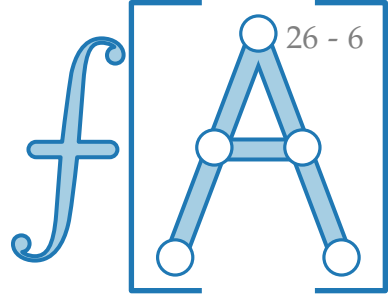
- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.
- Der beste bekannte FPT-Algorithmus hat Laufzeit $\mathcal{O}^*(1.2738^k)$.
[Chen, Kanj & Xia '06].
- Dieser löst Instanzen bis ca. $k = 190$ in annehmbarer Zeit
(**Klamwert** $20 / \log_{10} c \approx 190$ für $c = 1.2738$)
- Parametrisierte Komplexität =
neuer Werkzeugkasten für schwere Probleme:
Kernbildung, Tabellen, Suchbäume, ...

Fazit



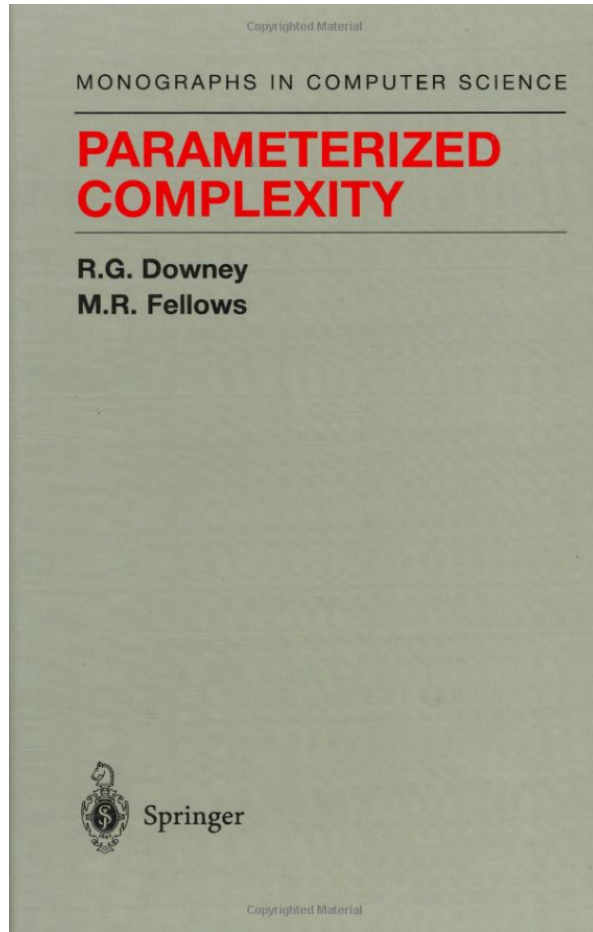
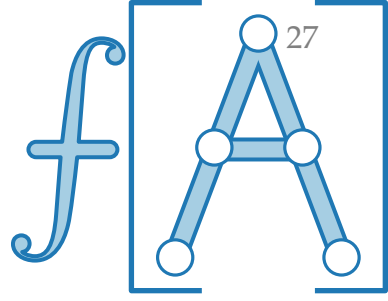
- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.
- Der beste bekannte FPT-Algorithmus hat Laufzeit $\mathcal{O}^*(1.2738^k)$.
[Chen, Kanj & Xia '06].
- Dieser löst Instanzen bis ca. $k = 190$ in annehmbarer Zeit
(**Klamwert** $20 / \log_{10} c \approx 190$ für $c = 1.2738$)
- Parametrisierte Komplexität =
neuer Werkzeugkasten für schwere Probleme:
Kernbildung, Tabellen, Suchbäume, ...
- Es ist immer sinnvoll, beschränkte Parameter zu identifizieren – FPT nutzt sie!

Fazit

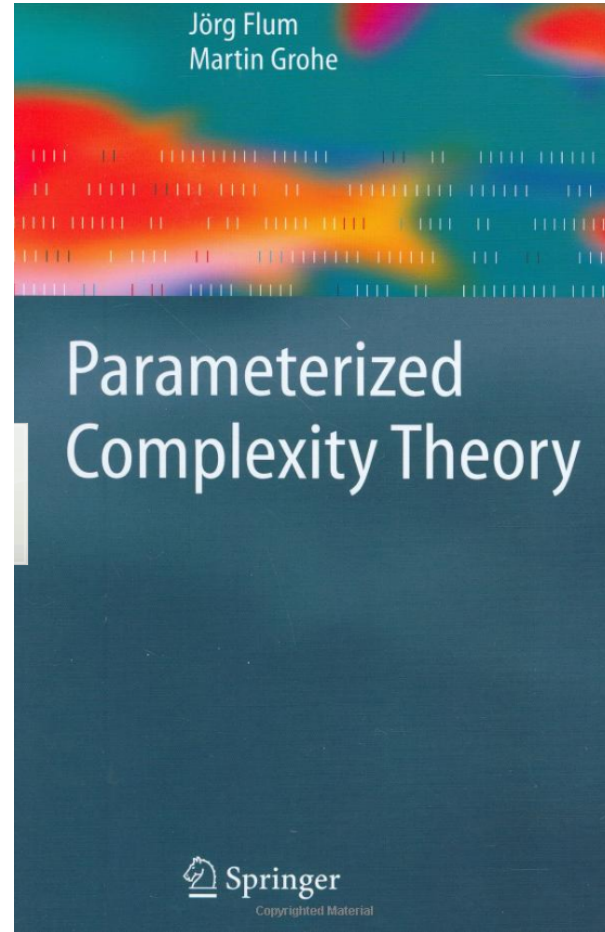


- k -VC kann in $\mathcal{O}^*(1.342^k)$ Zeit gelöst werden.
- Der beste bekannte FPT-Algorithmus hat Laufzeit $\mathcal{O}^*(1.2738^k)$.
[Chen, Kanj & Xia '06].
- Dieser löst Instanzen bis ca. $k = 190$ in annehmbarer Zeit
(**Klamwert** $20 / \log_{10} c \approx 190$ für $c = 1.2738$)
- Parametrisierte Komplexität =
neuer Werkzeugkasten für schwere Probleme:
Kernbildung, Tabellen, Suchbäume, ...
- Es ist immer sinnvoll, beschränkte Parameter zu identifizieren – FPT nutzt sie!
- Hoffnung:
„natürliches“ Problem $P \in \mathcal{FPT} \Rightarrow f(k)$ erträglich.

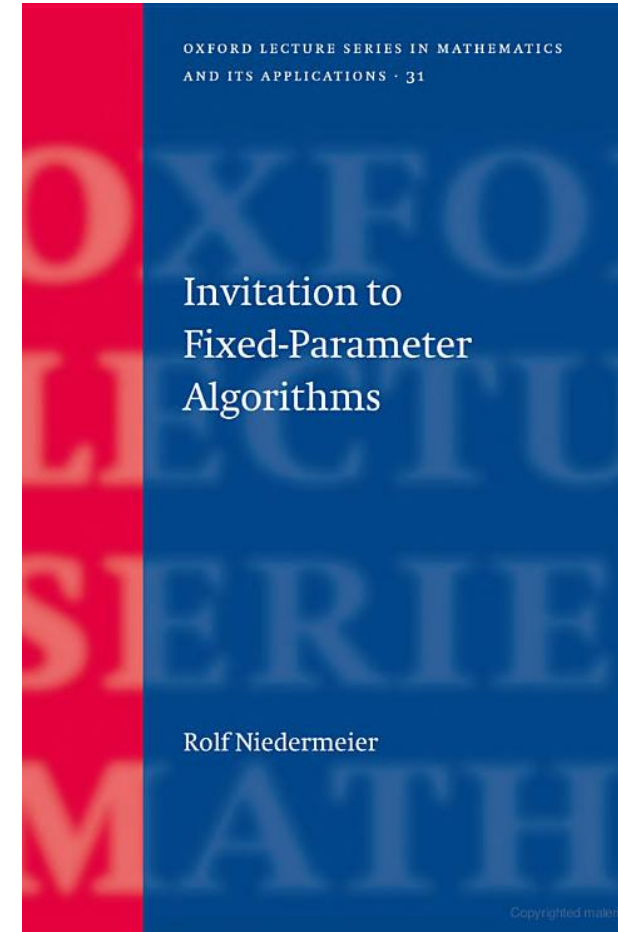
Bücher zum Thema



1999



2006



2006